

# **Parametrisierbare Lageroptimierung für eine agentenbasierte Fertigungsumgebung**

Hochschule Emden/Leer

**Master Thesis**

Studiengang Industrial Informatics (M.Eng.)

von Kristian Virkus

26. August 2010

Erstprüfer: Prof. Dr. rer. nat. Uwe Schmidtman  
Zweitprüfer: Prof. Dr.-Ing. Ewald Matull



**Institut I<sup>2</sup>AR**

Hochschule Emden/Leer

Fachbereich Technik

**Studiengang Industrial Informatics (M.Eng.)**

## **Master Thesis**

Thema der Master Thesis:

**Parametrisierbare Lageroptimierung für eine agentenbasierte Fertigungsumgebung**

Vorgelegt von:

Name, Vorname

geb. am:

Virkus, Kristian (5011544)

13.08.1984

Erstprüfer:

Prof. Dr. rer. nat. Uwe Schmidtman

Zweitprüfer:

Prof. Dr.-Ing. Ewald Matull

Datum der Ausgabe:

**08.03.2010**

Maximale Bearbeitungszeit:

24 Wochen

Spätester Abgabetermin:

**06.09.2010**

---

Die Arbeit wurde beim Prüfungsausschuss oder einer annahmefähigen Person eingereicht am:

#### ERKLÄRUNG (Official statement)

Soweit meine Rechte berührt sind, erkläre ich mich einverstanden, dass die Master-Arbeit Angehörigen der Hochschule Emden/Leer für Studium/Lehre/Forschung uneingeschränkt zugänglich gemacht werden kann.

#### EIDESSTATTLICHE ERKLÄRUNG (Affidavit)

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Master-Arbeit bis auf die offizielle Betreuung selbst und ohne fremde Hilfe angefertigt habe und die benutzten Quellen und Hilfsmittel vollständig angegeben sind.

---

Datum, Unterschrift (Kristian Virkus)

## Kurzfassung

Lager sind in ihren verschiedenen Ausprägungen das Rückgrat für Unternehmen in der Fertigungsindustrie. Ihre Verwaltungs- und Steuerungssoftware regelt nicht nur die Ein- und Auslagerungsvorgänge, sondern optimiert diese auch nach verschiedenen Gesichtspunkten, sodass vermeintlich wichtige Güter schnell am Lagerausgang zur Verfügung stehen. In dieser Arbeit werden Lagergrundlagen sowie Optimierungsstrategien aufgezeigt und die Umsetzung im Rahmen des LK<sup>3</sup>S-Projekts erläutert. Abschließend wird eine Leistungsmessung der Optimierungsstrategien durchgeführt und ausgewertet.

## Abstract

All kinds of warehouses state the backbone of manufacturing industry enterprises. Their management and control software does not only control the import and export operations but also optimises them in regard to various aspects, such that supposedly important goods are available quickly at the warehouse exits. This work presents warehouse basics as well as optimisation strategies and the realisation in scope of the LK<sup>3</sup>S project is being explained. Finally a benchmark will be performed and evaluated.

*Schlüsselworte (keywords):*

*Lager, Optimierung, Strategie, Agentenbasierte Fertigung,  
LK<sup>3</sup>S, Leistungsmessung  
Warehouses, Optimization, Strategy, Agent-Based Manufacturing,  
LK<sup>3</sup>S, Benchmark*



# Inhaltsverzeichnis

|                                                                  |             |
|------------------------------------------------------------------|-------------|
| <b>Abbildungsverzeichnis</b>                                     | <b>iv</b>   |
| <b>Tabellenverzeichnis</b>                                       | <b>vii</b>  |
| <b>Abkürzungsverzeichnis</b>                                     | <b>viii</b> |
| <b>1 Einleitung</b>                                              | <b>1</b>    |
| <b>2 Hochregallager</b>                                          | <b>4</b>    |
| 2.1 Allgemeines . . . . .                                        | 4           |
| 2.2 Digitale Fabrik . . . . .                                    | 11          |
| 2.2.1 Aufbau . . . . .                                           | 11          |
| 2.2.2 Hardware . . . . .                                         | 12          |
| 2.2.3 SPS-Programm . . . . .                                     | 15          |
| <b>3 LK<sup>3</sup>S</b>                                         | <b>21</b>   |
| 3.1 Stub . . . . .                                               | 23          |
| 3.2 Operator . . . . .                                           | 25          |
| 3.2.1 Architektur . . . . .                                      | 27          |
| 3.2.2 Grafische Darstellung . . . . .                            | 31          |
| 3.2.3 Bibliothek zur Nutzung . . . . .                           | 32          |
| 3.2.4 Konfiguration . . . . .                                    | 33          |
| 3.3 Agent . . . . .                                              | 34          |
| 3.3.1 JADE-Grundlagen . . . . .                                  | 35          |
| 3.3.2 LK <sup>3</sup> S-Agenten . . . . .                        | 45          |
| <b>4 Optimierungen</b>                                           | <b>60</b>   |
| 4.1 Analyse . . . . .                                            | 61          |
| 4.2 Optimierungsverfahren . . . . .                              | 63          |
| 4.2.1 Einlagerung – Äußere Randbedingungen . . . . .             | 64          |
| 4.2.2 Einlagerung – Abfragehäufigkeit vs. Verweildauer . . . . . | 66          |
| 4.2.3 Einlagerung – Anbruchlagereinheiten . . . . .              | 68          |
| 4.2.4 Einlagerung – Lagerplätze mit gleichem Gut . . . . .       | 69          |
| 4.2.5 Auslagerung – First In First Out . . . . .                 | 69          |
| 4.2.6 Auslagerung – Deadline . . . . .                           | 70          |
| 4.2.7 Auslagerung – Passendes Material . . . . .                 | 71          |

|          |                                                              |            |
|----------|--------------------------------------------------------------|------------|
| 4.2.8    | Auslagerung – Doppelspiele . . . . .                         | 72         |
| 4.2.9    | Multi-Aspekt-Optimierung . . . . .                           | 74         |
| <b>5</b> | <b>Lagerverwaltung und -steuerung</b>                        | <b>76</b>  |
| 5.1      | Datenhaltung . . . . .                                       | 77         |
| 5.1.1    | Datentypen . . . . .                                         | 78         |
| 5.1.2    | Verteilte Datenhaltung . . . . .                             | 87         |
| 5.2      | Logische Lagerverwaltung . . . . .                           | 88         |
| 5.2.1    | StorageFace . . . . .                                        | 89         |
| 5.2.2    | Database . . . . .                                           | 98         |
| 5.2.3    | SimulationBehaviour . . . . .                                | 101        |
| 5.3      | Physikalische Lagerverwaltung . . . . .                      | 102        |
| <b>6</b> | <b>Konfigurator</b>                                          | <b>109</b> |
| 6.1      | Konfigurator . . . . .                                       | 109        |
| 6.1.1    | Funktionen . . . . .                                         | 109        |
| 6.1.2    | Datenformat . . . . .                                        | 110        |
| 6.1.3    | Bedienung . . . . .                                          | 111        |
| 6.1.4    | Datenbankeinrichtung . . . . .                               | 117        |
| 6.1.5    | Umsetzung in Java . . . . .                                  | 119        |
| 6.2      | Visualisierung . . . . .                                     | 125        |
| 6.2.1    | Warehouse Operator Integration . . . . .                     | 125        |
| 6.2.2    | Warehouse Director Integration . . . . .                     | 130        |
| 6.3      | Benchmarking . . . . .                                       | 132        |
| 6.3.1    | Funktionalität . . . . .                                     | 132        |
| 6.3.2    | Umsetzung in Java . . . . .                                  | 135        |
| 6.3.3    | Benchmarking-Szenarien . . . . .                             | 138        |
| 6.3.4    | Kommunikationsprotokoll Agent – Warehouse Director . . . . . | 140        |
| <b>7</b> | <b>Benchmarking</b>                                          | <b>146</b> |
| 7.1      | Vorbereitungen . . . . .                                     | 146        |
| 7.2      | ABC – Sequentiell . . . . .                                  | 148        |
| 7.3      | ABC – Alternierend . . . . .                                 | 153        |
| 7.4      | Doppelspiel . . . . .                                        | 158        |
| 7.5      | Zufall . . . . .                                             | 163        |
| 7.6      | Berechnungszeiten . . . . .                                  | 163        |
| <b>8</b> | <b>Abschließende Betrachtungen</b>                           | <b>166</b> |
|          | Rückblick . . . . .                                          | 166        |
|          | Ausblick . . . . .                                           | 167        |
|          | <b>Literaturverzeichnis</b>                                  | <b>169</b> |
| <b>A</b> | <b>Konfigurationen für die Digitale Fabrik</b>               | <b>172</b> |
| A.1      | LK <sup>3</sup> S-Agent . . . . .                            | 172        |

|          |                                                         |            |
|----------|---------------------------------------------------------|------------|
| A.2      | OperatorsFace . . . . .                                 | 173        |
| A.3      | LinksFace . . . . .                                     | 173        |
| A.4      | TransportFace . . . . .                                 | 173        |
| A.5      | StorageFace . . . . .                                   | 173        |
| A.6      | Operator Skeleton – skeleton-warehouse.xml . . . . .    | 174        |
| A.7      | Warehouse Extension – extension-warehouse.xml . . . . . | 175        |
| A.8      | Warehouse Expert – expert-hrw.xml . . . . .             | 175        |
| A.9      | Simulation Expert – expert-simulation.xml . . . . .     | 176        |
| A.10     | Datenbank – cells . . . . .                             | 176        |
| <b>B</b> | <b>Genehmigung STILL</b>                                | <b>180</b> |

# Abbildungsverzeichnis

|      |                                                                                                                     |     |
|------|---------------------------------------------------------------------------------------------------------------------|-----|
| 2.1  | Hochregallager. Quelle: [2, 33] . . . . .                                                                           | 5   |
| 2.2  | Verschiedene Staplerausführungen. Quelle: [24] . . . . .                                                            | 7   |
| 2.3  | Zweimastiges Regalbediengerät mit Teleskopgabel für Europaletten. Quelle: [10] . . . . .                            | 8   |
| 2.4  | Frontansicht des Hochregallagers in der Digitalen Fabrik. . . . .                                                   | 12  |
| 2.5  | Layout des Hochregallagers in der Digitalen Fabrik. . . . .                                                         | 13  |
| 2.6  | Lastaufnahmemittel des Hochregallagers in der Digitalen Fabrik. . . . .                                             | 14  |
| 3.1  | Drei-Schichten-Modell im LK <sup>3</sup> S-Projekt. . . . .                                                         | 22  |
| 3.2  | Architektur des Stubs. Quelle: [29] . . . . .                                                                       | 24  |
| 3.3  | Beispielhaftes Sequenzdiagramm des Operator Skeleton. . . . .                                                       | 31  |
| 3.4  | Grafische Oberfläche des JADE-Frameworks. . . . .                                                                   | 35  |
| 3.5  | Grafische Oberfläche des Telegramm-Sniffers für das JADE-Framework. . . . .                                         | 36  |
| 3.6  | Grafische Oberfläche des JADE Directory Facilitators. . . . .                                                       | 37  |
| 3.7  | UML-Klassendiagramm der Agentenkonfiguration. . . . .                                                               | 56  |
| 3.8  | Vereinfachter Programmablaufplan der Methode <code>action</code> der Klasse <code>TransportFace</code> . . . . .    | 57  |
| 3.9  | Vereinfachter Programmablaufplan der Methode <code>routeItem</code> der Klasse <code>TransportFace</code> . . . . . | 58  |
| 3.10 | Vereinfachter Programmablaufplan der Methode <code>makeRoute</code> der Klasse <code>TransportFace</code> . . . . . | 59  |
| 4.1  | Links: Reihenfolge mit kurzen Wegen (Summe), rechts: eine andere Reihenfolge mit längeren Wegen . . . . .           | 63  |
| 5.1  | Anwendungsfalldiagramm eines Lagers. . . . .                                                                        | 76  |
| 5.2  | Architektur des LVS im Kontext des Agentensystems. . . . .                                                          | 77  |
| 5.3  | Illustration der Distanzbestimmung zwischen Lagerplatz und Zugang mit Hilfe des Satzes von Pythagoras. . . . .      | 82  |
| 5.4  | Ausgabe der maximal möglichen Distanz innerhalb einer Regalgasse (Zone). . . . .                                    | 82  |
| 5.5  | Stored Procedure <code>distancemax</code> . . . . .                                                                 | 83  |
| 5.6  | ER-Modell der Lagerdatenbank. . . . .                                                                               | 86  |
| 5.7  | UML-Klassendiagramm des <code>StorageFace</code> . . . . .                                                          | 90  |
| 5.8  | UML-Klassendiagramm für <code>Storage</code> und <code>Item</code> . . . . .                                        | 93  |
| 5.9  | Grafische Darstellung der Verknüpfung aller Einlagerungsstrategien. . . . .                                         | 100 |

|      |                                                                                                                                                              |     |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.10 | Grafische Darstellung der Verknüpfung aller Auslagerungsstrategien. . . .                                                                                    | 101 |
| 6.1  | Hauptfenster des Warehouse Director. . . . .                                                                                                                 | 112 |
| 6.2  | Konfigurations- und Visualisierungsansicht des Warehouse Director. . . .                                                                                     | 112 |
| 6.3  | Hochregallager-Koordinatensystem. . . . .                                                                                                                    | 115 |
| 6.4  | Dialogfenster zur Eingabe der Eigenschaften eines Lagerplatzes. . . . .                                                                                      | 116 |
| 6.5  | Detaillierte Ansicht eines Lagerplatzes im Lagerlayout des Warehouse<br>Directors. . . . .                                                                   | 117 |
| 6.6  | UML-Klassendiagramm der Lagerlayout-Datenstrukturen des Warehouse<br>Directors. . . . .                                                                      | 120 |
| 6.7  | Systemmodell der Visualisierungsfunktion. . . . .                                                                                                            | 126 |
| 6.8  | UML-Sequenzdiagramm zur Darstellung der Verarbeitung und Visuali-<br>sierung eines Einlagerungsauftrags. . . . .                                             | 142 |
| 6.9  | Programmablaufplan aller Methoden des Interface <code>IWarehouseExpert</code><br>für die spezielle Implementation im <code>SimulationExpert</code> . . . . . | 143 |
| 6.10 | Fenster zur Parametrierung eines Benchmarks. . . . .                                                                                                         | 143 |
| 6.11 | Systemmodell bei Simulationsfunktion. . . . .                                                                                                                | 144 |
| 6.12 | UML-Sequenzdiagramm zum Komponenten-Zusammenspiel. . . . .                                                                                                   | 145 |
| 7.1  | Entwicklung der relativen Anfragehäufigkeit bei zwei unterschiedlichen<br>Gütern. . . . .                                                                    | 148 |
| 7.2  | Zum Benchmarking verwendete Lagerlayouts. . . . .                                                                                                            | 149 |
| 7.3  | 5 × 5 ABC sequentiell, erstmaliges Füllen . . . . .                                                                                                          | 149 |
| 7.4  | 5 × 5 ABC sequentiell, Entnehmen . . . . .                                                                                                                   | 150 |
| 7.5  | 5 × 5 ABC sequentiell, erneutes Füllen . . . . .                                                                                                             | 151 |
| 7.6  | 10 × 10 ABC sequentiell, erstmaliges Füllen . . . . .                                                                                                        | 151 |
| 7.7  | 10 × 10 ABC sequentiell, Entnehmen . . . . .                                                                                                                 | 152 |
| 7.8  | 10 × 10 ABC sequentiell, erneutes Füllen . . . . .                                                                                                           | 152 |
| 7.9  | 5 × 5 ABC alternierend, erstmaliges Füllen . . . . .                                                                                                         | 154 |
| 7.10 | 5 × 5 ABC alternierend, siebtes Füllen . . . . .                                                                                                             | 154 |
| 7.11 | 5 × 5 ABC alternierend, neuntes Füllen . . . . .                                                                                                             | 155 |
| 7.12 | 5 × 5 ABC alternierend, letztes Füllen . . . . .                                                                                                             | 155 |
| 7.13 | 10 × 10 ABC alternierend, erstmaliges Füllen . . . . .                                                                                                       | 156 |
| 7.14 | 10 × 10 ABC alternierend, siebtes Füllen . . . . .                                                                                                           | 156 |
| 7.15 | 10 × 10 ABC alternierend, neuntes Füllen . . . . .                                                                                                           | 157 |
| 7.16 | 10 × 10 ABC alternierend, letztes Füllen . . . . .                                                                                                           | 157 |
| 7.17 | 5 × 5 Doppelspiel, Füllen . . . . .                                                                                                                          | 158 |
| 7.18 | 5 × 5 Doppelspiel, abwechselnd Ein- und Auslagern . . . . .                                                                                                  | 159 |
| 7.19 | 10 × 10 Doppelspiel, Füllen . . . . .                                                                                                                        | 159 |
| 7.20 | 10 × 10 Doppelspiel, abwechselnd Ein- und Auslagern . . . . .                                                                                                | 160 |
| 7.21 | 5 × 5 Doppelspiel mit ausschließlicher Gewichtung, Füllen . . . . .                                                                                          | 160 |
| 7.22 | 5 × 5 Doppelspiel mit ausschließlicher Gewichtung, abwechselnd Ein-<br>und Auslagern . . . . .                                                               | 161 |
| 7.23 | 10 × 10 Doppelspiel mit ausschließlicher Gewichtung, Füllen . . . . .                                                                                        | 161 |

|      |                                                                                               |     |
|------|-----------------------------------------------------------------------------------------------|-----|
| 7.24 | 10 × 10 Doppelspiel mit ausschließlicher Gewichtung, abwechselnd Ein- und Auslagern . . . . . | 162 |
| 7.25 | 5 × 5 Zufällig . . . . .                                                                      | 163 |
| 7.26 | 5 × 5 Zufällig . . . . .                                                                      | 164 |
| 7.27 | 10 × 10 Zufällig . . . . .                                                                    | 164 |
| 7.28 | 10 × 10 Zufällig . . . . .                                                                    | 165 |

# Tabellenverzeichnis

|     |                                                                                                         |     |
|-----|---------------------------------------------------------------------------------------------------------|-----|
| 2.1 | Matrixbeschreibung zur Adressierung einzelner Lagerplätze über die Codestellen C5001 bis C5003. . . . . | 17  |
| 2.2 | Beschreibung der Codestellen für Import und Export. . . . .                                             | 19  |
| 2.3 | Für Ablage und Aufnahme interessante Codestellen. . . . .                                               | 20  |
| 5.1 | Datenbankschema der Stammdaten-Tabelle <code>cells</code> . . . . .                                     | 105 |
| 5.2 | Datenbankschema der Güter-Tabelle <code>items</code> . . . . .                                          | 106 |
| 5.3 | Datenbankschema der Material-Tabelle <code>material</code> . . . . .                                    | 107 |
| 5.4 | Datenbankschema der Anfragen-Tabelle <code>requests</code> . . . . .                                    | 108 |
| 5.5 | Auswirkung eines zusätzlichen Werts je Produkt auf den Kommunikations-overhead. . . . .                 | 108 |
| 6.1 | Einstellungskombinationen für Durchlaufregale. . . . .                                                  | 116 |

# Abkürzungsverzeichnis

| <b>Abkürzung</b> | <b>vollständige Form</b>                        |
|------------------|-------------------------------------------------|
| AGV              | Automated Guided Vehicle                        |
| AGVS             | Automated Guided Vehicle System                 |
| AMS              | Agent Management System                         |
| ASRS             | Automated Storage And Retrieval System          |
| BGT              | Baugruppenträger                                |
| DBMS             | Database Management System                      |
| DC               | Dual Command (Doppelspiel)                      |
| DF               | Directory Facilitator                           |
| ERP              | Enterprise Resource Planning                    |
| FTF              | Fahrerloses Transportfahrzeug (s. AGV)          |
| FTS              | Fahrerloses Transportsystem (s. AGVS)           |
| HRL              | Hochregallager (s. HRW)                         |
| HRW              | High Rack Warehouse, auch High Bay Warehouse    |
| LVS              | Lagerverwaltungssystem (s. WMS)                 |
| MC               | Multiple Command (Mehrfachspiel)                |
| MES              | Manufacturing Execution System                  |
| mTSP             | multiple Travelling Salesman Problem            |
| RBG              | Regalbediengerät (s. ASRS)                      |
| SLS              | Staplerleitsystem                               |
| SPS              | Speicherprogrammierbare Steuerung               |
| UID              | Unique Identifier, (dt. eindeutiger Bezeichner) |
| VRP              | Vehicle Routing Problem                         |
| WMS              | Warehouse Management System                     |



# 1 Einleitung

Mit Just-in-Time und Just-in-Sequence unterstützt die Supply Chain die Produktion bei Automobilherstellern und auch anderen fertigenden Unternehmen. Gerade die großen Firmen haben Zuliefererketten etabliert, die die Lagerhaltung immer mehr reduzieren und stattdessen Waren direkt aus dem LKW oder der Bahn ans Fließband liefern. Dies ist ein wichtiger Punkt bei der Umsetzung der neuen Anforderungen an eine kundenorientierte, flexible Produktion. Dennoch gibt es Unternehmen und Bereiche, in denen Lagerhaltung notwendig ist. Sie bildet auch heute noch weiterhin das Rückgrat vieler Supply Chains, fabrikintern und unternehmensübergreifend. Eine besondere Ausprägung der Lager stellen die sog. Hochregallager dar. Sie besitzen eine besonders hohe Flächenintensität auf Grund ihrer Bauweise. Ein Hochregallager besteht in der Regel aus folgenden zum Betrieb notwendigen Komponenten:

- Regale,
- Bedienung,
- Vorlagerzone und
- Software.

Hinzu kommen Hilfsmittel zur Planung, Einrichtung und Inbetriebnahme. Im Rahmen dieser Arbeit wird ein Softwaretool entwickelt, das die strukturelle Planung am Bildschirm erlaubt, bei der Einrichtung behilflich ist und den Zustand im Betrieb erfassen und darstellen kann.

*„Im Sinne der Wertschöpfung erbringt der Lagerprozess außer beispielsweise durch Spekulationsgewinne oder bestimmte Reifeprozesse bei Lebensmitteln keine Leistung, sondern verursacht in erster Linie Kosten. Die Anzahl der Lagerprozesse und damit die Lagerzeit sollte deshalb möglichst minimiert werden.“ – SCHUMANN [23]*

Kostendruck und Zeitdruck im Kampf um die Kunden und gegen andere Unternehmen, aber auch gegen die zunehmend besorgniserregende Umweltsituation zwingen die Hersteller und Betreiber von Lagersystemen, Optimierungsverfahren bei ihren Lagern anzuwenden, um Kosten, Zeiten und Energieverbräuche zu minimieren. Bei Lagern kann dazu an verschiedenen Stellen angesetzt werden. Dabei können gewisse Faktoren bereits bei

der Lagerplanung berücksichtigt werden. Hierzu zählt die Dimensionierung des Lagers, wie z. B. das Höhe-Länge-Verhältnis der Regale. Im idealen Fall ist die maximale Distanz im Lager so zurückzulegen, dass beide Antriebe für vertikale und horizontale Fahrt gleichzeitig anfahren und mit maximaler Geschwindigkeit zeitgleich den Endpunkt erreichen. Arbeiten u.a. von SCHUMANN (s. [23]) und GLASS (s. [12]) beschäftigen sich auch mit diesem Thema. Ein anderer Punkt ist die Optimierung auf Hardwareebene mit der Analyse von Beschleunigungs- und Bremsrampen wie auch die Installation von Einrichtungen zur Energierückgewinnung bei der Bremsung von Abstiegen der Regalbediengeräte wie z. B. von BUSCH und OSTERMANN [6] näher beschrieben wurde. Diese Arbeit hingegen befasst sich mit der Optimierung der Fahrwege und der Zuweisung von Lagerplätzen bei Einlagerungsvorgängen bei bereits gegebenen Lagerstrukturen. Dabei spielen mehrere Optimierungsverfahren eine Rolle. Sie optimieren nach jeweils eigenen Gesichtspunkten und werden im Anschluss zu einer übergreifenden Entscheidungsfindung verknüpft. Dabei wird auf eine Zonierung des Lagers gänzlich verzichtet. Es herrscht stattdessen Chaos mit Strategie – chaotische Lagerplatzvergabe. Eine erste Arbeit hierzu wurde bereits von VIRKUS [28] verfasst. Im Rahmen dieser Arbeit werden einige Punkte erneut aufgegriffen, verfeinert und weiterentwickelt.

Dabei wird noch einmal sehr detailliert auf die Gegebenheiten im Technikum der Hochschule Emden/Leer, der Digitalen Fabrik, eingegangen. Einzelne Bereiche des Lagers in Hard- und Software werden erläutert und damit ein umfassender Überblick über die realen Umgebungsbedingungen gegeben.

Ein interessanter Punkt ist die Untersuchung der Ergebnisse der Lagerstrategien. Im Rahmen dieser Arbeit wird ein Simulationstool, respektive Benchmarkingtool, geschaffen, das sowohl Lageranforderungen simuliert wie auch die reale Lagerhardware. Damit können Vorgänge im Lager mit ihren Berechnungs- und grob nachgestellten Ausführungszeiten zur Auswertung festgehalten werden. Hierüber kann sowohl die Skalierbarkeit der Optimierungsverfahren mit wechselndem Lagerumfang wie auch die Qualität der Optimierungsergebnisse analysiert werden.

Diese Arbeit entstand als Unterstützung des LK<sup>3</sup>S-Projekts. Es befasst sich mit der Forschung im Bereich der agentenbasierten Fertigung, wozu auch eine beispielhafte Implementation entsteht. Sie dient in erster Linie zur Fertigungsdemonstration in der Digitalen Fabrik der Hochschule Emden/Leer und wird dabei vom Bundesministerium für Bildung und Forschung und von namhaften Industrieunternehmen unterstützt und gefördert. Dies zeigt die zunehmende Bedeutung von Agententechnologien auch außerhalb des Forschungssektors. Durch die enge Verzahnung des LK<sup>3</sup>S-Agentensystems mit dieser Arbeit und auch auf Grund einiger Besonderheiten sollte das System nicht nur am Rande erwähnt bleiben, sondern muss im Detail erläutert werden, um die Integration der entwickelten Lagerverwaltung zu verdeutlichen. So ist beispielsweise zum momentanen Zeitpunkt keine vorausschauende Planung für die Beschaffung und Einlagerung von Material verfügbar. Dies kann zwar die Implementation der Lagersoftware vorübergehend vereinfachen, fordert jedoch gleichzeitig eine gute Umsetzung der Lagerstrategien, da

Lageranfragen unerwartet kommen und in der Regel ohne Verzögerung bedient werden müssen. Wird neben dem Agentensystem ein herkömmliches *Enterprise-Resource-Planning*-System (ERP) eingesetzt, so ist eine entsprechende Anbindung notwendig, die dem ERP-System entweder die Steuerung der Lagerhardware ermöglicht oder die i.d.R. bereits integrierten Verwaltungsfunktionen ersetzt.

Zunächst werden in Kapitel 2 allgemeine Grundlagen zu Hochregallagern erläutert, um im Anschluss mit Abschnitt 2.2 auf die vorhandene Testumgebung, die Digitale Fabrik der Hochschule Emden/Leer, einzugehen. Daraufhin werden mit Kapitel 4 Einblicke in die Optimierung von Hochregallagern gegeben und die Grundlagen der Umsetzung im Rahmen dieser Arbeit dargelegt. Kapitel 3 stellt anschließend das LK<sup>3</sup>S-Projekt mit seiner Architektur vor, um den Leser mit den nötigen Hintergrundinformationen zu versorgen, die für das Verständnis der lagerspezifischen Implementation nötig sind. Im Folgenden werden die Datenhaltung und die speziellen Softwarekomponenten zum logischen und physikalischen Betrieb eines Lagers in Kapitel 5 vorgestellt. Kapitel 6 geht auf das besagte Softwaretool zur Einrichtung (Abs. 6.1), zur Visualisierung (Abs. 6.2) und zum Benchmarking (Abs. 6.3) ein. Schließlich werden einige Aufzeichnungen von Benchmarkingdurchläufen in Kapitel 7 ausgewertet und abschließend eine rückblickende Übersicht über diese Arbeit und einen Ausblick auf weiterführende Arbeiten gegeben.

## 2 Hochregallager

### 2.1 Allgemeines

Eine sehr effiziente Art der Lagerung von Gütern für Produktion und Vertrieb ist die Lagerung in *Hochregallagern* (HRL). Sie besitzen eine besonders hohe Flächenintensität, die sich auf die Anzahl der lagerbaren Paletten pro Quadratmeter bezieht. Üblicherweise werden Lager mit diskreten Lagerplätzen für Lagereinheiten ab einer Höhe von mindestens 12 m als Hochraumlager und bei der Verwendung von Regalen als Hochregallager bezeichnet (s. MARTIN [20]). Auf Grund ihrer diskreten Lagerplätze fallen sie in die Kategorie Stückgutlager. Güter können z. B. auf Paletten oder in Behältern gelagert werden. HRL können sowohl im Außenbereich als Freilager wie auch im Innenbereich als Gebäudelager betrieben werden. Während viele moderne Gebäudelager bereits vollautomatisch unter Verwendung automatischer Regalbediengeräte betrieben werden, eignen sich Freilager auf Grund der Umgebungsbedingungen eher für die manuelle Bedienung, beispielsweise mit Staplern. Oft bestehen Hochregallager aus mehreren Regalen, die auf einer Seite durch Gänge voneinander getrennt sind und auf der anderen Seite mit ihren Rücken aneinander stehen. Ist ein Lager in Bereiche gegliedert, spricht man auch von Lagerzonen. Über den Ort der Einlagerung eines Guts entscheidet die Lagersoftware anhand verschiedener Kriterien und gibt entsprechende Kommandos an ein Regalbediengerät oder ein Staplerleitsystem.

Ein Hochregallager besteht in der Regel aus mehreren Komponenten bzw. Systemen:

- Regale,
- Bedienung,
- Vorlagerzone und
- Software.

Abbildung 2.1a zeigt einen Blick auf mehrere Regale (vgl. MARTIN [20], S.332, Bild 9.2).

Im folgenden werden die einzelnen Komponenten näher erläutert.



(a) Blick auf Regale eines Hochregallagers. (b) Hochregallager im Bau.

Abbildung 2.1: Hochregallager. Quelle: [2, 33]

**Regale** Bei den verwendeten Regalen handelt es sich um stabile Konstruktionen (siehe Abb. 2.1b), die sich nach aktuellem Stand bis zu 55 m in die Höhe erstrecken. Sind mehr als zwei Regale im Lager vorhanden, so werden je zwei Regale durch eine Gasse voneinander getrennt. Sie stehen mit ihren Rückwänden, sofern Wände real vorhanden sind, aneinander und bilden damit ein *Doppelregal*. Kann ein Gut in einer Gasse in ein Regal eingeführt und in einer anderen Gasse wieder entnommen werden, so handelt es sich dabei um ein *Durchlaufregal*. Dabei finden Schwerkraftrollbahnen Einsatz, sodass ein eingesetztes Gut nur auf Grund seiner eigenen Masse auf der Rollbahn des Lagerplatzes zu gleiten beginnt, bis es am anderen Ende der Rollbahn entweder auf ein zuvor eingelagertes Gut stößt oder durch einen Endstopp aufgehalten wird<sup>1</sup>. Ist die Rollrichtung entgegen der Einfuhrriichtung, also in Richtung des Ganges, so werden bei der Einbringung alle bereits am Lagerplatz vorhandenen Güter weiter hinein geschoben und das neue Gut als erstes hinterlegt. Damit sind die auch aus anderen Bereichen bekannten Strategien *First In First Out* (FIFO) und *Last In First Out* (LIFO) realisierbar, beispielsweise zur Handhabung von verderblichen Gütern.

**Bedienung** Um Materialbewegungen im Lager und den Regalen durchzuführen, sind Mittel zur Bedienung notwendig, die manuell oder automatisch bei der Ein- und Auslagerung von Gütern behilflich sind. Sie können in zwei Klassen unterteilt werden: Stapler und Regalbediengeräte.

In Lagern mit großzügig dimensionierten Gängen kommen durchaus Gabelstapler bis zu einer Regalhöhe von 14 m zum manuellen Einsatz. Man unterscheidet hier Frontstapler, Schubmaststapler und Schmalgang-Hochregalstapler (keine Abb.), letztere oft in Kombination mit Kommissioniereinrichtungen (Kommissionierstapler bzw. Mann-oben-Schmalgangstapler). Gerade letzterer eignet sich für den Einsatz in Lagern mit schmalen Gängen, sodass hier die Flächenintensität gesteigert werden kann. Entsprechende

<sup>1</sup>Quelle: A. G. Regal, [http://www.ag-regal.com/produkte\\_fr.php?id\\_kategorie=5](http://www.ag-regal.com/produkte_fr.php?id_kategorie=5), 20.08.2010

Stapler sind ab einer Breite von 1,20 m erhältlich<sup>2</sup>. Sie unterstützen zum Teil auch Energierückgewinnung aus Bremsungen<sup>3</sup>. Die Steuerung der Stapler geschieht durch menschliche Bediener, ihre Führung ist jedoch heute oft durch ein *Staplerleitsystem* (SLS) realisiert. Ein SLS besteht aus einer Software mit Anbindung an das Lagerverwaltungssystem und einem Terminal in jedem Stapler. Der Staplerfahrer erhält seine Fahrhinweisungen auf dem Display angezeigt und kann auch entsprechend interagieren. Stand der Technik sind jedoch sogenannte Pick-by-Voice-Systeme, bei denen der Staplerfahrer sprachgesteuert mit einem Computer kommuniziert und auf diesem Wege seine Fahrhinweisungen erhält und Rückmeldungen geben kann. Dieses Verfahren wird oft bei Kommissionieraufgaben eingesetzt, da der Fahrer so ständig beide Hände für seine Tätigkeit frei hat. Pick-by-Light-Systeme zeigen Regalentnahmestellen für Kommissioniertätigkeiten visuell an den entsprechenden Lagerplätzen an. Betritt ein Kommissionierer einen Lagerbereich, fordert er das *Lagerverwaltungssystem* (LVS) z. B. durch Scannen eines auftragsgebundenen Barcodes auf, alle Entnahmestellen im Lagerbereich für ihn zu markieren. Über Displays an allen Entnahmestellen wird dem Kommissionierer die Anzahl der zu entnehmenden Teile angezeigt und er kann die Entnahme direkt am Regal gegenüber dem System bestätigen (s. MARTIN [20]).

Neben den in Hochregallagern hauptsächlich durch Stapler vertretenen manuellen Hilfsmitteln existieren mit *fahrerlosen Transportfahrzeugen* (FTF) nach VDI-Richtlinie 2510 auch vollautomatische Hilfsmittel. Sie ähneln zum Teil Staplern, werden jedoch ihrem Namen entsprechend vollständig durch einen Computer gesteuert. Ein menschlicher Bediener fährt nicht einmal als Beobachter mit. Fahraufträge werden im *fahrerlosen Transportsystem* (FTS) mit Hilfe eines FTS-Leitsystems übergeben. Aufgaben des Leitsystems sind die optimale Zuordnung von Fahraufträgen zu einzelnen FTF, um die insgesamt zurückgelegten Strecken im Lager zu minimieren. Die Problemstellung ähnelt der des *multiple Traveling Salesman Problem* (mTSP), bei dem mehrere Handlungsreisende von einem gemeinsamen Startpunkt eine definierte Menge Orte besuchen müssen, sodass jeder Ort genau ein einziges Mal von einem einzigen Handlungsreisenden besucht wurde, um dann wieder zum Startpunkt zurück zu kehren. Im Zusammenhang mit Logistik spricht man hier auch vom *Vehicle Routing Problem* (VRP), bei dem zusätzlich Kapazitätsbeschränkungen je FTF berücksichtigt werden müssen (s. WebVRP [8]). Ihre Orientierung im Lager erhalten die FTF anhand von Leitlinien optischer, magnetischer oder induktiver Natur. Sie können entweder kontinuierlich eingerichtet sein oder rasterförmig, sodass FTF zusätzlich zur Navigation zwischen zwei Rasterpunkten die Koppelnavigation nutzen. Außerdem können Methoden zur laserbasierten Navigation und über Funk (WLAN, Bluetooth, GPS) angewandt werden.

Die zweite Klasse der Hilfsmittel wird *Regalbediengerät* (RBG) genannt. Es handelt sich dabei um ein geführtes Hebezeug. Sein Fahrwerk bewegt sich entlang einer Führungsschiene und ist mit einem Mast gekoppelt. Der durch einen Seil- oder Kettentrieb über ihn

---

<sup>2</sup>Quelle: STILL, [http://still.de/downloads.php?filename=MX-X\\_MX-Q\\_HHd\\_2009\\_TD\\_web.pdf&backuri=mx-x.0.43.html&type=datasheet](http://still.de/downloads.php?filename=MX-X_MX-Q_HHd_2009_TD_web.pdf&backuri=mx-x.0.43.html&type=datasheet)

<sup>3</sup>Quelle: Hyster, <http://www.hyster.com/Americas/en-US/Products/ElectricNarrowAisle/V30-35ZMU.htm>



Abbildung 2.2: Verschiedene Staplerausführungen. Quelle: [24]

geführte Hubschlitten besitzt ein Lastaufnahmemittel, das mit dem Hubschlitten fest verbunden ist. Es ist entsprechend der physikalischen Gegebenheiten der Lagereinheit ausgeführt (s. Abb. 2.3). Als gassengebundene Version verfährt ein RBG genau in der Gasse zwischen zwei Regalen. Mit Hilfe von Umsetzern können RBG auch einen Gassenwechsel vollziehen. Zum Teil sind sie kurvengängig, sodass ein Gassenwechsel beispielsweise auch ohne Umsetzer über Weichen möglich ist. RBG existieren in manuellen, halb- und vollautomatischen Varianten. Bei manuellen RBG werden alle Einzelschritte der Hardwarereuerung durch einen Bediener initiiert, während im halbautomatischen Betrieb bereits Einzelschritte zu Teilfunktionen zusammengefasst worden sind. Im vollautomatischen Betrieb verfahren die RBG eigenständig und führen auch Ein- und Auslagerungsvorgänge in den Regalen eigenständig durch. Dabei geschieht die Positionsbestimmung im Regal entweder über Winkelcodierer oder mittels Laser-Distanzmessung. Gassen müssen beim Einsatz von RBG entsprechend nach EN 528 auf den Zutritt von Personen reagieren, um Gefahrensituationen auszuschließen (s. TEN HOMPEL [25], Wikipedia [32]).

**Vorlagerzone** Die Vorlagerzone kann auch als Schnittstelle des Lagers zu seiner Umgebung bezeichnet werden. Hier endet der Zuständigkeitsbereich der RBG, falls vorhanden. Ausgelagerte Ware kann hier einem menschlichen Kommissionierer zur Verfügung gestellt oder mit Hilfe von Staplern zum Versand in entsprechende Transportmittel verladen werden. Wird das Lager als Pufferlager zum Ausgleich von Schwankungen in Produktion und Materialversorgung genutzt, so stellt die Vorlagerzone oft eine automatisierte Lösung zur Versorgung der lokalen Produktionsstätten dar. Die Anbindung erfolgt dann beispielsweise über Rollenförderer.



Abbildung 2.3: Zweimastiges Regalbediengerät mit Teleskopgabel für Europaletten.  
Quelle: [10]

**Software** Software zur Lagerverwaltung ist ein wichtiger Bestandteil zur Realisierung der Supply Chain bzw. Logistik. Die Aufgaben eines Lagerverwaltungssystems umfassen viele Bereiche. Hierzu existiert bereits eine große Anzahl universeller Lösungen. Darunter befinden sich sowohl proprietäre wie auch Open-Source-Lösungen. Zu den Proprietären zählen LES/TRM bzw. die Neuentwicklung EWM-MFS (Extended Warehouse Management Material Flow System) von SAP, das vollständig in R/3 integriert ist. Ebenso ist das MMS.i-System von STILL als Gesamtsystem zu erwähnen. Damit ist STILL nicht nur Anbieter von Fahrzeugen zum Einsatz im Lager, sondern steigt damit auch in das Lagermanagement ein und führt sämtliche Informationsstränge von ERP bis SLS zusammen. Diese Bündelung von verschiedenen Kompetenzen und Services in einem allumfassenden Produkt ist eine aktuell zu beobachtende Entwicklung zur Steigerung des Mehrwerts und der Kundenbindung. Auf Seite der Open-Source-Lösungen ist an erster Stelle myWMS zu nennen, das jedoch nur ein Framework für die Erstellung eigener LVS ist. Hierauf basierend ist das größte und mit kommerziellen Produkten vergleichbare LinogistiX-Projekt entstanden. Es wurde in Java entwickelt und kann mit externen Modulen über RMI<sup>4</sup> und Webservices erweitert werden. Über Webservices geschieht auch die Anbindung an ein ERP-System.

Zu den häufigsten Aufgaben eines LVS gehören:

---

<sup>4</sup>Remote Method Invocation, siehe Wikipedia: [http://de.wikipedia.org/wiki/Remote\\_Method\\_Invocation](http://de.wikipedia.org/wiki/Remote_Method_Invocation)



**Wareneingang** Ein LVS bietet meist verschiedene Benutzerkonten zur Rollenverteilung dessen Bediener. So unterstützt das LVS im Wareneingang bei der Warenannahme. Es werden beispielsweise Handgeräte mit Barcode- oder RFID-Scanner unterstützt, um eingetroffene Ware zu identifizieren und im System für die weitere Verarbeitung und Lagerung zu hinterlegen.

**Warenausgang** Der Materialfluss in umgekehrter Richtung kann ebenfalls vom LVS gehandhabt werden. Dazu zählen Vorbereitungen vor und für den Abtransport der Waren. So wird Bedienern zum Teil bei der (Neu-)Etikettierung der Ware geholfen und die Übergabe an einen Logistikpartner in die Wege geleitet. Hierzu können auch Stammdaten der belieferten Empfänger hinterlegt werden.

**Kundenverwaltung** Wird ein Lager als Zwischenlager oder Distributionslager gemeinsam von mehreren Parteien betrieben oder die Lagerlogistik als Dienstleistung angeboten, so muss ein LVS auch Stammdaten bezüglich eigener Partner bzw. Kunden verwalten und bei der Ein- und Auslagerung Rücksicht auf die Zuordnung von Gütern zu Kunden nehmen sowie eventuell garantierte Kapazitäten berücksichtigen.

**Stammdatenpflege** Zu den Aufgaben eines LVS gehört auch die bekannt sein, wie die Vorlagerzone konfiguriert ist, wie viele Regale vorhanden sind, welche Mittel zur Materialbewegung zur Verfügung stehen und wie auch alle einzelnen Lagerplätze beschaffen sind. Hier muss bei der Einlagerung Rücksicht auf die Maße, Belastbarkeit und in speziellen Fällen z. B. auch auf eine garantierte Temperatur genommen werden. Hinzu kommen Informationen über Lagerstrategien wie LIFO, FIFO und *First Expire First Out* (FEFO).

**Lagerplatzzuordnung** Beim Zugang eines neuen Guts muss ein freier Lagerplatz gefunden werden, der im Anschluss mit dem neuen Gut belegt wird. Hierzu existieren unterschiedliche Strategien, die auch in Kombination sicherstellen sollen, dass der Lagerplatz einerseits den Anforderungen nach Maßen, Belastbarkeit und weiteren Umgebungsparametern genügt, andererseits aber auch der Abruf- oder Umschlaghäufigkeit des Guts gerecht wird. Zu den bekannten Strategien zählen die ABC- und Querverteilungsstrategie, die u.a. in VIRKUS [28] beschrieben wurden. Einfluss auf die Wahl eines Lagerplatzes, aus dem ein Gut ausgelagert werden soll, kann das Zusammenspiel mit Optimierungsstrategien wie dem Doppelspiel haben, wobei Ein- und Auslagerungen so kombiniert werden, dass möglichst kurze Wege gefahren werden. Um dies zu realisieren, ist es u.a. auch erforderlich, mehrere Anfragen für Ein- und Auslagerungen mit ihren spätesten Ausführungszeitpunkten zwischenspeichern. In der Regel wird die in der Branche anerkannte FEM 9.851 (siehe [26]) zum Leistungsnachweis für RBG-Spielzeiten herangezogen.

**Materialbewegungen** In einigen Fällen ist ein LVS für die Umsetzung der Materialbewegungen im Lager zuständig. Darunter fällt die Bedienung von RBG und auch die Übergabe von Anweisungen an ein SLS. Beides kann direkt im LVS integriert sein. So kann bei SAP EWM-MFS beispielsweise eine Zwischenschicht zwischen ERP und einer SPS entfallen. Auch das SLS kann direkt im LVS integriert sein.

**Visualisierung** Zur Überwachung des Gesamtsystems werden häufig Visualisierungen in Form von Modulen, eigenständigen Anwendungen oder Webinterfaces in ein LVS integriert. Hierüber ist dann auch eine manuelle Steuerung von Ein- und Auslagerungsvorgängen möglich. Einzelne Regale und die Zustände der RBG bzw. der im Einsatz befindlichen Stapler sowie anstehende Aufträge werden dem Bediener zum Teil in Echtzeit in 2D oder 3D präsentiert.

**Kommissionierung** Neben der Kommissionierung direkt in den Regalgassen, z. B. mit Hilfe von Mann-oben-Schmalgangstaplern, können Güter auch durch RBG in die Vorlagerzone verbracht werden. Mitarbeiter können hier dann Sendungen zusammenstellen und die zuvor empfangenen Lagereinheiten (z. B. Palette) wieder zurück in die Regale verbringen lassen. Ein LVS hilft dem Bediener hier nicht nur beim Holen und Verbringen der Waren aus dem Lager, sondern auch bei der Durchführung der Kommissioniertätigkeit durch Listen und Dialoge am Bildschirm. Zudem kann ein solches LVS auch berücksichtigen, ob Lagereinheiten der gleichen Waren bereits angebrochen wurden und diese bevorzugt zur Kommissionierung bereitstellen.

**Protokollierung** Zur lückenlosen Dokumentation des Herstellungsprozesses eines Produkts und dessen Transportwegs sind auch alle Materialbewegungen im Lager für jedes einzelne Gut zu protokollieren. Diese Informationen sind relevant, falls Kunden etwaige Regressansprüche auf Grund von Qualitätsmängeln geltend machen wollen. Das LVS führt diese Tätigkeit im Hintergrund aus.

**Auswertung** Ein fortschrittliches LVS wertet alle Materialbewegungen hinsichtlich der Umschlagleistung im Lager aus und hilft sich somit selbst, stetig Optimierungen einzubringen. Die Auswertungen können jedoch auch vom Bediener abgerufen werden, um Statistiken zu bestimmten Fragestellungen zu erhalten.

**Inventur** In der Bundesrepublik Deutschland ist jeder Kaufmann nach § 240 HGB und §§ 140, 141 AO verpflichtet, jährlich eine Inventur (Stichtagsinventur) zur Bestandsaufnahme im Unternehmen durchzuführen. In Bezug auf ein Lager kann das LVS bei der Inventur behilflich sein. Es kann eine Durchfahrt sämtlicher Lagerplätze erfolgen, um

so die Lagerbelegung festzustellen. Funktechnologie wie RFID kann diesen Vorgang unter Umständen erheblich beschleunigen. Alternativ kann auch eine permanente Inventur durchgeführt werden. Gerade vollautomatische Hochregallagersysteme sind hierfür geeignet, da kein manueller Eingriff im Lager erfolgt, sondern alle Vorgänge exakt über das LVS protokolliert und zur Inventur herangezogen werden können (s. HGB [5], AO [4]).

**Leergutverwaltung** Zu den Aufgaben eines LVS kann auch die Verwaltung von Leergut gehören. Dies umfasst in der Regel Sammlung, Verteilung und Abtransport leerer, nicht länger benötigter Paletten.

**Sonstiges** Einige LVS bieten Erweiterungen für bestimmte Anwendungsgebiete, z. B. zum Handling von Gefahrstoffen.

## 2.2 Digitale Fabrik

Nachdem der generelle Aufbau von Hochregallagern beschrieben wurde, soll in diesem Abschnitt dargestellt werden, welche Eigenschaften sich im HRL der Digitalen Fabrik wiederfinden. Es wurde über Jahre hinweg durch studentische Arbeiten in Zusammenarbeit mit der hauseigenen Werkstatt in Hard- und Software konzipiert und umgesetzt.

### 2.2.1 Aufbau

Das HRL der Digitalen Fabrik (s. Abb. 2.4) ist nach obigen Formulierungen mit weniger als 4 m Höhe kein Hochregallager. Da die Fabrik an sich jedoch eine Modellfabrik, zum Teil im Miniaturformat, ist, sei das HRL als Miniatur-HRL zu betrachten. In der Breite misst es ca. 2,5 m. Alle Lagerplätze besitzen eine Einheitsbreite von 18 cm und eine Einheitstiefe von 17,5 cm. Es ist von seiner Konstruktion ausschließlich für einfachtiefe Lagerung konzipiert. Die Lagerplätze liegen in drei verschiedenen Höhenklassen vor. Mit 32 cm existiert eine Schicht für besonders hohe Güter. Zwei Schichten mit je 20 cm Höhe können für Güter mit leicht größerer Höhe verwendet werden und alle übrigen Schichten besitzen eine Höhe von 15 cm. Damit hat ein Regal entsprechend 56 Lagerplätze. Das HRL der Digitalen Fabrik besitzt noch ein zweites, gegenüberliegendes Regal. Dieses besitzt jedoch weniger Lagerplätze, da hier die Lagerein- und ausgänge integriert sind. Auf Grund ihres Raumbedarfs kann ein Regalbereich von 3 Lagerplätze in der Breite und 4 in der Höhe nicht zu Lagerzwecken genutzt werden. Es bleiben somit noch 44 Lagerplätze in diesem Regal übrig, sodass das HRL insgesamt 100 Lagerplätze zur Verfügung stellt (s. Abb. 2.5). Beide Regale werden über ein einziges RBG bedient.

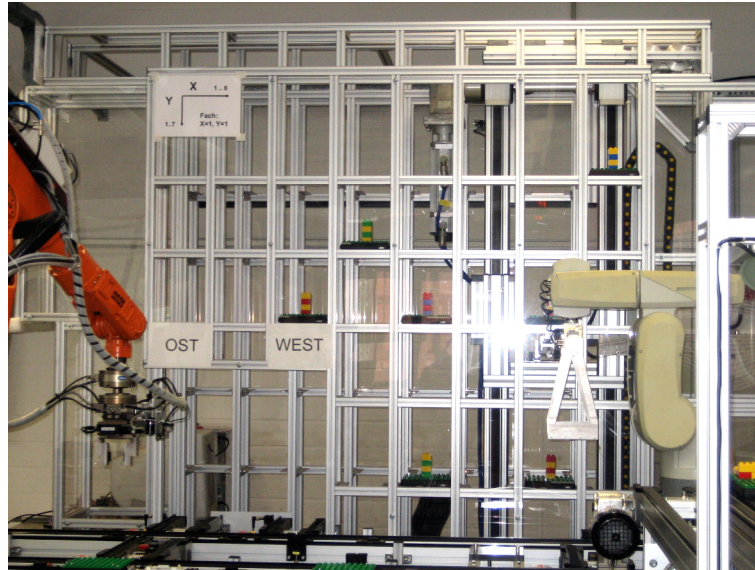


Abbildung 2.4: Frontansicht des Hochregallagers in der Digitalen Fabrik.

Die Lagerausgänge führen bei diesem HRL nicht in eine dedizierte Vorlagerzone, sondern direkt zu zwei Förderbändern, die sich bereits in der Produktionsumgebung befinden. Dies spiegelt sich auch in der Transportlogik wieder, denn die Steuerung der Bänder obliegt nicht dem LVS, sondern dem Hauptumlauf.

Das gesamte Lager wurde mit transparentem Plexiglas ummantelt, um die Betriebssicherheit zu erhöhen. Da die eingesetzten Antriebe sehr kraftvoll sind, wäre ein manuelles Eingreifen bei Bewegungen im Lager unter Umständen lebensgefährlich.

### 2.2.2 Hardware

Die Regale bzw. Fächer des HRL sowie der Mast des RBG sind durch Aluminiumprofile realisiert. Das RBG wird ausschließlich am Boden durch eine Zahnleiste geführt und bewegt. Über eine Schleppkette findet die Strom- und Signalzuleitung statt. Zum seilgetriebenen Hubschlitten existiert ein entsprechendes Gegengewicht, das jedoch in erster Linie das Gewicht des Hubschlittens und nicht das der geladenen Güter ausgleicht. Auf dem Hubschlitten ist ein Lastaufnahmemittel installiert, das in beide Richtungen rechtwinklig zum horizontalen und vertikalen Regalverlauf ausfahren kann. Durch entsprechende Sensorik wird die Position des Lastaufnahmemittels festgestellt und auf drei diskrete Zustände abgebildet. Es ist ausschließlich für einfachtiefe Lagerplätze vorgesehen und verfügt über eine spezielle Aufnahmevorrichtung (s. Abb.2.6), die genau auf die in der Digitalen Fabrik eingesetzten Baugruppenträger ausgerichtet ist. Die beiden anderen Achsen im Lager verfügen über Inkrementalgeber, sodass eine genauere Positionsbestimmung möglich ist.

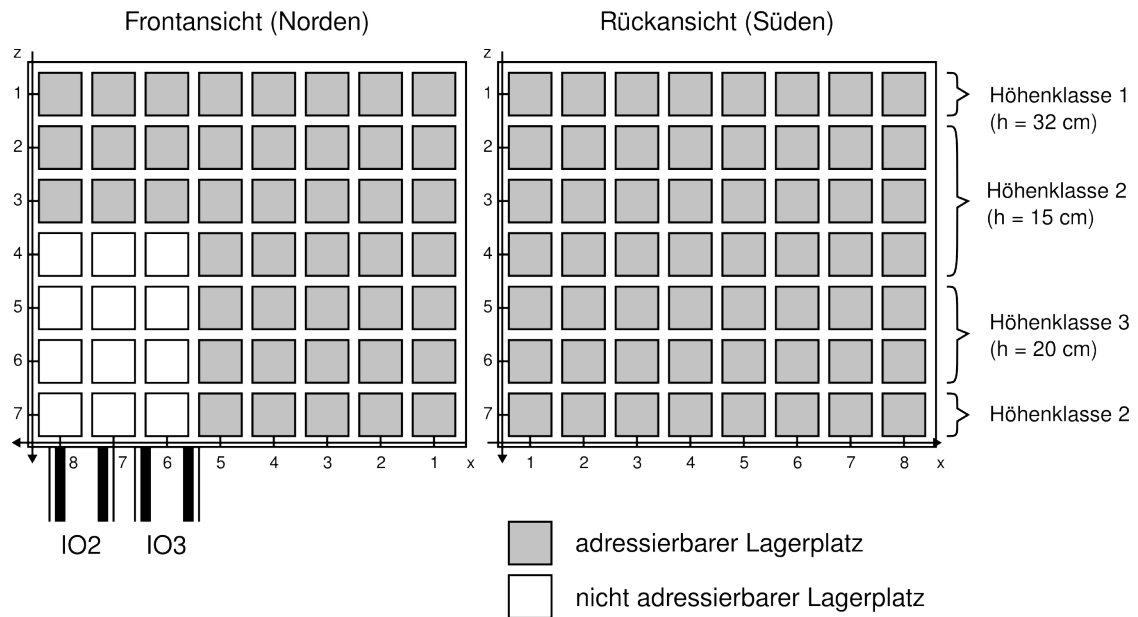


Abbildung 2.5: Layout des Hochregallagers in der Digitalen Fabrik.

Jede der drei Achsen wird über einen eigenen elektrischen Antrieb verfahren. Für die Horizontal- und Vertikalachse kommt jeweils ein Lenze-Antrieb mit 0,8 kW zum Einsatz, während das Lastaufnahmemittel mit einem wesentlich geringer dimensionierten Lenze-Antrieb mit 0,075 kW bedient wird. Ihre Ansteuerung wird von drei Lenze-Frequenzumrichtern des Typs „9300 servo plc“ realisiert. Letztere besitzen jeweils eine eigenständige SPS, die über das Lenze Drive Developer Studio programmiert werden kann. Mehr dazu im nächsten Abschnitt. Die Anbindung der Umrichter untereinander erfolgt über die seriennmäßige CAN-Schnittstelle, die einen Lenze-Dialekt als Protokoll nutzt. Im Rahmen dieser Arbeit galt es zudem, die Frequenzumrichter auch über eine optionale CANopen-Anbindung mittels Lenze 2178 Kommunikationsmodul in Betrieb zu nehmen. Eine entsprechende Vorarbeit wurde bereits von BUSCH und OSTERMANN (s. [6]) geleistet. Weiterhin bestehende Probleme konnten in Zusammenarbeit mit einem studentischen Projekt von WERMANN JR., LÜCKE und KYEK (s. [31]) sowie mit Herrn Wermann Senior lokalisiert und behoben werden. Nach genauer Betrachtung der CANopen-Telegrammheader stellte sich heraus, dass fälschlicherweise eine unzureichende Telegrammlänge angenommen wurde. Es traten Fehlpositionierungen auf, sobald Lagerplätze mit einem horizontalen oder vertikalen Index größer als sechs angefahren werden sollten. Da der Index zur Verarbeitung durch die SPS zuvor mit einem Faktor von zehntausend beaufschlagt wird, jedoch durch eine zu geringe Telegrammlänge nur 16 Bit (vorzeichenlos 0-65535) übertragen wurden, wurden alle Indizes größer sechs (entsprechend sechzigtausend) implizit durch das SPS-Programm der Frequenzumrichter auf einen anderen Wert zurückgeführt.

Weiterhin ist zu beachten, dass die Einschaltreihenfolge der Frequenzumrichter (1.) und des CANopen-Masters (2.) eingehalten werden muss. Der CANopen-Master ist in die-

sem Fall ein Industrie-PC, der mit einer entsprechenden Interface-Karte ausgestattet ist. Das Einschalten bezieht sich beim IPC auf den Beginn der Tätigkeit der Interface-Karte. Empfängt ein CANopen-Slave (SPS) ein Telegramm, bevor dieser seine Initialisierung abgeschlossen hat, so verfällt er häufig in einen Fehlerzustand, der nur durch Trennen des zugehörigen Frequenzumrichters vom Netz zurückzusetzen ist. Wird der CANopen-Master jedoch so eingerichtet, dass ein Netzwerk-Management-Telegramm zum globalen Start der CANopen-Kommunikation nicht mit Stromzufuhr der CANopen-Interface-Karte erfolgt, sondern erst nach erfolgtem Systemstart, dann sind alle Frequenzumrichter mit ihren Schnittstellenmodulen bereits hochgefahren und können mit einer sehr hohen Wahrscheinlichkeit in einen kommunikationsbereiten Zustand versetzt werden.

Als weiterer Punkt hat sich herauskristallisiert, dass eine Erhöhung der Übertragungsrate mit einer Verringerung der Telegrammverluste bei einer hohen Buslast einher geht. Die Ursache konnte noch nicht abschließend festgestellt werden. Da jedoch eine Telegramm-Pufferung sowohl im CANopen-Master wie auch in den CANopen-Slaves durchgeführt wird, liegt die Vermutung nahe, dass es zu einem Pufferüberlauf in den Slaves kommt. Dieser entsteht, da die Slaves eingehende Telegramme zwar schnell genug verarbeiten können, diese zum Senden jedoch in einen Puffer legen müssen, weil sie keinen exklusiven Buszugriff besitzen. In Versuchen mit unterschiedlichen Übertragungsraten hat sich herausgestellt, dass die maximal wählbare Übertragungsgeschwindigkeit von 1 Mbit/s auch über einen Zeitraum von mehreren Stunden keinen Fehlerzustand in den CANopen-Slaves hervorruft. Vor einem Einsatz in einer Produktivumgebung müsste dieser Ansatz jedoch über einen längeren Zeitraum getestet werden.

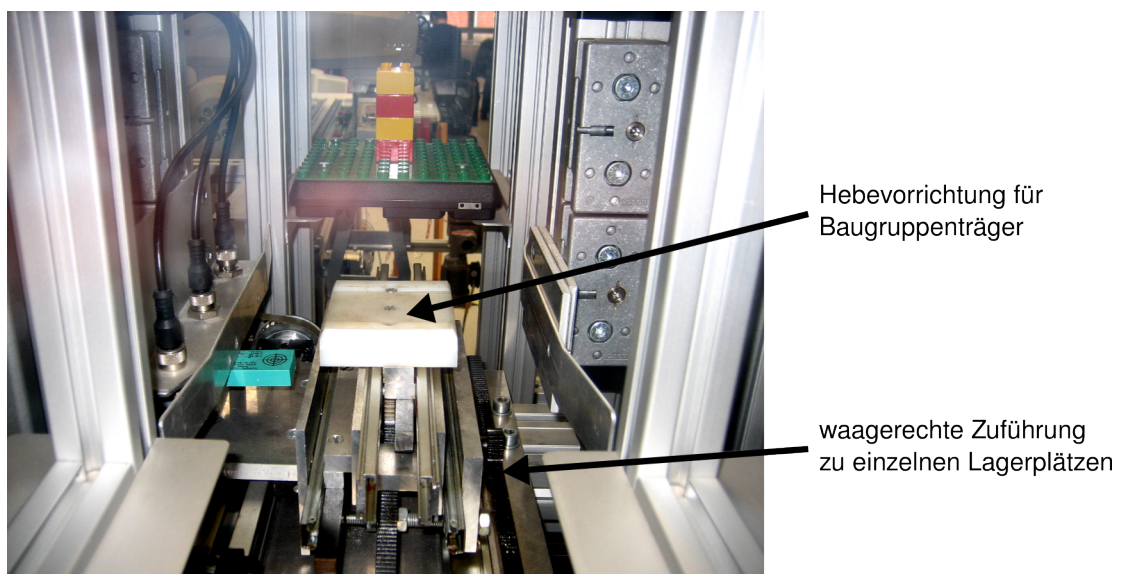


Abbildung 2.6: Lastaufnahmemittel des Hochregallagers in der Digitalen Fabrik.

### 2.2.3 SPS-Programm

Wie bereits erwähnt, sind die eingesetzten Frequenzumrichter jeweils mit einer SPS ausgestattet, sodass auch komplexe Steuerungsvorgänge ohne eine überlagerte Steuerung ausgeführt werden könnten. In diesem Fall werden die SPSen jedoch genutzt, um eine Schnittstelle zur Bedienung des Lagers für eine überlagerte Steuerung zur Verfügung zu stellen. Dabei werden einzelne Vorgänge gekapselt, sodass von außen lediglich eine abstrakte Sicht auf das Lager und seine Parameter möglich ist. Die Kommunikation über CANopen bezieht sich auf das Lesen und Schreiben von Parametern im CANopen-Slave. Viele Parameter bzw. Merker der SPS werden auf sogenannte Codestellen abgebildet. Damit sind sie von außen über CANopen je nach Konfiguration les- oder schreibbar. Da dies die einzige Möglichkeit ist, über CANopen und die verwendeten CANopen-Kommunikationsmodule mit den SPS-Programmen zu kommunizieren, sind auch Aktionen wie das Auslagern von einem Lagerplatz auf Codestellen abgebildet. Das SPS-Programm führt dazu eine Flankenerkennung der entsprechenden booleschen Merker bzw. Codestellen durch. Wird eine Codestelle über CANopen also von Null auf Eins gesetzt, kann dies eine Aktion in der SPS auslösen.

Neben CANopen beherrschen die Frequenzumrichter auch native Kommunikation über CAN. Diese Anbindung wird einerseits von Projekten unter Führung von Herrn Matull genutzt, andererseits jedoch auch zur Kommunikation der SPSen untereinander. Nicht nur die einzelnen Achsen der Lagersteuerung, sondern auch die zugehörige Logik wird je von einem eigenen Frequenzumrichter bearbeitet. Die SPS der Horizontaleinheit besitzt zudem Funktionen, die die Gesamtheit der Lagersteuerung betreffen. So geschieht die Steuerung des Lagers aus externer Sicht ausschließlich durch Kommunikation über Codestellen mit der SPS der Horizontaleinheit. Da die weiteren Beiden jedoch auch über CANopen erreichbar sind, können sie für eine feinere Überwachung der Vorgänge herangezogen werden (s. Tab.2.3, 2.2 u. Abschnitt 3.2).

Direkte Erfolgsmeldungen über abgeschlossene Aktionen gibt keine der SPSen aus. Jedoch kann auf einen Fehlerfall über die dafür vorgesehene Codestelle C5005 der SPS des Frequenzumrichters für die Horizontalachse geprüft werden. Eine zweite Codestelle C5015 enthält zusätzlich einen Fehlercode, über den die Ursache des Fehlers eingegrenzt werden kann. Intern ist das Programm `FailureOutput` dafür zuständig, den Fehlerzustand intern und extern zu signalisieren.

**Referenzfahrt** Nach jedem Hochfahren der Frequenzumrichter ist es zwingend erforderlich, eine Referenzfahrt über alle Achsen durchzuführen. Durch manuelles Verfahren der Achsen könnte die Position der beweglichen Teile verändert und die zuletzt berechneten Positionen ungültig geworden sein. Um bei der Wiederaufnahme des Lagerbetriebs Kollisionen mit den Regalen auszuschließen, wird erst mit erfolgreich beendeter Referenzfahrt eine Freigabe für die Ausführung anderer Fahrbefehle erteilt.

**Ablage und Aufnahme** Die Ein- und Auslagerung von Gütern ist durch das SPS-Programm im Frequenzumrichter der Horizontalachse in zwei Teile gegliedert:

1. Import aus bzw. Export in die Vorlagerzone
2. Ablegen in bzw. Aufnehmen aus einem Lagerplatz

Durch diese Trennung ist es somit auch möglich, Umlagerungen von Gütern durchzuführen, ohne dabei jedes Mal die Vorlagerzone anfahren zu müssen. Dies kann bei der Optimierung des Lagers hilfreich sein.

Die Ablage- und Aufnahme-Funktion umfasst die Fahrt zu einem entsprechend adressierten Lagerplatz sowie den Vorgang des Ablegens oder Aufnehmens eines Guts durch das Lastaufnahmemittel. Nach diesem Vorgang kehrt das Lastaufnahmemittel wieder in seine Ausgangsstellung zurück, sodass weiter in der Gasse verfahren werden kann.

Wie bereits beschrieben, werden alle Funktionen über Setzen von Codestellen realisiert. Für die Ablage bzw. Aufnahme eines Guts muss zunächst der Lagerplatz adressiert werden, auf dem die Aktion ausgeführt wird. Hierzu steht eine Matrix bereit, deren Elemente den Achsen im Hochregal entsprechen. Sie ist in Tabelle 2.1 dargestellt. Damit ist auch klar, dass die Steuerungssoftware ausschließlich einfachtiefe Lagerung erlaubt. Nach der Adressierung kann der durch das dritte Matricelement näher spezifizierte Vorgang ausgeführt werden, indem die Codestelle C5000 auf 1 gesetzt wird. In der Implementation von RHODEN [22] ist dies im Funktionsblock `CodeCommunication` umgesetzt. Es ist jedoch zu beachten, dass für jeden übertragenen Wert eine Multiplikation mit zehntausend zu erfolgen hat, da das standardmäßige Datenformat der Codestellen ein Festkommazahl-Datentyp mit vier Nachkommastellen ist (s. Lenze [16]). Im Folgenden sind alle Angaben über zu schreibende Werte noch nicht multipliziert und alle Angaben über gelesene Werte bereits durch zehntausend dividiert, falls nicht explizit anders angegeben.

Das SPS-Programm ist in der Lage, ungültige Werte für die einzelnen Matricelemente zu erkennen, beispielsweise wenn der Index eines Lagerplatzes angegeben wird, der rechnerisch außerhalb des Hochregals liegt. In solchen Fällen wird der Vorgang abgebrochen und die Fehler-Codestellen C5005 und C5015 entsprechend gesetzt. Weitere Verfahrensanweisungen werden zurückgehalten und erst ausgeführt, nachdem die Fehler-Codestelle C5005 manuell bzw. anwendungsseitig von Extern auf 0 zurückgesetzt wurde. Hierdurch wird verhindert, dass die Kontrolle über das Lager im Fehlerfall sowohl mechanisch wie auch logisch verloren geht.

Dennoch ist besonders darauf zu achten, dass alle Vorgänge erfolgreich abgeschlossen werden oder bei Fehlerzuständen sicherzustellen, dass die Lagerbelegung und die Belegung des Lastaufnahmemittels mit den Informationen der Steuerungsebene identisch sind. Die SPS-Steuerung des Hochregallagers der Digitalen Fabrik unterstützt keine Verwaltung der Lagerbelegung – sie obliegt ausschließlich der überlagerten Steuerung. Gegen Beschädigung der Lagerkonstruktion hat RHODEN eine Drehmomentüberwachung implementiert. Sie löst aus, sobald das Lastaufnahmemittel nicht voll ausfahren kann, z. B.



wenn es mit einer geladenen Plattform in einen bereits belegten Lagerplatz fährt. Somit können Beschädigungen der Anlage vermieden werden. Daher liegt eine Empfehlung nahe,

- jeweils drei Vorgänge zu kombinieren und als atomare Einheit zu behandeln
  - Aufnehmen – Positionieren – Ablegen
  - Importieren – Positionieren – Ablegen
  - Aufnehmen – Positionieren – Exportieren
- und zeitgleiche Zugriffe auf die SPSen nur über eine einzige, zentrale Komponente zu gestatten (zentrale Kommunikation mit der SPS).

Durch die Verteilung der Achsen auf drei unterschiedliche Frequenzumrichter ist eine Koordination dieser erforderlich. Da der Frequenzumrichter für die Horizontale die allgemeine Schnittstelle zur Außenwelt anbietet, ist er auch dafür zuständig, dass die anderen beiden Frequenzumrichter dem aktuellen Vorgang entsprechend angesteuert werden. Auch dies geschieht intern mit Hilfe der Codestellen, erfolgt jedoch nicht über CANopen, sondern über das native CAN-Interface. Interessante Codestellen zur Überwachung der Vorgänge sind in Tabelle 2.3 dargestellt.

| Element  | Beschreibung                                                                                   |
|----------|------------------------------------------------------------------------------------------------|
| 1, C5001 | Horizontale Position bzw. $x$ des zu adressierenden Lagerplatzes als Index                     |
| 2, C5002 | Vertikale Position bzw. $z$ (s. Abb. 6.3)                                                      |
| 3, C5003 | 1: Ablage, Nordseite<br>2: Aufnahme, Nordseite<br>3: Ablage, Südseite<br>4: Aufnahme, Südseite |

Tabelle 2.1: Matrixbeschreibung zur Adressierung einzelner Lagerplätze über die Codestellen C5001 bis C5003.

Beispiel:

Es gilt, einen sich bereits auf der Aufnehmereinheit befindlichen Baugruppenträger in Lagerplatz  $H=3$ ,  $V=5$ , Nord abzulegen. Dazu wird zunächst die Matrix wie folgt besetzt:

$C5001 \leftarrow 3$ ,  $C5002 \leftarrow 5$ ,  $C5003 \leftarrow 1$

Anschließend wird das System angewiesen, die Matrixwerte explizit zu übernehmen und den Vorgang auszuführen:  
 $C5000 \leftarrow 1$ .

**Import und Export** Um ein Gut zunächst in den Aufgabenbereich des Hochregallagers zu setzen, muss es über dessen Vorlagerzone, dem Hauptumlauf, zu einem der Übergabepunkte, einem bidirektionalen Ein- bzw. Ausgang, verbracht werden. Ihnen sind die Kardinalpunkte Ost und West zugeordnet. Die SPS-Software der Horizontaleinheit kapselt dabei als Import die Fahrt zum Eingang innerhalb der Regalgasse sowie den Vorgang der Aufnahme des Guts außerhalb des Regals und die anschließende Rückfahrt des Lastaufnahmemittels in die Gasse, ohne dabei jedoch einen Lagerplatz anzufahren.

Als Export hingegen wird zunächst ebenfalls die Fahrt zum gewählten Eingang durchgeführt, dann jedoch das sich auf dem Lastaufnahmemittel befindliche Gut aus der Regalgasse heraus getragen und auf der Fließbandzuführung abgesetzt. Das Lastaufnahmemittel fährt im Anschluss ebenfalls wieder in die Gasse hinein.

Beide Vorgänge unterscheiden sich in der Vertikalposition, in der sich die Aufnehmereinheit befindet und das Lastaufnahmemittel die Regalgasse verlässt. Zum Import ist sie niedrig, sodass es unter dem zu importierenden Gut hindurch fährt und es von unten anheben kann. Zum Export ist sie hoch, um nicht mit dem Baugruppenträger gegen das Fließbandende zu stoßen.

Um Kollisionen mit sich bereits auf dem Fließband befindlichen Baugruppenträgern zu vermeiden, sind hier Sensoren vorhanden, die jedoch der Steuerung des Fließbandsystems zugeordnet sind. Somit kann das SPS-Programm hierüber nicht wachen und kritische Vorgänge blockieren. Dieser Umstand macht es nötig, dass eine übergeordnete Steuerungsebene über diese Übergabepunkte wacht und sie zeitweise für Vorgänge sperrt:

- für Export sperren, falls sich dort ein einzulagerndes Gut befindet
- für Import sperren, falls dort ein auszulagerndes Gut abgelegt werden soll

Die Kombination von Import und Export mit östlichem und westlichem Übergabepunkt erfordert vier getrennte Codestellen des Frequenzumrichters der Horizontalachse zur Bedienung (s. Tab. 2.2). Wird ihnen der Wert 1 zugewiesen, so startet das Lagersystem den entsprechenden Vorgang wie oben beschrieben.

Beispiel:

Ein Gut ist vom westlichen Eingang in den Aufgabenbereich des Lagers zu importieren. Hierzu ist lediglich das Setzen einer einzigen Codestelle nötig:

C5010 ← 1

| <b>Codestelle</b> | <b>Beschreibung</b>     |
|-------------------|-------------------------|
| C5008             | Exportieren nach Westen |
| C5009             | Exportieren nach Osten  |
| C5010             | Importieren von Westen  |
| C5011             | Importieren von Osten   |

Tabelle 2.2: Beschreibung der Codestellen für Import und Export.

| FU <sup>a</sup> | R/W <sup>b</sup> | Codestelle | Beschreibung                                                 |
|-----------------|------------------|------------|--------------------------------------------------------------|
| H               | W                | C5000      | Matrix übernehmen und Vorgang ausführen (1)<br>(s. Tab. 2.1) |
| H               | W                | C5001      | Matrixwert Horizontalposition                                |
| H               | W                | C5002      | Matrixwert Vertikalposition                                  |
| H               | W                | C5003      | Matrixwert Vorgang                                           |
| H               | R                | C5017      | aktuelle Position in der Horizontalen                        |
| H               | R                | C5020      | Horizontalachse in Bewegung: ja (1), nein (0)                |
| V               | R                | C5017      | aktuelle Position in der Vertikalen                          |
| V               | R                | C5020      | Vertikalachse in Bewegung: ja (1), nein (0)                  |
| A               | R                | C5017      | aktuelle Position der Aufnehmereinheit                       |
| A               | R                | C5020      | Aufnehmereinheit in Bewegung: ja (1), nein (0)               |
| H               | W                | C5007      | Referenzfahrt für alle Achsen starten (1)                    |
| H               | R                | C5012      | Referenz für Horizontalachse vorhanden: ja (1),<br>nein (0)  |
| H               | R                | C5014      | Referenz für Vertikalachse vorhanden: ja (1),<br>nein (0)    |
| H               | R                | C5013      | Referenz für Aufnehmereinheit vorhanden: ja<br>(1), nein (0) |
| H               | R/W              | C5005      | Fehlerzustand vorliegend: ja (1), nein (0)                   |
| H               | R                | C5015      | Fehlercode: <sup>c</sup>                                     |

0: kein Fehler

1: externer Fehler (z. B. fehlende Steuer-  
spannung, mechanische Endlagen)

2: maximales Drehmoment erreicht (z. B.  
verkantetes Werkstück auf Aufnehme-  
reinheit)

3: Fehler bei Referenzfahrt, horizontal

4: Fehler bei Referenzfahrt, vertikal

5: Fehler beim Auslagern, falls Auslage-  
rungsanforderung bei bereits belegter  
Aufnehmereinheit empfangen wird

6: Matrixwerte adressieren keinen realen  
Lagerplatz

<sup>a</sup>Frequenzumrichter: Horizontal, Vertikal, Aufnehmereinheit

<sup>b</sup>Read (Lesen)/Write (Schreiben)

<sup>c</sup>nach RHODEN [22]

Tabelle 2.3: Für Ablage und Aufnahme interessante Codestellen.

### 3 LK<sup>3</sup>S – Leicht konfigurierbare Komponenten kollaborativer Systeme

Das an der Hochschule Emden/Leer durchgeführte Forschungsprojekt für leicht konfigurierbare Komponenten kollaborativer Systeme (LK<sup>3</sup>S), gefördert vom Bundesministerium für Bildung und Forschung (BMBF) sowie namhafter Industriepartner, ist im Bereich der Automatisierungstechnik angesiedelt. Ziel der industriellen Automatisierung ist im Endeffekt immer die Planung und letztendlich die verschiedenen Regeln unterlegene Steuerung von Aktorik und Sensorik z. B. zur Umwandlung von Rohmaterial in ein Endprodukt, das als Grundlage für andere Produkte oder zum Konsum dient. Die Planung erfolgt heutzutage üblicherweise über ein *Enterprise Resource Planning System* (ERP) und mit der Hilfe eines *Manufacturing Execution System* (MES). Die diskrete Fertigungsindustrie verwendet zur Prozessanbindung und Hardwaresteuerung größtenteils *Speicherprogrammierbare Steuerungen* (SPS), auf denen meist zyklische Programme ablaufen, die die Steuerlogik eines Prozesses umsetzen. Kommuniziert werden kann über Bussysteme sowohl mit der Aktorik und Sensorik wie auch mit anderen SPSen, Visualisierungen und dem MES.

Nach Ansicht verschiedener Expertengruppen ist die nahe Zukunft der Fertigungsautomation zunehmend von individuellen Kundenwünschen geprägt (s. EU [9], BOLLINGER et al. [3]). Dies impliziert das Verlangen nach höherer Flexibilität in den Fabrikhallen: Es wird öfter werden und Kapazitäten müssen nach Bedarf aufgestockt oder gekürzt werden. Eine Möglichkeit zur Umsetzung dieser Ziele ist die Neukonfiguration aller Komponenten statt ihrer Neuprogrammierung. Dies setzt mehr oder minder autonome Fertigungszellen voraus, die, jede für sich, eine definierte (konfigurierbare) Leistung erbringen und in das Gesamtgeschehen im laufenden Betrieb eingebunden oder entfernt werden können. Für diese Aufgabe sind Multiagentensysteme prädestiniert, die eine Einheit aus Hard- und Software bilden. Ein Agent besitzt nach allgemein verbreiteter Ansicht folgende Eigenschaften<sup>1</sup>:

**Autonomie** Ein Agent bildet in sich eine abgeschlossene Einheit, die in der Lage ist ein bestimmtes Ziel durch eigene Tätigkeit zu verfolgen.

**Proaktivität** Ein Agent ist in der Lage, eigenständig und ohne gezielte Eingaben aus seiner Umwelt Handlungen auszuführen.

---

<sup>1</sup>in Anlehnung an Wikipedia: <http://de.wikipedia.org/wiki/Software-Agent>

**Reaktivität** Ein Agent nimmt seine Umwelt wahr (andere Agenten, Sensorik etc.) und kann auf dortige Änderungen reagieren.

**Sozialität** Ein Agent ist in der Lage mit anderen Agenten zu kommunizieren und Verhandlungen zu führen, um sein eigenes Ziel zu verfolgen.

**Lernfähigkeit** Ein Agent kann aus den Ergebnissen seiner eigenen Handlungen und von seiner Umwelt lernen und die Ausführung zukünftiger Handlungen optimieren.

In diesem Kapitel soll nunmehr speziell auf das LK<sup>3</sup>S-Agentensystem eingegangen werden. Die entsprechenden Erweiterungen, die die Lagerfähigkeiten betreffen werden anschließend in Kapitel 5 dargelegt.

Das LK<sup>3</sup>S-Projekt definiert ein Drei-Schichten-Modell, das sich über alle Ebenen der Automatisierungspyramide (vgl. IEC 62264 [14]) erstreckt und sich an den klassischen Modelle orientiert. In Abbildung 3.1 ist dieses System dargestellt. Aufgaben, die klassischerweise einem ERP-System zufallen, gehen hier mit in das dreischichtige Modell über. Während sich das ERP-System klassischerweise in der obersten Schicht der Pyrami-

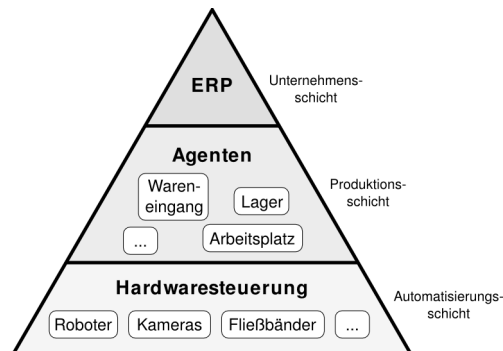


Abbildung 3.1: Drei-Schichten-Modell im LK<sup>3</sup>S-Projekt.

de befindet, entspricht dies im LK<sup>3</sup>S-Agentensystem nur noch bedingt der Realität, denn diese Schicht verschimmt zunehmend mit den Agenten in der Produktionsschicht, die auch die Funktion des MES übernehmen. Die Agenten in der mittleren Schicht sind für die logische Verbindung und Steuerung von Handlungen der Automatisierungseinrichtungen zuständig und Operatoren können elementare Aktionen wie Lesen und Schreiben von Sensor-/Aktorwerten zusammenfassen und den Agenten als abstraktere Dienste bereitstellen. Ein ähnliches Verfahren wurde von CACHAPA et al. [7] entwickelt. Sie setzen jedoch viele *Smart Devices*, also kleine, intelligente Geräte im Feld ein, die die Dienste zur Verfügung stellen. Das LK<sup>3</sup>S-Agentensystem setzt hier jedoch auf deren Zusammenführung (Clusterbildung) in weniger Systemen, die dann entsprechende Dienste anbieten und orchestrieren. Diese Systeme können auch in der Fabrik verteilt sein, besitzen jedoch auf Grund der Zusammenführung einen geringeren Stromverbrauch als viele kleine Smart Devices. Die Operatoren benötigen ihrerseits Hardwarezugriff. Auf Grund der optimalen Skalierbarkeit und der Möglichkeit Steuerungslogik von Prozessanbindung zu entkoppeln, werden hierfür Industrie PCs eingesetzt. Sie bieten den Operatoren mit

ihren Feldbus-Interfaces einen einfachen Dienst (Stub) an, der dennoch konfigurierbar ist und den Zugriff auf alle Aktoren und Sensoren über einen jeweils eigenen symbolischen Bezeichner gestattet. Sie bilden eine Subschicht der untersten Ebene, womit eine mehrschichtige Architektur entsteht, die zwischen jeweils zwei Schichten eine Interfaces einführt. Einen ähnlichen Ansatz hat auch ein Forschungsprojekt an der TU Wien gewählt, bei der Geräte mit unterschiedlicher Ansteuerung über einheitliche Schnittstellen höheren Schichten verfügbar gemacht werden (s. LEPUSCHITZ et al. [18]).

Zunächst sollen die näheren Erläuterungen mit der hardwarenähesten Komponente der untersten Schicht, dem Stub, in Abschnitt 3.1 begonnen werden. Danach folgen die Operatoren in Abschnitt 3.2 und schließlich die Agenten in der mittleren Schicht, in Abschnitt 3.3.

## 3.1 Stub

Zur Anbindung der Prozesse an die Produktionslogik steht mit dem Stub eine Linux-Software zur Verfügung, die aus mehreren Teilen besteht (s. Abb. 3.2):

**Hardwaretreiber** Zur Kommunikation mit einem Feldbus-Interface (u.a. PCI-Steckkarte, Ethernet, ...) wird ein Treiber benötigt, der die spezifischen Steuersequenzen und Datenformate zur Handhabung des Feldbus-Interface kapselt und einem Programmierer in Form eines *Application Programming Interface* (API) bereitstellt. Da die verwendete Architektur auf einem RTAI-Linux aufbaut, ist es notwendig, dass auch die Hardwaretreiber entsprechende Anpassungen erfahren.

**Kernelmodul** Die Treiber-API wird sodann von einem RTAI-Linux Kernelmodul des Stubs verwendet. Es umfasst einen, auf Grund des Echtzeitverhaltens, statischen Speicherbereich, der Sequenzen elementarer Ein- und Ausgabebefehle aus Sicht des Feldbus-Interface aufnehmen kann. Diese Sequenzen bilden eine Einheit und können als Ganzes in Echtzeit ausgeführt werden. Die möglichen Operationen werden im Anschluss im einzelnen erläutert. Da mehrere Sequenzen parallel ausführbar sein sollen, bedarf es einem Dispatcher (Umschalter, s. Abb. 3.2), der die Ausführung der voneinander unabhängigen Sequenzen steuert. Wird beispielsweise in einer Sequenz auf eine Wertänderung gewartet, kann der Dispatcher derweil ein Kommando einer anderen Sequenz zur Ausführung bringen. Das Kernelmodul kommuniziert über FIFOs (First-In-First-Out-Puffer) als Devicefile im Linux-Dateisystem) anhand eines selbst spezifizierten Protokolls mit einer anderen Anwendung, die das Verhalten des Kernelmoduls steuert.

**Userlandmodul** Die besagte Steuerung des Kernelmoduls geschieht über den Stub Daemon, der als Serveranwendung im Userland des RTAI-Linux-Systems arbeitet und keinerlei Echtzeitanforderungen unterliegt. Dieser öffnet einen Serversocket, so dass sich beliebige Clients zu diesem verbinden können. Ein einfaches Kommunikationsprotokoll regelt den Ablauf der Konversation. Hierüber ist es möglich,

symbolische Bezeichner („Symbole“) zur Benutzung anzufragen, Sequenzen zur Ausführung zu definieren und diese auch zu veranlassen. Diese Aktionen können zu jedem Zeitpunkt, auch während des laufenden Betriebs, durchgeführt werden. Listing 3.1 zeigt eine beispielhafte Registrierung symbolischer Bezeichner sowie die Definition einer Sequenz, die die Werte der zuvor registrierten Symbole ausliest. Dabei ist den vom Client gesendeten Zeilen ein > und den vom Stub Daemon gesendeten Zeilen ein < vorangestellt. Der Stub Daemon stellt bei der Verarbeitung der Kommunikation mit einem Client sicher, dass die angeforderten Symbole existieren, indem er seine Konfigurationsdatei konsultiert. Zudem können alle Symbole alleinig oder zur gemeinsamen Nutzung registriert werden. Der Stub Daemon stellt hier außerdem sicher, dass keine Widersprüche entstehen, die später Verklemmungen oder konkurrierende Zugriffe gestatten würden.

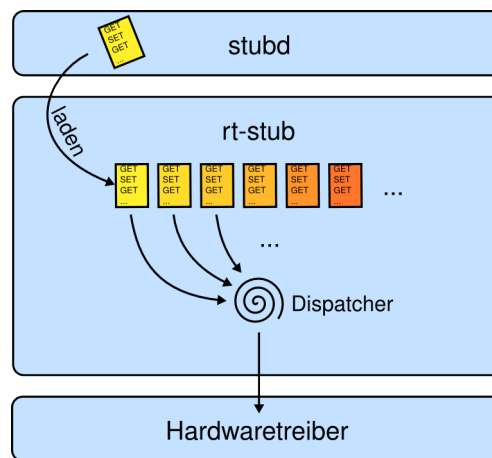


Abbildung 3.2: Architektur des Stubs. Quelle: [29]

Die Sequenzen, die ein Client definieren kann, bestehen aus drei elementaren Operationen:

**get** zum Lesen eines Datums vom angegebenen Symbol

**set** zum Schreiben eines definierten Datums in das angegebene Symbol

**wait** zum Warten, bis ein angegebenes Symbol einen definierten Wert besitzt

Zu jeder Operation wird eine Vor-Ausführung-Wartezeit definiert, die stets eingehalten wird. Ebenfalls wird ein Timeout definiert, in der die Ausführung einer Operation abgeschlossen sein muss. Im Fehlerfall wird dann die Bearbeitung der kompletten Sequenz abgebrochen. Die `set`- und `wait`-Operationen benötigen zudem je einen Wert, den es zu schreiben bzw. dessen Erreichen es abzuwarten gilt. Eine Operation wird wie folgt angegeben:

```
code [Wartezeit] get/set/wait [Symbolname] [Wert bei set/wait] [Timeout]
```

Anhand der Kombination dieser elementaren Operationen können aufwändige Abläufe, z. B. zur Steuerung der Fließbandanlagen in der Digitalen Fabrik, realisiert werden. Ist



jedoch eine bedingte Logik bei der Hardwaresteuerung erforderlich, so überschreitet dieses die Fähigkeiten des Stub. Konstrukte wie Bedingungen und Schleifen sind über diese einfache Sprache nicht möglich und müssen außerhalb der Echtzeitumgebung realisiert werden. Dennoch war es so einfach möglich, eine konfigurierbare, flexible und durch die symbolischen Bezeichner abstrahierende, echtzeitfähige Feldbuskommunikation auf RTAI-Linux-PC-Basis zu schaffen, die zudem im Netzwerk verteilt betrieben werden kann, z. B. auf Embedded PCs.

Da diese Methode der Hardwaresteuerung trotz der Abstraktion durch symbolische Bezeichner immer noch sehr detailliert ist, müssen Agenten jedoch sehr viele Komponenten und Parameter im Überblick behalten und sollten daher nach Möglichkeit nur Wünsche aus Manager-Sicht äußern. Daher bedarf es einer weiteren Softwareschicht, die genau diese Abstraktion gegenüber den Agenten für komplexe Hardwarevorgänge realisiert, die Operatoren.

Listing 3.1: Ausschnitt einer Kommunikation zwischen Stub Daemon und einem Client zur Registrierung von Symbolen und Definition einer Sequenz.

```
1 > exclusive TEMPERATURE-SENSOR
2 < ok exclusive TEMPERATURE-SENSOR
3 > shared PRESSURE-SENSOR
4 < ok shared PRESSURE-SENSOR
5
6 > begin demo
7 < ok begin demo
8 > code 0 get TEMPERATURE-SENSOR 0
9 < ok code 0 get TEMPERATURE-SENSOR 0
10 > code 0 get PRESSURE-SENSOR 0
11 < ok code 0 get PRESSURE-SENSOR 0
12 > end
13 < ok end
```

## 3.2 Operator

In einer Fabrik existieren viele unterschiedliche Maschinen, die oft Dienste zur Verfügung stellen, die in gleicher Form oder mit wenigen Parametern immer wieder in Anspruch genommen werden können. Die Operatoren setzen genau hier an und bieten der höheren Agentenschicht für jede Maschine entsprechende Dienste als parametrierbare Sequenz eines oder mehrerer Stubs an. Das von DE PUTTER et al. entworfene und im Laufe der Zeit von VIRKUS [27] weiterentwickelte und implementierte Protokoll zur Kommunikation zwischen Agenten und Operatoren macht intensiv von XML Gebrauch. Agenten benötigen eine Kenntnis, welche Dienste von einem Operator angeboten werden und welche Semantik sie besitzen. Mit dieser Kenntnis können sie die Ausführbarkeit eines Dienstes vom Operator prüfen lassen, Parameter übertragen und schließlich auch

die Ausführung eines Dienstes veranlassen. Der Operator wiederum hat die Möglichkeit, Ergebniswerte zurück an den Agenten zu übertragen. Hierzu zählen auch Angaben zu Ausführungszeit und Energieverbrauch des Vorgangs, was insbesondere bei der Auftragsplanung Anwendung findet, um Alternativen abzuwägen.

Um zu vermeiden, dass jeder Operator für eine andere Maschine auch den kompletten Protokoll-Stack neu implementieren muss, wurde ein erweiterbares Grundgerüst geschaffen, der Operator Skeleton. Ziel des LK<sup>3</sup>S-Projekts ist eine weitgehende Konfigurierbarkeit des Gesamtsystems. Bei der konkreten Realisierung einer Logik für die Bedienung von Maschinen usw. sind der Konfigurierbarkeit jedoch auch Grenzen gesetzt. Es wäre denkbar, eine eigene Programmier- respektive Skriptsprache zu erfinden und diese schlicht als Konfiguration statt Programmierung zu betiteln. Dies gliche jedoch eher einer Täuschung. Aus diesem Grund fiel die Entscheidung, die maschinenspezifischen Algorithmen, die für die Stub-Sequenzbildung und die Bedienung des maschinenspezifischen Stubs zuständig sind, als flexible Erweiterung des in Java entwickelten Operator Skeletons zu realisieren.

Der Operator Skeleton ist eine eigenständige Anwendung, die zunächst ein Framework für die Protokollverarbeitung zur Kommunikation mit Agenten bereitstellt. Über seine Konfiguration wird eine Java-Klasse definiert, die eine spezifische Implementation für eine bestimmte Maschine bereitstellt, die Operator Extension. Das entsprechende Java-Interface `IOperatorExtension`, das die Schnittstelle exakt definiert, schreibt dabei einige entscheidende Methoden vor, die jede spezifische Implementation zu besitzen hat:

**Initialisierung** Zu Beginn, nach Instanziierung der Operator Extension, wird zunächst eine Initialisierungsmethode der Extension aufgerufen und der Name einer Konfigurationsdatei als Argument übergeben, wie beim Start des Operators als Kommandozeilenargument angegeben. Damit besitzt die Operator Extension die Möglichkeit, anfängliche Vorbereitungen durchzuführen, bevor sie in einen betriebsbereiten Zustand übergeht.

**Anfrage** Bei einer Anfrage möchte ein Agent in Erfahrung bringen, ob der von ihm angegebene Dienst, auch Programm genannt, durch die Operator Extension ausgeführt werden kann und ob die mit der Anfrage übergebenen Parameter zusammen mit dem Programm sinnvoll sind und so übernommen werden können. Die Operator Extension hat bei nahezu jeder über das Interface definierten Methode die Möglichkeit, durch Java-Exceptions zu signalisieren, dass eine Auftrag nicht angenommen werden kann oder dass ein Fehler bei seiner Ausführung auftrat. Dies wird vom Operator Skeleton abgefangen, entsprechend verarbeitet und letztendlich über das Kommunikationsprotokoll auch an den Agenten weitergeleitet.

**Auftrag** Die Ausführung eines Auftrages unterscheidet sich von einer Anfrage nur in den Punkten, dass ein anderes Schlüsselwort des Protokolls verwendet wird und dass die Operator Extension dieses Mal wirklich eine Aktion auf der Hardware oder, wenn vorgesehen, rein virtuell ausführt.

### 3.2.1 Architektur

Der Operator Skeleton, als Grundlage jedes Operators, definiert zunächst einige Java-Packages:

**de.i2ar.operators.skeleton** Dies ist das Basis-Package des Operator Skeletons. Es enthält u.a. die Klasse `Main`, welche den Einsprungspunkt `main` enthält und zunächst die Skeleton-Konfiguration anhand eines Kommandozeilenarguments in ein Objekt der Klasse `SkeletonConfig` über dessen statische Methode `loadFromFile` lädt. Im Folgenden wird ein Wartungsthread (`MaintenanceThread`) aus dem gleichen Paket instanziiert, der auf einem in der Konfiguration angegebenen Port lauscht und das geplante Beenden des Operators erlaubt. Außerdem werden von `main` die anhand ihres Klassennamens konfigurierte Operator Extension mit Hilfe der Java Reflection API instanziiert und eine Referenz darauf vorgehalten. An dieser Stelle wird auch die im Operator Extension Interface `IOperatorExtension` spezifizierte Methode `setConfigurationFileName` aufgerufen und der als Kommandozeilenargument angegebene Dateiname der Operator Extension übergeben. Diese hat nun die Möglichkeit, ihre oben erwähnte Initialisierung vorzunehmen. Um mehrere Agenten parallel bedienen zu können (entsprechende Synchronisation hat durch die Operator Extension zu erfolgen), wird für jeden konfigurierten Agenten ein Objekt der Klasse `AgentInformation` erzeugt und jeweils einem zugehörigen, neu erstellen Objekt der Klasse `AgentConnectionThread` übergeben, das als Thread gestartet wird. Abschließend wird auf das Ende aller Threads gewartet, bevor die Operator Extension mit Aufruf ihrer Methode `finish` zum Abschluss aufgefordert wird.

**de.i2ar.operators.skeleton.exceptions** Hier sind Exceptions definiert worden, die zur Signalisierung aufgetretener Fehlerzustände dienen und von einzelnen Methoden zur Beeinflussung der Protokolloperationen ausgelöst werden können. Wann die einzelnen Exceptions auftreten können, wird zusammen mit dem folgenden Package erläutert. Wird ein String als nähere Beschreibung mit übergeben, wird dieser mit in die Agentenkommunikation integriert, sodass dieser die Meldung theoretisch ausgeben oder auswerten kann.

**de.i2ar.operators.skeleton.interfaces** Dieses Package enthält das für Erweiterungen notwendige Java-Interface `IOperatorExtension` zur Spezifikation der Schnittstelle einer jeden Operator Extension. Dabei werden alle Protokollspezifikation über entsprechende Methoden, ihre Parameter und die möglichen, auslösbaren Exceptions umgesetzt.

**version** Diese Methode muss einen spezifischen Versionsstring zurückliefern.

**setConfigurationFileName** dient zur Initialisierung der Operator Extension. Als Argument wird ein Dateiname übergeben, der die Konfigurationsdatei der Extension angibt. Beendet die Methode regulär, gilt die Konfiguration als erfolgreich abgeschlossen. Durch Auslösen einer Exception des Typs `FileNotFoundException`

kann signalisiert werden, dass keine Datei zum übergebenen Dateinamen existiert und mit der Exception `InvalidConfigurationException`, dass die Konfiguration ungültig ist oder ein anderer Fehler bei der Initialisierung auftrat. Diese Methode realisiert keinen Teil des Kommunikationsprotokolls.

**getRegistrationInformation** wird vom Operator Skeleton aufgerufen, um Informationen von der Operator Extension abzufragen, die zur Anmeldung des Operators am Agenten notwendig sind. Hierzu zählen ein eindeutiger Bezeichner des Operators im Bereich des Agenten, ein Operator-Typ, eine textuelle Beschreibung und eine Liste mit Parametern (Schlüssel-Wert-Paare). Diese Informationen werden gebündelt als Objekt der Klasse `OperatorInformation` aus dem Package `de.i2ar.operators.common.registration` als Ergebnis der Methode zurückgegeben und vom Operator Skeleton im Rahmen des Kommunikationsprotokolls verarbeitet und an den Agenten weitergeleitet.

**queryJob** Diese Methode wird ausgeführt, sobald ein Agent eine Auftragsbeschreibung (abgebildet u.a. in Klasse `Jobs` des Pakets `de.i2ar.operators.common.jobs`, siehe auch VIRKUS [27]) an den Operator als Anfrage überträgt. Dabei soll der Auftrag lediglich evaluiert werden, hinsichtlich Ausführbarkeit und Kosten in Zeit und Energie. Als Rückgabewert dient `null`, falls kein Angebot abgegeben werden kann oder ein Objekt der Klasse `JobQueryResults`. Sie enthält Felder zur Angabe von Zeit und Energieverbrauch. Ist der übergebene Auftrag inhaltlich fehlerhaft, so sollte die Methode `queryJobs` eine Exception des Typs `InvalidInformationException` ausgelöst werden.

**executeJob** Im Gegenteil zu `queryJobs` sollte bei Aufruf dieser Methode nach Prüfung des übermittelten Auftrags eine reale Ausführung des Auftrags stattfinden. Der Auftrag wird dabei ebenfalls als Objekt der Klasse `de.i2ar.operators.common.jobs.Jobs` als Argument übergeben. Nach erfolgreicher Ausführung hat die Methode ein Objekt der Klasse `JobExecutionResults` zurückzugeben, das eine Liste Schlüssel-Wert-Paare enthält, wobei jedoch keine speziellen Schlüssel vorgeschrieben sind. Alternativ kann auch `null` zurückgegeben werden, um eine erfolgreiche Ausführung zu signalisieren. Kann ein Auftrag aus beliebigen Gründen gar nicht erst zur Ausführung gebracht werden, so ist eine Exception des Typs `JobRejectException`, bei einem potentiell reparablen Fehler während der Ausführung eine Exception des Typs `JobFailureException` auszulösen. Trat hingegen ein Hardwareproblem auf, der die Ausführung weiterer Anfragen verhindern sollte, so ist eine Exception des Typs `StubException` auszulösen.

**abortJob** kann veranlassen, dass ein bereits über `executeJob` bereits in Ausführung befindlicher Auftrag abgebrochen wird. Ist dies nicht möglich oder tritt dabei ein Fehler auf, kann eine Exception des Typs `JobFailureException` oder des Typs `StubException` bei Hardwareproblemen ausgelöst werden.

**finish** ist eine Methode, die direkt vor Beendigung des Operators aufgerufen wird, um abschließende Aktionen durchzuführen, z. B. eine Hardware zurück in einen definierten Ausgangszustand versetzen. Da der Operator in jedem Fall beendet wird, ist

es nicht vorgesehen, über einen Rückgabewert oder die Auslösung von Exceptions Fehler zu signalisieren.

**de.i2ar.operators.skeleton.agents** enthält u.a. die Klasse `AgentInformation`. Sie enthält lediglich Felder für die IP-Adresse und den Port, über die sich der Operator zu einem hierdurch beschriebenen Agenten zu verbinden hat. Ein weiteres Feld `connected` dient zur Speicherung, ob eine Verbindung zum Agenten besteht. Die Klasse `AgentConnectionThread` erbt von der Klasse `Thread` der Java-API. Jede Instanz dieser Klasse ist allgemein ausgedrückt für die Kommunikation des Operators mit genau einem Agenten zuständig. Dabei wird zunächst bei Ausführung des Threads (Methode `run`) versucht, eine Verbindung zum Agenten der übergebenen Daten herzustellen. Schlägt dies fehl oder wird die Verbindung im Betrieb unterbrochen, wird dieser Vorgang nach einer Pause beliebig wiederholt. Anschließend wird in einen Zustand übergegangen, in dem der Operator zyklisch auf eine Anweisung vom Agenten wartet, die Anfrage verarbeitet und ausführt und danach eine Antwort an den Agenten sendet. Dies ist noch in der Methode `run` realisiert, die dazu die Methode `waitForJob` aufruft. Die Methode `waitForJob` liest eine Zeile vom Socket des Agenten ein und kann nach Anfragen, Aufträgen und Abbrüchen unterscheiden sowie eine entsprechende Methode zur Bearbeitung der Anfrage aufrufen. Für das Lesen vom Socket ist ein Timeout eingerichtet, nach dessen Ablauf – sowie nach erfolgtem Lesen – geprüft wird, ob währenddessen ein bereits gestarteter Auftrag abgeschlossen wurde. Zuvor findet jedoch eine Prüfung statt, ob der Operator möglicherweise beendet werden soll, z. B. auf Grund eines Fehlerzustandes. Zur Bearbeitung abgeschlossener Aufträge wird die Liste `answerQueue` durchlaufen, die Objekte der Klasse `JobExecutionHandle` enthält. Diese besitzt Felder zur Speicherung eines Auftrages sowie dessen Ergebnis. Liegt nun ein Eintrag in `answerQueue` vor, so wird geprüft, ob der zugehörige Auftrag erfolgreich abgeschlossen wurde, fehlschlug oder nicht akzeptiert werden kann. Die Antwortnachricht an den Agenten wird vorbereitet, gesendet und der abgeschlossene Auftrag schließlich wieder aus den internen Datenstrukturen des Operators entfernt.

Bisher wurde noch nicht die Verarbeitung der unterschiedlichen Anfragen dargelegt. Diese sind prinzipiell hinsichtlich ihrer Asynchronität zu unterscheiden, denn nur die Ausführung eines Auftrags kann asynchron zum restlichen Programmablauf erfolgen. Geht eine einfache Anfrage ein, so blockiert dies die weitere Kommunikation mit dem gleichen Agenten, bis eine Antwort an diesen gesendet wurde. Die Verarbeitung der reinen Anfragen geschieht in der Methode `processQuery`. Hier wird zu Beginn geprüft, ob neue Anfragen überhaupt zugelassen sind (`Main.isNewRequestsAllowed`). Anschließend erfolgt ein Parsen der Anfrage mit Hilfe des SAX-XML-Parsers und der Klasse `de.i2ar.operators.common.jobs.JobsXMLParser`. Die Ergebnisse werden in einer Instanz der Klasse `Jobs` (s.o.) gespeichert. Abschließend wird die Methode `queryJobs` der Operator Extension aufgerufen und das `Jobs`-Objekt übergeben sowie deren Antwort als `JobQueryResults`-Objekt als XML-Dokument dargestellt und eine Antwort an den anfragenden Agenten gesendet.

Bei real auszuführenden Aufträgen wird stattdessen die Methode `processExecute` aufgerufen. Hier erfolgt die Bearbeitung ähnlich, jedoch wird nach Parsen in ein `Jobs`-Objekt ein Verwaltungsobjekt für diesen Auftrag von der Klasse `JobExecutionHandle` erzeugt. Da reale Aufträge parallel ausgeführt werden, muss eine spätere Zuordnung der Ausführungsergebnisse zu einem Auftrag zwecks Antwort erfolgen. Daher sieht das Protokoll eine ID vor, die mit dem `Jobs`-Objekt in dem Verwaltungsobjekt gespeichert wird. Zudem wird mit Hilfe der Methode `generateExtendedId` eine weitere ID erzeugt, die aus der ID aus dem Protokoll sowie dem Namen des zugehörigen Agenten besteht. Diese Informationen werden im Anschluss global gespeichert, falls zur Zeit kein Auftrag mit der gleichen erweiterten ID vorliegt. Dies hat den Vorteil, dass alle Agenten die gleichen IDs verwenden können, sie sich also untereinander nicht abstimmen müssen. Der Auftrag kann nun ausgeführt werden, wozu ein Objekt der Klasse `JobExecutionThread` erzeugt und als Thread gestartet wird. Dabei wurden das zuvor erstellte `JobExecutionHandle`-Objekt und eine Referenz auf die zu verwendende Queue für die Ausführungsergebnisse (`answerQueue`) angegeben. Zuletzt wird der aufrufende Agent darüber benachrichtigt, dass sein Auftrag zur Ausführung gebracht wurde. Die abschließende Ergebnisverkündung wird vom eben genannten Thread veranlasst.

Dieser Thread gibt konstante Ergebnisse zurück, falls in der Operator Skeleton Konfiguration der Dummy Execution Modus gesetzt wurde. Die Ausführung aller protokollverarbeitenden Schritte ist hier also gegeben, nur wird keine reale Ausführung auf der Hardware vorgenommen. Ist dieser Modus hingegen nicht aktiviert, so wird zunächst die Methode `executeJob` der Operator Extension ausgeführt und ihr Rückgabewert direkt im zum Auftrag gehörenden `JobExecutionHandle`-Objekt gespeichert. Trat bei der Ausführung keine Exception auf (siehe `Package interfaces`), ist davon auszugehen, dass die Ausführung erfolgreich war und es kann das Feld `success` des `JobExecutionHandle` gesetzt werden, die Fehlermeldung im Feld `errorMsg` kann null sein. Im Fehlerfall hingegen wird `success` auf `false` gesetzt, die Meldung der aufgetretenen Exception als `errorMsg` gespeichert und im Falle einer Ablehnung statt eines Fehlers zudem das Feld `rejected` auf `true` gesetzt. Als Letztes wird das angepasste `JobExecutionHandle`-Objekt in die anfangs übergebene `answerQueue` eingefügt, sodass es später in der Methode `waitForJobs` ausgewertet wird.

Bei Auftragsabbrüchen wird die Methode `processAbort` aufgerufen. Sie prüft zunächst, ob ein Auftrag mit der übermittelten ID für den entsprechenden Agenten existiert. Ist dies der Fall, wird die Methode `abortJob` der Operator Extension aufgerufen, mit dem `Jobs`-Objekt als Parameter. Die Operator Extension hat selbst dafür zu sorgen, dass der korrekte Auftrag abgebrochen wird. Erst nach Rückkehr aus der Methode `abortJobs` wird fortgefahren und der Auftrag aus der globalen Auftragsliste entfernt. Dieser wird hingegen nicht entfernt, wenn `abortJobs` eine Exception ausgelöst hat.

Es ist zu bemerken, dass jede aufgetretene Exception des Typs `StubException` zum Setzen der Flagge `stubError` in der Klasse `Main` führt, das wiederum zur Beendigung des Operators bei nächstmöglicher Gelegenheit führt.

### 3.2.2 Grafische Darstellung

Nachdem die Aufgaben der einzelnen Java-Klassen nun bereits erläutert wurden, soll zur Veranschaulichung ein beispielhafter Ablauf einer Ausführung des Operators anhand eines Sequenzdiagramms (s. Abb. 3.3) dargestellt werden.

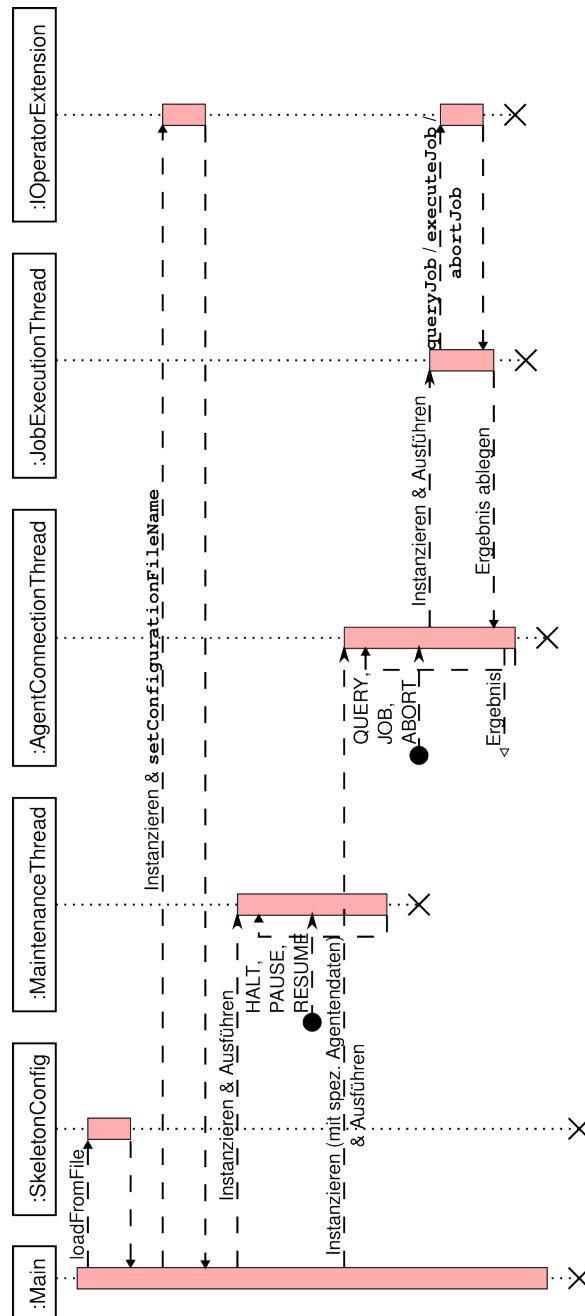


Abbildung 3.3: Beispielhaftes Sequenzdiagramm des Operator Skeleton.

### 3.2.3 Bibliothek zur Nutzung

Mit dem Java-Package `de.i2ar.operators.libop` steht eine einfach zu bedienende Java-Bibliothek zur Anwendung aller Protokollfunktionen des Agenten-Operator-Protokolls zur Verfügung. Dabei muss ein Agent lediglich die Klasse `Operators` instanziiieren und als Thread zur Ausführung bringen. Dabei wird bei der Erzeugung angegeben, auf welchem Port sich die Operatoren zu diesem Agenten verbinden sollen, wie groß die Verbindungswarteschlange (Backlog) ist, über welches Objekt sie bezüglich dem Ende des Agenten benachrichtigt werden sollen (Klasse `de.i2ar.operators.common.misc.Notification`), wie viele Operatoren sich maximal zeitgleich anmelden dürfen und ob ein Operator Listener existiert, der bei der An- und Abmeldung eines Operators benachrichtigt wird.

Bei Ausführung des Threads wird zunächst ein Serversocket mit oben genannten Eigenschaften geöffnet, über den sich Operatoren verbinden können. Für jeden verbundenen Operator wird ein neuer Thread der Klasse `Operator` erzeugt und ausgeführt. Dieser erhält über den Konstruktor das `Socket`-Objekt des angemeldeten Operators sowie weitere Daten. Der neue Operator wird schließlich in einer lokalen Liste gespeichert. Die Klasse `Operator` besitzt eine Liste anstehender Operationen von Seiten des Agenten und noch zu verarbeitende Antworten von Seiten des verbundenen Operators.

Zu jeder vom Agenten ausgelösten Operation gehört einerseits ein `Jobs`-Objekt zur näheren Spezifikation, was der Operator auszuführen hat und andererseits eine Referenz auf eine `LinkedBlockingQueue`. In diese wird nach Abschluss der Ausführung das Ergebnis gelegt. Dieses Vorgehen wurde gewählt, da die Operator-Kommunikation komplett asynchron abläuft, also kein Aufruf blockierend ist. Um die Methoden zur Nutzung der Protokolloperationen blockierend zu gestalten, wartet die aufgerufene Methode (`query` oder `execute`) auf einen neuen Eintrag in einer von ihr erzeugten `LinkedBlockingQueue`, direkt nachdem sie ihren Auftrag an die `Operator`-Klasse weitergegeben hat. Für einfache Anfragen existiert die private Klasse `RequestQuery` und für reale Aufträge die private Klasse `RequestExecute`, jeweils mit den Feldern `query` bzw. `execute`, `id` und `queue`. Das Feld `id` wird dabei vor der Ausführung über die Methode `getRequestId` mit einer für den Agenten mit jeder Anfrage fortlaufenden ID versehen, um asynchron eintreffende Antworten vom Operator wieder einer Anfrage zuordnen zu können. Objekte dieser Klasse werden in die oben genannte Liste zur Ausführung eingefügt.

Die `run`-Methode des Threads ist zyklisch aufgebaut und prüft in jedem Zyklus zunächst, ob neue Einträge in besagter Liste vorliegen. Ist dies der Fall, wird unterschieden nach anliegender Anfrage, Auftrag oder eingegangener Antwort vom Operator. Bei Anfrage oder Auftrag wird eine entsprechende Nachricht generiert und gemäß dem Agenten-Operator-Protokoll (s. VIRKUS [27]) an den Operator gesendet und bei vorliegender Antwort wird diese ausgewertet und ein entsprechendes `JobQueryResults`- oder `JobExecutionResults`-Objekt erzeugt. Dieses wird dem Benutzer der Bibliothek über die anfänglich angegebene `LinkedBlockingQueue` bereitgestellt. Im Fehlerfall ist der hinzugefügte Eintrag einfach der Wert `null`. Lag keine Anfrage oder eingegangene Nachricht vor oder



wurde sie bearbeitet, geht die `run`-Methode in eine zweite Phase über. Hier wird mit einem Timeout auf eine neue Nachricht vom Operator über den Socket gewartet. Trifft eine Nachricht ein, dann wird zunächst geprüft, ob eine laufende Anfrage mit der ID der Antwort vorliegt. Ist dies der Fall, erfolgt eine entsprechende Verarbeitung mit Parsen des eventuell enthaltenen XML-Dokuments und die Informationen werden als Objekt der Klasse `JobQueryResults` bzw. `JobExecutionResults` in die mit der Anfrage bereitgestellte `LinkedBlockingQueue` eingefügt und die Anfrage anschließend entfernt, sodass weitere Antworten des Operators zur gleichen ID nicht weiter zugeordnet würden.

Ein Anwender kann die Methoden `query` und `execute` der Klasse `Operator`, jeweils in blockierender und nicht-blockierender Variante, direkt für einen Operator aufrufen (Liste der Operator kann über die Methode `getAllOperators` der Klasse `Operators` ermittelt werden) oder die gleichnamige Methode der `Operators`-Klasse nutzen. Dabei erwarten beide Methoden explizit die Angabe, durch welchen Operator die Operation ausgeführt werden soll. Die Methode `query` erlaubt sogar, eine Liste von Operatoren zu übergeben. Der Rückgabewert ist hier eine Liste mit Wertepaaren, die aus einer Referenz auf einen Operator und auf ein `JobQueryResults`-Objekt bestehen, sodass diese Liste nun eine Zuordnung von je einem Angebot auf einen befragten Operator enthält, sofern dieser eines abgegeben hat.

Die Methode `getOperatorByName` ermittelt das `Operator`-Objekt, dessen verbundener Operator sich mit einem als Argument gegebenen Namen beim Agenten angemeldet hat. Mit der Methode `setInitializer` kann eine Implementation des Interfaces `IOperatorListener` zugewiesen werden, sodass dieses bei An- und Abmeldung des entsprechenden Operators konsultiert wird, respektive die implementierten Methoden aufgerufen werden. Das Interface schreibt die Methoden `onOperatorRegisters` und `onOperatorQuits` vor. Erstere wird aufgerufen, sobald sich ein Operator beim Agenten angemeldet hat. Die konkrete Implementation des Interfaces kann dem übergebenen `OperatorInformation`-Objekt alle Registrierungsinformationen entnehmen und die Methode mit dem Rückgabewert `true` verlassen, falls die Anmeldung zugelassen werden soll oder mit `false`, falls die Anmeldung abgelehnt werden soll. Die Methode `onOperatorQuits` hingegen besitzt ausschließlich informativen Charakter, sodass auf das Verlassen eines Operators reagiert werden kann.

### 3.2.4 Konfiguration

Die Konfiguration für den Operator Skeleton ist für jede einzelne Operator Extension einzeln vorzunehmen, da hierüber letztendlich die Funktion des Operators definiert ist. Die Maximalkonfiguration eines Operator Skeletons ist in Listing 3.2 abgedruckt. Die vom Operator Skeleton erwartete Umgebung ist durch das XML-Element `<skeleton-config>` in den Zeilen 2 und 13 gegeben. Für eine flexible Konfiguration muss hier mit dem Element `xsd-files` (Zeile 3) der Pfad zu den vom Operator benötigten XSD-Dateien (eXtensible Stylesheet Document) definiert werden. Alle Operatoren können auf den gleichen Pfad verweisen. Die beiden folgenden Zeilen definieren mit `extension-class` den

vollständigen Klassennamen der Operator Extension, die beim Start des Operators geladen werden soll und mit `extension-config` einen Dateinamen zur Konfiguration der Extension. Die Auswertung dieser Angabe obliegt ausschließlich der Operator Extension. Über das Element `dummy-execution` kann mit den Werten `true` und `false` spezifiziert werden, ob die Ausführung von Aufträgen simuliert werden soll (`true`). Es erfolgt dann keine Ausführung der Methode `executeJobs` der Operator Extension. Über das Element `maintenance` wird definiert, auf welchem Port `port` der Operator auf eingehende Wartungsverbindungen lauschen soll und wie lang die Verbindungswarteschlange `backlog` ist. In der Umgebung `agents` (Zeile 8 und 12) werden neben den Timeouts `agent-registration-timeout` zur Wartezeit zwischen zwei Verbindungsversuchen zu einem Agenten und `agent-job-timeout` (zur Zeit ungenutzt) auch die Agenten mit IP und Portnummer spezifiziert, zu denen sich der Operator selbstständig verbinden soll. Hierzu dient das Element `agent` und dessen Attribute `ip` und `port`. Das Element kann beliebig oft in der `agents`-Umgebung auftreten.

Listing 3.2: Beispielkonfiguration eines Operator Skeleton.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <skeleton-config>
3   <xsd-files>/usr/local/etc/lk3s/xsd</xsd-files>
4   <extension-class>
5     de.i2ar.warehouseop.ExtensionWarehouse
6   </extension-class>
7   <extension-config>
8     /usr/local/etc/lk3s/operators/ap5/warehouse/extension.xml
9   </extension-config>
10  <dummy-execution>false</dummy-execution>
11  <maintenance port="10005" backlog="1" />
12  <agents>
13    <agent-registration-timeout>
14      1000
15    </agent-registration-timeout>
16    <agent-job-timeout>0</agent-job-timeout>
17    <agent ip="192.168.157.220" port="11005" />
18  </agents>
19 </skeleton-config>
```

### 3.3 Agent

Die logische Verwaltung und Planung der Fertigung mit ihren einzelnen Fertigungsaufträgen wird von einem Multiagentensystem übernommen. Hier spielen mehrere Agenten mit ihren jeweils ganz eigenen Fähigkeiten zusammen. Um das Rad nicht neu erfinden zu müssen, wird als Grundlage auf das Java Agent DEvelopment Framework (JADE) zurückgegriffen. Es stellt bereits eine Laufzeitumgebung für Agenten bereit und bietet

unter anderem auch Funktionen für den Nachrichtenaustausch unter Agenten. Dabei kommen Protokolle zum Einsatz, die den Richtlinien der Foundations of Intelligent Physical Agents (FIPA) entsprechen. Ein weiteres Feature von JADE ist die Migration von Agenten zwischen physikalischen Systemen. Dabei kann ein Agent auf einem System schlafen gelegt und auf einem anderen mit seinem gesamten Kontext wieder zum Leben erweckt werden.

### 3.3.1 JADE-Grundlagen

#### Agent Management System

Das Rückgrat des JADE-Frameworks bildet das *Agent Management System* (AMS), bei dem sich alle Agenten bei ihrem Start anmelden. Damit besitzen sie einen im AMS eindeutigen Namen und können mit dem AMS, anderen Agenten und Komponenten kommunizieren. In einem System und in einer ganzen Umgebung können mehrere AMS parallel existieren. Durch einen Namenszusatz der Agenten ist stets eindeutig, welchem AMS ein Agent zugeordnet ist. Dies ist für die Kommunikation über AMS-Grenzen hinweg und für die Migration der Agenten von einem zu einem anderen AMS nötig. Die Migration ist Bestandteil des LK<sup>3</sup>S-Forschungsprojekts und wurde im Rahmen der Kooperation mit der Fachhochschule Osnabrück erfolgreich erprobt. Lokal auf die Digitale Fabrik bezogen findet die Migration jedoch keine Anwendung. Stattdessen liegt hier der Fokus auf der Inter-Agenten-Kommunikation, die in einem der folgenden Abschnitte näher erläutert wird. Interessant ist die Migrationsfunktion jedoch für externe Zulieferer mit eigenen Agentensystemen, um die Bestellung von Material abzuwickeln.

Das AMS ist selbst auch ein Agent und verwaltet sich und andere Agenten in sog. Containern. Mehrere Agenten können in diesen Containern zusammengefasst sein (s. Abb. 3.4).

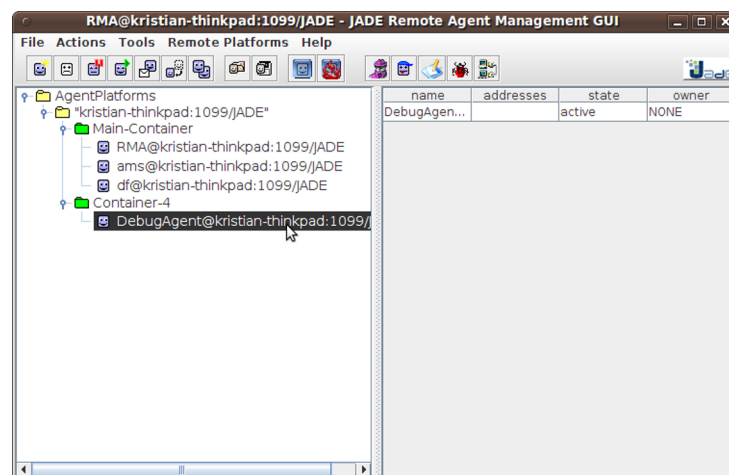


Abbildung 3.4: Grafische Oberfläche des JADE-Frameworks.

Über die GUI kann die GUI eines Sniffers aufgerufen werden. Dieser registriert sich im Agentensystem und bekommt dann die Telegramme ausgewählter Agenten ebenfalls zugestellt, um sie grafisch darzustellen, wie in Abb. 3.5 zu sehen ist. Hierdurch wird eine grafische Form des Debuggings möglich. Leider kann der Sniffer erst dann Nachrichten von Agenten mitschneiden, sobald der Benutzer die entsprechenden Agenten gewählt hat. Senden diese bereits bei ihrem Start Telegramme, so werden diese für den Benutzer nicht sichtbar sein, da Agenten erst nach ihrem Start im Sniffer aufgeführt werden. Die dargestellten Informationsflüsse können näher betrachtet und der Inhalt der Telegramme untersucht werden. Andere bzw. eigene Agenten können sich ebenfalls für den Empfang fremder Telegramme anmelden.

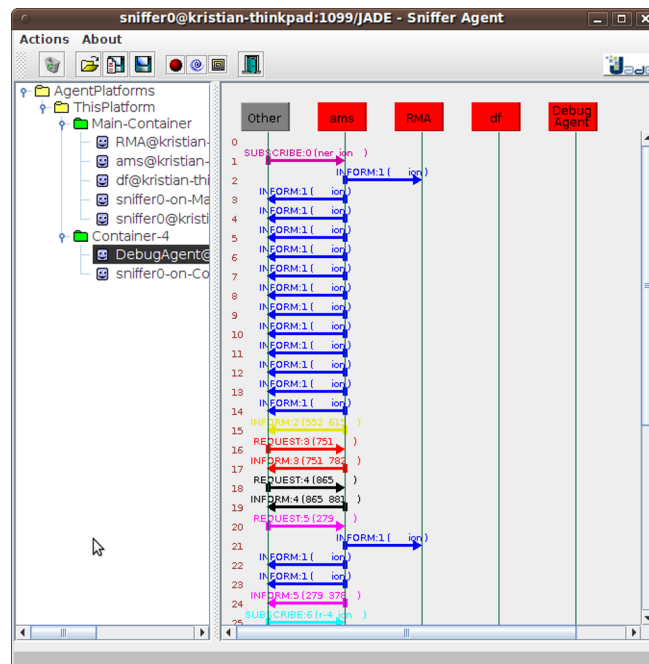


Abbildung 3.5: Grafische Oberfläche des Telegramm-Sniffers für das JADE-Framework.

### Directory Facilitator

Eine weitere Kernkomponente des JADE-Frameworks ist der sog. Directory Facilitator (DF). Dieser hat die Funktion eines Gelben-Seiten-Verzeichnisses, ist also eine zentrale Anlaufstelle für alle Agenten, die ihre Dienste selbst anbieten möchten und für alle Agenten, die andere suchen, die bestimmte Dienste anbieten. Die Registrierung der Dienste eines Agenten umfasst den eindeutigen Namen des Agenten sowie ein einheitliches Stichwort des angebotenen Dienstes und eine entsprechende Beschreibung. Die erforderlichen Aufrufe sind in Listing 3.3 abgedruckt. Zunächst wird eine Instanz der Klasse `DFAgentDescription` erzeugt und in Zeile 2 der eindeutige Name des eigenen Agenten zugewiesen. Im Anschluss wird eine `ServiceDescription` erzeugt, die die Beschreibung eines vom Agent bereitgestellten Dienstes repräsentiert. Genauer spezifiziert

wird diese Beschreibung in den Zeilen 4 und 5, in denen zunächst der einheitliche Typ des Dienstes und dann seine textuelle Beschreibung festgelegt wird. Anschließend wird in Zeile 6 als Dienst hinzugefügt und in Zeile 7 dem Directory Facilitator des Agentensystems bekannt gegeben.

Listing 3.3: Registrierung des Dienstes links eines Agenten.

```

1  DFAgentDescription dfd = new DFAgentDescription();
2  dfd.setName(myAgent.getAID());
3  ServiceDescription sd = new ServiceDescription();
4  sd.setType("links");
5  sd.setName("Links");
6  dfd.addServices(sd);
7  DFService.register(myAgent, dfd);

```

Die Suche nach einem Agenten mit einem speziellen angebotenen Dienst hingegen ist in Listing 3.4 dargestellt. Der Aufruf von `DFService.search` in Zeile 6 liefert letztendlich ein Array mit Objekten der Klasse `DFAgentDescription`, dem zu entnehmen ist, welche Agenten zu der Suchanfrage passen.

Listing 3.4: Suche nach Agenten, die den Dienst links anbieten.

```

1  template = new DFAgentDescription();
2  ServiceDescription sd = new ServiceDescription();
3  sd.setType("links");
4  template.addServices(sd);
5  /* myAgent ist eine Referenz auf den suchenden Agenten. */
6  DFAgentDescription[] result = DFService.search(myAgent,
    template);

```

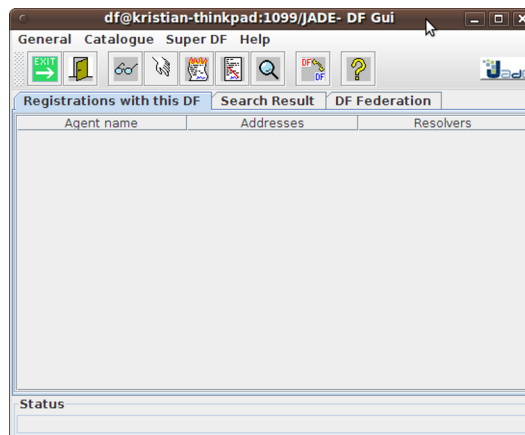


Abbildung 3.6: Grafische Oberfläche des JADE Directory Facilitators.

## Agenten-Grundgerüst

Ein minimaler Agent ist in Listing 3.5 dargestellt. Dabei ist es zunächst entscheidend, dass die eigene Klasse von der Klasse `Agent` erbt. Damit stehen auch alle Methoden zur Verfügung, um entsprechende verwaltende und kommunizierende Aktionen mit dem Agenten durchzuführen. Die Klasse `Agent` implementiert u.a. die Methoden `setup()` und `takeDown()`. In der Regel werden Sie von einem eigenen Agenten implementiert. `setup` wird dabei beim Start des Agenten automatisch durch das Agentensystem aufgerufen. Es empfiehlt sich also, hier initialisierende Aktionen durchzuführen. `takeDown` hingegen wird vom Agentensystem aufgerufen, sobald der Agent beendet werden soll. Hier besteht die Möglichkeit, abschließende Aktionen durchzuführen und den Agent kontrolliert zu beenden, um beispielsweise Datenverlust zu vermeiden. Üblicherweise werden in der `setup`-Methode Sprachen und Ontologien registriert, soweit diese bereits bekannt sind und es werden Behaviours eingerichtet, um den weiteren Programmablauf des Agenten zu realisieren. Ein Agent besitzt keine `main`-Methode. Stattdessen wird ein JADE-Boot-Code aufgerufen und der Name sowie die Klasse des Agenten als Parameter übergeben. Das Agentensystem instanziiert dann diese Klasse und bringt den Agenten zur Ausführung. Der Konsolenbefehl zur Ausführung sieht wie folgt aus, wenn davon ausgegangen wird, dass alle benötigten Bibliotheken und Klassen bereits im Java-Classpath bekannt sind:

```
$java jade.Boot -container eindeutiger Agentename:Java-Klassenname
```

Der Parameter `-container` besagt hierbei, dass der Agent in einem neuen Container ausgeführt werden soll. Bei Weglassen dieser Angabe würde das System versuchen, den Main-Container erneut zu erzeugen, was bei einem bereits laufenden JADE-Agentensystem zu einem Fehler führen würde. Optional kann der Parameter `-gui` eingefügt werden, um die in Abb. 3.4 anzuzeigen.

Listing 3.5: Minimaler Agent.

```
1 import jade.core.Agent;
2 public class MinimalAgent extends Agent {
3     @Override
4     public void setup() {
5     }
6 }
```

## Behaviours

Unter Behaviours werden einzelne Bestandteile eines Agenten verstanden, deren Ausführung zur Verfolgung des Ziels des Agenten dienen. Dabei besitzt jede Behaviour eine Referenz auf den Agenten, sodass Gebrauch von den zur Verfügung stehenden Kommunikationsmechanismen und weiteren Fähigkeiten der Agenten gemacht werden kann.

Es existieren unterschiedliche, vordefinierte Behaviour-Modelle. Als Grundlage dient die Behaviour. Sie ist eine Klasse, die die abstrakten Methoden `action` und `done` deklariert. Sie müssen daher von jeder konkreten Behaviour-Implementation definiert werden. Es gibt bereits u.a. die vordefinierten Behaviours `OneShotBehaviour` sowie `CyclicBehaviour`. Eine `OneShotBehaviour` terminiert nach genau einer Ausführung ihrer Methode `action`, eine `CyclicBehaviour` wird unbegrenzt oft wiederholt. Diese Funktionalität ist jeweils durch eine bereits vorgenommene Implementation der Methode `done` realisiert worden und diese Behaviours dienen nur zur Vereinfachung der Nutzung, könnten jedoch auch vom Entwickler selbst implementiert werden.

Die Methode `action` ist für die spezifische Aufgabe der Behaviour vorgesehen und muss in jedem Fall vom Entwickler mit der beabsichtigten Funktion implementiert werden.

Die Einbindung der Behaviours erfolgt programmiertechnisch durch Aufruf der Methode `addBehaviour` des Agenten. Soll eine Behaviour als Thread ausgeführt werden, so kann diese mit Hilfe der Methode `wrap` einer `ThreadedBehaviourFactory` gekapselt und dann wiederum mit `addBehaviour` eingebunden werden.

Typisch für Behaviours ist die Kommunikation mit anderen Komponenten. Da Filter auf den Telegrammempfang angewandt werden können, ist es somit möglich in mehreren Behaviours gleichzeitig auf den Eingang von Nachrichten unterschiedlicher Ausprägung zu warten.

### Agentenkommunikation

Mit Hilfe einer Sprache wird die Syntax eines Nachrichtenaustauschformats definiert. Eine Ontologie beschreibt die Struktur der Nachrichtennutzdaten. Die Sprache wird durch einen eindeutigen Bezeichner und einer entsprechenden Klasse mit Hilfsfunktionen repräsentiert. In diesem Projekt wird dafür FIPA-SL verwendet, eine Sprache auf Klartext-Basis. Alle Nachrichten können also auch direkt durch einen menschlichen Betrachter verstanden werden. Dies erleichtert die Fehlersuche. Diese Sprache liegt als sog. Codec (Abk. für *coder-decoder*) in Form der Klasse `SLCodec` vor. Sie wird beim Start eines Agenten oder spätestens vor Benutzung der Sprache instanziiert und beim Agenten über dessen `ContentManager` registriert, indem das Kommando

```
getContentManager().registerLanguage(new SLCodec(),  
    FIPANames.ContentLanguage.FIPA_SL0);
```

ausgeführt wird. Durch Zuweisen dieses Codecs zu einem zu versendenden Telegramm, kann das Agentensystem diese Telegramme automatisch beim Versenden entsprechend dem Codec mit dessen Hilfsfunktionen kodieren und der empfangende Agent kann sie wieder dekodieren. Dazu ist es nötig, dass der Codec bei allen beteiligten Agenten registriert ist.

Eine Ontologie zur Beschreibung der Struktur der Telegramm-Nutzdaten besteht aus zwei wesentlichen Teilen: Zum einen eine Klasse, deren Klassenvariablen mitsamt Getter-/Setter-Methoden die Nutzdaten repräsentieren und zugänglich machen und zum anderen eine weitere Klasse, die die Ontologie implementiert (siehe Beispiel mit Listing 3.6, 3.7). In Listing 3.7 ist zu sehen, dass die Klasse `BeispielOntologie` von der JADE-Klasse `Ontology` abgeleitet ist. Durch diese Wiederverwendung können alle Ontologien nicht nur vom Agentensystem gleich behandelt werden, in der Implementation der spezifischen Ontologie-Klasse stehen auch entsprechende, von `Ontology` geerbte Methoden zur Erzeugung der Nutzdatenstruktur bereit. In Zeile 8 wird zunächst der Name der Ontologie als Konstante festgehalten wird. Zur Realisierung eines Singleton, denn mehrere Instanzen der selben Ontologie wären irreführend, wird ein privater Konstruktor ab Zeile 15 definiert und eine Instanz in Zeile 9 erzeugt und der privaten Klassenvariablen `instance` zugewiesen, deren Wert mit Hilfe der öffentlichen Methode `getInstance` (Zeile 11) abgefragt werden kann. Der private Konstruktor ab Zeile 15 veranlasst zunächst den Aufruf des Super-Konstruktors mit dem Ontologie-Namen und einem weiteren Parameter. Vor einer Nutzung bedarf es jedoch der Registrierung der einzelnen Felder, die aus der in Listing 3.6 in die Nutzdatenstruktur einbezogen werden sollen, denn diese Klasse kann durchaus auch für diesen Zweck ungenutzte Felder und Methoden enthalten. Für dieses Beispiel wird als erstes die Klasse `Beispiel` als `PredicateSchema` in der Zeile 19 registriert. Eine ein `Predicate` implementierende Klasse dient zur Definition einer Nutzdatenstruktur, mit der eine Handlung ausgeführt und parametrisiert werden soll. Dazu wird ein String als Name verwendet, der in diesem Fall dem Klassennamen `Beispiel` entsprechen muss. Der zweite Parameter des Aufrufs ist eine Referenz auf die Klasse in Listing 3.6. Mittels der `getSchema`-Methode (Zeile 20) kann das soeben erzeugte `PredicateSchema` wieder abgerufen werden und die relevanten Felder der Klasse `Beispiel` über die Methode `add` (Zeilen 21 u. 22) eingebunden werden. Hierzu ist als erstes Argument wieder der Name des Feldes als String notwendig, als zweites der Typ des Feldes, kodiert als bereits vordefiniertes Schema im JADE-Framework. Hierzu stehen u.a. für die Typen `String` und `Integer` vordefinierte Schemata über die Klasse `BasicOntology` bereit, die wie alle anderen Ontologien über `getSchema` anhand eines eindeutigen Strings abgerufen werden können. Die bereits Vordefinierten müssen in den Typ `PrimitiveSchema` transformiert werden. Optional kann als drittes Argument eine Optionalität der Belegung Feldes mit einem Wert angegeben werden: obligatorisch, optional, beliebig viele (entspricht einer Liste). Zudem können Felder eigener Typen bzw. Klassen eingebunden werden. Dies erfordert die Implementation des Interfaces `Concept` bei diesen Klassen. Beim Hinzufügen des Feldes zur Ontologie darf dann als Typ kein vordefiniertes Schema aus `BasicOntology` verwendet werden. Stattdessen muss das neue Schema wie oben zunächst mit seinen Feldern registriert worden sein, jedoch nicht als `PredicateSchema`, sondern als `ConceptSchema`. Dann kann das Schema anhand seines Klassennamens mit Hilfe von `getSchema` abgerufen und der Rückgabewert in ein `ConceptSchema` transformiert und als Typ des Feldes verwendet werden. Es gilt zu beachten, dass der oben erwähnte FIPA-SL-Codec nur eingeschränkt in der Lage ist, Listen zu verarbeiten und das Auftreten unbesetzter optionaler Felder in der Mitte einer Struktur nicht unterstützt. Da gerade auch diese Kombination des Öffneren auftritt, ist es an einigen Stellen notwen-



dig, eine Hilfskonstruktion zu verwenden, um die einfache Debugfähigkeit des Protokolls zu erhalten. Hierzu wurden XML-Kodierungen gewählt, die als einfacher String in die Ontologien eingefügt wurden. Die entsprechenden Klassen (z. B. `Item`) besitzen dafür eigene Kodier- und Dekodiermethoden. Da einige der aufgerufenen Methoden Exceptions auslösen können, ist der Einschluss aller Aufrufe in einen try-catch-Block notwendig.

Wie auch bei Sprachen, muss eine Ontologie vor ihrer Benutzung bei jedem mit ihr konfrontierten Agenten über dessen `ContentManager` registriert werden. Dies geschieht über den Aufruf:

```
getContentManager().registerOntology(Ontologie-Instanz,  
                                     Ontologie-Bezeichner);
```

also beispielsweise:

```
getContentManager().registerOntology(BeispielOntologie.  
    getInstance(), "Beispiel-Ontologie");
```

Listing 3.6: Beispiel.java

```
1 import jade.content.Predicate;  
2  
3 public class Beispiel implements Predicate {  
4     private String name;  
5     private Integer age;  
6  
7     public String getName() {  
8         return name;  
9     }  
10  
11     public void setName(String name) {  
12         this.name = name;  
13     }  
14  
15     public Integer getAge() {  
16         return age;  
17     }  
18  
19     public void setAge(Integer age) {  
20         this.age = age;  
21     }  
22 }
```

Listing 3.7: BeispielOntologie.java

```
1 import jade.content.onto.BasicOntology;  
2 import jade.content.onto.Ontology;  
3 import jade.content.onto.OntologyException;
```

```
4 import jade.content.schema.PredicateSchema;
5 import jade.content.schema.PrimitiveSchema;
6
7 public class BeispielOntologie extends Ontology {
8     public static final String ONTOLOGY_NAME = "Beispiel-Ontologie";
9     private static Ontology instance = new BeispielOntologie();
10
11     public static Ontology getInstance() {
12         return instance;
13     }
14
15     private BeispielOntologie() {
16         super(ONTOLOGY_NAME, BasicOntology.getInstance());
17         PredicateSchema ps;
18         try {
19             add(new PredicateSchema("Beispiel"), Beispiel.class);
20             ps = (PredicateSchema) getSchema("Beispiel");
21             ps.add("name", (PrimitiveSchema) getSchema(BasicOntology.STRING));
22             ps.add("age", (PrimitiveSchema) getSchema(BasicOntology.INTEGER));
23         } catch (OntologyException e) {
24             e.printStackTrace();
25         } catch (Exception e) {
26             e.printStackTrace();
27         }
28     }
29 }
```

Das JADE-Framework stellt sog. ACLMessages als Telegramme der Agentenkommunikation zur Verfügung. Sie bestehen u.a. aus Feldern für:

- Absender,
- Empfänger,
- Sprachbezeichnung,
- Ontologiebezeichnung,
- Gesprächs-ID,
- Antwort-ID,
- erwarte Antwort mit ID,
- Performative und
- Inhalt gemäß Ontologie.

Absender und Empfänger sind Objekte der Klasse `AID` (*Agent Identifier*). Dieser gibt Auskunft über den Namen des Agenten und das JADE-Laufzeitsystem, in dem er ausgeführt wird. Typisch ist die Darstellung als String in dieser Form:

```
agentname@host:1099/JADE
```

Um auch die system- und agentenplattformübergreifende Telegrammvermittlung zu ermöglichen ist bei manueller Erzeugung einer `AID`, z. B. anhand einer eingelesenen Zeichenkette, darauf zu achten, dass auch eine sog. Adresse über die Methode `addAddresses` der `AID` hinzugefügt wird. Sie besitzt die Form:

```
http://host:7778/acc
```

Es existieren das Feld `language` für einen eindeutigen Bezeichner einer Agentensprache, z. B. `fipa-sl` für den FIPA-SL-Codec, und das Feld `ontology` für einen eindeutigen Bezeichner einer Ontologie. Diese Angaben gestatten nicht nur dem Absender die korrekten Methoden zur protokollgerechten Kodierung des Telegramms auszuführen, sondern auch dem Empfänger, nach Erhalt des Telegramms sofort die korrekten Methoden zur Dekodierung aufzurufen.

Zusätzlich kann eine Gesprächs-ID (`conversationId`) festgelegt werden. Sie entspricht „Mein Zeichen“ eines formellen Briefs. In diesem Zuge kann auch eine Antwort-ID (`inReplyTo`, entspricht „Ihr Zeichen“) gesetzt werden, um den Bezug einzelner Nachrichten klarzustellen. Erwartet der Absender, dass der Empfänger mit einer bestimmten Antwort-ID antwortet, so kann er dies mit dem Feld `replyWith` vorgeben.

Jeder Nachricht ist zudem eine Performative (Definition, Übersetzung s. [17]) zugeordnet, die Auskunft über die Absicht des Telegramms gibt. Dies kann eine Anfrage sein, ein Angebot, eine Auskunft oder ähnliches. Die Klasse `ACLMessage` definiert hierzu die in FIPA [11]) definierten Konstanten. Die folgende Auflistung zeigt die im entwickelten Agentensystem genutzten Performativen und ihre Anwendung:

**ACCEPT PROPOSAL** Annehmen eines zuvor eingeholten Angebots.

**CFP** (Call for Proposal) Ein Angebot einholen.

**CONFIRM** Bestätigen einer zuvor getätigten Anfrage.

**DISCONFIRM** Entkräften einer zuvor getätigten Anfrage.

**FAILURE** Signalisierung eines eingetretenen Fehlers im Betrieb.

**INFORM** Benachrichtigen über ein eingetretenes Ereignis oder einen Zustand.

**PROPOSE** Unterbreitung eines Angebots.

**REFUSE** Zurückweisen eines Auftrages vor dessen Ausführung.

Die Nutzdaten (Inhalt gemäß Ontologie) eines Telegramms können als String manuell vorgegeben werden. Alternativ kann vom `ContentManager` des Agenten Gebrauch gemacht werden. Mit seiner Hilfe kann ein Objekt, das das Interface `Predicate` oder

Concept implementiert, anhand der für das Telegramm vorgegebenen Sprache und Ontologie automatisch als String kodiert und als Nutzdaten gesetzt werden. Zum Erzeugen des kodierten Inhalts ist folgender Aufruf vorgesehen:

```
getContentManager().fillContent(ACLMessage-Objekt, Predicate-/  
Concept-Objekt);
```

Zum Dekodieren des Inhalts einer *ACLMessage* kann folgender Ausdruck verwendet werden:

```
getContentManager().extractContent(ACLMessage-Objekt);
```

Der Rückgabewert der Methode ist ein Objekt des Typs *ContentElement*, das zur weiteren Verwendung in den erwarteten *Predicate-/Concept*-Typ transformiert werden muss.

Bei beiden Methoden können Exceptions auftreten, z. B. wenn die angegebene Sprache oder Ontologie nicht zum angegebenen *Predicate-/Concept*-Objekt passt oder wenn Restriktionen der Ontologie nicht eingehalten wurden.

Das Versenden eines Telegramms geschieht durch den Aufruf

```
send(ACLMessage-Objekt);
```

als Methode des Agenten. Es ist nicht möglich, den erfolgreichen Versand festzustellen, es sei denn, es wird explizit auf Fehler-Nachrichten des Agent Management System (AMS) gelauscht. Diese Nachrichten werden als normale Telegramme versandt. Empfangen werden können sie wie alle anderen, indem eine der folgenden Methoden des Agenten aufgerufen wird:

```
receive()  
receive(Muster)  
blockingReceive()  
blockingReceive(Muster)  
blockingReceive(Timeout)  
blockingReceive(Muster, Timeout)
```

Die ersten beiden Methoden rufen ein bereits empfangenes, neues Telegramm ab oder geben eine Null-Referenz zurück, falls momentan keine Telegramme anliegen. Die übrigen vier Methoden blockieren solange, bis ein neues Telegramm empfangen wurde. Über ein *Timeout*-Argument kann die maximale Blockierdauer vorgegeben werden. Nach deren Ablauf gibt die Methode *blockingReceive* ebenfalls eine Null-Referenz zurück. Da ein Aufruf von *blockingReceive* den weiteren Programmablauf des Agenten und aller seiner Behaviours vorübergehend blockiert, ist in Erwägung zu ziehen, dass die entsprechenden Programmteile in Threads mit Hilfe der *ThreadedBehaviourFactory* ausgelagert werden. Sowohl bei *receive* wie auch bei *blockingReceive* kann ein *Muster* vorgegeben werden, das durch Methoden der Klasse *MessageTemplate* generiert werden kann. Dies führt zu einer Annahme von Nachrichten, nur wenn sie dem vorgegebenen, durch ein

MessageTemplate beschriebenes Muster entsprechen. Das Muster kann aus logisch verknüpften Übereinstimmungen von Telegrammfeldern wie Absender oder Gesprächs-ID bestehen. Folgendes Beispiel zeigt die Erzeugung eines Musters für folgende umgangssprachliche Bedingung:

*Der Absender der Nachricht heißt „Absender“ und die Performative ist entweder ACCEPT\_PROPOSAL oder REFUSE.*

```
MessageTemplate muster = MessageTemplate.and(
    MessageTemplate.MatchSender("Absender"),
    MessageTemplate.or(
        MessageTemplate.MatchPerformative(ACLMMessage.ACCEPT_PROPOSAL),
        MessageTemplate.MatchPerformative(ACLMMessage.REFUSE)
    )
);
```

Der Vorteil dieses Konzepts ist die Möglichkeit, in mehreren Programmteilen parallel Nachrichten unterschiedlicher Form empfangen zu können. Zudem können Kommunikationsabläufe daher auch ohne Verwendung von Zustandsautomaten realisiert werden.

### 3.3.2 LK<sup>3</sup>S-Agenten

Bei den Agenten, die im Rahmen des LK<sup>3</sup>S-Projekts entwickelt werden, handelt es sich um Agenten, die auf der gleichen Softwarebasis aufsetzen. Es existiert dabei ein gemeinsamer minimaler Agent, der anhand einer Konfigurationsdatei seine eigentliche Funktionalität mit Hilfe der Java Reflection API nachlädt. Diese Funktionalitäten kapseln jeweils ganze Aufgabenbereiche in wiederverwendbare Module. Sie verleihen dem Agenten damit auch nach außen verschiedene Gestalten und Schnittstellen. Aus diesem Grund werden diese Gestalten einer deutschen Redewendung nach mit dem Begriff *Gesicht* (engl. *face*) bezeichnet. Im Folgenden soll nun eine Übersicht über die im LK<sup>3</sup>S-Projekt standardmäßig vorhandenen Gesichter gegeben werden und wie sie in die Agenten eingebunden werden.

Beim Start eines LK<sup>3</sup>S-Agenten, der von der Klasse LK3SAgent2 (s. Abb. 3.7) abgeleitet ist, wird eine als Startargument angegebene Konfigurationsdatei eingelesen und ausgewertet. Sie besitzt den in Listing 3.8 exemplarisch dargestellten Aufbau.

Listing 3.8: Beispielhafte Konfiguration eines LK<sup>3</sup>S-Agenten.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <agent-config>
3     <name>WarehouseAgent</name>
4     <description>Warehouse Agent</description>
```

```
5   <xsd-files>/usr/local/etc/lk3s/xsd</xsd-files>
6   <logging>
7       <host>192.168.157.220</host>
8       <name>logging</name>
9       <table>logbook</table>
10      <user>logger</user>
11      <password>logger</password>
12  </logging>
13  <faces>
14      <face class="DFFederation" config="/usr/local/etc/lk3s/
15          agents/ap5/face-dffederation.xml" />
16      <face class="operators" config="/usr/local/etc/lk3s/
17          agents/ap5/face-operators.xml" />
18      <face class="links" config="/usr/local/etc/lk3s/agents/
19          ap5/face-links.xml" />
20      <face class="transport" config="/usr/local/etc/lk3s/
21          agents/ap5/face-transport.xml" />
22      <face class="storage" config="/usr/local/etc/lk3s/agents
23          /ap5/face-storage.xml" />
24      <face class="orders" config="/usr/local/etc/lk3s/agents/
25          ap5/face-orders.xml" />
26  </faces>
27 </agent-config>
```

Zunächst wird in Zeile 3, in der Umgebung `agent-config`, der Name des Agenten definiert. Da beim Start des Agenten über den JADE-Loader `jade.Boot` bereits ein Agentenname festgelegt werden muss, ist das Feld `name` zunächst ohne Verwendung. Das Feld `description` ist ebenfalls nicht von operativer Bedeutung. Beide Informationen können hingegen für den geplanten Agentenkonfigurator von Bedeutung sein. Im Anschluss daran wird in Zeile 5 über das Feld `xsd-files` ein Pfad definiert, in dem ein Agent nach XSD-Dateien suchen soll, die z. B. für die Durchführung der Operator-Kommunikation notwendig sind. Somit ist es möglich, dass alle Agenten die gleichen XSD-Dateien nutzen. Unter `logging` finden sich Angaben zu einer Datenbankverbindung, die zur Protokollierung der Produktion genutzt werden soll. In der Umgebung `faces` (Zeile 6) werden nun alle Gesichter des jeweiligen Agenten aufgeführt, die für das Erreichen der spezifischen Ziele des Agenten benötigt werden. Da die Ziele eines Agenten, wie bereits erwähnt auf Behaviours abgebildet werden, so muss auch jedes Agentengesicht von einer Behaviour abgeleitet sein, damit es später als solche ausgeführt werden kann. Jedes Gesicht wird anhand seines Klassennamens und einer optionalen Konfigurationsdatei spezifiziert.

Wie oben bereits erwähnt, wird beim Start eines Agenten dessen Methode `setup` aufgerufen. So auch bei dem `LK3SAgent2`, der dort zunächst den Debuglevel für Bildschirmausgaben setzt und im Anschluss die Methode `init` aufruft, die zunächst die Agentenkonfigurationsdatei lädt und als Objekt des Typs `LK3SAgentConfig` speichert. Für jedes dort konfigurierte Gesicht werden eine Instanz der Klasse `LK3SAgentFaceConfig` erzeugt und

die Werte aus der Agentenkonfiguration übernommen. Jede Gesichtskonfiguration enthält hierzu vier Felder:

**agentConfig** ist eine Referenz auf das LK3SAgentConfig-Objekt des Agenten. Es enthält die Felder der Agentenkonfigurationsdatei (XML). Dabei sind alle Gesichter in einer Map *faces* (über Getter *getFaces* zu nutzen) mitsamt ihrer entsprechenden Gesichtskonfiguration gespeichert. Hierneben existiert eine zweite Map *capabilities* (*getCapabilities*), in die spezielle Erweiterungen des *OrdersFace* (s.u.) die Bearbeitungsfähigkeiten der an den Agenten angeschlossenen Hardware eintragen können. Zudem enthält es eine Referenz auf das LK3SAgent2-Objekt selbst. Es ist notwendig, um bereits beim Laden der Gesichter, also noch vor Ausführung ihrer Behaviours, eine Referenz auf den Agenten zu besitzen, um z. B. die Existenz anderer, benötigter Gesichter prüfen zu können.

**clazz** enthält den Namen der Klasse, die dieses Agentengesicht repräsentiert. Im weiteren Verlauf wird versucht, anhand dieses Namens die entsprechende Klasse zu instanziiieren.

**configFileName** enthält den Namen der Konfigurationsdatei laut Agentenkonfiguration.

**instance** wird nach dem Laden der Konfigurationen eine Referenz auf das erzeugte Objekt dieses Gesichts vorhalten. Zunächst handelt es sich dabei um ein Objekt des Typs LK3SAgentLoadableFace, das zunächst einmal nur ein Interface ist und die Methoden *load*, *getConfig* und *unload* zur Implementation vorschreibt. Mit der Kenntnis des Gesichts, das hinter diesem Interface implementiert ist, kann eine Transformation des Datentyps vorgenommen werden, sodass auch auf die speziellen Methoden und Felder eines Gesichts Zugriff besteht, falls dieses solche definiert.

In *setup* wird auch der *IdManager* initialisiert, der unter Berücksichtigung des Agentennamens eindeutige Bezeichner für z. B. Konversations-IDs erzeugen kann. Abschließend wird über die im LK3SAgentConfig enthaltenen Gesichtskonfigurationen iteriert und angenommen, dass jede Instanz eines Gesichts zugleich eine Behaviour darstellt, sodass jedes Gesicht mit Hilfe einer *ThreadedBehaviourFactory* als neue Behaviour zum Agenten hinzugefügt und damit zur Ausführung gebracht wird.

## OperatorsFace

**Beschreibung** Verwaltet Verbindungen zu lokalen Operatoren. Erlaubt die Anfrage und Ausführung von Operator-Programmen.

**Konfiguration** Port des Serversockets, Backlog, max. Anzahl Operatoren

**Abhängigkeiten** keine

Mit Hilfe des `OperatorsFace` besitzt ein Agent eine einfach zu handhabende Schnittstelle, um von beliebigen Operatoren des LK<sup>3</sup>S-Agentensystems Gebrauch zu machen. Es instanziiert dabei während der Initialisierung ein Objekt der Klasse `Operators` der Bibliothek `libop`. Dabei werden als Parameter die Werte übergeben, die aus der Konfigurationsdatei des `OperatorsFace` entnommen werden (s. Listing A.2). Es findet die Protokollspezifikation der Agenten-Operator-Kommunikation Anwendung (s. VIRKUS [27]) und die Methoden `query`, `execute`, `getAllOperators`, `getOperatorByName`, `addOperator`, `getBestOperator` und `setOperatorInitializer` stellen dem Entwickler den Zugang hierzu bereit. Die auszuführenden Anfragen und Befehle werden anhand eines `Jobs`-Objekts gekapselt und die Protokollumsetzung wird vom `OperatorsFace` durchgeführt. Auch die Ergebniswerte werden je nach Ausführung (Anfrage, Vorbereitung oder Befehl) als Objekt der Klasse `JobQueryResults` oder als `JobExecutionResults` für die letzten beiden Varianten zur Verfügung gestellt. Die Implementation erlaubt die parallele Ausführung mehrerer Anfragen und Befehle, da eine eindeutige Zuordnung über die in jedem Telegramm enthaltene ID möglich ist. Zudem existiert eine Möglichkeit, die Anmeldungen der Operatoren am Agenten zu validieren, bevor diese akzeptiert werden. Hierzu wurde das Interface `IOperatorListener` eingeführt. Erstellt ein Entwickler eine Implementation dieses Interfaces, so kann er diese über die Methode `setOperatorInitializer` des `OperatorsFace` hinzufügen und wird fortan über anstehende Anmeldungen informiert. Hierzu wird die Methode `onOperatorRegisters` des Interfaces aufgerufen. Gibt die Ausführung der Methode den Wert `true` zurück, so wird die Anmeldung gestattet, gibt sie `false` zurück, so wird sie verhindert. Meldet sich ein Operator ab, so wird dies durch Aufruf der entsprechend implementierten Methoden `onOperatorQuits` signalisiert, die jedoch das Abmelden des Operators nicht unterbinden können.

## **LinksFace**

**Beschreibung** Verwaltet logische und physikalische Verbindungen zu Nachbaragenten.

**Konfiguration** ja

**Abhängigkeiten** keine

Aufgabe des `LinksFace` ist es, sicherzustellen, dass alle bekannten logischen und physikalischen Verbindungen zu anderen Agenten aufgebaut und aufrecht erhalten werden. Die Verbindungen der Agenten untereinander werden über die Konfigurationsdatei für dieses Gesicht festgelegt. Sie ist in Listing A.3 abgedruckt. In den Zeilen 2 bis 6 werden dabei alle Links mit ihrer fabrikweit eindeutigen Link UID festgehalten. Jedem Link kann auch ein Haltepunkt über das Attribut `stop-uid` zugeordnet werden, falls über den Link Transportaufgaben durchführbar sein sollen. Über die booleschen Attribute `incoming` und `outgoing` kann darüber hinaus spezifiziert werden, ob an einem Link Güter angenommen bzw. abgegeben werden können.

Die Klasse `LinksFace` legt bei ihrer ersten Ausführung für jeden konfigurierten Link ein Objekt der Klasse `Link` mit sämtlichen konfigurierten Informationen und in Abhängigkeit



dessen Materialflussrichtung eine eigene Behaviour an. Die Klasse `Link` gehört zur Ontologie `Link-Ontology`. Für ausgehende Links ist dies die `RequestLinkBehaviour`. Sie fragt zyklisch beim Directory Facilitator des Agentensystems, welche anderen Agenten ebenfalls Links unterstützen. Dazu ist es notwendig, dass all diese Agenten bzw. die `LinksFace` Implementation für alle Agenten einen entsprechenden Eintrag im Directory Facilitator erzeugen. An alle gefundenen Agenten wird eine Nachricht gesendet, mit der Bitte, alle konfigurierten Links bekanntzugeben. Wurde die Antwort empfangen, so wird diesem eine Nachricht mit Bitte der Bestätigung der gemeinsamen Links gesendet und anschließend auf eine Antwort gewartet. Eben für die Beantwortung der Anfragen bezüglich eingehender Links ist auf Seite des anderen Agenten die zyklische `ReplyLinkBehaviour` zuständig. Sie beantwortet Anfragen positiv, falls der angefragte Link hier wirklich eingehend ist. Erhält der anfragende Agent keine oder eine negative Antwort, so wird die Verlinkung zum anderen Agenten entfernt. Da die `RequestLinkBehaviour` eine `CyclicBehaviour` ist, wird nach Ablauf eines Timeouts erneut nach einem entsprechenden Partner für einen Link gesucht. Diese erneute Suche findet im Übrigen auch nach Ablauf der Gültigkeitsdauer eines Links statt. Dieses Verfahren dient zum Aufspüren bislang unerkannter, abgebrochener Verbindungen zu anderen Agenten. Die ebenfalls bei der ersten Ausführung des `LinksFace` als Behaviour erzeugte `MonitorLinksBehaviour` ist hingegen dafür zuständig, die eingehenden Anfragen bezüglich der Bekanntgabe der vom Agenten verwalteten Links zu beantworten. Da diese Anfragen beliebig häufig eintreffen können, handelt es sich dabei um eine zyklische Behaviour.

Die Methode `lookupLink` stellt zudem eine Suchfunktion bereit, mit der die gespeicherten `Link`-Objekte nach einer Link-UID durchsucht werden können.

## TransportFace

**Beschreibung** Bedient die Anfrage und Ausführung von lokalen und agentenübergreifenden Transportaufträgen.

**Konfiguration** Lokale Haltepunkte

**Abhängigkeiten** `OperatorsFace`, `LinksFace`

Das `TransportFace` besitzt eine sehr komplexe Aufgabe und wird von mehreren Behaviours genutzt, um Transportaufgaben sowohl innerhalb wie auch agentenübergreifend zu evaluieren und auszuführen. Da für das `LinksFace` nur diejenigen anfahrbaren Haltepunkte definiert werden, ist es nötig, dem `TransportFace` über eine Konfigurationsdatei bekanntzugeben, welche internen Haltepunkte im Bereich des Agenten existieren, einschließlich denen bereits über das `LinksFace` konfigurierten. Die Konfiguration besitzt den in Listing A.4 dargestellten Aufbau. Hier ist zudem in Zeile 3 über das Element `accept-all-items` mit den Werten `true` und `false` zu definieren, ob der Agent bei Untätigkeit dazu genutzt werden kann, um einfache Routingvorgänge von Materialien und Gütern durchzuführen. Demnach würden alle angebotenen Güter akzeptiert und über

das arbeitsplatzinterne Transportsystem zu einem Link zum nächsten Agenten weitergeleitet.

Es kann ganz oberflächlich gesagt werden, dass das `TransportFace` einen Weg durch das Agentensystem anhand der miteinander verbundenen Links sucht. Dabei findet ein rekursives Verfahren Verwendung und es werden jeweils die internen Fahrtkosten der Agenten aufsummiert und an die nächsthöhere Stufe in der Rekursion zurückgemeldet. Jeder Agent befragt dazu alle eigenen, direkten Nachbarn, sofern sie in der bisherigen Route noch nicht auftreten. Zunächst wird das spätere Angebot für die Route mit einem konkreten direkten Nachbaragenten mit den Fahrtkosten vom eigenen Übernahmeort des Guts zum Link des Nachbarn belegt. Diese werden mit Hilfe des Path Operators berechnet, also außerhalb des Agenten. Die Transportanfrage wird dann zum gewählten Nachbaragenten weitergeleitet, der obige Schritte ebenfalls durchführt und schließlich an den anfragenden Agenten mit einem Angebot berichtet. Dieser addiert die Kosten des Angebots auf die internen Fahrtkosten zum Link des anderen Agenten. Nachdem alle eigenen Nachbaragenten auf diese Weise befragt wurden, wird das Angebot mit den insgesamt günstigsten Fahrtkosten ausgewählt und an den Agenten berichtet, der in der Rekursion an nächsthöherer Stufe steht. Die folgenden Absätze beschreiben dieses Vorgehen detaillierter.

Nach Laden des `TransportFace`, erwartet das `TransportFace` eingehende Nachrichten über die `action`-Methode (siehe auch Abb. 3.8). Es erfolgt nun eine Dekodierung, sodass im Anschluss ein Objekt der Klasse `Transport` der `Transport-Ontology` vorliegt, das alle Details zur Transportanfrage enthält. Diese sind:

- Vollständige Beschreibung des zu transportierenden Guts
- Priorität der Transportanfrage
- Global eindeutige UID eines Links, an dem das Gut vom nächsten Agenten übernommen werden soll (bei erstem Agenten leer, da sich das Gut bereits in dessen Zugriffsbereich befindet)
- Zielagent für den Transport
- Zielhaltepunkt für den Transport, falls bekannt, ansonsten leer
- Ursprungsagent des Transports
- Ursprungshaltepunkt des Transports
- Flagge, ob das transportierte Gut frei ist oder einem bestimmten Auftrag angehört
- Angebot, falls es sich um eine Anfrageantwort handelt
- Alle bisher an der Anfrage beteiligten Agenten, um Schleifen zu vermeiden und einen Determinismus im Algorithmus zu erhalten

Nach Eintreffen einer Transportanfrage wird zunächst geprüft, ob im `Transport`-Objekt der Wert des Feldes `originAgent` dem Bezeichner des eigenen Agenten entspricht. Ist

dies der Fall ist, dann ist davon auszugehen, dass ein interner Vorgang oder ein anderer Agent einen Transportauftrag mit dem eigenen Agenten als Startpunkt durchführen möchte. Daher ist auch davon auszugehen, dass der Haltepunkt `originStop` bereits bekannt und in der Transportanfrage eingetragen ist, an dem sich das mit dem Feld `item` beschriebene Gut befindet. Für den Fall, dass der Ursprung der Transportanfrage nicht im eigenen Agenten liegt, so ist davon auszugehen, dass der eigene Agent Teil einer Verkettung von Agenten zur Durchführung der Transportaufgabe ist und nicht an dessen Anfang steht. Daher muss angenommen werden, dass die Transportanfrage im Feld `handoverLink` den Bezeichner eines globalen Links enthält, der auch dem eigenen Agenten zugeordnet ist. Im Anschluss wird geprüft, ob die aus der Anfrage ermittelten Werte Gültigkeit besitzen und das weitere Vorgehen wird durch Aufruf der privaten Methode `routeItem` eingeleitet, falls das beschriebene Gut beim eigenen Agenten erwartet wird (`ItemManager.isAwaitingItem`) oder falls das `TransportFace` so konfiguriert ist, dass es bei Untätigkeit beliebige Güter zur Durchleitung akzeptiert. Tritt bis zu diesem Punkt ein Fehler auf, so wird ein `REFUSE`-Telegramm an den Absender zurückgeschickt, um zu signalisieren, dass die Anfrage nicht bearbeitet werden kann.

**routeItem** In der Methode `routeItem` wird zunächst bei einem Transportbefehl (`isQuery = false`, keine Anfrage) die Liste der bereits beim Routing beteiligten Agenten entfernt und dabei auf einen leeren String gesetzt (s. Abb.3.9). Dieser Schritt ist zwingend erforderlich, da die für Transporte versendeten Telegramme wiederverwendet werden und nur der Empfänger angepasst wird und sich seit erstmaliger Routenplanung Änderungen in der Belegung und Verfügbarkeit der Zwischenstationen bzw. -agenten ergeben haben können. Anschließend erfolgt, unabhängig davon, ob es sich um eine reine Anfrage oder um einen Transportbefehl handelt, eine erneute Routenplanung, unter Berücksichtigung der bereits in der Route befindlichen (Anfrage) Zwischenstationen. Hierzu wird die private Methode `makeRoute` aufgerufen, die eine aus Kommunikationssicht rekursive Planung der Transportwege von Agent zu Agent durchführt. Sie wird gleich im Anschluss näher erläutert. War der Aufruf der Methode erfolgreich (Ergebnis ungleich `null`) und wurde mindestens eine Zwischenstation neben dem eigenen Agenten gefunden, dann kann fortgefahren werden, ansonsten wird dem Anfragenden Agenten eine Ablehnungsinformation (`REFUSE`) zugestellt und die Methode verlassen. Die Zwischenstation kann auch der eigene Agent sein, z. B. falls der eigene Agent das Ende der Route markiert (Feld `destinationAgent`). War die Anfrage hingegen erfolgreich, so ist die Variable `route` nun eine Referenz auf ein `Transport`-Objekt, das u.a. noch einmal alle Anfrageinformationen enthält und zudem im Feld `offer` das günstigste Angebot aller direkten Nachbarn. Das Feld `handoverLink` enthält nun den Bezeichner des globalen Links, über den der Agent mit dem günstigsten Angebot zu erreichen ist. Da Links stets nur genau zwei Agenten verbinden, ist diese Information somit eindeutig. Es erfolgt eine Fallunterscheidung danach, ob es sich lediglich um eine Transportanfrage oder um einen Transportbefehl handelt. Ist es eine Anfrage, so wird ein Antworttelegramm erzeugt und an den in der Rekursion nächsthöheren Agenten adressiert, also an denjenigen, der auch die zur Zeit bearbeitete Anfrage geschickt hat. Sie enthält das

Transport-Objekt inklusive der Angebotsdaten, die von der Methode `makeRoute` ermittelt wurden. Handelt es sich jedoch um einen Transportbefehl, so wird die Liste der Zwischenstationen im Transport-Objekt `route` näher untersucht. Ist nur eine einzige Zwischenstation verzeichnet, so ist der eigene Agent der letzte im Weg des Transports. Fehlt nun eine Angabe des Zielhaltepunkts, der durch die Methode `makeRoute` hätte eingesetzt werden sollen (ein beliebiger freier Haltepunkt), dann muss der Transport leider mit einem `FAILURE`-Telegramm an den Anfragenden abgebrochen werden, falls der eigene Agent keine Lagereinrichtungen besitzt (es ist kein `StorageFace` konfiguriert). Ansonsten wird versucht, durch Aufruf der Methode `store` das noch zu empfangende Gut über interne Wege zu einem Lager-Übergabepunkt zu befördern und dann die Einlagerung durchzuführen. Schlägt dies fehl, wird ebenfalls ein `FAILURE`-Telegramm versandt, ansonsten ein `INFORM`-Telegramm. An dieser Stelle ist die Integration des Lagers in das LK<sup>3</sup>S-Agentensystem besonders hoch, da eine Abhängigkeit im `TransportFace` besteht, die nicht durch reine Konfiguration herbeigeführt werden kann. Wurde hingegen von der Methode `makeRoute` ein Zielhaltepunkt eingesetzt – dies wird bei nicht vorhandenem `StorageFace` zumindest versucht –, dann wird ein interner Transport zu eben diesem Zielhaltepunkt über die Methode `TransportManager.moveItemFromTo` durchgeführt. Anschließend wird Ursprungsagent über den ordnungsgemäß abgeschlossenen Transport informiert und der Algorithmus damit beendet.

**makeRoute** Die Methode `makeRoute` (s. Abb. 3.10) erstellt zunächst eine Kopie des Transport-Objekts, das sie als Argument übergeben bekommen hat, um später direkt Anpassungen darin vornehmen zu können und es dann als Ergebnis zurückzugeben. Im Anschluss wird der Liste der Zwischenstationen im Ergebnis der eigene Agent angefügt, da dieser unabhängig der weiteren Schritte am Weg beteiligt sein wird, falls das Transportziel über diesen Agenten zu erreichen ist. Auf Grund von Schwierigkeiten bei der Verwendung von Listen in Ontologien (s. Abs. 3.3.1) sind alle Zwischenstationen in einem String kodiert, getrennt durch das Pipe-Symbol. Nun folgt eine Fallunterscheidung, ob der eigene Agent dem Zielagenten der Transportanfrage entspricht, oder nicht. Ist dies der Fall, wird geprüft, ob ein Zielhaltepunkt angegeben wurde. Wenn nicht, erfolgt eine Prüfung, ob überhaupt ein beliebiger Haltepunkt innerhalb der Transportwege des Agenten frei ist. Wenn nicht, bricht die Methode `makeRoute` mit dem Rückgabewert `null` ab. War hingegen ein Zielhaltepunkt angegeben, so wird dieser zunächst als Wert der lokalen Variablen `stop` übernommen und es wird geprüft, ob der Haltepunkt noch nicht belegt ist. Ist er belegt, bricht die Methode mit dem Rückgabewert `null` ab. Ansonsten wird der interne Transport des Guts vom Übernahmeort aus dem Transport-Argument und `stop` geprüft und die Fahrkosten in die lokale Variable `internalCosts` geschrieben. Wurde hingegen gar kein Zielhaltepunkt in der Transportanfrage angegeben, so wird versucht, einen freien Haltepunkt im Transportsystem des Agenten über die Methode `findFreeReachableStop` zu ermitteln. Dabei werden alle noch nicht reservierten Haltepunkte durchlaufen und die Erreichbarkeit, ausgehend vom Übernahmeort des Guts, geprüft. Die Fahrkosten werden auch in einer lokalen Variable `internalCosts` gespeichert und der ermittelte Haltepunkt in der Ergebnisvariablen der Methode als Zielhal-

tepunkt `destinationStop` festgehalten. Wurde kein erreichbarer Haltepunkt gefunden, terminiert die Methode `makeRoute` mit dem Rückgabewert `null`. War ein erreichbarer Zielhaltepunkt angegeben oder konnte ein solcher ermittelt werden, so wird die Ergebnisvariable mit den Fahrkosten und das Feld der Zwischenstationen mit dem zu Beginn aktualisierten Wert belegt und die Ausführung der Methode mit diesem Ergebnis beendet. Im anderen Fall, dass dieser Agent nur eine Zwischenstation auf dem Weg zum Zielagenten darstellt, werden alle ausgehenden Links des Agenten geprüft. Um Schleifen bei der Wegfindung zu vermeiden, wird als erstes sichergestellt, ob der über den gewählten Link verbundene Agent noch nicht in der mitgeführten Liste der Zwischenstation aufgeführt ist. Mit Hilfe der privaten Methode `TransportManager.moveItemFromTo` werden anschließend die Kosten zum Transport des Guts über die internen Transportwege zum gewählten Link berechnet und wie oben in einer Variablen `internalCosts` gespeichert. Im Anschluss daran wird eine Transportanforderung nach gleichem Schema wie die empfangene an den jeweils verbundenen Agenten gesendet. Ist nach einem Timeout keine Antwort oder eine negative Antwort auf die Anfrage eingegangen, so wird der Link übersprungen und die Wegfindung beim nächsten Link fortgesetzt. Gibt es für einen Link schließlich eine positive Antwort (PROPOSE) des verbundenen Agenten – es gibt also einen Weg zum Zielagenten/-haltepunkt –, dann werden die in der Antwort enthaltenen Kosteninformationen mit denen des internen Transport aufaddiert und einzeln über ein Objekt der Klasse `EfficiencyInformation` gemeinsam mit dem Transport-Objekt der Antwort in einen `SimpleEfficiencyDecisionMaker` gegeben. Nach Prüfung aller Links wird über die Methode `decide` des `SimpleEfficiencyDecisionMaker` das günstigste Angebot nach Zeit und Energie bestimmt und die Methode `makeRoute` mit dem zugehörigen Transport-Objekt als Ergebnis beendet.

**findFreeReachableStop** Die Methode `findFreeReachableStop` sucht einen beliebigen freien Haltepunkt im internen Transportsystem eines Agenten. Dazu werden alle nicht gesperrten Haltepunkte durchlaufen und diejenigen näher betrachtet, die dabei zugleich keine Links sind, um die Zu- und Abwege der verbundenen Agenten nicht zu versperren. Für einen potentiellen Haltepunkt wird sodann eine Transport-Anfrage über den eigenen Path Operator gestartet, um die Erreichbarkeit des Haltepunkts zu prüfen. Sobald ein erreichbarer Haltepunkt gefunden wurde, wird die Methode `findFreeReachableStop` beendet und der Bezeichner des Haltepunkts als Rückgabewert ausgegeben. An dieser Stelle besteht noch Optimierungsbedarf, um die Haltepunkte so zu wählen, dass sie die internen Transportwege eines Agenten nicht für weitere Transporte oder Materiallagerungen versperren. Im Rahmen der Digitalen Fabrik der Hochschule Emden/Leer besteht jedoch kein Problem, wenn für jeden Arbeitsplatz nur jeweils ein Haltepunkt je zu-/abführendem Link genutzt wird und diese Haltepunkte intern nicht verbunden werden.

Entscheidend ist, dass für das Versenden jeder Anfrage und jedes Befehls an einen Nachbaragenten eine neue Conversation-ID gesetzt wird, da an unterschiedlichen Programmstellen auf Antworttelegramme gewartet wird, was schließlich dazu führt, dass Telegram-

me an falschen Programmstellen angenommen würden. Da die Conversation-ID jedoch stets beim Empfang als Filter dient, ist eine eindeutige Zuordnung hierüber möglich.

## **OrdersFace**

**Beschreibung** Wickelt Materialbestellungen und Bauaufträge ab.

**Konfiguration** Spezifische Steuerungsprogramme

**Abhängigkeiten** LinksFace, TransportFace, LinksFace (, StorageFace)

Die Architektur des OrdersFace ähnelt der des Operator Skeletons. Das Gesicht besitzt Funktionalität, die allen auftragsverarbeitenden Agenten gemein ist. Dies umfasst die prioritätsbasierte Annahme von Aufträgen nach VIRKUS [30] und eine Ablaufsteuerung in einem zyklischen Verfahren. Für jeden Arbeitsplatz und jeden auftragsgesteuerten Agenten müssen ein oder mehrere sog. Order Handler implementiert werden. Diese haben auf dem Java-Interface IOrderHandler zu basieren. Damit ist ein Satz Methoden vorgeschrieben, die von der zyklischen Auftragssteuerung genutzt werden:

**init** Initialisiert einen Order Handler. Das LK3SAgentFaceConfig-Objekt des OrdersFace wird als Argument erwartet, über das Zugriff auf den Agenten und damit auch auf alle anderen Agentengesichter besteht sowie ein Dateiname, der bei Bedarf als Verweis auf eine Konfigurationsdatei dient. Da init bei der Initialisierung des OrdersFace aufgerufen wird, empfiehlt es sich hier auch, Fähigkeiten (capabilities) beim Agenten anzumelden, falls diese durch den Order Handler hinzukommen. Alle angemeldeten Fähigkeiten werden nach Abschluss der Agentengesichter-Initialisierung gemeinsam beim für den Agenten zuständigen Directory Facilitator bekanntgegeben, sodass andere Agenten nach Agenten mit diesen Fähigkeiten suchen können. Auch sollte hier die Auftragsart in das Feld requirements des OrdersFace eingetragen werden, um eine Zuordnung einer bestimmten Auftragsart zu diesem Order Handler zu hinterlegen.

**isAcceptingOrders** Diese Methode wird aufgerufen, um zu prüfen, ob ein als Argument übergebener Auftrag von diesem Order Handler zum aktuellen Zeitpunkt ausgeführt werden kann.

**isAcceptingMultipleOrders** Nicht jeder Agent kontrolliert Hardware in der Fabrikhalle oder die Hardware ist in der Lage mehrere Aufträge gleichzeitig auszuführen. Mit dieser Methode wird abgefragt, ob dies möglich ist, falls das OrdersFace einen Auftrag vergeben will, aber feststellt, dass der Order Handler bereits einen Auftrag verarbeitet.

**isExecuting** gibt zurück, ob ein als Argument übergebener Auftrag gerade ausgeführt wird.

**executeOrder** Hierüber wird ein als Argument übergebener Auftrag zur sofortigen Ausführung gebracht, falls dies möglich ist. Jedoch soll über die Implementation dieser Methode lediglich die Ausführung vorbereitet werden.

**nextStep** Es wird angenommen, dass ein übermittelter Auftrag in den meisten Fällen in mehrere Schritte untergliedert werden kann. Dies ist z. B. beim Bau von LEGO-Duplo-Häusern der Fall, wobei jedes Setzen eines Steins einen Schritt darstellt. Diese Methode wird zyklisch vom `OrdersFace` für jeden laufenden Auftrag aufgerufen.

**isDone** prüft, ob ein als Argument übergebener Auftrag abgeschlossen ist. In diesem Fall würde er vom `OrdersFace` aus der aktuellen Auftragsliste entfernt werden und es würde die Methode `finished` des Order Handlers aufgerufen.

**finished** veranlasst den Abschluss eines als Argument übergebenen Auftrags. Dies kann z. B. das automatisierte Aufräumen des Arbeitsplatzes oder eine Bewegung in den Ausgangszustand sein.

Jeder Auftrag besteht aus einem Bauplan. Dieser kann für eine Materialbeschaffung, aber beispielsweise auch für einen Fertigungsauftrag stehen. Jeder Bauplan besitzt eine Angabe, welche Einrichtung benötigt wird, um ihn umzusetzen. Diese Angabe findet sich im Feld `require` des Bauplans:

```
<?xml version="1.0" encoding="UTF-8"?>
<blueprint require="assembly" ..."
```

Trifft ein Auftrag von einem ERP-System oder einem anderen Agenten z. B. zur Materialbestellung ein, kann somit umgehend der für diesen Bauplan zuständige Order Handler gewählt und das erwähnte zyklische Verfahren zur Abarbeitung des Bauplans durchgeführt werden.

Die Order Handler werden über die Konfigurationsdatei des `OrdersFace` eingerichtet, wie in Listing 3.9 gezeigt. Dabei ist der vollständige Klassenname des Order Handlers und optional der Name einer zugehörigen Konfigurationsdatei anzugeben.

Da sich das `OrdersFace` zum Zeitpunkt dieser Ausführungen noch intensiv in der Entwicklung befindet, soll es an dieser Stelle nicht näher erläutert werden.

Listing 3.9: Beispielhafte Konfiguration des `OrdersFaces` eines Arbeitsplatzes.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <orders-config>
3   <extra-handlers>
4     <handler class="de.i2ar.agents.faces.orders.
        OrderHandlerMaterialOnly" config="" />
5   </extra-handlers>
6 </orders-config>
```

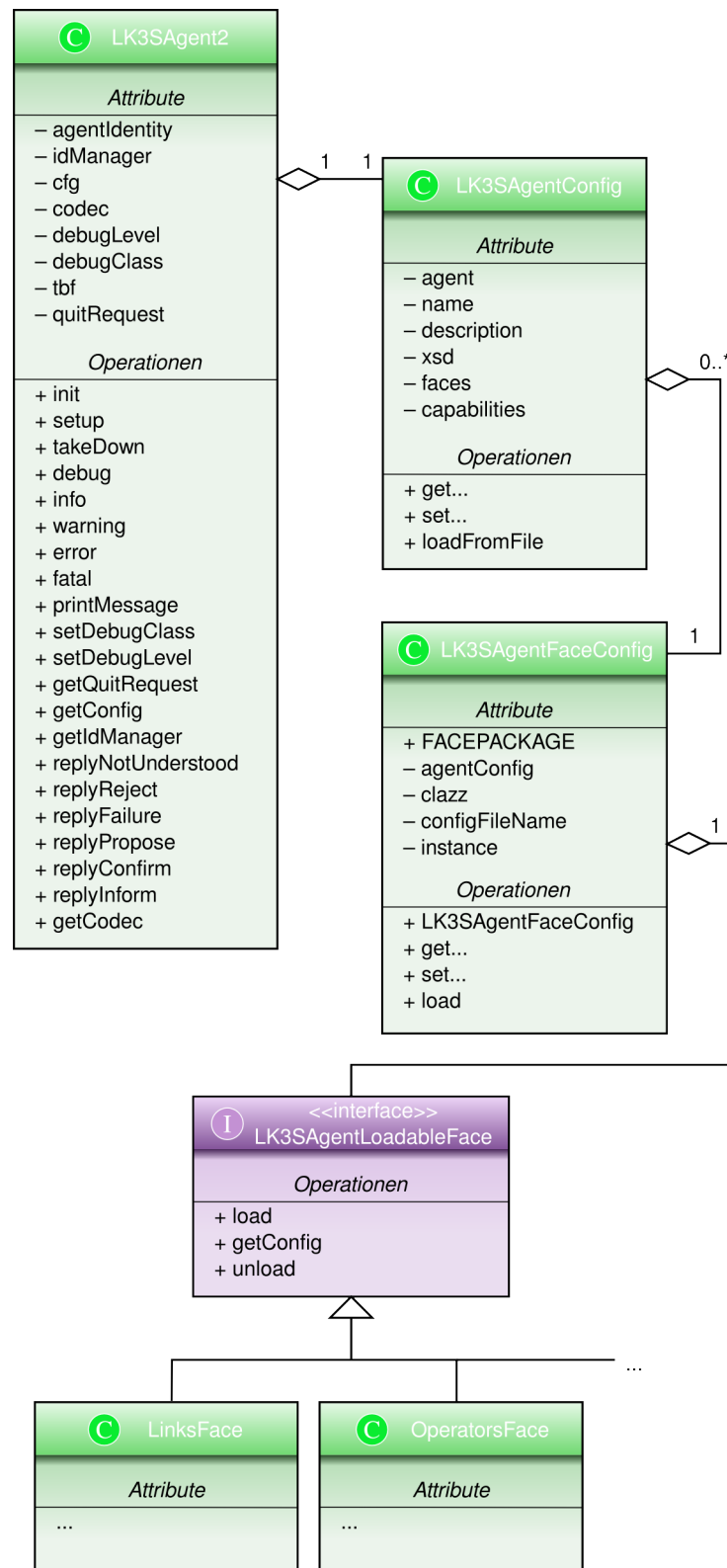


Abbildung 3.7: UML-Klassendiagramm der Agentenkonfiguration.



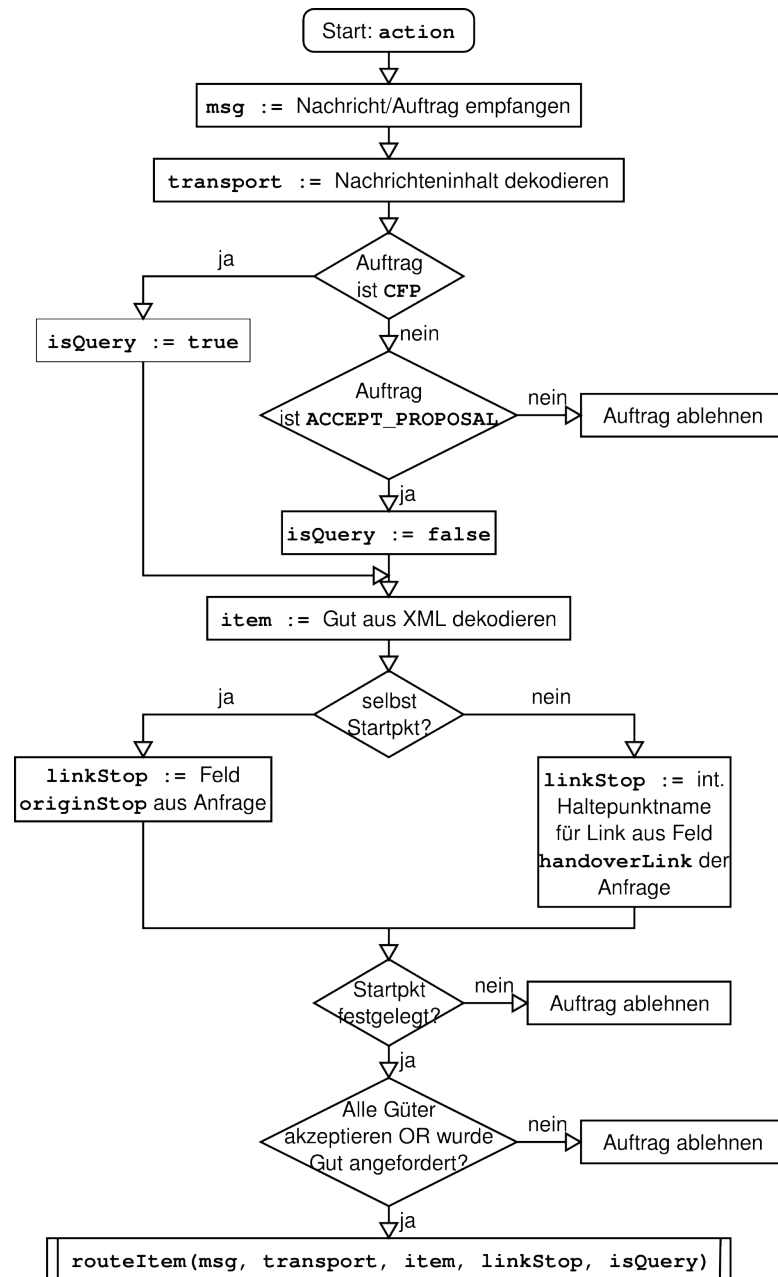


Abbildung 3.8: Vereinfachter Programmablaufplan der Methode `action` der Klasse `TransportFace`.

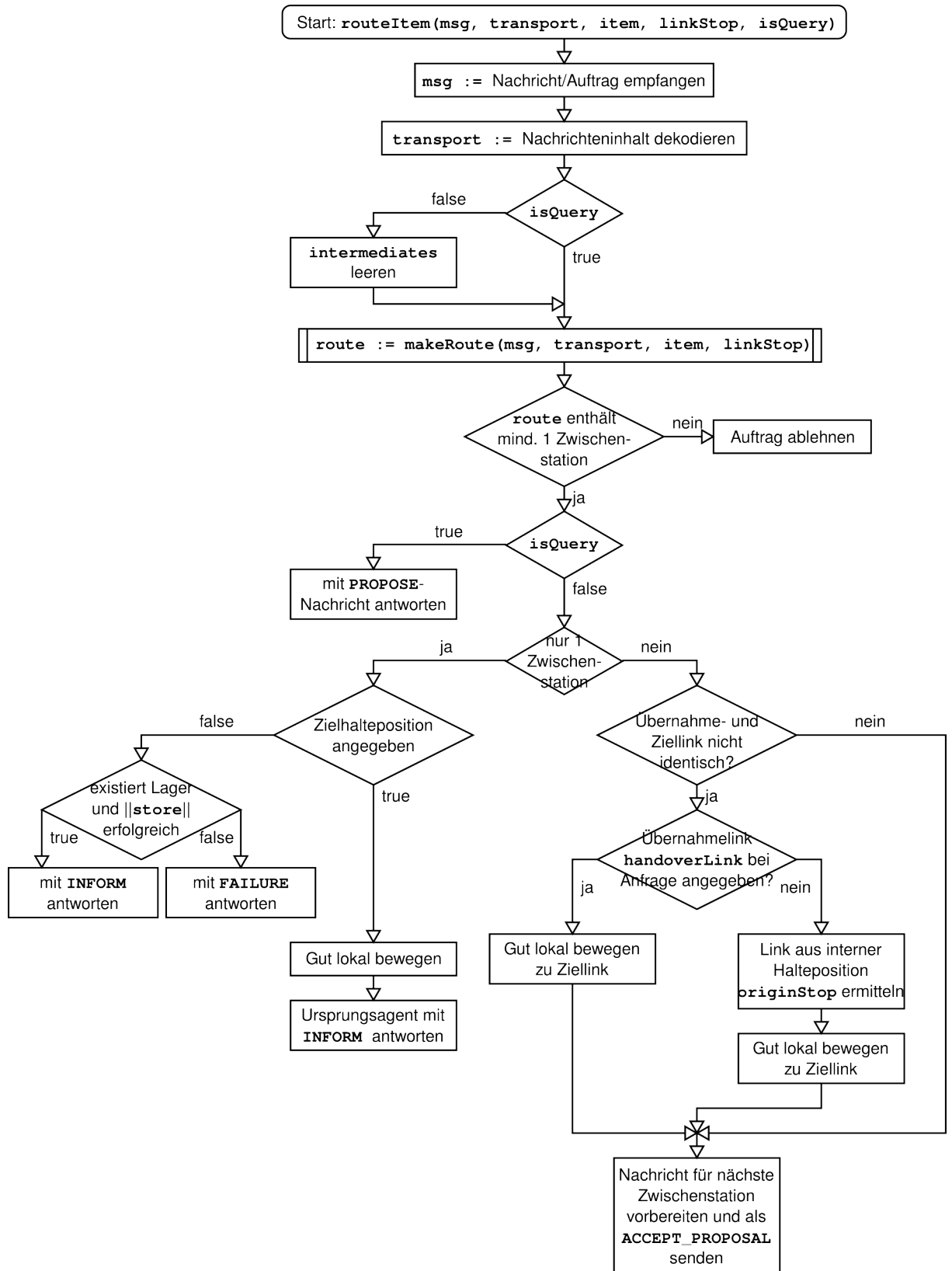


Abbildung 3.9: Vereinfachter Programmablaufplan der Methode `routeItem` der Klasse `TransportFace`.

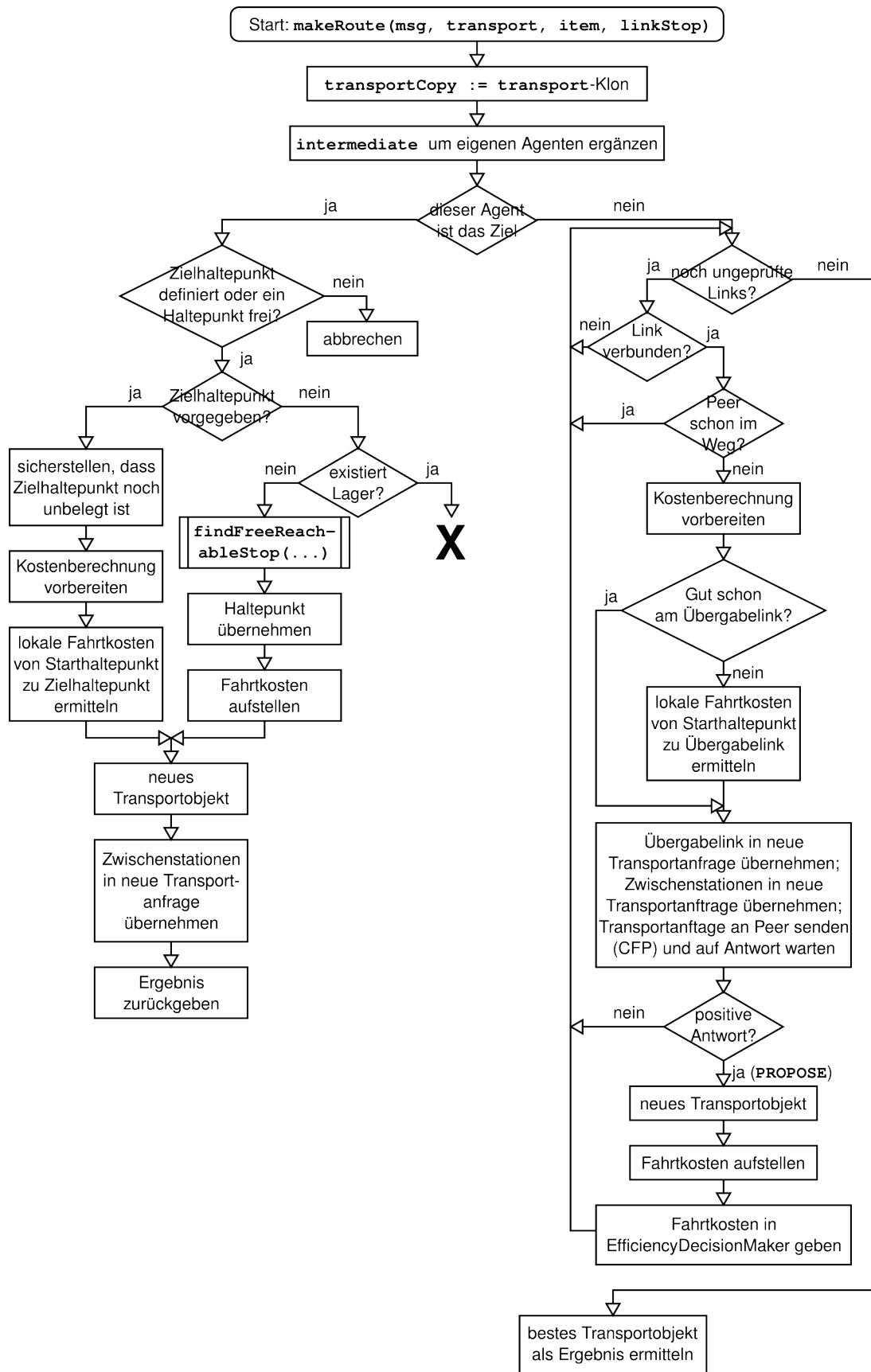


Abbildung 3.10: Vereinfachter Programmablaufplan der Methode `makeRoute` der Klasse `TransportFace`.

## 4 Optimierungen

Bei der Optimierung von Hochregallagern können zwei unterschiedliche Aspekte angesprochen werden. Ein Aspekt bezieht sich auf die Dimensionierung des Lagers. So schreibt GLASS (s. [12]), dass es stets das Ziel sein sollte, Lagerkapazitäten zu minimieren. Dies führt nicht nur zu geringeren Kosten für Bau und Betrieb des Lagers, sondern auch automatisch zu einer höheren Umschlagleistung auf Grund geringerer maximaler Fahrwege bei Ein- und Auslagerungen. Jedoch kann die Lagerkapazität nicht beliebig klein gehalten werden. Sie ist auch abhängig von der erwarteten Auftrags- und Bestandslage. Sind alle zu lagernden Güter nicht positiv korreliert, so werden niedrigere Bestände eines Guts die höheren Bestände eines anderen ausgleichen. GLASS nennt den Spielzeuggroßhandel als Beispiel, wo zur Jahresmitte Bestände für das Weihnachtsgeschäft aufgebaut werden. Hierneben ist auch die Lagerstrategie entscheidend. Es wird in der Regel zwischen fester und chaotischer bzw. dynamischer Lagerplatzvergabe unterschieden. Während bei ersterer Variante definierte Bereiche des Lagers für genau eine Klasse von Gut reserviert und von keiner anderen genutzt werden darf, so existiert diese Restriktion bei der dynamischen Lagerplatzvergabe nicht. Dafür bedarf es anderer Strategien, um alle Güter entsprechend ihrer Relevanz hinsichtlich einer oder mehrerer Faktoren im Lager zu positionieren. Eine Minimierung der Lagerkapazität bei Einsatz der festen Lagerplatzvergabe ist selbstredend schwieriger möglich, als stünden alle Lagerplätze zur allgemeinen Verfügung. In diesem Zusammenhang ist auch die Strategie der Anbruchlagereinheiten von Interesse. Dabei werden diejenigen Lagereinheiten, bzw. Güter, die aus mehreren eigenständigen Gütern bestehen, bevorzugt zur Auslagerung adressiert, sobald die Lagereinheit nicht mehr in ihrem vollständigen Ursprungszustand ist. Dieses Vorgehen kann jedoch auch mit anderen Lagerstrategien wie der FIFO-Strategie im Konflikt stehen. Eine Priorisierung oder Gewichtung der einzelnen Optimierungsansätze ist daher vonnöten.

Ist die Dimensionierung und Konfiguration eines Lagers erst einmal abgeschlossen, so können andere Strategien greifen. Sie beschäftigen sich nicht mit der statischen Einteilung des Lagers in Bereiche oder dessen Dimensionierung, sondern mit Verfahren, die je nach aktueller Konfiguration, Auslastung und den betroffenen Gütern auf Anfrage nach möglichst optimalen Lagerplätzen suchen. Hierüber wird die sog. Grenzleistung des Lagers optimiert. Sie wird bei maximaler Umschlagleistung erreicht und zur Umsetzung kommen sog. Schnellläuferstrategien zum Einsatz. Abstrakt betrachtet, beschreibt eine Schnellläuferstrategie die Nutzung der vorhandenen Mittel in einer Art und Weise sowie eine Kombination untereinander, sodass sich die Umschlagleistung vergrößert.

Wie in VIRKUS [28] beschränkt sich die Lageroptimierung auf die dynamische Lagerplatzvergabe. Im Folgenden wird zunächst mit einer Analyse begonnen, welche Parame-

ter zur Hochregallageroptimierung zur Verfügung stehen und sinnvoll sind. Ausgehend von der Analyse wird ein universelles Optimierungsverfahren entwickelt, das analytisch und mathematisch beschrieben wird.

### 4.1 Analyse

Mit der Hochregallageroptimierung soll im Wesentlichen die Umschlagleistung erhöht werden. Hierzu ist zunächst zu analysieren, welche Mittel zur Umsetzung dieses Ziels herangezogen werden können. Erst im Folgenden werden Optimierungsverfahren entwickelt, die sich der Eigenschaften der Mittel zur Umsetzung bewusst sind und diese sinnvoll einsetzen.

Für Hochregallager charakteristisch sind die vorhandenen Komponenten:

1. Regale,
2. Regalbediengeräte,
3. Übergabepunkte zwischen RBG,
4. Vorzone und
5. Lagerverwaltungssystem

Diese Arbeit befasst sich mit der Entwicklung eines LVS und der Optimierung seiner Funktion, nicht jedoch mit Fragen bezüglich der Dimensionierung des Lagers. Damit sind Diskussionen bezüglich der Regale nicht Bestandteil dieser Arbeit. Die Dimensionierung der RBG sind es ebenfalls nicht, jedoch wird ihr Gebrauch optimiert. Auch die Übergabepunkte der RBG bedürfen einer Berücksichtigung in den entwickelten Algorithmen. Die Vorlagerzone hingegen bleibt unberücksichtigt.

Der Raum zur Einlagerung von Gütern erstreckt sich mit den Regalen in drei Raumdimensionen (s. Abb. 6.3). Damit ist die erste Größe bekannt. Sie kann beispielsweise in Metern angegeben werden. Eine Einheit soll an dieser Stelle nicht vorgeschrieben werden, es gilt jedoch zu beachten, dass bei der Parametrierung aller Komponenten die gleichen Einheiten einzusetzen sind. Angaben beziehen sich auf die Positionen der einzelnen Lagerzellen innerhalb des Raums und im Detail beispielsweise auf die Lagerzellenmitte. Dabei werden nach GLASS [12] alle Lagerzellen im Gegensatz zu ihrer diskreten Position im Raum über eine Abbildungsfunktion als kontinuierlich betrachtet. Die Abbildungsfunktion wird jedoch nur mit diskreten ganzzahligen Werten, den entsprechenden Indizes zur Adressierung der Lagerplätze, berechnet. Diese Werte sind nach Projektierung, Einrichtung und Vermessung des Lagers statisch und zählen zu den Lagerstammdaten. Sie sind für die RBG von Bedeutung, um jede einzelne Lagerzelle korrekt anfahren zu können. Die Rückwärtsabbildung der Indizes auf reale Raumpositionen übernimmt eine hardwarenahe Schicht.

Nicht statisch hingegen sind Größen, die von nicht-deterministischen Vorgängen abhängig sind. Hierzu zählt die Distanz zwei nacheinander durch ein RBG anzufahrende Positionen im Raum, denn beide Positionen ergeben sich erst im Laufe des Betriebs und sind abhängig von aktueller Lagerbelegung und aufgetretenen externen Anforderungen.

Für das Zurücklegen der Distanz zwischen zwei Positionen benötigt das RBG eine Zeitdauer. Diese Größe allein kann noch nicht optimiert werden, da bei gleichbleibenden Umgebungsbedingungen stets die gleiche Fahrzeit erreicht wird, also eine statische Matrix der Fahrzeiten zwischen zwei Positionen erstellt werden kann. Erst die Aneinanderreihung von mehr als zwei Positionen bietet Optimierungsspielraum über der Reihenfolge der Positionen. Abb. 4.1 zeigt zwei unterschiedliche Reihenfolgen und welche Auswirkungen sie auf die erzielte zurückzulegende Gesamtdistanz beim Anfahren aller Positionen haben. Die Distanzen wurden anhand der Diagonalen rechtwinkliger Dreiecke (nach Pythagoras) bestimmt und je Reihenfolge aufsummiert. Als Grundlage wird die Distanz 1 beim Übergang einer Lagerzelle zu ihrem horizontalen oder vertikalen Nachbarn genutzt.

Die Fahrzeit zwischen zwei Lagerzellen ist abhängig von deren Distanz, der Beschleunigungs- und Bremsrampe des eingesetzten Antriebssystems (siehe SCHUMANN [23]) sowie der Höchstgeschwindigkeit  $v_{max}$  des RBG. Zur Vereinfachung werden die Beschleunigungen bei allen folgenden Betrachtungen nicht weiter berücksichtigt und seien hiermit nur aus Gründen der Vollständigkeit erwähnt. Es wird für die Geschwindigkeit ein konstantes  $v = v_{max}$  angenommen. Die Fahrzeit  $t$  für eine Distanz  $d$  ergibt sich nach

$$t = \frac{d}{v}, \quad t = \left[ \frac{\text{m}}{\frac{\text{m}}{\text{s}}} \right] = [\text{s}].$$

Eine weitere Größe, die sich direkt im Zusammenhang mit den eingesetzten Antriebssystemen und den zurückzulegenden Distanzen ableiten lässt, ist die kinetische Energie, die bei Fahrten aufgewendet werden muss:

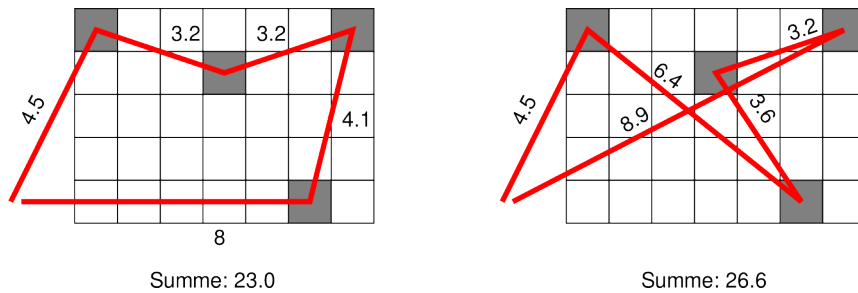
$$W = \frac{1}{2}mv^2.$$

Es geht die Masse  $m$  des zu bewegenden Guts zuzüglich der bewegten Teile des RBG in die Rechnung ein, wie auch die Fahrgeschwindigkeit. Wird  $W$  zusätzlich multiplikativ mit  $t$  beaufschlagt, so ergibt sich die für die Fahrt um eine Distanz  $d$  aufgewendete kinetische Energie. Real ist jedoch auch der Wirkungsgrad der Antriebe und dessen Beschleunigungs-/Bremsrampe heranzuziehen. Hierauf wird jedoch abermals verzichtet.

Es können nach obigen Ausführungen somit zwei Ziele verfolgt werden:

1. Fahrzeiten minimieren durch:

- Verringerung der Distanzen einer Reihenfolge anzufahrender Positionen
- Maximierung der Fahrgeschwindigkeit



Optimierungsverfahren eine Bewertung – im Folgenden auch Metrik genannt – im Intervall  $[0, 1]$  berechnet. Wie diese Bewertungen zu einer verfahrensübergreifenden, optimalen Lagerplatzbestimmung verknüpft werden, ist in Abschnitt 4.2.9 dargelegt. Erläuterungen zu der Implementation der Verfahren werden in den folgenden Kapiteln gegeben.

### 4.2.1 Einlagerung – Äußere Randbedingungen

Unter den äußeren Randbedingungen werden die physikalischen Eigenschaften der Güter und der Lagerstrukturen verstanden, die bei der Einlagerung von Gütern berücksichtigt werden müssen. Hierzu zählen u.a.

- Breite,
- Höhe,
- Tiefe,
- Gewicht bzw. maximale Gewichtsbelastung,
- Maximaltemperatur bzw. garantierte Temperatur sowie
- weitere Eigenschaften z. B. bei der Lagerung von Gefahrgut

der Güter bzw. Lagerplätze. Dabei sind die Ausmaße der Güter Mindestwerte für die Ausmaße der Lagerplätze, welche jedoch die der Güter auch übersteigen dürfen, sodass weiterhin eine Lagerung ermöglicht wird. Ebenso verhält es sich mit dem Gewicht – die maximale Belastbarkeit der Lagerplätze darf das Gewicht des einzulagernden Guts übersteigen. Nicht jedoch die Temperatur. Hier kann für ein Gut eine maximale Lagertemperatur definiert sein, die aus Gründen der Qualitätssicherung nicht überschritten werden darf (häufig bei Lebensmitteln). In diesem Fall darf das Lager diesen Wert im gewissen Rahmen unterschreiten, ohne dabei das Gut negativ zu beeinträchtigen und die Lagerung somit unmöglich zu machen.

Bei der in dieser Arbeit beschriebenen Implementation wird sich auf die Eigenschaften Breite, Höhe, Tiefe und Gewicht beschränkt, da diese Eigenschaften auch im Lager der Digitalen Fabrik abgebildet sind. Ziel dieses Optimierungsverfahrens ist es, einen Lagerplatz für die Einlagerung eines Guts zu finden, der mindestens den Größen des Guts entspricht, jedoch möglichst knapp.

$$\text{Gut } \vec{G} = \begin{pmatrix} \text{Breite} \\ \text{Höhe} \\ \text{Tiefe} \\ \text{Gewicht} \end{pmatrix}, \quad \text{Lagerplatz } \vec{L} = \begin{pmatrix} \text{Breite} \\ \text{Höhe} \\ \text{Tiefe} \\ \text{Belastbarkeit} \end{pmatrix}$$

Die Menge  $\mathcal{L}$  beschreibt alle der  $p$  Lagerplätze  $L$ , die für eine Einlagerung unter obiger Beschränkung geeignet sind:



$$\mathcal{L}(G) = \{L_s | s = 1, \dots, p \wedge G_i \leq L_i\}$$

Zum Erzielen einer optimalen Ausnutzung der geometrischen Lagergegebenheiten ohne Verschwendung von Lagerraum ist unter allen der Bedingung genügenden Lagerplätzen derjenige zu wählen, der die geringsten Größen aufweist und somit möglichst nah an der Größe des einzulagernden Guts liegt:  $\min(\mathcal{L})$ .

Um eine Einstufung in Abhängigkeit der spezifischen Parameter des einzulagernden Guts und der des betrachteten Lagerplatzes durchführen zu können, ohne großen Aufwand der Datenbanksuche zu betreiben, werden vier diskrete Klassen gebildet, in die jede potentielle Lagerzelle  $L$  eingestuft wird. Dabei werden die vier Attribute getrennt betrachtet, sodass eine Einstufung auch anhand eines einzelnen Kriteriums erfolgt, wenn dieses besonders heraussticht.

1. jedes Lagerzellenattribut besitzt weniger als oder genau das  $1\frac{1}{2}$ -fache der geforderten Größe
2. ein Lagerzellenattribut besitzt mehr als  $1\frac{1}{2}$ -fache der geforderten Größe
3. ein Lagerzellenattribut besitzt mehr als 2-fache der geforderten Größe
4. ein Lagerzellenattribut besitzt mehr als  $2\frac{1}{2}$ -fache der geforderten Größe
5. ein Lagerzellenattribut besitzt mehr als 3-fache der geforderten Größe

Da mit zunehmender Überdimensionierung des Lagerplatzes dessen Sinnhaftigkeit für das einzulagernde Gut abnimmt, ist bei steigender Überdimensionierung eine höhere Metrik anzusetzen. Nach obigem Schema erhalten diejenigen Lagerplätze eine Metrikzuschlag von 0.25, die mindestens das  $1\frac{1}{2}$ -fache der geforderten Größe in einem der vier geprüften Attribute aufweisen. Für jeden weiteren Fall (3. bis 5.) wird ein weiterer Zuschlag von 0.25 angesetzt. Die Fälle werden also geprüft und die Zuschläge addiert. Unterschreiten die Größen aller Attribute des Lagerplatzes die korrespondierende  $1\frac{1}{2}$ -fache Größe des einzulagernden Guts, dann wird die minimale Metrik 0 angesetzt und der Lagerplatz ist damit optimal. Damit ist eine Metrik im Intervall  $[0, 1]$  zu erreichen.

Während einige Optimierungskriterien wünschenswert, eine Einlagerung jedoch auch ohne diese durchführbar sind, sind die äußeren Randbedingungen ein Ausschlusskriterium, wenn Lager und Güter nicht beschädigt werden sollen. Daher werden die hierüber ermittelten, potentiellen Lagerplätze auch des Öfteren herangezogen, wenn es darum geht, potentielle Lagerplätze für weitere Optimierungen zu ermitteln.

Für eine exakte Anpassung der vier Grenzen zur Kategorisierung der geometrischen Eigenschaften eines Lagerplatzes in Bezug auf ein Gut ist eine Untersuchung der verschiedenen Lagerplätze und die Betrachtung aller zukünftig einzulagernden Güter nötig. Hieraus können sich sinnvolle Grenzwerte ergeben. So können bei Gütern mit einer Standardhöhe von 20 cm und einer zusätzlichen Höhe, die im Bereich 0 – 1 cm variiert, Grenzen im einstelligen Prozentbereich verwendet werden, während eine Variation im Bereich

0 – 100 cm andere Grenzen erfordert. Neben der manuellen Analyse könnte natürlich auch eine Selbstoptimierung zum Ziel führen und die Grenzen dynamisch anpassen. Für das im Rahmen dieser Arbeit entwickelte Verfahren werden jedoch feste Grenzen angenommen. Folgendes Beispiel zeigt die Metrikberechnung anhand der Situation in der Digitalen Fabrik.

Beispiel:

Es gilt, einen Lagerplatz für ein 14 cm hohes Gut zu finden. Es existieren Lagerplätze in den drei Höhenklassen 15 cm, 20 cm und 32 cm. Alle anderen Attribute werden vernachlässigt.

Für alle Lagerplätze mit 15 cm Höhe ergibt sich somit eine Metrik von 0, da 15 cm kleiner gleich  $14 \text{ cm} \times 1\frac{1}{2} = 21$  ist. Ebenso verhält es sich mit allen Lagerplätzen mit 20 cm Höhe, also mit  $20 \leq 21$ . Erst die Lagerplätze mit 32 cm Höhe sind mehr als  $1\frac{1}{2}$  mal so hoch, wie das einzulagernde Gut. Da sie auch höher sind als das 2-fache des einzulagernden Guts (28 cm), jedoch niedriger als das  $2\frac{1}{2}$ -fache, erhalten sie eine Metrik von 0.5.

#### **4.2.2 Einlagerung – Abfragehäufigkeit vs. Verweildauer**

GLASS [12] rät von der Verwendung der Abfragehäufigkeit zum Vorteil der Verweildauer einzelner Güter im Lager ab. Dies liegt darin begründet, dass die Verweildauer die Abfragehäufigkeit implizit mit abdeckt. So ist die durchschnittliche Lagerdauer eines im Lager vorhandenen Guts einer Sorte, ausgehend von einem Zeitpunkt  $t_0$ , geringer, wenn viele davon abgefragt werden, als die durchschnittliche Lagerdauer von Gütern einer weniger gefragten anderen Sorte. Die Verweildauerstrategie deckt hierüber hinaus jedoch auch Fälle kurzzeitiger Zwischenlagerung ab, die von der Abrufhäufigkeitsstrategie jedoch nur erfasst werden können, wenn diese häufig stattfinden, während die Verweildauerstrategie nur die Lagerzeiten ins Verhältnis setzt. Nachteilig bei letzterer ist jedoch die Tatsache, dass sich die Lagerdauer der unterschiedlichen Güter im Mittel nicht über die Zeit ändert. Existieren nun einige wenige Schnelldreher (nur kurzzeitig eingelagerte Güter), so werden entsprechend gut gelegene Lagerplätze bevorzugt mit diesen Gütern belegt. Dies wird jedoch zum Nachteil, wenn Schnelldreher nur selten auftreten, wie es in der Digitalen Fabrik der Fall ist. Dann würden die häufig benötigten Güter schlechter positioniert, da die Lagerplatzzuordnung nur nach den Verweildauern stattfindet. Aus diesem Grund ist im Rahmen dieser Arbeit die Wahl auf die Abfragehäufigkeitsstrategie gefallen. Sollte sich herausstellen, dass die Verweildauerstrategie dennoch notwendig ist, so kann diese im Rahmen der Multi-Aspekt-Optimierung (s. Abs. 4.2.9) leicht nachträglich implementiert werden.

Das Prinzip dieser Strategie ist knapp zu beschreiben: Diejenigen Güter, die am häufigsten Abgefragt werden, erhalten die beste Positionierung im Lager. Werden mehrere Güter

gleich häufig abgefragt, erhalten sie die gleiche Positionierung im Lager. Die am seltensten abgefragten Güter erhalten automatisch die schlechteste Positionierung im Lager. Dabei werden jedoch nie Lagerplätze für entsprechende Güter reserviert. Durch dieses Verfahren findet immer nur eine Bewertung aller Lagerplätze statt, hinsichtlich Sinnhaftigkeit hinsichtlich der berechneten Abfragehäufigkeit.

Zur Berechnung wird die Abfragehäufigkeit als Statistik vergangener Abfragen gebildet. Liegt noch keine Abfrage des einzulagernden Guts vor, so wird zunächst die schlechteste Positionierung angenommen. Dies kann sich jedoch bereits bei der nächsten Einlagerung in das Gegenteil gekehrt haben, da dieses Verfahren sehr dynamisch ist. Da die Nachfrage einzelner Güter u.a. saisonaler Schwankungen unterliegt, ist es erforderlich, ein Zeitfenster für die Bestimmung der Abrufhäufigkeiten zu definieren. Hierzu kommen typischerweise Verfahren wie das gewichtete gleitende Mittel zum Einsatz, das zudem die unterschiedliche Gewichtung einzelner Zeitfenster erlaubt. Der aus der Informatik bekannten *Least-Frequently-Used-Algorithmus* (LFU) könnte ebenfalls Anwendung finden, jedoch arbeitet dieser zumindest klassischerweise mit einer begrenzten Menge zu betrachtender Ereignisse. Dies macht ihn abhängig von der Gesamtumschlagsleistung eines Lagers, was jedoch nicht beabsichtigt ist.

Neben der Abrufhäufigkeit des einzulagernden Guts werden in dieser Arbeit auch die Abrufhäufigkeiten aller anderen Güter in einem parametrierbaren Zeitfenster mit gleicher Gewichtung berechnet, sodass darunter auch die geringste und die höchste Abrufhäufigkeit ermittelt wird. Die Abrufhäufigkeit des einzulagernden Guts wird hier nun eingeordnet, indem die geringste und die häufigste ein Intervall  $[0, 1]$  bilden. Existiert ein Vektor  $\vec{A}$  der Abrufhäufigkeiten für alle bereits abgerufenen Güter, dann erfolgt die Normierung der Abrufhäufigkeiten auf das Intervall  $[0, 1]$  als  $\vec{A}'$  wie folgt:

$$a'_i = (a_i - \min(\vec{A})) \cdot \frac{1}{\max(\vec{A}) - \min(\vec{A})}$$

Im Folgenden ist eine Abbildung dieser relativen Abrufhäufigkeit  $a_i$  für das einzulagernde Gut auf die relative Entfernung im Lager, ausgehend vom Zugangspunkt der Einlagerung, notwendig. Dazu wird für alle Lagerplätze ebenfalls eine relative Entfernung zwischen dem Zugangspunkt und dem im Lager am weitesten entfernten Punkt berechnet. Für diese Arbeit wird dabei auf eine einfache Distanzberechnung mit Hilfe des Satzes des Pythagoras zurückgegriffen (s. Abb. 5.3). Dies ist jedoch nur möglich, da die Positionen aller Zugänge und Lagerplätze als kontinuierlich angesehen und über Indizes abgebildet werden und die in der konkreten Anwendungsumgebung, der Digitalen Fabrik im Technikum der Hochschule Emden/Leer, auftretenden Gewichtsbelastungen im Verhältnis zur Leistung der eingesetzten Antriebe marginal sind. Daher ist der Energieaufwand für die beiden Hauptachsen als gleichwertig anzusehen und entspricht unter Vernachlässigung der Beschleunigungen dem Vielfachen des zeitlichen Aufwands. Die absolute Distanz eines Lagerplatzes wird durch die maximale Distanz im Lager (idealisiert) dividiert, um somit die relative Position jeder einzelnen Lagerzelle zu erhalten.

Um die relative Abrufhäufigkeit und die relative Lagerplatzposition zu korrelieren, wird

die Abweichung beider Werte voneinander bestimmt. Ihr Betrag kann niemals größer als 1 sein, sodass sich wiederum Werte in einem Intervall  $[0, 1]$  ergeben, die die relative Angemessenheit einer Einlagerung eines Guts an einem Lagerplatz beschreiben. Nach Abs. 4.2.9 ist ein geringer Wert wieder optimal.

### 4.2.3 Einlagerung – Anbruchlagereinheiten

Eine weitere Optimierungsstrategie, die dem Verfahren der Anbruchlagereinheiten ähnelt (s. GLASS [12]), berücksichtigt den Belegungsstand jedes einzelnen Lagerplatzes. Dabei wird nicht auf den Anbruch einzelner Lagereinheiten eingegangen, sondern auf die Auslastung von Lagerplätzen, die eine höhere Kapazität als 1 aufweisen. Die Idee hinter dieser Strategie lautet wie folgt: Existieren Lagerplätze mit einer Kapazität  $> 1$ , die bereits teilbelegt sind, so sind diese zunächst weiter zu belegen, bevor noch gänzlich unbelegte Lagerplätze zur Einlagerung herangezogen werden. Daher ist zunächst eine Fallunterscheidung zu treffen, unter der Annahme, dass vollbelegte Lagerplätze bereits ausgeblendet worden sind:

1. Lagerplatz ist gar nicht belegt
2. Lagerplatz ist teilbelegt

Hiernach wird eine Bewertung der Lagerplätze  $\vec{L}$  nach Abs. 4.2.9 durchgeführt. Dabei wird den gar nicht belegten Lagerplätzen eine Metrik  $m$  von 1 zugewiesen, also die schlechteste Beurteilung, sodass sie automatisch bei der Auswahl eines Lagerplatzes benachteiligt werden. Allen teilbelegten Lagerplätzen wird hingegen der Belegungsgrad als Quotient ihrer belegten und ihrer maximalen Kapazität zugewiesen.

$$m(\vec{L}) = \begin{cases} 1 & \text{falls Lagerplatz } \vec{L} \text{ noch unbelegt} \\ \frac{\text{used}(\vec{L})}{\text{capacity}(\vec{L})} & \text{falls Lagerplatz } \vec{L} \text{ teilbelegt} \end{cases}$$

Dadurch ergibt sich ein Schema, nach dem schwach belegte Lagerplätze als erstes weiter aufgefüllt werden, während bereits stark belegte Lagerplätze erst später weiter bis zu ihrer kompletten Füllung belegt werden. Somit wird der Aufwand bei Auslagerungen möglichst gering gehalten, falls auf ein Gut im hinteren Bereich eines Lagerplatzes zugegriffen werden soll.

Zu beachten sind die Seiteneffekte dieser Strategie. Die Anwendung ohne die im Folgenden dargestellte Strategie für Lagerplätze mit gleichem Gut kann dazu führen, dass beliebige Güter in einem Lagerplatz gruppiert werden. Daher ist es empfehlenswert, diese Optimierungsstrategie nicht einzeln zu verwenden. Auch sollte abgewägt werden, mit welcher Gewichtung diese Strategie in die Gesamtbewertung eines Lagerplatzes eingeht.

Beispiel:

Im schlimmsten Fall könnte ein Lagerplatz mit diesem Verfahren sehr gut (z. B. mit einer Metrik von 0.1) bewertet werden, mit dem Verfahren für gleiche Güter sehr schlecht (1, da anderes Material bereits im Lagerplatz) und auf Grund der Entfernung zum Lagerzugang wieder sehr gut (z. B. mit einer Metrik von 0.1). Damit ergäbe sich eine Metrik von 1.2 für diesen Lagerplatz. Ein anderer Lagerplatz ist noch unbelegt (Metrik 1), enthält demnach auch kein Gut derselben Sorte (0), liegt jedoch geringfügig weiter vom Lagerzugang entfernt (0.3). Damit ergäbe sich eine Metrik von 1.3 für diesen Lagerplatz. Der Vorzug würde also dem zuerst beschriebenen Lagerplatz gegeben, der jedoch in der Folge einen höheren Aufwand verursachen würde, falls eines der früher eingelagerten Güter abgerufen wird.

Für die Zukunft sollte in Betracht gezogen werden, mit dieser Strategie zugleich diejenigen Lagerplätze herauszufiltern, die andere Güter enthalten, um damit eine Sortenreinheit der Lagerplätze sicherzustellen. Diese Sortenreinheit wird jedoch mit einer im Zweifelsfall geringeren maximalen Lagerkapazität erkauft.

### 4.2.4 Einlagerung – Lagerplätze mit gleichem Gut

Wie im vorigen Abschnitt bereits erwähnt, existiert auch eine Lagerstrategie, die die Einlagerung eines Guts bei mehrfach tiefen Lagerplätzen hinsichtlich gleicher Güter bewertet. Im Gegenteil zur Vorigen findet nur eine Betrachtung statt, ob Güter der gleichen Sorte wie die des einzulagernden Guts im betrachteten Lagerplatz vorliegen oder ob dieser komplett unbelegt ist. Dies führt zu einer harten Klassifizierung. Lagerplätze, die komplett frei sind oder zumindest ein Gut der gleichen Sorte besitzen, werden mit einer Metrik von 0 eingestuft, was sie attraktiv für den Einlagerungsvorgang macht. Ist hingegen keines der vorhandenen Güter von der gleichen Sorte, so wird mit der Metrik 1 signalisiert, dass dieser Lagerplatz für die anstehende Einlagerung eher ungeeignet ist.

In Kombination mit der Optimierungsstrategie der Anbruchlagereinheiten lässt sich somit eine gute Nutzung des Lagers realisieren. Gleichartige Güter werden bei mehrfach tiefer Lagerung gebündelt und viele schwach besetzte Lagerplätze vermieden.

### 4.2.5 Auslagerung – First In First Out

Ein sehr verbreitetes Verfahren zur Optimierung der Lagerhaltung ist das *First-In-First-Out*-Verfahren (FIFO), das auch aus anderen Bereichen wie z. B. der Systemprogrammierung bekannt ist. Dabei geht es darum, diejenigen Güter zuerst auszulagern, die als erstes

eingelagert wurden. Damit wird erreicht, dass die Liegedauer aller Güter im Durchschnitt gesenkt wird, was natürlich nur möglich ist, wenn auch eine Nachfrage nach allen Gütern besteht.

Allerdings ist dieses Verfahren relativ in Bezug auf die Lagerdauer. Es existieren keine Deadlines oder Ähnliches, sondern es wird lediglich die Reihenfolge der Einlagerungszeitpunkte betrachtet. Damit ist es nur begrenzt nützlich, um verderbliche Güter zu verwalten, was durch die Produktionsplanung des LK<sup>3</sup>S-Systems jedoch ohnehin nicht vorgesehen ist.

Zur Umsetzung der Optimierungsstrategie muss eine Rangfolge der Güter zur Auslagerung gebildet werden. Zur besseren Handhabung wird jedem Lagerplatz  $\vec{L}$  dazu ein Wert im Intervall  $[0, 1]$  zugewiesen, mit 0 falls er das älteste und 1 falls er das jüngste der betrachteten und für die Auslagerungsanfrage überhaupt relevanten Güter enthält. Der Wert kann dann zugleich als Metrik in das in Abs. 4.2.9 beschriebene Verfahren einfließen. Dabei wird der Wert in den Zeitbereich der frühesten und der spätesten Einlagerung des aktuellen Bestandes eingegliedert.

$$m(\vec{L}) = \frac{t_{\text{Einlagerung}}(\vec{L}) - \min(t_{\text{Einlagerung}})}{\max(t_{\text{Einlagerung}}) - \min(t_{\text{Einlagerung}})}$$

Ähnliche und hiermit verwandte Verfahren sind das *First-Expired-First-Out*-Verfahren (FEFO), das *Highest-In-First-Out*- (HIFO) und das *Last-In-First-Out*-Verfahren (LIFO). Dabei orientiert sich das FEFO-Verfahren an absoluten Zeitvorgaben, also dem Namen nach an Verfallsdaten. Damit ist es prädestiniert zum Einsatz in Umgebungen mit verderblichen Gütern. Es kann ebenso implementiert werden wie das FIFO-Verfahren, es muss lediglich ein anderes Datum für die Auswertung betrachtet werden. Das HIFO-Verfahren verfolgt hingegen einen anderen Ansatz, der so mit dem LK<sup>3</sup>S-System nicht verfolgt werden kann: Es wird versucht, diejenigen Güter mit dem höchsten Wert als erstes wieder auszulagern, was aus betriebswirtschaftlicher Sicht erstrebenswert ist, jedoch vom System zum aktuellen Zeitpunkt nicht berücksichtigt werden kann. Die Anwendung des ebenfalls bekannten LIFO-Verfahrens macht bei der Rangfolgebildung für Auslagerungen keinen Sinn, da es nicht im Interesse der Lagerverwaltung ist, alte Güter zu horten und nur die jüngsten Neuzugänge wieder auszulagern.

#### 4.2.6 Auslagerung – Deadline

Die Bearbeitung von Auslagerungsanforderungen mit bestimmten zeitlichen Restriktionen kann erforderlich sein, um den laufenden Produktionsprozess nicht hinauszuzögern. Existiert eine Obergrenze für die Belieferung mit einem Gut (Deadline), dann kann ein LVS mehrere Anfragen zwischenspeichern und die Rangfolge der Auslagerungen hinsichtlich ihrer spätesten Fälligkeit neu sortieren. In diesem Zusammenhang kann die Optimierungsstrategie mit der der Doppelspiele (s. Abs. 4.2.8) kombiniert werden.

Um die Einhaltung der Deadlines zu forcieren, kann eine stärkere Gewichtung der Metrik für diese Optimierungsstrategie gewählt werden. Die Metrik wird als relative Position der Deadline  $d$  einer Anforderung im Bereich zwischen der frühesten und der spätesten Deadline aller noch nicht erfüllten Anforderungen für jeden einzelnen potentiellen Lagerplatz  $\vec{L}$  angegeben. Diese Angabe ist ein Wert im Intervall  $[0, 1]$ , der gleichzeitig als Metrik  $m$  genutzt werden kann.

$$m(\vec{L}) = \frac{d(\vec{L}) - \min(d)}{\max(d) - \min(d)}$$

Diese Strategie wurde erläutert und zum Teil implementiert. Jedoch ist die Produktionsplanung des LK<sup>3</sup>S-Agentensystems so ausgelegt, dass auch Materialbeschaffungen spontan ausgeführt werden, sodass sich weiterführende Planungen und damit auch die Pufferung mehrerer Aufträge auf Grund der stets sofortigen Deadlines erübrigen.

#### 4.2.7 Auslagerung – Passendes Material

Ein besonderes Verfahren, das Einsatz bei der Lagerung von mehrerem Material in einer Lagereinheit findet, kann bei der Verringerung des Kommissionieraufwands helfen, falls eine Kommissionierung vorgesehen ist. Dabei müssen Auslagerungsanforderungen die Anzahl der gewünschten Einheiten eines Guts beinhalten. Dann kann bereits bei der Wahl des Lagerplatzes zur Auslagerung berücksichtigt werden, wie genau die Anzahl des Materials in der Lagereinheit mit der Anforderung übereinstimmt. Im Idealfall sind diese gleich, denn dann kann die komplette Lagereinheit ohne nachgelagerte Kommissionierung aus dem Lager verbracht werden.

Das Verfahren zur Bestimmung der Übereinstimmung von real vorhandenem Material in einem Lagerplatz  $\vec{L}$  und der Anforderung sieht eine Differenzbestimmung vor. Dazu ist es nicht relevant, welcher Wert von welchem subtrahiert wird, da im Anschluss der Differenzbetrag zur weiteren Rechnung genutzt wird. Es wird der Wert 1 zum Betrag addiert, um im Folgenden einer Division durch Null vorzubeugen. Die Zahl 1 wird durch die Summe dividiert, sodass sich bei einer zuvor berechneten Differenz von 0 nun ein Wert 1 ergibt. Mit zunehmender Differenz von angeforderten und in einem Lagerplatz vorhandenen Material verringert sich dieser Quotient und geht dabei gegen 0. Um diese Rechnung direkt als Metrik  $m$  nutzen zu können, ist es erforderlich den Quotienten von 1 zu subtrahieren, da eine steigende Metrik mit einer Verschlechterung der Bewertung eines Lagerplatzes gleichzusetzen ist. Die Differenz von 0 ist jedoch das Optimum, sodass die Semantik invertiert werden muss. Nach der Subtraktion entspricht die Metrik 0 einer Anzahldifferenz von 0 und die Metrik 1 würde einer unendlich großen Differenz entsprechen. Die möglichen Metrikwerte liegen hier also im Intervall  $[0, 1]$ .

$$m(\vec{L}) = 1 - \frac{1}{1 + |n_{angef.} - \text{Verfügbar}(\vec{L})|}$$

Allerdings kann bei diesem Verfahren ein Konflikt auftreten, falls eine mehrfachtiefe Lagerung in den Regalen betrieben wird. Dann würde die Berechnung ohne weitere Anpassungen sämtliches im mehrfachtiefen Lagerplatz befindliches Material berücksichtigen. Es könnte angenommen werden, dass eine Lagereinheit genau die erforderliche Menge an Material enthält, diese sich jedoch über weitere Lagereinheiten erstreckt. Ohne eindeutige Lösung dieses Umstands ist von der Nutzung dieser Optimierungsstrategie bei mehrfachtiefer Lagerung abzusehen.

#### **4.2.8 Auslagerung – Doppelspiele**

Eine gänzlich andere Optimierungsstrategie ist das Doppelspiel (s. SCHUMANN [23], AMATO et al. [1]). Hierbei werden zwei Lageroperationen miteinander verknüpft, um eine Wegverkürzung des RBG zu erzielen (s. Abb. 4.1). Oftmals handelt es sich dabei um die Verknüpfung eines Ein- und eines Auslagerungsvorgangs, bei denen die betroffenen Lagerplätze besonders nah beieinander liegen. Dies können jedoch auch Umlagerungsvorgänge innerhalb des Lagers sein. Kann ein RBG mehrere Güter aufnehmen, so können z. B. auch mehrere Auslagerungen verknüpft werden.

Es existieren mehrere Schemata, nach denen Doppelspiele ausgeführt werden können:

- Das RBG behält seine letzte Position bei und wartet auf den nächsten Auftrag. Dies ermöglicht Doppelspiele auch ohne Auftragslisten, also auf Abruf. Allerdings geht ein wichtiger Vorteil der Doppelspiele hierbei verloren: Es besteht keine Auswahlmöglichkeit zur Kombination verschiedener Vorgänge.
- Als Ergänzung zum eben vorgestellten Schema können Auftragslisten, jeweils für Einlagerungsvorgänge und Auslagerungsvorgänge angelegt werden. Jeder Eintrag gehört dabei zu einer Bestellung und kann im Rahmen definierter zeitlicher Beschränkungen (Deadline) zur Ausführung disponiert werden. Dann ist es möglich, dass das LVS z. B. zu jedem Einlagerungsvorgang automatisch einen wartenden Auslagerungsvorgang auswählt und zur Ausführung bringt. Dabei muss geprüft werden, welche Lagerplätze mit jeder einzelnen wartenden Auslagerungsanforderung korrelieren, denn letztere beschreiben nicht zwangsläufig einen konkreten Lagerplatz, sondern meistens ein Material, das angefordert wird und das durchaus in mehr als einem Lagerplatz vorliegen kann. Es wird also ein Lagerplatz für den folgenden Auslagerungsvorgang gewählt, der das benötigte Material enthält und dem Einlagerungsplatz zudem möglichst nahegelegen ist.
- Neben der Ausführung eines Auslagerungsvorgangs als Nachfolger eines Einlagerungsvorgangs kann auch ein umgekehrtes Schema Anwendung finden. Auch die Kombination mit dem ersten vorgestellten Schema ist möglich, sodass dann ein Schwellwert bei der Beurteilung des zweiten Lagerplatzes bei einem Doppelspiel zum Einsatz kommt, anhand dessen entschieden werden kann, ob ein Doppelspiel überhaupt ausgeführt wird, oder ob nach dem ersten Vorgang abgebrochen wird.



- Als Ergänzung zu obigen Schemata sei erwähnt, dass einerseits die Wahl des zweiten Vorgangs in Abhängigkeit des ersten stattfinden kann, wie auch die Wahl des ersten Vorgangs anhand des zweiten und auch die Wahl beider Vorgänge in gegenseitiger Abhängigkeit. Letztere Variante erhöht den Aufwand maßgeblich, bietet jedoch auch das größte Optimierungspotential.

Der Literatur nach genießt das zweite Schema die größte Verbreitung. Ebenso ist es möglich, nicht nur zwei Vorgänge miteinander zu verknüpfen, sondern theoretisch auch beliebig viele. Dies wird dann im Englischen Multiple Command genannt und beispielsweise von MALMBORG et al. beschrieben (s. [19]). Dabei steigt der Aufwand jedoch erheblich, da für jeden potentiellen Lagerplatz eines Vorgangs wiederum alle potentiellen Lagerplätze für den darauf folgenden Vorgang untersucht werden müssen usw.

Die Implementation im Rahmen dieser Arbeit macht von der ersten Variante Gebrauch. Es finden somit zwar keine automatischen Aus- oder Einlagerungen infolge eines gegensätzlichen Vorgangs statt, doch bei einem entsprechenden Auftrag wird die letzte Position des RBG in Betracht gezogen, wenn ein Lagerplatz zur Auslagerung gesucht wird. Dies ist eine sinnvolle Variante, da das LK<sup>3</sup>S-System zum momentanen Zeitpunkt keine Vorausplanung der Produktion durchführt, sondern spontan reagiert. Damit würden zwischengespeicherte Auslagerungsanfragen nur sehr selten auftreten bzw. ihre Deadlines wären minimal, da die Produktion bei Auftragsstellung bereits pausiert und auf das nächste Material wartet. Einlagerungen werden zudem auch stets sofort durchgeführt, da auf Grund der Gegebenheiten der Digitalen Fabrik keine Möglichkeiten zur Zwischenlagerung der Güter auf Fließbändern o.ä. bestehen, ohne andere Prozesse zu behindern.

Auf Grund der kontinuierlichen indizierten Betrachtungsweise der Lagerplätze und der einfachen, bereits beschriebenen, Distanzberechnung zwischen zwei Lagerplätzen ist auch die Bestimmung einer Metrik für die Wahl eines geeigneten Lagerplatzes für die Auslagerung in einem Doppelspiel vereinfacht. Dazu bedient sich der Algorithmus wieder dem Satz des Pythagoras und berechnet die Distanz vom zuletzt angefahrenen Lagerplatz zu allen potentiellen Lagerplätzen, die das gesuchte Gut enthalten. Normiert über dem Intervall  $[0, 1]$  sind die Distanzen, die im Bereich  $(0, m]$ , mit  $m$  als maximaler Entfernung im Lager, bestimmt wurden bereits als Metrik angebar.

Zur weiterführenden Optimierung des Doppelspiels kann auch der Winkel der Fahrtstrecke zwischen den anzufahrenden Lagerplätzen in Betracht gezogen werden. So ist in energetischer und möglicherweise auch zeitlicher Hinsicht zwischen Horizontal- und Vertikalfahrt zu unterscheiden (s. BUSCH und OSTERMANN [6]).

Das Doppelspielprinzip (auch Multiple Command) findet nicht nur im Bereich der Lageroptimierung seinen Einsatz. Auch in anderen, alltäglichen Bereichen existieren vergleichbare oder ähnliche Verfahren. Hierzu zählt z. B. das sog. *Fahrstuhlprinzip* (s. KOEHLER et al. [15]). Es wird beim Rufen eines Fahrstuhls berücksichtigt, ob bewegte Fahrstühle sich von der aktuellen Etage entfernen oder sich ihr annähern. So werden angefangene Fahrten zunächst fortgesetzt und neue Anforderungen, deren Etagen jedoch auf dem

aktuellen Weg liegen und der gewünschten Richtung entsprechen, integriert. Auch die Verlängerung der Route nach einer Anforderung für die entgegengesetzte Richtung ist denkbar, da sich u. U. nur eine geringe Verzögerung ergibt. Ein weiteres Anwendungsgebiet ist das sog. *Native Command Queueing* (NCQ), das als Technologie zur Sortierung und Optimierung der Zugriffsreihenfolge auf Festplattensektoren bei SATA-Festplatten zuständig ist (s. Intel et al. [13], siehe auch *Tagged Command Queueing* (TCQ) bei SCSI-Festplatten). Auch bei dieser Technologie werden Anforderungen (Lesen und Schreiben) in einer Warteschlange verwaltet und entsprechend der Bewegung des langsamsten Gliedes einer Festplatte, dem Kopf, angeordnet.

### 4.2.9 Multi-Aspekt-Optimierung

Die vorgestellten Verfahren können jeweils für sich allein optimale oder zumindest bessere Situationen herstellen, als die zufällige Wahl von Lagerplätzen. Natürlich ist es entscheidend, diese Verfahren so zu kombinieren, dass auch die Gesamtsituation im Lager möglichst optimal wird. Dabei sollten sich die erreichten Optimierungen nach Möglichkeit nicht wieder gegenseitig relativieren. Da die Verfahren jedoch unterschiedliche Aspekte der Optimierung betrachten, ist dieser Effekt allerdings unumgänglich, sodass eine Bewertung der Ergebnisse der Verfahren von Nöten ist.

Zu diesem Zweck wird eine Metrik eingeführt, die eine Gesamtwertung über alle Verfahren hinweg zulässt. Bei der Einrichtung können dabei Prioritäten zur Gewichtung der Ergebnisse vergeben werden. Die Metrik setzt sich aus den Einzelbewertungen der Verfahren für jede einzelne Lagerzelle additiv zusammen. Bei Multiplikation der Metriken würde bereits eine einzige Null-Metrik die Vergleichbarkeit zumindest der übrigen Metriken verhindern, sodass hier Optimierungspotential ungenutzt bliebe.

Um eine Vergleichbarkeit der Verfahren untereinander zu garantieren, dürfen deren Bewertungsfunktionen nur Werte in einem definierten Intervall zurück liefern. Dieses ist standardmäßig  $[0, 1]$  wird jedoch direkt multiplikativ durch die Gewichtungparameter der einzelnen Optimierungskriterien beeinflusst. Per Definition sollte eine Lagerzelle zur Ein- oder Auslagerung dann bevorzugt werden, wenn die Metrik einen kleineren Wert besitzt, als alle anderen Lagerzellen.

Es existieren, wie in Abschnitt 4.2.1 bereits erwähnt, insgesamt  $p$  Lagerplätze  $L$ , von denen

$$\mathcal{L}(G) = \{L_s | s = 1, \dots, p \wedge G_i \leq L_i\}$$

potentielle Lagerplätze für eine Einlagerung und

$$\mathcal{L}(G) = \{L_s | s = 1, \dots, p \wedge G \text{ lagert in } L\}$$

potentielle Lagerplätze für eine Auslagerung sind.

$\mathcal{M}$  sei die Menge aller  $q$  Optimierungsfunktionen, die einen  $q$ -dimensionalen Optimierungsraum aufspannen und

$$m^q(s) \rightarrow [0, 1]$$

sei die Metrik der  $q$ -ten Optimierungsfunktion für Lagerplatz  $L_s$  mit  $\alpha_q$  als Gewichtungsfaktor einer Optimierungsfunktion. Die Relevanz bzw. Gesamtmeterik  $c(s)$  eines Lagerplatzes  $L_s$  zur Ein- bzw. Auslagerung eines Guts  $G$  ergibt sich aus

$$c(s) = \sum_{k=1}^q \alpha_k m^k(s), \quad L_s \in \mathcal{L}(G).$$

Das Minimum

$$\min_{L_s \in \mathcal{L}} c(s) = c(s_0)$$

liegt damit bei Lagerplatz  $s_0$  und dieser sollte daher zur Ein- bzw. Auslagerung herangezogen werden. Die Gewichtungsfaktoren sind u.U. abhängig von lokalen und zeitlichen Lagerstrukturen und -anforderungen, sodass diese für jedes Lager erneut eruiert werden müssen oder auch dynamisch während der Laufzeit verändert werden können.

Die Komplexität dieses gesamten Vorgangs ist besonders hoch, da diese Evaluation für alle Lagerzellen durchgeführt werden muss, die für eine Einlagerung zur Verfügung stehen, also noch freie Kapazität aufweisen, bzw. die das für eine Auslagerung angeforderte Material enthalten. Zudem entsteht beim Einsatz von Doppelspielen die Anforderung, alle Bewertungen  $E$  der noch freien Lagerzellen mit den Bewertungen  $A$  der für eine Auslagerung zulässigen zu korrelieren. Mathematisch entspricht dies dem Term

$$M_{i,j} = E_i + A_j, \quad \text{mit } i : \text{Zeile, } j : \text{Spalte.}$$

Da die Dimension der Matrizen je nach Größe und Belegung des Lagers variiert, kann der für die Berechnung nötige Aufwand stark variieren.

Je nach eingesetzten Algorithmen kann eine Optimierung z. B. auch ausschließlich mit Hilfe eines DBMS geschehen (siehe auch Abschnitt 5.1). Sobald zu den in der Datenbank verarbeiteten Informationen weitere aus anderen Quellen – z. B. andere Datenbanken oder Systeme – hinzu kommen, ist ein zeit- und kommunikationsintensiver Datenaustausch nötig (s. Abs. 5.1.2).

## 5 Lagerverwaltung und -steuerung

In Kapitel 4 wurden bereits diejenigen Optimierungsverfahren vorgestellt, die auch tatsächlich im Rahmen dieser Arbeit implementiert werden sollen. Doch zunächst soll in Abb. 5.1 durch ein Anwendungsfalldiagramm noch einmal deutlich gemacht werden, was die Anwendungsfälle sind, die u.a. für ein Lager existieren. Alle in diesem Kapitel disku-

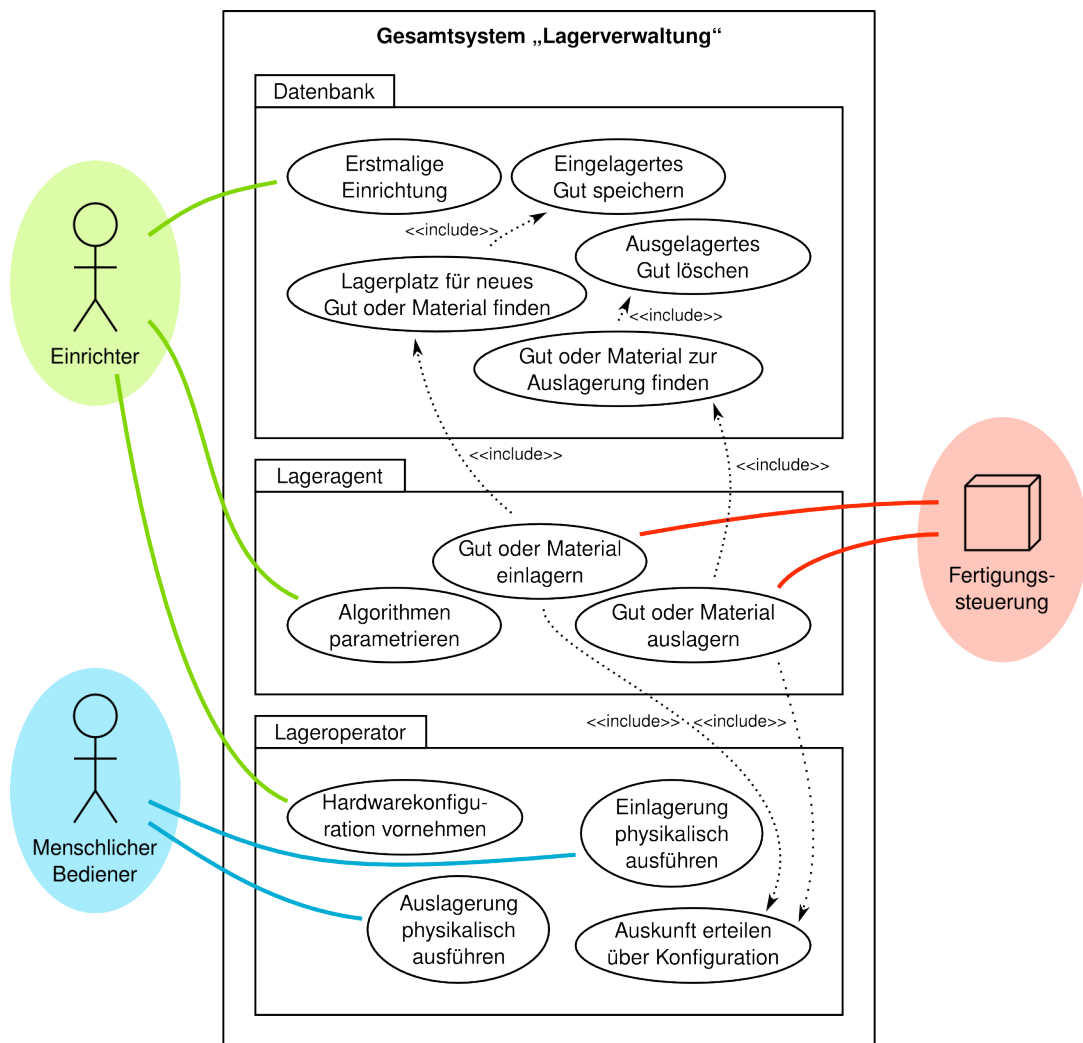


Abbildung 5.1: Anwendungsfalldiagramm eines Lagers.

tierten Themen besitzen bereits konkreten Bezug zur Implementation im Rahmen dieser

Arbeit. Da die Datenhaltung das Rückgrat einer Lagerverwaltung darstellt, soll hierauf auch zunächst eingegangen werden. Anschließend wird ein Überblick über die zentralen Komponenten des Lagermanagementsystems gegeben und Kernbereiche werden näher erläutert. Am Ende dieses Kapitels wird abschließend auf die individuelle Konfiguration der Komponenten eingegangen. In Abbildung 5.2 ist die Architektur des LVS im Kontext des Agentensystems dargestellt.

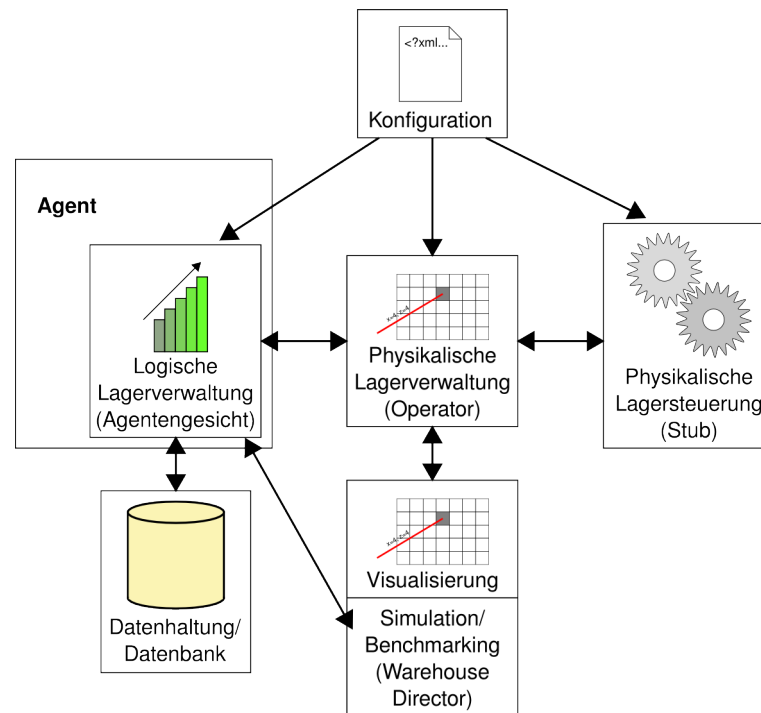


Abbildung 5.2: Architektur des LVS im Kontext des Agentensystems.

## 5.1 Datenhaltung

Daten sind der Dreh- und Angelpunkt eines jeden Lagers. Es werden die Stammdaten des Lagers erfasst und bei jeder Aktion herangezogen, die Daten der Güter werden erfasst und Statistiken werden erstellt und ausgewertet. Durch die langjährige Entwicklung der gängigen Database Management Systems (DBMS) und der höheren Datensicherheit beim Absturz der Lagersoftware ist ein dediziertes, ausgereiftes DBMS, z. B. auf SQL-Basis, gegenüber einer eigenen Lösung zu favorisieren. Wird das DBMS auf einem anderen Rechner ausgeführt wie die Lagersoftware, so ist jedoch bei der Wahl der Algorithmen bzw. generell des Vorgehens zur Verwaltung der Ein- und Auslagerungen zu beachten, dass entsprechende Datenmengen anfallen können, die teils oder ganz zwischen dem DBMS und der Lagersoftware (verteilte Datenhaltung) ausgetauscht werden müssen, was über Systemgrenzen hinweg einen erhöhten Aufwand darstellt. In Abschnitt 5.1.2 wird

auf die Auswirkungen in einem Beispiel eingegangen. Die Schnittstelle zwischen Lagersoftware und DBMS bildet beispielsweise die Abfragesprache SQL. Hierüber werden Anweisungen zur Zusammenstellung der gewünschten Daten ausgedrückt.

### 5.1.1 Datentypen

Es existieren drei unterschiedliche Arten von Daten, die vom DBMS gehandhabt werden müssen, um die Struktur des Lagers und seine Güter zu verwalten. Diese sind:

- Stammdaten
- Belegungsdaten
- Bewegungsdaten

**Stammdaten** Bei den Stammdaten handelt es sich um Informationen zum Lager und den einzelnen Lagerplätzen. Sie umfassen geometrische Rahmenbedingungen, die maximale Belastbarkeit, aber auch Metainformationen wie Zugehörigkeit zu einer Regalgasse oder ähnliches. Eine einmalige Parametrierung und Konfiguration dieser Daten ist vor Aufnahme des Produktivbetriebs notwendig und kann größtenteils mit Hilfe des ebenfalls im Rahmen dieser Arbeit entwickelten Warehouse Director durchgeführt werden (s. Kapitel 6). Er unterstützt bei der Abbildung der physikalischen Lagerstruktur in elektronische Stammdaten.

Es wird eine Tabelle namens `cells` für die Haltung der Stammdaten erzeugt. Sie enthält alle Informationen zu den einzelnen Lagerplätzen. Nach ihrer initialen Erzeugung, z. B. mit Hilfe des Warehouse Directors, unterliegt sie keinen Änderungen durch den Lagerbetrieb, ist somit also statisch. Ihre Daten werden insbesondere bei der Einlagerung von Gütern herangezogen. So kann beispielsweise geprüft werden, ob und wie gut die geometrischen Attribute eines Guts zu denen eines Lagerplatzes passen und damit eine Rangfolge der geeignetsten Lagerplätze aufgestellt werden. Zu jeder Lagerzelle werden Informationen gespeichert, die sich auf ihre äußeren Eigenschaften (Geometrie usw.) beziehen, aber auch Informationen, die zur Verwaltung des Lagers dienen (z. B. ein eindeutiger Bezeichner, UID). Diese sind im Detail in Tabelle 5.1 zusammen mit datenbankspezifischen Eigenschaften angeführt.

Die Spalten `width`, `height`, `depth` und `maxload` stellen physikalische Größen dar, die die Geometrie (Breite, Höhe, Tiefe) und die maximale Gewichtsbelastung des Lagerplatzes näher beschreiben. Auf diese Werte wird bei der Einlagerung zurückgegriffen, um bereits eine Vorauswahl der potentiellen Lagerplätze zu erstellen. Verständlicherweise muss ein Lagerplatz, dessen Dimensionen denen des einzulagernden Guts nicht gerecht werden diskussionslos in weiteren Betrachtungen unberücksichtigt bleiben. Das Ignorieren dieser Restriktion könnte zu schweren Schäden der Lagerhardware und anderen eingelagerten Gütern führen: Das einzulagernde Gut könnte gequetscht werden, das Regal gestaut

oder gekippt werden und herabfallende Güter könnten andere Güter beschädigen. Zudem wäre mindestens das betroffene Regal vor einer Reparatur nicht länger automatisiert bedienbar, da bei einer Beschädigung angenommen werden muss, dass auch geometrische Änderungen des Regals herbeigeführt wurden und geteachte Positionen und Distanzen nun ihre Gültigkeit verloren haben.

Mit Hilfe der Spalte `capacity` können Lagerplätze für eine mehrfachtiefe Einlagerung definiert werden. Bei der Optimierung einer Einlagerung wird dieser Wert mit denen anderer Datenbanktabellen korreliert, um den Füllgrad aller potentiellen Lagerplätze zu bestimmen. So können z. B. bereits teilgefüllte Lagerplätze mit ähnlichen Gütern aufgefüllt werden, bevor weitere Lagerplätze belegt werden.

Die Spalten `x`, `y`, `z` enthalten für jeden Lagerplatz eine entsprechende Positionsangabe, gemäß dem in Abbildung 6.3 dargestellten Koordinatensystem. Diese Werte werden bei der Ausführung von Vorgängen auf der Lagerhardware an die entsprechende Hardwaresteuerung weitergereicht, welche dann letztendlich die detaillierten absoluten oder relativen Koordinaten bei der Steuerung berücksichtigt. Aus diesem Grund ist es nicht zwingend notwendig, dass diese Positionsangaben in der Datenbank der realen, physikalischen Position des Lagerplatzes entsprechen, sie stellen vielmehr die Indizes einer dreidimensionalen Lagerplatz-Matrix dar.

Die Spalten `action`, `strategy`, `flowzoneuid` und `flowcelluid` stellen zusammen eine Möglichkeit dar, Durchlaufregale zu realisieren. Über `action` kann definiert werden, ob ein Lagerplatz zur Einlagerung, Auslagerung oder für beides genutzt werden kann. In Verbindung mit `strategy` (LIFO, FIFO) können zumindest bereits die Ein- und Ausgänge eines Durchlaufregals definiert werden. Mit Hilfe der Spalten `flowzoneuid` und `flowcelluid` wird zudem definiert, wo Güter entnommen werden können, wenn sie in den Lagerplatz als Durchlaufregal-Eingang eingegeben werden. Auf Grund des hohen Realisierungsaufwands und mangelnder realer Testumgebungen kann dies nur in weiterführenden Arbeiten näher untersucht und umgesetzt werden.

Um einen Lagerplatz von Einlagerungsvorgängen auszuschließen, besteht die Möglichkeit, der Spalte `reserved` eines Lagerplatzes mit einem Wert zu besetzen. Dieser könnte vom Warehouse Agent ausgewertet werden, z. B. sobald eine Anfrage bezüglich eines bestimmten Auftrages eintrifft, dessen Auftragskennung in der Spalte abgelegt wurde. Vor unerwünschter Auslagerung von Gütern schützt nur die Kennzeichnung als Item mit einem eindeutigen Bezeichner, da diese nur auf direkten Abruf ausgelagert werden und bei reinen Materialanfragen keine Berücksichtigung finden. Zur Zeit erfolgt eine Berücksichtigung bei der Einlagerung, da die Fertigungssteuerung jedoch momentan keine Reservierung von Gütern vorsieht, bleibt das Feld im automatischen Produktionsbetrieb bisher ungenutzt.

Die Lagerzugänge werden nicht in Tabellen verwaltet. Hierzu existieren individuelle *Stored Procedures*, die somit auch zu den Stammdaten zählen. Es sind Programme, die innerhalb des DBMS definiert sind und auch dort ausgeführt werden. Sie können Argumente

bei ihrem Aufruf übernehmen und Ergebnisse als Rückgabewert bereitstellen. Aufgerufen werden sie als Teil einer SQL-Datenbankabfrage und ihr Rückgabewert kann als Spaltenwert angesehen werden. Das verwendete DBMS PostgreSQL 8.4<sup>1</sup> bietet Unterstützung für verschiedene Programmiersprachen (Procedural Languages, PL) zur Realisierung der Stored Procedures. Hierzu zählen PL/pgSQL, die Eigenentwicklung von PostgreSQL, PL/Tcl, PL/Perl und PL/Python. Es ist zudem möglich, eigene Sprachen zu entwickeln und/oder einzubinden (s. POSTGRESQL [21]). Mit Hilfe der Stored Procedures sollen Anfragen um Werte ergänzt werden, die ansonsten nur mit einem komplexeren SQL-Ausdruck erzeugt werden könnten.

In Listing 5.1 ist die vom Warehouse Director automatisch für ein eingängiges Lager mit einem einzigen Zugang (Gate) generierte Stored Procedure `distancegate` abgebildet, die die Distanz von einem Lagerplatz zu einem Zugang berechnet. Als Grundlage wird davon ausgegangen, dass alle Zugänge genau wie Lagerplätze über ihre ganzzahligen Positionsindizes adressiert werden können und diese auch als Positionsangaben in die Distanzberechnung eingehen. Befinden sich die Zugänge real nicht an den angegebenen Positionen, so sind diese Angaben als *virtuelle Positionen* zu interpretieren. Unter diesen Voraussetzungen kann ein einfacher Pythagoras-Ansatz zur Berechnung der Distanz  $d$  gewählt werden (vgl. Abb. 5.1, Zeile 17):

$$d = \sqrt{(x_{gate} - x_{cell})^2 + (z_{gate} - z_{cell})^2} + |y_{gate}| + |y_{cell}|$$

Diese Formel gilt unter der Annahme, dass die Achse, horizontal ( $x$ ) oder vertikal ( $z$ ), mit der weitesten Distanz als Leitachse dient, also die andere Achse so verfahren wird, dass sie zeitgleich mit der Leitachse startet und auch zeitgleich stoppt. Ansonsten würde sich ein anderer Weg ergeben, der insgesamt in jedem Fall länger als die hier dargestellte Diagonale zwischen Start- und Endpunkt ist. Dass die Fahrt in die Regale hinein ( $y$ ) gesondert betrachtet wird, liegt darin begründet, dass alle übrigen Achsen für die Dauer der  $y$ -Fahrt gesperrt sind. Eine Beschädigung des Regals wäre ansonsten möglich.

Da die Gewichtsbelastung durch LEGO-Duplo-Steine in der Digitalen Fabrik sehr gering ist, kann auch die energetische Betrachtung der Vorgänge durch diese Formel abgebildet werden. Ebenso wird nach ihr auch der Weg zwischen zwei Lagerplätzen für Umlagerungen berechnet (ohne Abdruck, da zur Zeit nicht verwendet).

Listing 5.1: Stored Procedure `distancegate`.

```
1 CREATE OR REPLACE FUNCTION distancegate("zone" character varying
2     ,
3     gate character varying, x integer, y integer, z integer)
4 RETURNS double precision AS
5 $BODY$
6 DECLARE gx INT;
```

---

<sup>1</sup>siehe <http://www.postgresql.org/>



```

6 DECLARE gy INT;
7 DECLARE gz INT;
8 DECLARE d FLOAT;
9 BEGIN
10     IF zone = 'main' THEN BEGIN
11         IF gate = 'G' THEN BEGIN gx := 0; gy = 0; gz = 0; END;
12         ELSE RAISE EXCEPTION 'Invalid gate UID.';
13     END IF;
14 END;
15 ELSE RAISE EXCEPTION 'Invalid zone UID.';
16 END IF;
17 d := SQRT(POWER(gx - x, 2) + POWER(gz - z, 2));
18 RETURN d;
19 END$BODY$
20 LANGUAGE 'plpgsql';

```

In Abbildung 5.1 wird zunächst in den Zeilen 1-4 und 19-20 die Prozedur beschrieben. Mit den Zeilen 5-8 werden die Variablen *gx*, *gy*, *gz* und *d* deklariert. Es folgt eine Fallunterscheidung (Zeile 10-16) und zugleich Prüfung der der Prozedur übergebenen Argumente auf Plausibilität. Bei ungültigen Werten wird die Ausführung mit einem Fehler abgebrochen (Zeile 12 und 15). Je nach übergebenem Zonennamen (*zone*) und Zugangsnamen (*gate*) werden die Variablen *gx*, *gy* sowie *gz* mit Werten belegt, die spezifisch für den jeweiligen Zugang *gate* in der Gasse *zone* sind. Da es sich hierbei um (virtuelle) Positionsangaben handelt, kann im Folgenden mit Hilfe des Satzes des Pythagoras in Zeile 17 die Distanz *d* berechnet und als Wert zurückgegeben werden (Zeile 18). Ein weiterer Grund für den Einsatz von Stored Procedures an dieser Stelle ist, dass nicht in jedem Lager sichergestellt ist, dass eine Distanzberechnung der Lagerplätze über eine solch einfache Formel wie dem Satz des Pythagoras erfolgen kann. Mit Hilfe einer Stored Procedure sind auch umfangreichere Algorithmen zur Berechnung von Distanzen anhand der in der Datenbank hinterlegten Stammdaten und den Aufrufargumenten der Prozedur möglich. Dies rechtfertigt auch die Existenz der Stored Procedure *distancemax*. Sie gibt Auskunft über die maximal zurücklegbare Entfernung in einer Regalgasse. Dies erspart eine Berechnung bei jeder Nutzung des Wertes und macht nutzende SQL-Ausdrücke daher auch übersichtlicher. Die Stored Procedure ist in Abbildung 5.5 dargestellt. Sie findet Anwendung bei der Einlagerungsoptimierung. Es wird die maximale Distanz ausgegeben, die zwischen den Lagerplätzen in einer Lagerzone existiert. Anhand dieser Angabe können die einzelnen Distanzen auf ein Intervall  $[0, 1]$  normiert werden. Je weiter ein Lagerzugang in der geographischen Mitte des Lagers liegt, desto mehr entfernt sich die berechnete relative Distanz für einen Lagerplatz zum Lagerzugang von 1 (Intervall  $[0, 1)$ ), da von der maximalen Strecke im Lager und nicht vom maximalen Radius um den gewählten Lagerzugang ausgegangen wird (s. Abb. 5.4). Als Alternative könnte die Berechnung der Distanzen aller Lagerzellen zu allen Lagerzugängen im Voraus durchgeführt werden.

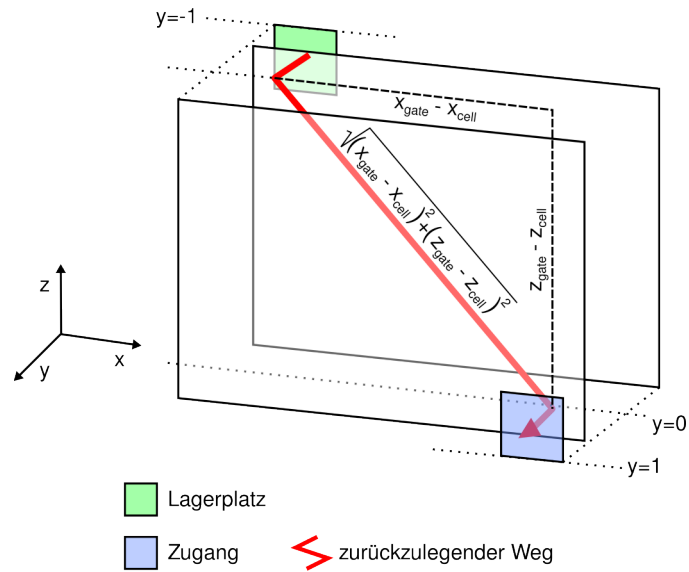


Abbildung 5.3: Illustration der Distanzbestimmung zwischen Lagerplatz und Zugang mit Hilfe des Satzes von Pythagoras.

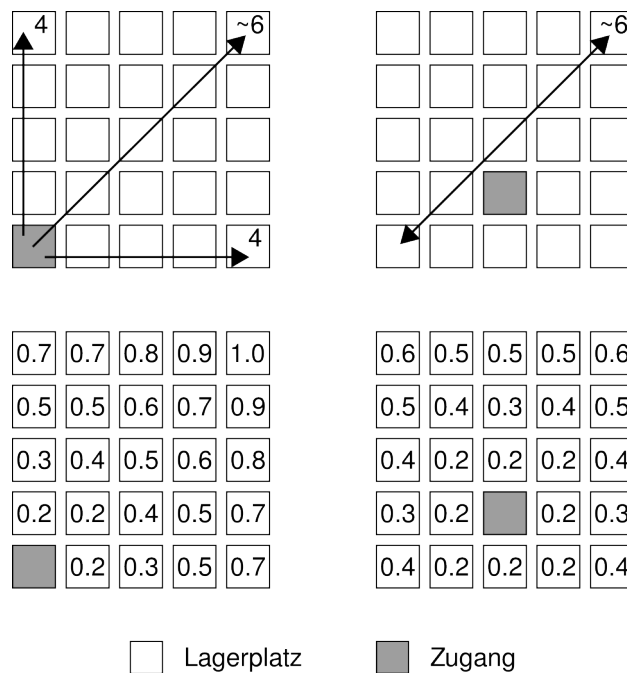


Abbildung 5.4: Ausgabe der maximal möglichen Distanz innerhalb einer Regalgasse (Zone).

**Belegungsdaten** Die Belegungsdaten sind temporäre Informationen über den aktuellen Belegungsstand des Lagers und seiner einzelnen Lagerplätze. Sie umfassen den eindeutigen Bezeichner eines Baugruppenträgers mit dem zugehörigen Lagerplatz sowie den Inhalt des BGT und dessen physikalische Gegebenheiten. Werden diese Daten inkonsis-

```
1 CREATE OR REPLACE FUNCTION distancemax("zone" character varying)
2 RETURNS double precision AS
3 $BODY$
4 BEGIN
5     IF zone = 'main' THEN RETURN 5.656854249;
6     ELSE RAISE EXCEPTION 'Invalid zone UID.';
7     END IF;
8 END$BODY$
9 LANGUAGE 'plpgsql';
```

Abbildung 5.5: Stored Procedure `distancemax`.

tent, z. B. durch manuelle Lagerbedienung, fehlerhafte Datenbankabfragen oder andere fehlerhafte Lagersoftware, so ist entweder ein manueller Eingriff nötig, um die aktuelle Belegungssituation zu prüfen und ggf. zu korrigieren oder es könnte beispielsweise eine Lagerdurchfahrt mit RFID-Lesegeräten ausgeführt werden, falls das Lager entsprechend ausgerüstet und die eingelagerten Güter bzw. BGT hierfür geeignet sind.

Die Belegungsdaten umfassen hier die spezifischen Eigenschaften eines BGT und des enthaltenen Materials, sofern das Material einzeln abrufbar sein soll. Zur Beschreibung der Daten finden zwei Tabellen `items` und `material` Verwendung. Während `items` Informationen zu den eingelagerten BGT enthält, werden die Detailinformationen über enthaltenes Material erwartungsgemäß in `material` gespeichert.

Die Daten, die in `items` hinterlegt werden, sind in Tabelle 5.2 im Detail dargestellt. Wie auch bei den Stammdaten gliedert sich diese Tabelle spaltenweise in unterschiedliche Bereiche. Einige Spalten gehen auf die Eigenschaften des BGT und dessen Inhalt ein, während andere Spalten administrative Eigenschaften des Guts beschreiben.

Bei der Belegung von Lagerplätzen wird zwischen zwei Arten Güter unterschieden. Ein BGT kann mehrere Objekte als Material enthalten. Ist dies der Fall, so kann das enthaltene Material bei Material-Auslagerungsanforderungen auch gefunden werden. Im zweiten Fall wird ein BGT trotz des enthaltenen Materials als Einheit betrachtet, z. B. als ein bereits vollständig montiertes Objekt. Es ist in diesem Fall nicht vorgesehen, dass das Objekt zur Materialgewinnung wieder demontiert wird, was unter Umständen auch gar nicht mehr möglich ist. Da anhand der Belegung eines BGT nicht bestimmt werden kann, ob es sich um allgemein abrufbares Material oder um ein nur gänzlich zu verwendendes Objekt handelt, existiert eine entsprechende Spalte `material`, die für jedes Gut (item) definiert, in welchem Zustand sich das Gut befindet.

Zu den güterspezifischen Eigenschaften zählen noch die Datumsangaben `entered`, `left` und `expires`. Die ersten beiden Spalten halten fest, zu welchen Zeitpunkten ein Gut die Fabrik betreten und wann es sie wieder verlassen hat. Die Speicherung auch nach der Verarbeitung kann zur Dokumentation der Produktion herangezogen werden. Mit `expires`

hingegen kann ein güterspezifisches Verfallsdatum definiert werden. Es findet bei den Optimierungsverfahren Anwendung. So können mehrfachtiefe Lagerplätze mit Last-In-First-Out-Strategie beispielsweise zeitlich unabhängig mit mehreren Gütern belegt werden, die das gleiche Verfallsdatum besitzen, ohne dabei einen Verfall der Güter zu riskieren.

Zur Identifikation und Verwaltung der Güter existieren weitere Spalten in der Tabelle `cells`. So wird ein systemweit eindeutiger Bezeichner für die Spalte `itemid` erwartet, den ein Gut über seine gesamte Lebenszeit trägt. Selbst nach Auslieferung, also dem Entfernen aus der Fertigungsumgebung, bleibt dieser Bezeichner gültig und kann im Fall von Reklamationen im Rahmen der Fertigungsdokumentation herangezogen werden. Hinzu kommt eine Spalte `chipuid`, deren Name auf den RFID-Chip anspielt, den jeder BGT besitzt. Über sie soll es möglich sein, gezielt Baugruppenträger anhand eines in der Produktionsumgebung einmaligen Bezeichners im Lager aufzufinden. Mit der Spalte `priority` wird der Fertigungssteuerung Rechnung getragen. Jedes mit einem Auftrag verknüpfte Gut trägt mindestens dieselbe Priorität wie der eigentliche Fertigungsauftrag, um Materialbewegungen innerhalb der Produktionsumgebung den Aufträgen entsprechend durchführen zu können. Wird ein Gut vorübergehend eingelagert, so kann seine letzte Priorität in dieser Spalte festgehalten und bei späterer Auslagerung unter entsprechenden Umständen berücksichtigt werden. Es kann zudem eine Zuordnung zu einem bestimmten Auftrag durch Nutzung der Spalte `orderid` erfolgen sowie die Zuweisung einer Autorität, also einer Systemkomponente, die die Verantwortung für ein Gut trägt, beispielsweise das ERP-System oder ein Agent, der die Einlagerung des Guts ausgelöst hat.

Rein administrativer Natur sind die Spalten `celluid` und `zoneuid`. Hierüber wird der berechnete Aufenthaltsort eines Guts festgehalten. Dabei repräsentiert der Wert der Spalte `celluid` den innerhalb der über `zoneuid` definierten Lagerzone eindeutigen Bezeichner des Lagerplatzes, in dem sich das Gut zu befinden hat.

Für alle Güter, die bei Materialanfragen berücksichtigt werden sollen, ist die Tabelle `material` relevant. Sie enthält Angaben zu den einzelnen Materialien, die auf dem BGT vorhanden sind bzw. zusammen ein Gut bilden.

Die Zuordnung von Einträgen der Tabelle `material` geschieht über den eindeutigen Bezeichner `itemid`. Damit ist das Gut bekannt, dem das Material zugehörig ist und es kann im Anschluss der entsprechende Lagerplatz des Guts mit der `itemid` ermittelt werden.

Die übrigen Spalten der Tabelle `material` sind sehr spezifisch und auf das LK<sup>3</sup>S-Projekt abgestimmt. Im Rahmen dieser Arbeit wurde versucht, Hochregallager möglichst allgemein darzustellen und auch auf die entwickelte Software abzubilden. Jedoch sind dieser Arbeit auch Grenzen gesetzt, sodass an dieser Stelle leider nicht auf die vielfältigen Materialspezifika eingegangen werden konnte. Stattdessen werden die projektspezifischen Materialattribute zur Beschreibung von LEGO-Duplo-Elementen verwendet. Das dies nicht ausschließlich Bausteine umfassen muss, zeigt die Präsenz der Spalte `typeid`, die es ermöglicht einen Materialtyp anzugeben. Somit kann beispielsweise zwischen Bausteinen und Fahnen – die z. B. mit Hilfe des Scara-Roboters verarbeitet werden könnten

– unterschieden werden. Hierzu dienen die diskreten Positionsangaben eines Steins auf einem BGT `x`, `y` und `z`. Zudem muss das System in der Lage sein, auch Steine zu verarbeiten, die keine einheitliche Breite aufweisen. In der Testumgebung stehen Steine mit Breiten von 2, 4, 6 und 8 Noppen zur Verfügung. Diese Breite wird mit Hilfe der Spalte `width` abgebildet. Ebenso muss die Farbe eines LEGO-Duplo-Steins berücksichtigt werden, da der Kunde über einen Konfigurator nicht nur die Form, sondern auch die Farbgebung eines Hauses definieren kann. Die Spalte `width` nimmt einen der genannten Werte auf. Abschließend können Steine auf dem BGT noch um ein Vielfaches von 90° gedreht sein, sodass eine letzte Spalte `angle` über den Drehwinkel eines Steins Auskunft gibt. Sollte mit der Spalte `typeid` kein LEGO-Duplo-Baustein beschrieben sein, so haben die Spalten `width` und `colour` Werte anzunehmen, die im System definiert sind, jedoch nicht zwingend einer Unterscheidung der Materialien dienen. Somit wird jedoch die unveränderte Nutzbarkeit der Optimierungsalgorithmen sichergestellt. Tabelle 5.3 gibt einen Überblick über die Spalten der Tabelle.

**Bewegungsdaten** Unter Bewegungsdaten werden nicht-flüchtige Informationen zu Materialbewegungen innerhalb des Lagers verstanden. Dabei wird festgehalten, zu welchem Zeitpunkt welche Anfrage eingetroffen ist und wie sie bearbeitet wurde. Die so gesammelten Daten können bei Bedarf verdichtet und als Statistik genutzt werden, um zukünftige Vorgänge weiter zu optimieren. Hiervon macht das *Earliest-Deadline-Optimierungsverfahren* bei Auslagerungen Gebrauch. Da dies das einzige Verfahren ist, das auf diese Statistik zurückgreift, werden die anfallenden Daten bezüglich dem Material auch ausschließlich mit den für das Earliest-Deadline-Optimierungsverfahren notwendigen Daten angereichert. Diese umfassen lediglich die Deadline, bis zu der eine Anforderung ausgeführt sein muss.

Tabelle 5.4 stellt die einzelnen Spalten der Relation `requests` dar. Jede Anfrage wird mit einer eindeutigen `id` versehen. Das System ist so konzipiert, dass diese alle Anfragen ausschließlich in dieser Relation abgelegt und dann bei Bedarf, z. B. im Doppelspiel, vom LVS zur Ausführung gebracht werden. Da die Optimierung auch für die Doppelspiel-Strategie auch in der Datenbank abgewickelt wird, ist es notwendig, dass dem LVS nicht nur mitgeteilt wird, aus welchem Lagerplatz ein Gut zu entnehmen ist, sondern auch welcher ursprünglichen Anfrage einer anderen Systemkomponente dieses Gut zuzuordnen ist. Da das Mitführen aller Informationen zu einer Anfrage in den Optimierungsergebnissen unnötig ist, wird stattdessen zu jedem Ergebnis die entsprechende `id` mitgeführt. Das LVS ist somit in der Lage, gezielt notwendige Parameter dieser Anfrage nachzuladen.

Eine gespeicherte Anfrage gibt Auskunft, ob ein konkretes Gut über eine `itemid` angefragt wurde, dann sind die materialspezifischen Spalten leer, oder ob ein Material angefordert wurde. Hierzu werden die Spalten `typeid` zur Spezifikation des Materialtyps, `width` zur Definition der erforderlichen Materialbreite, `colour` zur Angabe der gewünschten Materialfarbe sowie `amount` für die angeforderte Anzahl des über die anderen Attribute beschriebenen Materials belegt. Anhand der Spalten `width` und `colour`

ist klar, dass diese Tabelle `requests` wieder sehr spezifisch hinsichtlich der beabsichtigten Anwendung des LVS ist. Selbstverständlich können die betroffenen Spalten wie oben bereits beschrieben auch mit Pseudo-Werten belegt werden, falls andere Materialien verwaltet werden sollen.

Administrativen Charakter besitzen die folgenden Spalten: `date` hält den Zeitpunkt fest, zu dem die vorliegende Anfrage in das System gegeben wurde und `deadline`, als spezifische Spalte für das Earliest-Deadline-Optimierungsverfahren, den spätesten Ausführungszeitpunkt. Die Spalte `authority` enthält den Namen des Auftraggebers für diese Anfrage. Hierüber ist dem LVS bekannt, welche Systemkomponente mit dem ausgelagerten Gut letztendlich zu beliefern ist. Die Spalte `done` dient den Optimierungsverfahren zur schnellen Erkennung, ob es sich bei der Anfrage um eine noch zu bedienende handelt, oder ob sie lediglich noch zur Bildung der Statistik hinsichtlich der Abfragehäufigkeit von Materialien in der Datenbank präsent ist.

Angewandte SQL-Ausdrücke zur Auswertung der Informationen und Optimierung der Ein- und Auslagerungsvorgänge sind in Abschnitt 5.2.2 beschrieben. Zusammenfassend sind alle Relationen der Lagerdatenbank und ihre Beziehungen untereinander in Abbildung 5.6 als ER-Modell dargestellt.

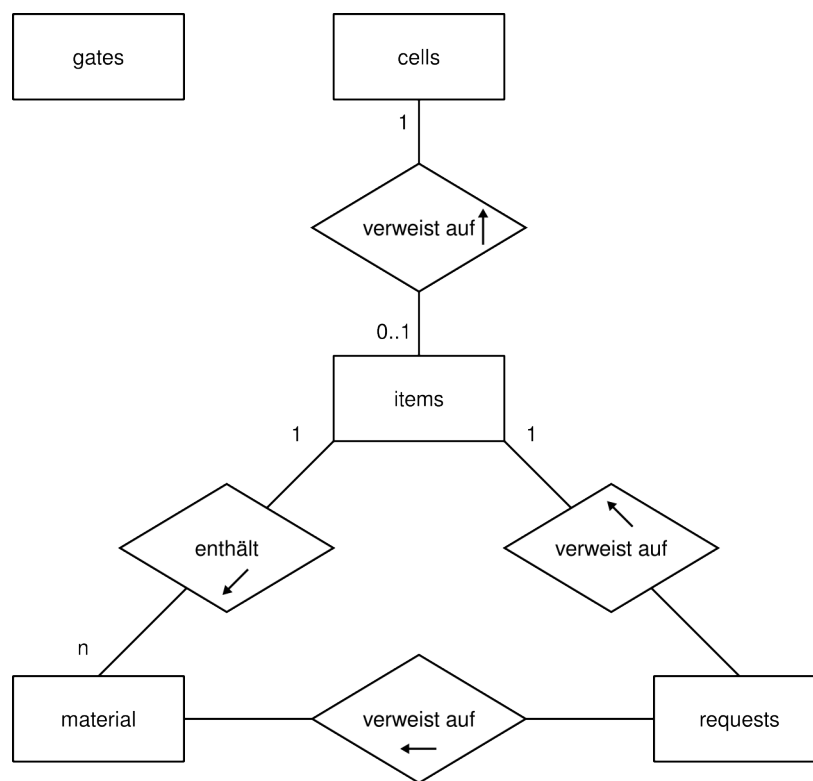


Abbildung 5.6: ER-Modell der Lagerdatenbank.

### 5.1.2 Verteilte Datenhaltung

Datenbankanfragen können individuelle Werte enthalten. Wie bereits in Abschnitt 4.2.9 erwähnt, steigert sich jedoch die zu übertragene Datenmenge erheblich, wenn zu jedem der aus der Datenbank ermittelten Datensätze ein oder mehrere Werte hinzukommen, die innerhalb des DBMS nicht bekannt sind, sondern nur von der Lagersoftware verwaltet werden. In diesem Fall würde bereits die initiale Datenbankanfrage aufgebläht, wie das folgende Beispiel zeigt.

**Beispiel** Eine Datenbank besitzt eine Tabelle mit den Spalten `Produkt` und `Preis`. Sie enthält folgende Einträge:

| Produkt          | Preis |
|------------------|-------|
| Schokoladeneis   | 1,99  |
| Vanilleeis       | 1,99  |
| Zitronenkuchen   | 1,39  |
| Lakritzschnecken | 0,89  |
| Schaumküsse      | 0,99  |

Es soll der Umsatz zu jedem Produkt berechnet werden. Hierzu ist zu jedem Produkt ein weiterer Wert `Verkaufsmenge` nötig. Wenn davon ausgegangen wird, dass die Verkaufsmenge aller Produkte ausschließlich in einer Software außerhalb des DBMS verwaltet wird, so muss diese nun bei der Datenbankabfrage zunächst mit zur Datenbank übertragen werden (siehe Listing 5.2). Dabei entstehen für dieses Beispiel ein festes Mehr-Datenvolumen<sup>2</sup> von 21 Byte zur syntaktischen Erweiterung der Abfrage um Zusatzdaten<sup>3</sup> und für jedes zusätzlich integrierte Datum weitere 66 Byte (für das erste Datum 60 Byte). In Tabelle 5.5 ist die Rechnung nach diesem Schema für unterschiedlich viele Produkte fortgesetzt. Es ist also ersichtlich, dass die Vermengung von Daten, die sich innerhalb und außerhalb des DBMS befinden, mit zunehmendem Umfang zu einem erheblichen Kommunikationsoverhead zwischen Lagersoftware und DBMS führt. Hinzu kommt die Tatsache, dass die Anfragehäufigkeit am DBMS zunehmen wird, je umfangreicher die Produktpalette, respektive das Lager, ist. Da zum Erstellungszeitpunkt dieser Arbeit jedoch keine realen Daten hierzu vorliegen, könnten lediglich Vermutungen zu den Auswirkungen geäußert werden, was jedoch an dieser Stelle nicht erfolgen soll.

Dieses einfache Beispiel kann sicher auch mit vertretbarem Aufwand durch eine eigene Anwendung umgesetzt werden. Es ist jedoch die Möglichkeit zu bedenken, dass weitere Daten mit den Ergebnissen verknüpft und beispielsweise sortiert werden müssen. Die Verwendung eines dedizierten DBMS wäre in diesem Fall wiederum empfehlenswert.

---

<sup>2</sup>im Vergleich zur Speicherung der Verkaufsmenge in der gleichen Tabelle als Teil jedes Datensatzes (s. Listing 5.3). Bei Speicherung in einer anderen Tabelle oder generell bei komplexeren Beziehungen entsteht jedoch nur ein vergleichsweise geringer fixer Mehraufwand

<sup>3</sup>es wird von Produktnamen mit durchschnittlich 13 Zeichen und einer dreistelligen Verkaufsmenge ausgegangen

Listing 5.2: Umsatzauskunft bei externer Haltung der Verkaufsmengen.

```

1 SELECT "Produkt", ("Preis" * "Verkaufsmenge") AS "Umsatz" FROM
2   (SELECT "Produkt", "Preis" FROM "Katalog") AS intern
3   LEFT JOIN
4   (
5     SELECT 'Schokoladeneis' AS "Produkt", 4 AS "Verkaufsmenge"
6     UNION
7     SELECT 'Vanilleeis' AS "Produkt", 7 AS "Verkaufsmenge"
8     UNION
9     SELECT 'Zitronenkuchen' AS "Produkt", 2 AS "Verkaufsmenge"
10    UNION
11    SELECT 'Lakritzschnecken' AS "Produkt", 10 AS "Verkaufsmenge"
12    UNION
13    SELECT 'Schaumküsse' AS "Produkt", 8 AS "Verkaufsmenge"
14  ) AS extern USING ("Produkt");

```

Listing 5.3: Umsatzauskunft bei komplett DBMS-seitiger Datenhaltung.

```

1 SELECT "Produkt", ("Preis" * "Verkaufsmenge") AS "Umsatz"
2   FROM "Katalog";

```

Aufgrund dieser Ergebnisse wurde das Ziel verfolgt, alle Optimierungsschritte innerhalb des DBMS durchzuführen und so wenig Parameter in die entsprechenden Datenbankabfragen einzubringen wie möglich.

Dieses Vorgehen (Algorithmen in DB laufen lassen) bietet sich an, da die für ein Lager verwalteten Daten einen erheblichen Umfang annehmen können. Wird auf ein DBMS verzichtet, müsste eine entsprechend effiziente Datenhaltung manuell implementiert werden.

## 5.2 Logische Lagerverwaltung

Aufgabe der logischen Lagerverwaltung ist die Abstimmung der Ein- und Auslagerungsvorgänge sowie die Durchführung der Optimierungsvorgänge auf Basis der zuvor beschriebenen Datenbank. Softwareseitig liegt die logische Lagerverwaltung in Form des Lageragenten (engl. *Warehouse Agent*) vor. Dieser handelt im Auftrag anderer Agenten und sorgt für die Umsetzung der Bestimmung auszulagernder Güter anhand von Bestellungen, für die Einlagerung von Gütern auf optimalen Lagerplätzen und für die Hardwareresteuerung.

Da das in Kapitel 3 vorgestellte Agentensystem auf einem modularen Ansatz basiert, werden Ein- und Auslagerungsaktionen auf reguläre Transport- und Bestellaktionen abgebildet. Die Abwicklung einer Einlagerung ursächlich einer Transportanfrage ist im Agentensystem fest integriert, spricht jedoch erst an, sobald auch die Lagererweiterung konfiguriert wurde. Dieser Vorgang ist für den Initiator der Aktion transparent. Zwar ist davon



auszugehen, dass sich dieser bewusst ist, dass er ein Gut zum Zwecke der Aufbewahrung innerhalb der Fabrikumgebung versendet, jedoch hat er sich zu keinem Zeitpunkt mit den Einlagerungsvorgängen selbst zu befassen. Hierbei ist lediglich zu beachten, dass eine lagerfähige Komponente versucht, eine Einlagerung vorzunehmen, wenn in der Transportanforderung zwar der Ziel-Agent, jedoch kein Ziel-Haltepunkt angegeben ist. Da dem Agenten somit die Wahl eines noch freien Haltepunkts überlassen wird, ist es vorgesehen, dass dann eine Lagerzone angefahren wird, die noch freie Kapazität besitzt und den Anforderungen des Materials genügt. Ähnlich verhält es sich mit Auslagerungen. Sie resultieren entweder aus einer allgemeinen Materialanfrage oder auf Anforderung eines durch einen bestimmten Bezeichner definierten Guts. Alle im System beteiligten Komponenten können auf eine solche Anfrage mit einem Angebot antworten, falls sie im Besitz des angeforderten Materials sind oder dieses beschaffen können. In Konkurrenz zu anderen Komponenten, die Güter erst über den Einkauf beschaffen oder durch Materialumwandlung erzeugen müssten, könnte sich das Lager durchsetzen und ein günstigeres Angebot abgeben.

### 5.2.1 StorageFace

Auf Grund des modularen Systems ist der Lageragent nicht als eigenständige Komponente vorhanden, sondern wird als zusätzliches Gesicht `StorageFace` des Basisagenten konfiguriert und gliedert sich damit wie folgt in die Konfigurationsdatei des Basisagenten ein:

```
<face class="storage" config="file.xml" />
```

Per Definition muss daher die Klasse `de.i2ar.agents.faces.StorageFace` existieren, damit dieses Gesicht beim Start des Basisagenten geladen werden kann. Diese Klasse implementiert das Interface `de.i2ar.agents.faces.LK3SAgentLoadableFace` mit dessen Methoden `load`, `getConfig` und `unload`. Dies ist in Abbildung 5.7 als UML-Klassendiagramm dargestellt. Zudem handelt es sich bei `StorageFace` um eine `CyclicBehaviour`. Daraus folgt, dass die Methode `action` automatisch durch das JADE-Agentensystem immer wieder neu aufgerufen wird, sobald ihre Ausführung terminiert ist.

**load** Diese Methode wird bei der Initialisierung des `StorageFace` vom Basisagenten aufgerufen, wie bereits in Abschnitt 3.3 erläutert wurde. Zunächst wird durch die Methode geprüft, ob eine Konfigurationsdatei angegeben wurde. In ihr sind verschiedene Parameter zur Datenbankbindung, die Gewichtungsfaktoren der einzelnen Optimierungsverfahren sowie Zuordnungen der Lagerzugänge zu allgemeingültigen Links der Fertigungsanlage beschrieben. Nähere Erläuterungen werden in Abschnitt 6.1 gegeben. Diese Parameter werden im Laufe der Ausführung von `load` in ein Objekt des Typs `StorageConfig` geladen, das im Paket `de.i2ar.agents.faces.storage` definiert ist. Dazu steht in der Klasse `StorageConfig` eine statische Methode `loadFromFile` zur Verfügung, die den Dateinamen der Konfigurationsdatei als Argument erwartet. Mit Hilfe der Klasse `DOMHelper`

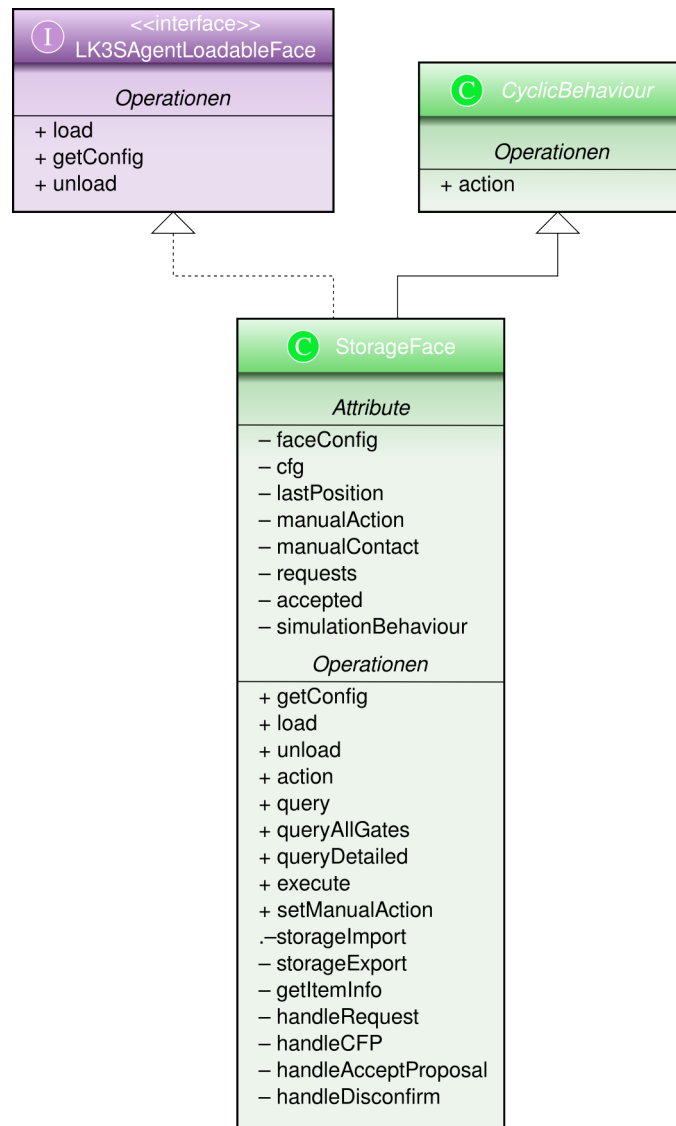


Abbildung 5.7: UML-Klassendiagramm des StorageFace.

des Pakets `de.i2ar.operators.common.misc` wird die angegebene XML-Datei geladen und anhand der vordefinierten Elemente ausgewertet. Treten dabei Fehler auf, so wird eine Exception vom Typ `DocumentProcessingException` ausgelöst, die bis zum Aufrufer der `load`-Methode weitergeleitet wird.

Im Anschluss wird geprüft, ob alle konfigurierten Links auch den übrigen Komponenten des Agenten (`LinksFace`) bekannt sind und ob sie jeweils sowohl als ein- wie auch ausgehend markiert sind, da diese Unterscheidung in der Lagerkonfiguration nicht unterstützt wird.

Zudem wird geprüft, ob alle bestehenden Abhängigkeiten erfüllt sind. Hier gilt es insbesondere zu beachten, dass die Lageroperationen einer Zu- und Abführung der Güter

bedürfen, also einen Anschluss des Lagers bzw. der Zugänge an das Transportsystem der Fertigungsanlage. Zunächst wird eine allgemeine Verwaltung für Links benötigt, die über das Gesicht `LinksFace` bereitgestellt wird. Dieses besitzt keine Abhängigkeiten, so dass es als erstes geladen werden kann. Lediglich die Konfiguration muss entsprechend dem Lager angepasst sein bzw. umgekehrt: Alle Links, die für das Lager als Zuordnung zu Lagerzugängen definiert wurden, müssen auch dem `LinksFace` bekannt sein. Neben der Verwaltung von Nachbar-Agenten über das `LinksFace` ist auch die Anbindung an das Transportsystem mit Hilfe des Gesichts `TransportFace` nötig. Es verwendet ebenfalls Funktionen des Gesichts `LinksFace` und hat somit in der Initialisierungsreihenfolge hinter diesem zu stehen. Wie bereits erwähnt, wird die durch das `TransportFace` bereits zur Verfügung gestellte Funktionalität genutzt, um Güter transparent in das Lager einzubringen. Auch nach erfolgtem Export wird es genutzt, um ein Gut seinem Bestimmungsort zuzuführen. Der Export bedarf jedoch, wie ebenfalls oben bereits erwähnt, auch der Bestellfunktionalität des Agentensystems. Dies ist durch das Gesicht `OrdersFace` realisiert. Es ist also notwendig, auch dieses zu Laden, bevor das `StorageFace` geladen wird. Da es vom `TransportFace` und vom `LinksFace` abhängig ist, kann es erst jetzt geladen werden. Letztendlich fehlt noch das Gesicht `OperatorsFace`, ohne das keine Ausführung der Lagervorgänge auf der Hardware realisiert werden könnte. Das `OperatorsFace` ist von keinem der übrigen Gesichter abhängig, sodass es an beliebiger Stelle vor dem `StorageFace` geladen werden kann.

Am Ende der Methode `load` wird die URI erzeugt, die zum Herstellen der Datenbankverbindung verantwortlich zeichnet. Die notwendigen Benutzerdaten werden dem zuvor erzeugten Konfigurationsobjekt entnommen. Die Datenbankverbindung wird zu diesem Zeitpunkt noch nicht hergestellt, jedoch bereits ein Objekt vom Typ `Database` erzeugt und einem privaten Klassen-Member zugewiesen, sodass im späteren Verlauf für alle Zugriffe ein Null-Pointer ausgeschlossen ist.

**getConfig** Zur Abfrage der Konfiguration durch eine andere Komponente kann diese Methode aufgerufen werden. Sie liefert ein `Object` zurück, dass vom Aufrufer in ein entsprechendes `StorageConfig`-Objekt transformiert werden muss. Dies setzt voraus, dass sich der Aufrufer bewusst ist, welcher Konfiguration genau er sich bedient.

**unload** Falls beispielsweise beim Beenden des Agenten für ein Gesicht Aktionen durchgeführt werden müssen, so können diese im Rahmen der Methode `unload` implementiert werden.

**action** Diese Methode wird von der `CyclicBehaviour` abstrakt deklariert und hier in Klasse `StorageFace` überschrieben. Zunächst wird bei erstmaligem Aufruf eine weitere Initialisierung durchgeführt: Eine neue, zyklische `Behaviour` wird eingerichtet. Sie stellt einen Server-Socket bereit, an den nach Verbindungsaufbau Lagerbefehle gesandt werden

können (siehe Abschnitt 5.2.3). Bei jedem Aufruf von `action` wird eine neue Nachricht abgefragt, falls vorhanden, im Anschluss dekodiert und die entsprechende Aktion durch Aufruf der Methode `execute` ausgeführt.

**storageImport** Neben den bereits vorgestellten Methoden, die alle von Interfaces bzw. abstrakten Klassen übernommen und implementiert wurden, existieren drei sehr wichtige Methoden: `storageImport`, `storageExport`, `query` und `execute`. Zunächst soll auf die Erste eingegangen werden.

Die Methode `storageImport(Storage storage, Item item)` erwartet als Argumente einerseits ein `Storage`-Objekt, das die gesamte Anfrage mit allen Parametern enthält, und andererseits das bereits aus genannter Anfrage dekodierte `Item`-Objekt. Dieses darf

1. ein Gut anhand einer `itemId` ohne Angabe von Material (über `contains`, siehe Abb. 5.8) oder
2. ein Material als `Item`-Objekt eingebettet als enthaltenes Material (`contains`) eines übergeordneten Items jedoch ohne Nennung einer `itemId`. Das Material wird über die bereits erwähnten Felder `typeId`, `width` und `colour` beschrieben.

Es werden als erstes Referenzen auf die Instanzen des `LinksFace` und des `OperatorsFace` zwischengespeichert, um sicherzustellen, dass diese in der Zwischenzeit nicht entladen wurden. Im Anschluss wird mit Hilfe des `LinksFace` Linkverwaltung sichergestellt, dass der in `storage` angegebene Übergabepunkt (Link) auch für den Warehouse Agent konfiguriert wurde. Anhand dieses Links wird nun der entsprechende, interne Haltepunkt (Stop) ermittelt. Nach seiner erfolgreichen Reservierung und Belegung mit dem ebenfalls übergebenen `item` werden die zu dem Übergabepunkt gehörende Lagerzone und der zugehörige Lagerzugang ermittelt und für den späteren Zugriff vorgehalten. Dies ist eine besondere Beschränkung der verwendeten Testumgebung. Auf Grund der Gesamtkomplexität dieser Arbeit wurde davon abgesehen, andere Lösungen wie z. B. eine Vorlagerzonensteuerung zu implementieren.

Nun ist es notwendig, eine Verbindung zur Datenbank herzustellen. Das Datenbank-Objekt `db` wurde wie bereits erwähnt schon mit der Initialisierung des `storageFace` erzeugt und die URI (`dbUri`) zum Verbindungsaufbau zusammengestellt. Da `db` und `dbUri` Klassenfelder sind, kann direkt auf sie zugegriffen werden. Um Benchmarking zu erlauben, wird an dieser Stelle der erste Schritt zu einer Zeitmessung durchgeführt: Die Zwischenspeicherung der aktuellen Systemzeit in Millisekunden. Dabei ist die Auflösung abhängig von der verwendeten Hardware und dem verwendeten Betriebssystem. Nach Starten der Zeitmessung kann die Suche nach Lagerplätzen beginnen, die für die Einlagerung des vorgegebenen Guts `item` geeignet sind. Hierzu wird die Methode `import_findCellsForImport` der Klasse `Database` (s. Abschnitt 5.2.2) aufgerufen. Sie führt die Suche anhand der übergebenen Parameter auf dem derzeitigen Bestand der verbundenen Datenbank aus:

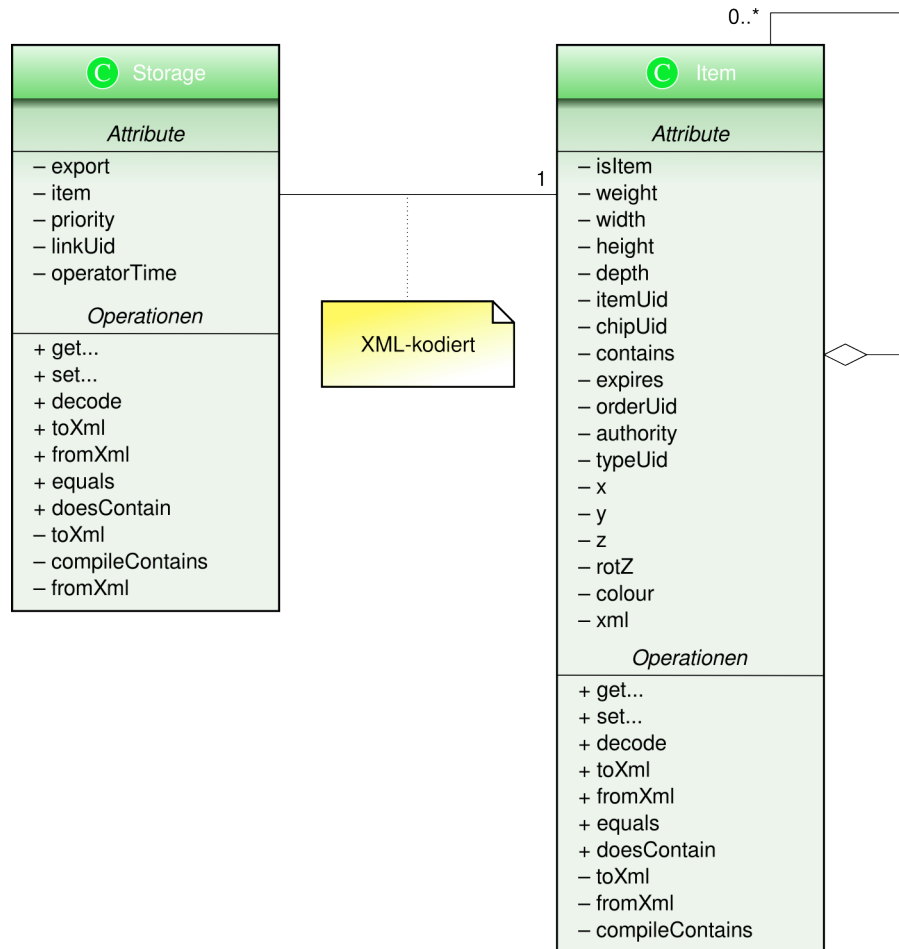


Abbildung 5.8: UML-Klassendiagramm für Storage und Item.

**zoneUid** da die Lagerzone durch den Übergabepunkt definiert ist, kann die Lagerplatzsuche auf diese Lagerzone beschränkt werden,

**gateUid** dient zur Vorgabe des zu verwendenden Lagerzugangs,

**item** beschreibt das einzulagernde Gut (s. Abb. 5.8),

**daysBack** verweist auf die Anzahl der Tage, die vom aktuellen Zeitpunkt an rückwärts für das Optimierungsverfahren in Betracht gezogen werden sollen, um eine statistisch korrekte Positionierung zu erhalten,

**weightGeometry** ist der Gewichtungsfaktor für die Optimierung der geometrischen Ausbeutung der Lagerplätze,

**weightFillDegree** ist der Gewichtungsfaktor für die Vermeidung vieler gering belegter Lagerplätze mit höherer Kapazität als eins,

**weightSameMaterial** ist der Gewichtungsfaktor für die Lagerplatzwahl unter Berücksichtigung teilbelegter Lagerplätze mit gleichem (eventuell keinem) Verfallsdatum der betrachteten Güter,

**weightStatisticalCellPosition** ist der Gewichtungsfaktor für die Bewertung aller potentiellen Lagerplätze im Vergleich zur statistisch berechneten, idealen Lagerposition des einzulagernden Guts.

Als Rückgabewert liefert die Methode eine Liste von Object-Tupeln. Diese Tupel bestehen aus zwei Elementen, deren erstes ein Objekt vom Typ `String` ist, und die UID eines Lagerplatzes enthält. Das zweite Element entspricht der ermittelten Bewertung respektive Metrik des Lagerplatzes. Sie wird jedoch nicht ausgewertet, da die Liste bereits aufsteigend nach ihr sortiert ist und direkt das erste Listenelement als Grundlage zur Einlagerung verwendet werden kann. Der von `import_findCellsForImport` zurückgegebene Wert bei keinen gefundenen, geeigneten Lagerplätzen ist `null`. Entsprechende Prüfungen sind an dieser Stelle nötig. Zuvor wird jedoch die Zeitmessung gestoppt. Das heißt, es wird die Differenz zwischen aktueller Systemzeit in Millisekunden und der zuvor festgehaltenen Startzeit gebildet. Anschließend werden wiederum unter Mitwirken der Datenbank über die Methode `getCellForCellUid` der Klasse `Database` die Stammdaten des durch die UID beschriebenen Lagerplatzes ermittelt und in einem Objekt des Typs `Cell` zurückgegeben. Sie enthalten die Koordinaten des Lagerplatzes, die nun zur Zusammenstellung eines Kommandos für einen Operator genutzt werden können. Dazu wird zunächst ein Instanz `jpe` der Klasse `JobProgrammeExecution` erzeugt, mit der dem Operator die Ausführung eines vordefinierten, jedoch parametrierbaren Programms signalisiert wird. Durch Setzen des Programmnamens `import` wird die Aufgabe des Operators definiert. Die Parameter werden mit Hilfe der Methode `addParam` der erzeugten Instanz bekanntgegeben. Vor Einholung von Angeboten durch mehrere verbundene Operatoren, wird ein Objekt der Klasse `Jobs` erzeugt und mit einem Objekt der Klasse `Job` belegt, das als einzige Kindelement die zuvor definierte Programmausführung beinhaltet. Die Methode `getBestOperator` des `OperatorsFace` ermittelt nun anhand eines Anfrage-Angebot Verfahrens unter allen mit dem Warehouse Agent verbundenen Operatoren, ob sie in der Lage sind, die `Jobs` auszuführen und welche Kosten jeweils individuell damit verbunden sind. Spätestens nach dem hardkodierten Timeout des `OperatorsFace` wird die Methode als Rückgabewert eine Liste liefern, die aus Tupeln aus Objekten der Typen `LK3SOperator` und `EnergyEfficiencyInformation` bestehen. Sie ist bereits sortiert nach dem zeitlich günstigsten Operator. Wurde mit dem Rückgabewert `null` signalisiert, dass kein Operator die `Jobs` ausführen kann, so muss die Einlagerung abgebrochen werden. Im Erfolgsfall kann einfach das erste Element der Liste entnommen werden und die `Jobs` können direkt an diesen einen Operator geleitet werden, in dem die Methode `execute` des `OperatorsFace` aufgerufen wird. Ihr Rückgabewert gibt Aufschluss über den Erfolg der Ausführung. Ist er `null` oder enthält nicht genau eine Ausführungsstatistik, so kann nicht ausgeschlossen werden, dass ein Fehler oder Problem im Zusammenhang mit der Hardware aufgetreten und das Eingreifen eines menschlichen Bedieners nötig ist. Daher wird der Wert des Klassenfeldes `manualContact` auf `true` gesetzt, um für weitere Ein- und Auslagerungsanfragen zu signalisieren, dass bis auf Weiteres kein automatisierter Lagerbetrieb möglich ist. Dieser Wert wird am Ende der Methode `action` des `StorageFace` geprüft, bevor es zum Aufruf der Anfrageverarbeitung kommt. War die Einlagerung hingegen erfolgreich, so kann über die Klasse `Database` eine Aktualisierung

der Lager-Datenbank anhand der zuvor abgefragten Stammdaten des Lagerplatzes (Klasse `Cell`) und des bekannten Objekts des Typs `Item` erfolgen. Hierzu wird die Methode `import_placeItem` mit entsprechenden Argumenten aufgerufen.

Da der physikalische Vorgang nun abgeschlossen ist, wird die Halteposition wieder für neue Güter freigegeben. Zudem wird der angefahrene Lagerplatz festgehalten, um bei der Auslagerung Anhaltspunkte für die Optimierung des Doppelspielverfahrens zu besitzen. Schlussendlich sollte die Ausführungsstatistik Auskunft über die benötigte Ausführungsdauer des physikalischen Lagerzugriffs geben. Ist dies nicht der Fall, so wird zwar ein Fehler gemeldet, jedoch ist dabei nicht die Konsistenz von logischem und physikalischem Lager gefährdet. Konnte die Ausführungsdauer der Statistik entnommen werden, so wird ein Rückgabewert als `Object`-Array gebildet, bestehend aus:

1. UID des Lagerplatzes, in dem eingelagert wurde,
2. Metrik des Lagerplatzes in Bezug auf die Anfrageparameter,
3. X-Position des Lagerplatzes,
4. Y-Position des Lagerplatzes,
5. Z-Position des Lagerplatzes,
6. Ausführungsdauer des Optimierungsalgorithmus sowie
7. physikalische Ausführungsdauer des Einlagerungsvorgangs.

**storageExport** Auf Grund der unterschiedlichen Optimierungsansätze bei Ein- und Auslagerungen müssen diese getrennt betrachtet und implementiert werden. Für den Export wird anfänglich ebenfalls geprüft, ob die in Abhängigkeit stehenden, anderen Agentengesichter geladen wurden. Anhand des in der Anfrage enthaltenen Übergabelinks – hierdurch ist gleichzeitig die Lagerzone definiert, die für die Auslagerung betrachtet wird – wird nun die UID des Lagerzugangs ermittelt, die mit der Link-UID per Konfiguration (s. Listing A.5) verknüpft ist. Hiernach kann ein neues Objekt der Klasse `DatabaseExportRequest` instanziiert und dessen Felder `authority`, `gateUid` und `material` mit den bereits vorhandenen Werten belegt werden. An dieser Stelle muss eine Fallunterscheidung erfolgen: Ist das angefragt Gut ein Material, also nicht an eine Item-UID gebunden, oder ist es eine konkrete Anfrage bezüglich eines Gutes mit einer festen Item-UID.

Im ersten Fall (Material) wird zunächst eine Machbarkeitsanalyse durchgeführt, es wird also geprüft, ob das Material überhaupt im Lager vorrätig ist. Zu dieser Prüfung dient die Methode `checkFeasability` der Klasse `Database`. Sie wird mit dem zuvor erzeugten `DatabaseExportRequest`-Objekt als Argument aufgerufen und liefert einen booleschen Rückgabewert, der mit `true` die Machbarkeit der Anfrage bestätigt. Damit kann diese in der Datenbank gespeichert werden. Der darauf folgende Aufruf der `Database`-Methode `export_findItemsForMaterial` führt schließlich die Suche nach dem gesuchten Gut auf der Datenbank aus. Dabei besitzen die Argumente die folgenden Bedeutungen:

**weightFifo** ist der Gewichtungsfaktor für die Beachtung der FIFO-Strategie bei Auslagerungen,

**weightDeadlines** ist der Gewichtungsfaktor zur Unterscheidung der Dringlichkeiten einzelner Auslagerungsanfragen in Abhängigkeit ihrer Deadline,

**weightDualCommand** ist der Gewichtungsfaktor, um die Lagerplätze hinsichtlich der Doppelspielstrategie zu bewerten und

**weightMaterial** ist der Gewichtungsfaktor, der die Übereinstimmung des in Lagerplätzen vorhandenen Materials in Bezug auf die angefragten Materialien berücksichtigt.

**zoneUid** Da die Lagerzone durch den Übergabepunkt bzw. den Lagerzugang definiert ist, kann die Materialsuche auf diese Lagerzone beschränkt werden.

**dualCommandX** ist die zuletzt angefahrne Position des RBG in X-Richtung,

**dualCommandY** ist dasselbe in Y-Richtung und

**dualCommandZ** in Z-Richtung. Sie werden bei der Wahl von Doppelspielen berücksichtigt.

**specificRequestOnly** Ist dieses Argument nicht `null`, so spezifiziert es eine ID einer bereits in der Datenbank vorliegenden Anfrage. In diesem Fall wird die Auslagerung sofort und ohne Prüfung veranlasst, ob (andere) Doppelspiele machbar sind.

**excludelItems** Falls über das `StorageFace` einige Güter anhand ihrer Item-UID reserviert worden sind, so können diese über eine Liste vor einer Auslagerung geschützt werden.

Der Rückgabewert der Methode `export_findItemsForMaterial` ist eine Liste mit Item-UIDs der Güter, die für die Auslagerung in Frage kommen, der Metrik des Lagerplatzes, in dem sich das Gut befindet, und der ID der Anfrage, für die dieser Lagerplatz ausgewählt wurde. Diese Werte sind in einem `Object`-Array gespeichert. Die ID einer Anfrage muss mit jedem Listeneintrag gespeichert werden, da bei Einsatz der Doppelspielstrategie dem `StorageFace` an dieser Stelle nicht klar sein muss, welche Anfrage zur Ausführung kommt. Dies kann nur durch explizite Spezifikation der Anfrage-ID beim Aufruf von `export_findItemsForMaterial` erreicht werden.

War kein Material anhand seiner Eigenschaften, sondern ein Gut anhand seiner eindeutigen Item-UID angefragt, so besteht die Machbarkeitsanalyse in der Abfrage der Eigenschaften des Lagerplatzes, in dem sich das Gut befindet. Wenn kein entsprechender Lagerplatz gefunden wurde, wird eine `Exception` ausgelöst, um dies dem Aufrufer zu signalisieren.

Damit die Hardwaresteuerung im Erfolgsfall später für beide Fälle gleich geschehen kann, wird für das eine auszulagernde Gut ebenfalls eine Liste erzeugt, die jedoch nur die einzige korrespondierende Item-UID des Guts enthält. Damit kann im Anschluss, unabhängig der Anfrage, der Lagerplatz ermittelt werden, in dem sich das erste gefundene Gut befindet. Dazu wird die Methode `getCellForItemUid` der Klasse `Database` aufgerufen. Nun



wird ein `Jobs`-Objekt für die Ermittlung des am besten für diese Auslagerung geeigneten Operators erzeugt und dann als Argument in den Aufruf von `getBestOperator` des `OperatorFace` gegeben. Handelte es sich nur um eine Anfrage (das Argument `isQuery` ist `true`), dann wird einfach die vom Operator zurückgegebene Ausführungszeit in die lokale Variable `timeLong` übernommen. War es keine reine Anfrage, so ist es notwendig, zunächst nähere Informationen über den der Lagerzone zugehörigen Zugang abzurufen (Methode `getGateForGateUid` der Klasse `Database`). Zudem wird der Haltepunkt der Übergabe von Lager zu Transportsystem mit dem auszulagernden Gut belegt, in dem `TransportManager.lockStop` aufgerufen wird. Die Ausführung der Lageroperation erfolgt schließlich durch Ausführen von `execute` des `OperatorsFace` mit dem zuvor erstellten `Jobs`-Objekt und dem ermittelten Operator. Durch Aufruf von `export_removeExportedItem` der Klasse `Database` wird nach erfolgter Auslagerung sowohl die Information zum ausgelagerten Gut wie auch die Auslagerungsanfrage aus der Datenbank gelöscht respektive als ausgeführt markiert. Für weitere Doppelspiele muss nun die aktuelle Position des RBG im Lager gespeichert werden. Den Ergebnissen der Operator-Ausführung wird als letztes der Wert des erwarteten Parameters `time` entnommen und wie bei einer reinen Anfrage in der lokalen Variable `timeLong` zwischengespeichert.

Abschließend wird, ob Anfrage oder Ausführung, ein `Object`-Array als Rückgabewert der Methode `storageExport` bestehend aus folgenden Komponenten gebildet:

1. Item-UID des Guts, das ausgelagert wurde
2. Metrik des Lagerplatzes in Bezug auf die Anfrageparameter,
3. X-Position des Lagerplatzes,
4. Y-Position des Lagerplatzes,
5. Z-Position des Lagerplatzes,
6. Ausführungsdauer des Optimierungsalgorithmus sowie
7. physikalische Ausführungsdauer des Auslagerungsvorgangs.

**query** Die beiden Methoden `storageImport` und `storageExport` sind als privat ausgezeichnet. Um also von einem anderen Agentengesicht auf die Lagerfunktionalität zugreifen zu können, existiert für reine Anfragen die `query`-Methode. Sie erwartet als Argument ein `Storage`-Objekt, das Angaben enthält, ob es eine Ein- oder Auslagerungsanfrage ist, welches Gut betroffen ist und welche Priorität der Vorgang und das Material besitzen. Zudem muss eine Link-UID angegeben werden, die den Link und damit auch den Lagerzugang beschreibt, über den der Vorgang abgewickelt werden soll. Die Methode `query` ruft dann entsprechend Ein- oder Auslagerung die private Methode `storageImport` oder `storageExport` auf und gibt mit `true` als booleschen Rückgabewert an, dass die Anfrage erfolgreich war oder mit `false`, dass sie fehlschlug. Wenn detailliertere Informationen zu den Ergebnissen der Anfrage gewünscht sind, kann stattdessen die Methode `queryDetailed` aufgerufen werden. Ihr Rückgabewert entspricht dem internen

Rückgabewert der aufgerufenen Storage-Methode. Durch die Angabe des Links für den Lagerzugang ist bereits eine Lagerzone vordefiniert worden. Wenn jedoch eine Anfrage über alle Lagerzonen hinweg geschehen soll, so kann dazu die Methode `queryAllGates` genutzt werden. Auch sie erwartet ein `Storage`-Objekt als Argument, ruft jedoch die `queryDetailed`-Methode für jeden konfigurierten Lagerzugang auf. Jedes Ergebnis der Prüfung wird einem `EfficiencyDecisionMaker` übergeben, der am Ende der Methode zum Sortieren der Ergebnisse veranlasst wird. Das erste Element der Sortierung enthält dann Details der zu bevorzugenden Lagerzone. Die Methode gibt dann u.a. die UID des Links der bevorzugten Lagerzone zurück.

**execute** Neben reinen Anfragen muss auch die Ausführung von Lageroperationen für Nutzer des `StorageFace` möglich sein. Diese rufen hierzu die Methode `execute` mit einem `Storage`-Objekt als Argument auf. Je nachdem, ob Ein- oder Auslagerung gefordert ist, wird die entsprechende private Methode aufgerufen und die in der Anfrage als enthaltene Gut-/Materialbeschreibung durch die Methode `Item.fromXml` geparkt und als `Item`-Objekt übergeben. Der Rückgabewert ist die interne Ausgabe der jeweils aufgerufenen Methode.

## 5.2.2 Database

Die Klasse `Database` ist zuständig für die gesamte Kommunikation mit der Lagerdatenbank. In ihr sind auch die Optimierungsalgorithmen implementiert. Neben den Methoden `openDatabase` und `closeDatabase`, die eine Verbindung zur Datenbank herstellen bzw. abbauen, existieren auch Methoden, die Informationen zu einer gegebenen Item-UID oder einer Gate-UID (Lagerzugang) sowie zu einer Cell-UID (Lagerplatz) aus der Datenbank abrufen und als Java-Objekte bereitstellen. Da ihre Funktionen einen einfachen, erwartungsgemäßen Aufbau besitzen, sollen sie nicht weiter erläutert werden. Von größerem Interesse hingegen sind die Methoden `import_findCellsForImport` und `export_findItemsForMaterial`. Die Methode `import_placeItem` besitzt Parameter für ein Gut, eine Lagerzonen-UID und eine Lagerplatz-UID. Sie wird die Informationen des übergebenen `Item`-Objekts einschließlich der enthaltenen Materialien in die Datenbank einfügen. `export_removeExportedItem` hingegen wird ein Gut aus der Datenbank entfernen, falls der Methode eine entsprechende Item-UID übergeben wird und es wird eine gespeicherte Anfrage entfernen, falls eine Anfrage-ID in einem anderen Argument übergeben wurde. Durch Aufruf von `export_placeRequest` kann ein zuvor erstelltes `DatabaseExportRequest`-Objekt entsprechend in der Datenbank abgelegt und bei folgenden Auslagerungsanforderungen berücksichtigt werden. Die dem Datenbank-eintrag automatisch zugeordnete fortlaufende ID wird im Anschluss abgefragt und im übergebenen `DatabaseExportRequest`-Objekt vermerkt, sodass der Aufrufer der Methode Zugriff hierauf hat und beispielsweise im Anschluss direkt die Auslagerung des dadurch spezifizierten Materials veranlassen kann. Für die Prüfung, ob eine Auslagerungsanfrage überhaupt auf dem aktuellen Lagerbestand ausgeführt werden kann, ist die

Methode `export_checkFeasability` zu verwenden. Sie prüft, ob das angeforderte Material in Eigenschaft und Stückzahl im Lager vorhanden ist.

**`import_findCellsForImport`** ist eine Methode, die für das Finden und Bewerten freier Zellen für ein als Argument übergebenes Gut (Item-Objekt) in einer gegebenen Lagerzone und für einen gegebenen Lagerzugang zuständig ist. Weitere Argumente parametrieren die Optimierungsstrategien. Aus abstrakter Sicht kann die Methode auf die Ausführung eines SQL-Ausdrucks reduziert werden, der sich aus der Kombination mit `UNION ALL` von Unterausdrücken für jede Optimierungsstrategie zusammensetzt. Der äußerste SQL-Ausdruck ist in Listing 5.4 mit Platzhaltern für die Strategien abgedruckt.

Listing 5.4: Äußerster SQL-Ausdruck zur Einlagerungsoptimierung.

```
1 SELECT celluid, SUM(metric) AS metric FROM (  
2   -- Ausdruck für äußere Randbedingungen  
3 UNION ALL  
4   -- Ausdruck für Anbruchlagereinheiten  
5 UNION ALL  
6   -- Ausdruck für Lagerplätze mit gleichem Gut  
7 UNION ALL  
8   -- Ausdruck für Abfragehäufigkeit  
9 ) AS sub1 GROUP BY celluid ORDER BY metric ASC;
```

Die Ausdrücke für die Platzhalter werden von jeweils einer eigenen Methode erzeugt. Die einzige Abhängigkeit, die sie untereinander besitzen, liegt in den Ergebnissen der Optimierungsstrategie der Anbruchlagereinheiten (s. Abs. 4.2.3). Sie werden auch bei allen anderen Optimierungsstrategien als Grundlage der zu durchsuchenden Lagerstrukturen herangezogen, da nur diejenigen Lagerplätze in das Ergebnis eingehen, die noch nicht voll belegt sind und genau diese werden von der Anbruchlagereinheiten-Strategie ausgeschlossen. Es ist dabei auch zu bemerken, dass hier bereits sichergestellt wird, dass nur diejenigen Lagerplätze in das Ergebnis eingehen, die die Mindestanforderungen für die Einlagerung hinsichtlich Geometrie und Belastbarkeit erfüllen (siehe auch Abb. 5.10). Daher können die Ergebnisse aller Optimierungsstrategien einfach mit `UNION ALL` aneinander gekettet werden.

**`export_findItemsForMaterial`** Während bei der Einlagerung nur ein einziges Ausschlusskriterium (Geometrie und Belastbarkeit) existiert und die Ergebnisse der einzelnen Strategien einfach zusammengefasst werden können, sind die Strategien respektive die Datenbankabfragen bei Auslagerungen stärker miteinander verwoben. Dies resultiert z. B. aus der erhöhten Nutzung der `MIN`- und `MAX`-Funktion zur Bestimmung von Wertebereichen. Da bei diesen Wertebereichen zum Zwecke der Normierung jedoch nur über deren Elemente normiert werden darf, sind die bisherigen potentiellen Lagerplätze als Input (Unterabfrage) des Normierungsausdrucks notwendig. Listing 5.5 und 5.10 zeigen die vergleichsweise starke Verschachtelung. Trotzdem können beliebige Strategien

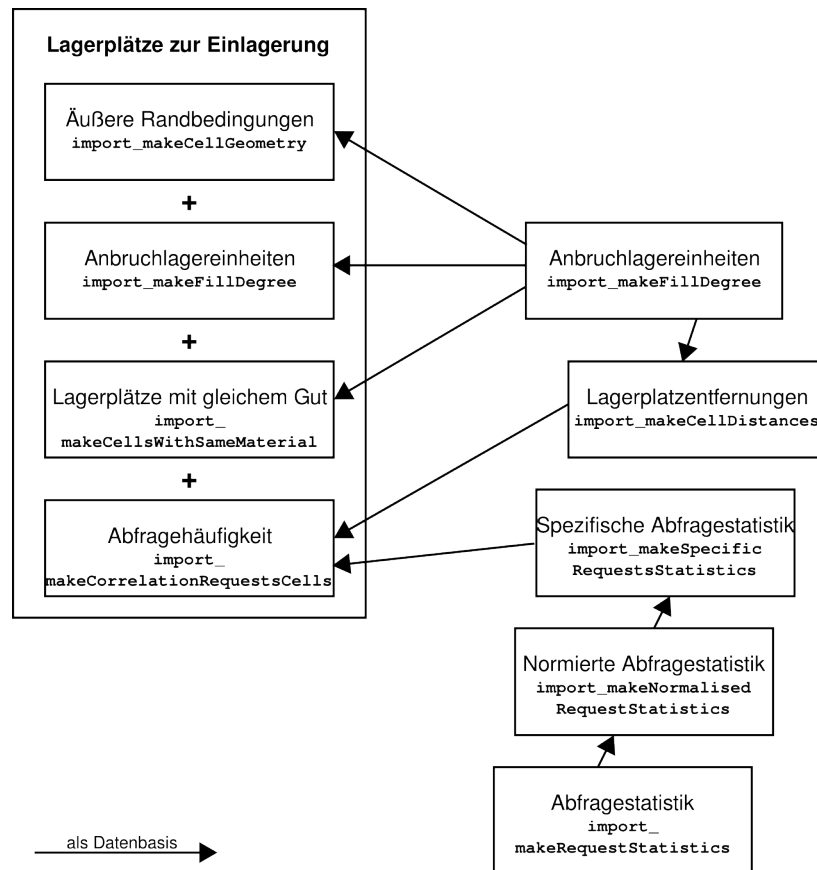


Abbildung 5.9: Grafische Darstellung der Verknüpfung aller Einlagerungsstrategien.

ausgelassen werden, da jeder SQL-Ausdruck die gleichen Ergebnisspalten (Item-UID des Guts, Metrik und korrespondierende Anfrage-ID), besitzt und die Gesamtmetrik dennoch als Summe aller einzelnen Metriken gebildet wird. Im Gegenteil zu der Einlagerungsoptimierung wird hier keine UID der Lagerplätze abgerufen, sondern die Item-UID jedes potentiellen Guts, sodass der Anwender vor der Auslagerung zunächst die zum Gut gehörenden Lagerplatzdaten abrufen muss, um den Warehouse Operator mit den nötigen Koordinaten speisen zu können.

Listing 5.5: Äußerster SQL-Ausdruck zur Auslagerungsoptimierung.

```

1 -- Ausdruck für Doppelspiel
2 SELECT ... FROM (
3   -- Ausdruck für passendes Material
4   SELECT ... FROM (
5     -- Ausdruck für Deadlines
6     SELECT ... FROM (
7       -- Ausdruck für FIFO-Strategie
8       SELECT ... FROM (
9         -- Ausdruck für Material der Anfrage entsprechend
10        SELECT ... FROM ( ... )

```

```

11      )
12    )
13  )
14 )

```

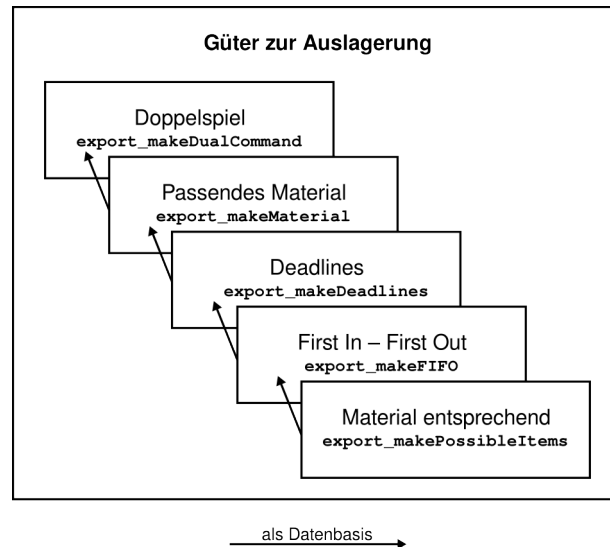


Abbildung 5.10: Grafische Darstellung der Verknüpfung aller Auslagerungsstrategien.

### 5.2.3 SimulationBehaviour

Die `SimulationBehaviour` wird durch das `StorageFace` als `threaded Behaviour` erzeugt, sobald es das erste Mal vom Agenten als JADE-Behaviour aufgerufen wird (Methode `action`). Die zyklisch ausgeführte Methode `action` der `SimulationBehaviour` öffnet einen Serversocket auf dem durch das Element `simulation-port` in der `StorageFace`-Konfiguration angegebenen Port. Diese Angabe ist im Übrigen auch notwendig, damit die `SimulationBehaviour` überhaupt erzeugt wird.

Nach Erstellen des Sockets wird auf eingehende Verbindungen gewartet, die dann durch die Methode `handleClient` verarbeitet werden. Sie sorgt zunächst für eine Meldung `HELLO` an den verbundenen Client und wartet sodann auf eingehende Nachrichten. Das Kommunikationsprotokoll ist simpel gehalten. So existieren hierin nur zwei Kommandos, `IMPORT` und `EXPORT`, die durch ein Leerzeichen getrennt die Ausgabe der Methode `toXml` eines `Storage`-Objekts erwarten. Ist eine mit `IMPORT` oder `EXPORT` beginnende Nachricht eingegangen, wird die Methode `handleImportExport` aufgerufen. Sie extrahiert ein `Storage`-Objekt aus dem String und ein `Item`-Objekt aus dem im `Storage`-Objekt enthaltenen String.

Da zur Simulation die gleichen Algorithmen verwendet werden wie im produktiven Betrieb, muss an dieser Stelle für die Sperrung bzw. Freigabe von Haltepunkten je nach

Anfrage gesorgt werden. Soll eine Auslagerung stattfinden, muss der Haltepunkt, der das ausgelagerte Gut aufnehmen soll, bedingungslos freigegeben werden. Handelt es sich um eine Einlagerung, so muss dieser Haltepunkt zunächst bedingungslos mit dem einzulagernden Gut gesperrt werden. Andernfalls würden Fehler bei der Abarbeitung der Algorithmen in `storageImport` und `storageExport` auftreten.

Sowohl für Ein- wie auch Auslagerungsanforderungen wird die Methode `execute` des `StorageFace` mit dem zuvor extrahierten `Storage`-Objekt aufgerufen. Der Rückgabewert der Methode wird in einen String transformiert, der nun über den Socket an den Client gesendet wird. Dieser kann hieraus Informationen über die Rechendauer der Optimierungsalgorithmen sowie der verwendeten Lagerplätze gewinnen.

**Es ist unbedingt darauf zu achten, dass die Simulation der Lagerfunktionen niemals in einer Produktivumgebung durchgeführt wird. Zum einen wird die Datenbank mit sämtlichen Informationen zu gelagerten Gütern unbrauchbar und zum anderen kann es auf Grund der erzwungenen Sperrung und Freigabe von Haltepunkten zu Beschädigungen der Lagerhardware kommen. Idealerweise wird daher der Dummy-Warehouse-Expert eingesetzt, der keinerlei Zugriff auf die Lagerhardware besitzt. Er befindet sich im Java-Package `de.i2ar.warehouseop.experts.dummy.WarehouseExpert` und wird statt dem Warehouse Expert `de.i2ar.warehouseop.experts.hrw.WarehouseExpert` o.ä. in der Konfiguration des Warehouse Operators angegeben.**

## 5.3 Physikalische Lagerverwaltung

Für die physikalische Lagerverwaltung zeichnet ein Operator verantwortlich. Seine Aufgabe besteht darin, vom Agenten stammende Befehle zur Umsetzung von Lageroperationen in Fahrbefehle des RBG umzusetzen. Hierzu wurde mit der Java-Klasse `de.i2ar.warehouseop.ExtensionWarehouse` eine Operator Extension für den Operator Skeleton erstellt, der Warehouse Operator, die somit das Interface `IOperatorExtension` implementiert. Sämtliche Lageroperationen sind, abstrakt betrachtet, wie folgt reduzierbar:

- Referenzfahrt durchführen
- Gut aufnehmen und einlagern
- Gut auslagern und ablegen
- Gut auslagern und an anderer Stelle wieder einlagern

- Geschwindigkeitsregulierung zu Energiesparzwecken

Da das entwickelte LVS so unabhängig wie möglich von einer konkreten Lagerhardware sein soll, wurde für obige Operationen ein Java-Interface erstellt. Die eigentliche Steuerung der Lagerhardware übernimmt damit nicht der Warehouse Operator, sondern ein Warehouse Expert – eine Implementation des Interfaces –, der die Steuerung der Lagerhardware im Detail beherrscht. Dieser ist die einzige Komponente, die für jedes unterschiedliche Lager neu implementiert werden muss. Zu Verwaltungszwecken sind noch einige weitere Methoden notwendig, so z. B. zur Initialisierung des Warehouse Expert als Software. Welcher Warehouse Expert beim Start des Warehouse Operators geladen wird, ist in der Konfigurationsdatei des letzteren festgehalten. Der Warehouse Operator verarbeitet alle eingehenden Anfragen und Aufträge zunächst, bevor sie an den Warehouse Expert weitergeleitet werden. Hierzu sind die bekannten Funktionen des Interfaces `IOperatorExtension` implementiert (s. Abs. 3.2.1). Ist die Syntax der Anfragen korrekt, werden die extrahierten Parameter direkt bei der Ausführung der entsprechenden Methode des Warehouse Expert übergeben. Die Rückgabewerte der Methode werden anschließend wieder in ein Objekt der Klasse `JobQueryResults` bei einfachen Anfragen bzw. `JobExecutionResults` bei realen ausgeführten Aufträgen transformiert und dieses Objekt als Ergebnis der vom Operator Skeleton aufgerufenen Warehouse-Operator-Methode (Extension) zurückgegeben.

Im Folgenden sollen kurz die gestatteten Befehle bzw. Programme für den Warehouse Operator vorgestellt werden:

**inithw** Veranlasst die Durchführung einer Referenzfahrt auf der Lagerhardware. Parameter werden ignoriert. Es wird die Methode `initHW` des `WarehouseExpert` aufgerufen.

**reportzones** Der Warehouse Operator wird aufgefordert, die ihm durch den Warehouse Expert gegebenen Lagerzonen bekanntzugeben. Hierzu wird die Methode `reportZones` des `WarehouseExpert` aufgerufen.

**reportentrances** Über dieses Programm werden alle durch den Warehouse Expert definierten Lagereingänge bekanntgegeben. Dabei wird die Methode `reportEntrances` des `Warehouse Expert` aufgerufen.

**reportexits** Wie `reportEntrances`, jedoch bezogen auf Lagerausgänge. Aufgerufen wird `reportExits` des `Warehouse Experts`.

**import** Führt den Import eines Guts aus der Vorlagerzone durch. Erforderlich sind die Parameter `x`, `y` und `z`, die die Koordinaten des bei der Einlagerung anzufahrenden Lagerplatzes angeben sowie die Parameter `entrance` zur Definition des Lagereingangs und `zone` als Beschreibung der zu verwendenden Lagerzone als Unterscheidung bei mehreren Zonen und Operatoren (Zuordnung aller Werte nach Definition des Warehouse Expert). Die Hardwaresteuerung wird über die Methode `doImport` des `Warehouse Experts` veranlasst.

**export** Eine Auslagerung wird ausgeführt. Die Parameter entsprechend denen der Einlagerungsanforderung, jedoch wird mit `exit` statt `entrance` ein Lagerausgang spezifiziert. Es wird die Methode `doExport` des Warehouse Experts aufgerufen.

**relocate** Umlagerungen innerhalb des Lagers sind mit diesem Programm möglich. Dazu werden als Parameter `x1`, `y1`, `z1`, `x2`, `y2`, `z2` und `zone` erwartet, wobei die ersten drei genannten Parameter den Ursprungslagerplatz und die nächsten drei Parameter den Ziellagerplatz definieren. `zone` wird zur Zuordnung der Anfrage zu einer Lagerzone benötigt. Auch hier erfolgt die Zuordnung aller Werte anhand der Definition des Warehouse Experts. Aufgerufen wird die Methode `doRelocate` des Warehouse Experts.

Die Implementation des Warehouse Experts wurde für das HRL der Digitalen Fabrik der Hochschule Emden/Leer mit der Klasse `de.i2ar.warehouseop.experts.hrw.WarehouseExpert` durchgeführt. Bei der Initialisierung über die Methode `initExpert`, die im Interface `IWarehouseExpert` deklariert ist, wird zunächst eine Konfigurationsdatei anhand ihres als Argument übergebenen Namens geöffnet und ausgelesen. Sie enthält Angaben über die Verbindung zum zuständigen Stub. Da ein Stub nur Befehle ausführt, ist sicherzustellen, dass hier die Verbindung zum korrekten Stub angegeben wird. Nach Einlesen der Konfiguration wird versucht, die Verbindung zum Stub herzustellen und im Anschluss alle für die Lageroperationen benötigten symbolischen Bezeichner zu reservieren. Sie werden als *shared* reserviert, daher ist sicherzustellen – am besten physikalisch –, dass kein weiteres System Netzwerkzugriff auf den Stub besitzt. Es werden abschließend alle Sequenzen für die Ausführung der Lageroperationen im Stub definiert. Die im Interface `IWarehouseExpert` deklarierten Methoden lassen diese Sequenzen bei ihrem Aufruf durch den Stub ausführen, ggf. nach vorheriger Parametrierung z. B. für die Koordinaten bei Ein-, Aus- und Umlagerungsoperationen. Die Koordinaten werden mit den Lagerein- und ausgangsbezeichnungen sowie den Lagerzonenbezeichnungen bei Aufruf der Methoden auf Plausibilität geprüft.



| Spaltenname | Datentyp          | Beschreibung                                                                                                                                                                                                                              |
|-------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| zoneuid     | CHARACTER VARYING | UID der Lagerzone, zu der dieser Lagerplatz gehört                                                                                                                                                                                        |
| celluid     | CHARACTER VARYING | UID des Lagerplatzes innerhalb der angegebenen Lagerzone                                                                                                                                                                                  |
| reserved    | CHARACTER VARYING | Reservierungsinformation, NULL für nicht reserviert                                                                                                                                                                                       |
| action      | CHARACTER VARYING | IN: nur Einlagerung in diesen Lagerplatz erlaubt, OUT: nur Auslagerung, INOUT: Ein- und Auslagerung erlaubt                                                                                                                               |
| x           | INTEGER           | Horizontaler/X-Index des Lagerplatzes                                                                                                                                                                                                     |
| y           | INTEGER           | Tiefe/Y-Index des Lagerplatzes                                                                                                                                                                                                            |
| z           | INTEGER           | Vertikaler/Z-Index des Lagerplatzes                                                                                                                                                                                                       |
| width       | INTEGER           | physikalische Breite des Lagerplatzes                                                                                                                                                                                                     |
| height      | INTEGER           | physikalische Höhe des Lagerplatzes                                                                                                                                                                                                       |
| depth       | INTEGER           | physikalische Tiefe des Lagerplatzes                                                                                                                                                                                                      |
| maxload     | INTEGER           | maximal zulässige Gewichtsbelastung des Lagerplatzes                                                                                                                                                                                      |
| strategy    | CHARACTER VARYING | LIFO: Last-In-First-Out-Strategie, Verwendung hauptsächlich in Regalen ohne Durchlauf, FIFO: First-In-First-Out-Strategie, Verwendung in Durchlaufregalen und anderen, bei denen ein Gut an anderer Stelle im Lager wieder entnommen wird |
| capacity    | INTEGER           | Kapazität des Lagerplatzes für $n$ Lagereinheiten                                                                                                                                                                                         |
| flowzoneuid | CHARACTER VARYING | ist die Strategie des Lagerplatzes FIFO, wird hier bei Durchlaufregalen angegeben, in welcher Lagerzone die Güter wieder entnommen werden können, wenn sie in diesen Lagerplatz eingelagert wurden                                        |
| flowcelluid | CHARACTER VARYING | ist die Strategie des Lagerplatzes FIFO, wird hier bei Durchlaufregalen angegeben, aus welchem Lagerplatz die Güter wieder entnommen werden können, wenn sie in diesen Lagerplatz eingelagert wurden                                      |

Tabelle 5.1: Datenbankschema der Stammdaten-Tabelle `cells`.

| Spaltenname | Datentyp          | Beschreibung                                                                                                                              |
|-------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| itemuid     | CHARACTER VARYING | UID des Guts, gilt produktionsweit und bleibt auch nach abschließender Verarbeitung des Guts zu Zwecken des Qualitätsmanagements erhalten |
| chipuid     | CHARACTER VARYING | UID des Lagerplatzes innerhalb der angegebenen Lagerzone                                                                                  |
| authority   | CHARACTER VARYING | Systemkomponente, die die Einlagerung des Guts veranlasst hat; kann z. B. bei Rückfragen angesprochen werden; kann auch NULL sein         |
| orderid     | CHARACTER VARYING | UID des Auftrags für den das Gut eingelagert wurde; kann auch NULL sein                                                                   |
| entered     | BIGINT            | UNIX-Timestamp, zu dem das Gut die Fabrik betreten hat bzw. erstmalig als eigenständiges Gut geführt wurde                                |
| left        | BIGINT            | UNIX-Timestamp, zu dem das Gut die Fabrik verlassen hat, NULL wenn noch aktiv                                                             |
| expires     | BIGINT            | UNIX-Timestamp, zu dem das Gut verfällt (Ablauf-/Verfallsdatum), NULL bei keinem                                                          |
| import      | BIGINT            | UNIX-Timestamp, zu dem das Gut eingelagert wurde                                                                                          |
| export      | BIGINT            | UNIX-Timestamp, zu dem das Gut ausgelagert wurde; NULL wenn noch im Lager                                                                 |
| priority    | INTEGER           | Priorität des Guts in der Produktionsumgebung wie von authority übergeben                                                                 |
| zoneuid     | CHARACTER VARYING | UID der Lagerzone, in der sich das Gut befindet                                                                                           |
| celluid     | CHARACTER VARYING | UID des Lagerplatzes in der Lagerzone zoneuid, in der sich das Gut befindet                                                               |
| material    | BOOLEAN           | true falls das Gut bei Materialanforderungen berücksichtigt werden soll, false andernfalls                                                |

Tabelle 5.2: Datenbankschema der Güter-Tabelle items.

| Spaltenname | Datentyp          | Beschreibung                                                                                                                   |
|-------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------|
| itemuid     | CHARACTER VARYING | UID des Guts, dem ein Material zugeordnet ist                                                                                  |
| typeuid     | CHARACTER VARYING | UID des Materials, das mit Hilfe der anderen Spalten weiter beschrieben wird                                                   |
| x           | INTEGER           | X-Position des Materials auf dem BGT des Guts ( <i>itemuid</i> )                                                               |
| y           | INTEGER           | Y-Position des Materials auf dem BGT des Guts ( <i>itemuid</i> )                                                               |
| z           | INTEGER           | Z-Position des Materials auf dem BGT des Guts ( <i>itemuid</i> )                                                               |
| width       | BIGINT            | Breite des LEGO-Duplo-Bausteins; falls nicht zutreffend, muss ein definierter Wert angenommen werden (z. B. 0)                 |
| colour      | CHARACTER VARYING | Farbangabe des LEGO-Duplo-Bausteins; falls nicht zutreffend, muss ein definierter Wert angenommen werden (z. B. <i>keine</i> ) |

Tabelle 5.3: Datenbankschema der Material-Tabelle `material`.

| Spaltenname | Datentyp          | Beschreibung                                                                                                               |
|-------------|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| id          | BIGINT            | fortlaufende ID unter den Anfragen                                                                                         |
| itemuid     | CHARACTER VARYING | UID des Guts, das abgerufen werden soll (alternativ typeuid usw. angeben)                                                  |
| typeuid     | CHARACTER VARYING | UID des Materials, das mit Hilfe der anderen Spalten weiter beschrieben wird                                               |
| width       | BIGINT            | Breite des LEGO-Duplo-Bausteins; falls nicht zutreffend, muss ein definierter Wert angenommen werden (z. B. <i>Fahne</i> ) |
| colour      | CHARACTER VARYING | Materialfarbe                                                                                                              |
| amount      | INTEGER           | gewünschte Anzahl                                                                                                          |
| date        | BIGINT            | Zeitpunkt der Anfrage als UNIX-Timestamp                                                                                   |
| deadline    | BIGINT            | spätester Lieferzeitpunkt als UNIX-Timestamp                                                                               |
| authority   | CHARACTER VARYING | Auftraggeber                                                                                                               |
| gateuid     | CHARACTER VARYING | UID des Lagerzugangs, an dem das Gut erwartet wird                                                                         |
| done        | BOOLEAN           | true falls bereits ausgeführt                                                                                              |

Tabelle 5.4: Datenbankschema der Anfragen-Tabelle `requests`.

| Produktanzahl | zusätzlich übertragene Bytes |
|---------------|------------------------------|
| 1             | 81                           |
| 2             | 147                          |
| 3             | 213                          |
| 10            | 675                          |
| 100           | 6615                         |
| 1000          | 66015                        |
| 10000         | 660015                       |

Tabelle 5.5: Auswirkung eines zusätzlichen Werts je Produkt auf den Kommunikationsoverhead.

## 6 Konfigurator

Neben möglichst optimaler Nutzung der vom Hochregallager bereitgestellten Ressourcen, ist auch eine Visualisierung Teil dieser Arbeit. Die Aufgaben des entsprechenden Tools umfassen:

- Erzeugung von Lagerkonfigurationen,
- grafische Abbildung von Lagerstrukturen,
- Anzeige der aktuellen Vorgänge im Lager und später
- Simulation.

Um eine möglichst gute Integration in das bestehende WMS (s. VIRKUS [28]) zu erzielen, werden die bereits vorhandenen Schnittstellen so genutzt, dass alle mit dieser Arbeit erzeugten Erweiterungen für das bestehende System transparent sind. Außerdem erfolgt die Umsetzung ebenfalls in Java, sodass bereits existierende Java-Schnittstellen direkt und ohne Zwischenschicht genutzt werden können.

Zunächst werden die dem Lagerkonfigurator zugrunde liegenden Funktionen und das entwickelte Datenformat erläutert, sodass im Folgenden kurz auf die Bedienung des Tools eingegangen werden kann. Anschließend wird die Visualisierungs- und Bedienfunktion vorgestellt und erläutert, wie die Anbindung an den Warehouse Operator vorgenommen wird. Abschließend wird auf die Realisierung der Simulationsfunktion und deren Bedienung eingegangen.

### 6.1 Konfigurator

Der Lagerkonfigurator dient einem Inbetriebnehmer oder Entwickler zur einmaligen Einrichtung eines Hochregallagers hinsichtlich seiner physikalischen Eigenschaften.

#### 6.1.1 Funktionen

Zu den Funktionen des Lagerkonfigurators zählen die Parametrierung des Lagers und seiner Lagerzellen über eine grafische Benutzeroberfläche. Die Eingaben können im Anschluss in einer XML-Datei gespeichert und auf Wunsch direkt in einer Datenbank für

die spätere Nutzung festgehalten werden. Mit dem Konfigurator ist es möglich, auch umfangreiche Lager schnell zu erzeugen, was insbesondere für die Vorbereitung von Simulationen von Vorteil ist.

Folgende Werte können parametrisiert werden:

1. Generell:
  - Lagerbezeichnung
  - Beschreibung
  - Autor
2. Lagerzonen
3. Lagerplätze:
  - Position vertikal/horizontal/Regalseite
  - Ausmaße Breite/Höhe/Tiefe
  - Kapazität
  - maximale Gewichtsbelastung
  - Strategie LIFO/FIFO mit korrespondierender Entnahmezelle
4. Lagerzugänge:
  - virtuelle Position

### 6.1.2 Datenformat

Wie bereits erwähnt, werden alle obigen Eingaben des Benutzers in einer XML-Datei hinterlegt. Der Aufbau einer solchen Datendatei ist Listing 6.1 zu entnehmen.

Listing 6.1: Beispiel einer XML-Lagerkonfiguration, wie sie vom Lagerkonfigurator erzeugt wird.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <hrw-config>
3 <name></name>
4   <author></author>
5   <description></description>
6   <database-host>null</database-host>
7   <database-name>null</database-name>
8   <database-user>null</database-user>
9   <zones>
10     <zone uid="main">
11       <gates>
12         <gate uid="I03" virtual-x="7" virtual-y="1"
13           virtual-z="5" />
14         <gate uid="I02" virtual-x="6" virtual-y="1"
```

```
15     virtual-z="5" />
16   </gates>
17   <cells>
18     <cell uid="6-3S">
19       <x>6</x>
20       <z>3</z>
21       <southside />
22       <action>INOUT</action>
23       <width>180</width>
24       <height>150</height>
25       <depth>175</depth>
26       <capacity>1</capacity>
27       <max-load>1000</max-load>
28       <strategy>LIFO</strategy>
29     </cell>
30     <cell uid="7-3S">
31       <x>7</x>
32       <z>...
33     </cell>
34   </cells>
35 </zone>
36 </zones>
37 </hrw-config>
```

### 6.1.3 Bedienung

Nach dem Start der Anwendung wird das Hauptfenster der GUI präsentiert (s. Abb. 6.1). Neben der Menüleiste sind zwei Registerkarten zu sehen, von denen diejenige mit der Beschriftung „General“ bereits aktiviert ist. Hier sollte der Einrichter die Möglichkeit nutzen und allgemeine Informationen zur Lagerkonfiguration eingeben. Dazu gehören eine Bezeichnung („Name“) und eine Beschreibung („Description“). Auch sollte der Name des Autors aus Gründen der Vollständigkeit eingetragen werden.

Das Lagerlayout ist über die Registerkarte „Visualisation“ erreichbar (s. Abb. 6.2). Wie in der Abbildung zu sehen, besteht diese Ansicht aus vier Bereichen, die näher beleuchtet werden sollen.

**Menüleiste** Viele Funktionen sind über die Menüleiste aufrufbar. Insbesondere das Gestalten des Lagers ist jedoch nur mit Hilfe einer Maus möglich. Die Menüleiste besteht aus folgenden Menüpunkten:

**File** Allgemeine Dateioperationen

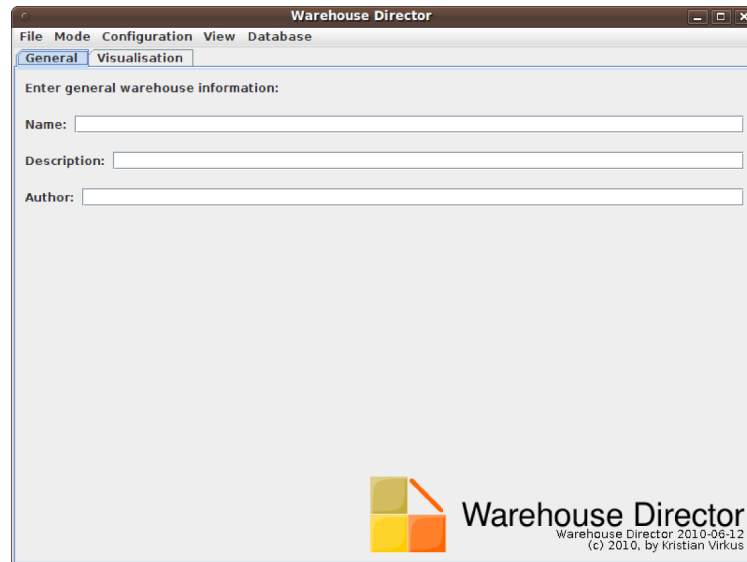


Abbildung 6.1: Hauptfenster des Warehouse Director.

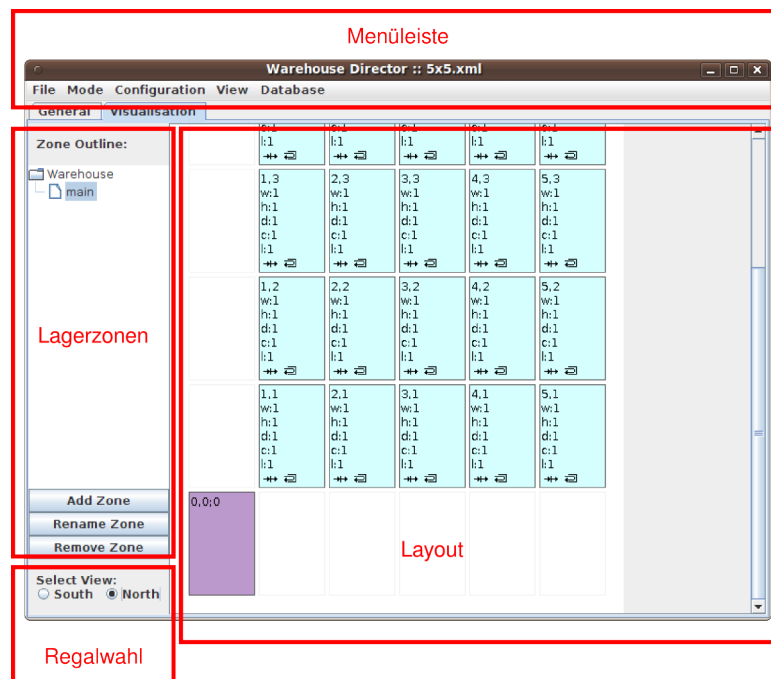


Abbildung 6.2: Konfigurations- und Visualisierungsansicht des Warehouse Director.

**New** Erstellt eine neue Lagerkonfiguration.

**Open** Lädt eine bestehende Lagerkonfiguration aus einer Datei.

**Save** Speichert die aktuelle Lagerkonfiguration in die bereits geladene Datei.

**Save as** Speichert die aktuelle Lagerkonfiguration in eine Datei, die zunächst vom Benutzer gewählt werden muss.



**Quit** Beendet den Warehouse Director.

**Mode** Programmmodus

**Configuration** Ruft den Konfigurationsmodus auf.

**Visualisation** Ruft den Visualisierungsmodus auf.

**Benchmarking** Ruft den Benchmarkingmodus auf.

**Configuration** Konfigurationshilfe

**Generate** Erzeugt ein rechteckiges Lagerlayout anhand eingegebener Größen.

**View** Ansichtsoptionen

**Tiny** Wechselt zu einer minimalistischen Darstellung des Lagerlayouts.

**Detailed** Wechselt zu einer großen, um Details bereicherten Darstellung des Lagerlayouts.

**Mirror horizontally** Spiegelt die Darstellung des Lagerlayouts horizontal.

**Mirror vertically** Spiegelt die Darstellung des Lagerlayouts vertikal.

**Show grid** Stellt die Anzeige des Rasters des Lagerlayouts an oder ab.

**Show gates** Stellt die Anzeige der Lagerzugänge im Lagerlayout an oder ab.

**Show cells** Stellt die Anzeige der Lagerplätze im Lagerlayout an oder ab.

**Database** Datenbankeinrichtung, siehe auch Abs. 6.1.4

**Manual Database Creation** Gibt Hinweise zur manuellen Grundeinrichtung der Lagerdatenbank.

**Connect** Stellt eine Verbindung zur Lagerdatenbank her.

**Create Tables** Erzeugt die Tabellen in der Datenbank anhand der aktuellen Lagerkonfiguration.

**Create Procedures** Erzeugt die Stored Procedures in der Datenbank anhand der aktuellen Konfiguration.

**Write Master Data** Schreibt die Stammdaten der aktuellen Konfiguration in die Datenbank.

**Create Regular Users** Erzeugt die zum Betrieb des LVS notwendigen Benutzer in der Datenbank.

**Disconnect** Baut die Datenbankverbindung wieder ab.

**Lagerzonen** In diesem Bereich befindet sich eine Baumansicht, die die im Lager vorhandenen Lagerzonen darstellt. Sie werden getrennt voneinander modelliert. Es bestehen die Optionen zum Hinzufügen, Umbenennen und Entfernen einer Lagerzone. Durch Klick auf den Namen einer Lagerzone wird die Layoutansicht im rechten Teil des Fensters neu und entsprechend der gewählten Lagerzone aufgebaut.

**Regalwahl** Jede Lagerzone besteht prinzipiell immer aus zwei Regalhälften. Eine Hälfte befindet sich links und die andere rechts vom RBG, das diese Lagerzone bedient. Somit stehen immer zwei Regalhälften mit ihren Rücken aneinander, womit ihre Öffnungen entgegengesetzt liegen, also zum jeweils zugehörigen RBG. Mit den beiden Radiobuttons „South“ und „North“ kann die Ansicht auf die linke bzw. rechte Regalhälfte gewechselt werden. Für die Zuordnung zur real vorhandenen Regalhälfte wird  $y = -1$  für die linke bzw.  $y = +1$  für die rechte verwendet. Je nach örtlicher Gegebenheit muss es nicht immer links und rechts sein. Der für die Hardwaresteuerung zuständige Warehouse Expert muss dies entsprechend umsetzen. In der Digitalen Fabrik der Hochschule Emden/Leer entspricht  $-1$  der Süd- und  $+1$  der Nordseite des Lagers.

**Layout** Der Layoutbereich des Hauptfensters stellt das aktuelle Layout des konfigurierten Lagers dar. Dabei wird standardmäßig ein Raster angezeigt, an dem sich alle konfigurierten Lagerplätze auf Grund ihrer indizierten Positionsangaben (s. Abs. 4.1) orientieren. Der Anzeige liegt ein Koordinatensystem zugrunde, das seinen Ursprung links unten besitzt. Dabei spannt die Achse  $x$  den Raum in horizontaler Richtung und die Achse  $z$  in vertikaler Richtung auf – sowohl auf dem Bildschirm wie im Lager (s. Abb. 6.3). Zwar entspricht diese Zuteilung nicht allen theoretischen und auch praktisch in der Digitalen Fabrik der Hochschule Emden/Leer umgesetzten Lösungen, allerdings ist die Hardwaresteuerung in den SPSen der Frequenzumrichter mit dieser Zuordnung umgesetzt und soll daher auch so übernommen werden.

Neben der Anzeige der Lagerplätze und Lagerzugänge können diese im Modus „Configuration“ auch modifiziert werden. Dazu existiert ein Kontextmenü das sich bei einem Rechtsklick in den Layoutbereich öffnet. Hierbei muss zwischen drei Regionen unterschieden werden: es wird auf einen freien Bereich, auf einen Lagerplatz oder auf einen Lagerzugang geklickt. Je nachdem werden spezielle Menüpunkte angezeigt.

Wird ein Rechtsklick in einen freien Bereich ausgeführt, so bietet das Kontextmenü Optionen, um einen neuen Lagerplatz oder einen neuen Lagerzugang einzufügen. Je nach Benutzerauswahl öffnet sich ein entsprechendes Dialogfenster. Für Lagerplätze sind eindeutige Bezeichner innerhalb der Lagerzone, Dimensionen, Maximalbelastung usw. einzugeben (s. Abb. 6.4). Hierbei ist es für den Warehouse Director irrelevant, in welcher Einheit die Größen angegeben werden. Entscheidend ist ausschließlich, dass alle Komponenten des Systems (implizit) von gleichen Einheiten ausgehen. Ähnlich verhält es sich bei der Erstellung eines neuen Lagerzugangs. Hier ist neben einer eindeutigen Bezeichnung innerhalb der Lagerzone die indizierte Position des Zugangs anzugeben, die

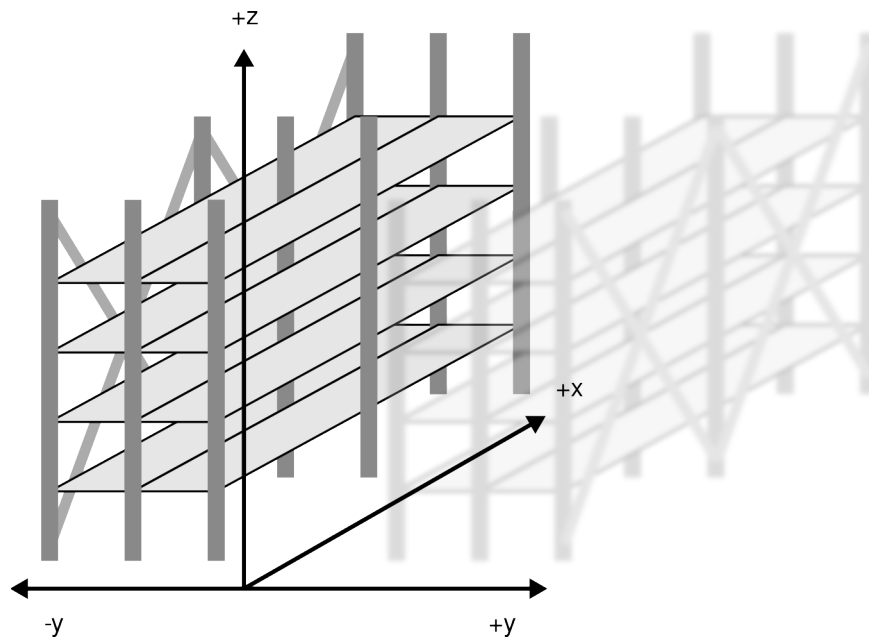


Abbildung 6.3: Hochregallager-Koordinatensystem.

der realen Position am nächsten kommt. Diese *virtuellen Koordinaten* wurden auf Grund einer wesentlich vereinfachten Rechnung eingeführt. Sie sind so zu wählen, dass der Bewegungsaufwand des RBG dem gleichen Aufwand entspricht, als würde eine Fahrt zwischen zwei Lagerplätzen durchgeführt, bei denen der eine an den virtuellen Koordinaten des Lagerzugangs liegt. Bei beiden Einfügen-Menüpunkten werden automatisch die Positionsindizes berechnet, die der Position des Mauszeigers im dargestellten Lagerlayout entsprechen und es wird die zur Zeit ausgewählte Regalwahl übertragen. Es ist hierbei zu beachten, dass je Regalseite und Position nur ein Lagerplatz oder ein Lagerzugang angeordnet werden kann. Zudem kann kein Lagerplatz an einer Position eingefügt werden, an der sich bereits ein Lagerzugang befindet, wenn dieser mittig liegt, also zwischen den Regalen ( $y = 0$ ). Mit dem Warehouse Director können auch Lager entworfen werden, die über sog. Durchlaufregale verfügen. Dazu stehen die Lagerplatzeigenschaften „Action“, „Strategy“ und „Flow zone/cell“ zur Verfügung. Tabelle 6.1 zeigt die dafür möglichen Einstellungskombinationen. Es ist jedoch zu bemerken, dass Durchlaufregale zwar mit dem Warehouse Director konfiguriert werden können, diese jedoch zur Zeit noch nicht vom entsprechenden Lageragenten und -operator unterstützt werden. Dies ist ein Punkt, der im Rahmen weiterer Forschungen aufgegriffen werden könnte.

Wurde im Lagerlayout ein Rechtsklick statt auf einen freien Bereich auf einen bestehenden Lagerplatz durchgeführt, so werden die Optionen „Properties“ zum Aufruf des bereits bei der Erstellung angezeigten Dialogfensters zur Konfiguration des Lagerplatzes, „Delete“ zum Entfernen des Lagerplatzes aus der Konfiguration und „Copy“ zum Kopieren des Lagerplatzes mit seinen Eigenschaften in eine Zwischenablage angezeigt.

Wurde zuvor ein Lagerplatz in die Zwischenablage kopiert, wird im Kontextmenü der

Abbildung 6.4: Dialogfenster zur Eingabe der Eigenschaften eines Lagerplatzes.

| Action | Strategy | Flow zone/cell | Erläuterung                                                                      |
|--------|----------|----------------|----------------------------------------------------------------------------------|
| INOUT  | LIFO     | –              | einfachtiefer Lagerplatz ohne Durchlauf                                          |
| IN     | FIFO     | X              | einfach- oder mehrfachtiefer Lagerplatz zur Einlagerung in ein Durchlaufregal    |
| OUT    | FIFO     | –              | einfach- oder mehrfachtiefer Lagerplatz zur Auslagerung aus einem Durchlaufregal |

Tabelle 6.1: Einstellungskombinationen für Durchlaufregale.

Punkt „Paste“ angezeigt, mit dem eine Kopie des Lagerplatzes an der Position unter dem Mauszeiger eingefügt werden kann.

War der Rechtsklick auf einen Lagerzugang platziert, so ist wie bei Lagerplätzen ein Menüpunkt zum Bearbeiten der Eigenschaften und zum Entfernen des Zugangs vorhanden. Copy & Paste kann mit Lagerzugängen nicht durchgeführt werden.

Ist die Ansichtsoption „Detailed“ eingestellt, so werden Lagerplätze und -zugänge als Rechtecke dargestellt. Abbildung 6.5 beschreibt die einzelnen angezeigten Werte und Symbole. Die Ansicht kann auch auf kleine Quadrate zur Repräsentation der Lagerplätze und -zugänge umgeschaltet werden. Dazu muss im Menü „View“ der Punkt „Tiny“ gewählt werden. Zur Unterscheidung der Lagerplätze und -zugänge wurde eine Farbgebung mit Blautönen gewählt. Erstere sind dabei hellblau eingefärbt, während letztere einen hellen Violetton besitzen. Stärkere Farbunterschiede konnten hier nicht gewählt

werden, da die Farben rot, grün und gelb bereits zur Differenzierung der unterschiedlichen Füllgrade der Lagerplätze in den anderen Modi Anwendung finden. Für den Fall, dass ein Benutzer die Farbunterschiede der Blautöne nicht wahrnehmen kann, existiert die Möglichkeit, entweder alle Lagerplätze oder alle Lagerzugänge über den Hauptmenüpunkt „View“ auszublenden.

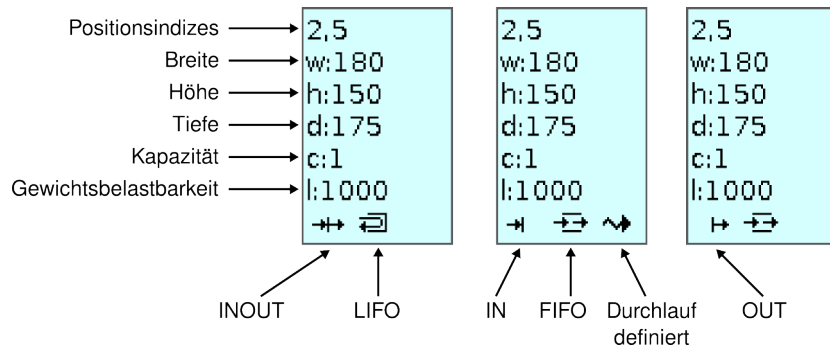


Abbildung 6.5: Detaillierte Ansicht eines Lagerplatzes im Lagerlayout des Warehouse Directors.

### 6.1.4 Datenbankeinrichtung

Eine wichtige Funktion des Warehouse Directors ist die nahezu vollständig automatisierte Einrichtung der Lagerdatenbank. Sie setzt eine bereits eingerichtete, leere PostgreSQL-8.4-Datenbank und einen Benutzer voraus, der die vollen Zugriffsrechte auf diese Datenbank besitzt. Eine Kurzanleitung hierfür findet sich im „Database“-Menü unter „1. Manual Database Creation“. Sie empfiehlt zur Erzeugung eines administrativen Benutzers und einer Datenbank den in Listing 6.2 dargestellten SQL-Ausdruck. Hier ist der Warehouse Director soweit behilflich, dass der gewünschte Name der Datenbank und des Benutzers abgefragt und anschließend in den gezeigten SQL-Ausdruck eingefügt wird. Einzig die Einsetzung des gewünschten Passworts bleibt dem Anwender überlassen. Zudem wird darauf hingewiesen, dass es unter Umständen erforderlich ist, dem DBMS PostgreSQL mitzuteilen, dass sich der neue Benutzer mit Kennwort vom lokalen System und/oder anderen Systemen anmelden darf. Dies geschieht auf UNIX-Systemen i.d.R. über die Datei `pg_hba.conf` im PostgreSQL-Konfigurationsverzeichnis.

Listing 6.2: SQL-Ausdruck zur Erzeugung der Datenbank und des administrativen Benutzers.

```

1 CREATE USER <user> WITH CREATEUSER PASSWORD '<password>';
2
3 CREATE DATABASE <name> WITH OWNER=<user> ENCODING='UTF8';
4
5 GRANT CREATE ON SCHEMA public TO <user>;

```

Nachdem dies durchgeführt wurde, kann mit dem soeben erzeugten Benutzer eine Verbindung zur Datenbank hergestellt werden, indem der Menüpunkt „2. Connect“ gewählt wird. Der Anwender wird nach Datenbankhost, Datenbankname, dem Benutzernamen und dessen Kennwort gefragt. Erscheint keine Fehlermeldung, so war der Verbindungsaufbau erfolgreich.

Nun können alle notwendigen Tabellen in der Datenbank erzeugt werden. Dazu wird Menüpunkt „3. Create Tables“ angewählt. Bevor jedoch Änderungen vorgenommen werden, muss der Anwender wählen, ob die bestehenden Tabellen zunächst gelöscht werden sollen. Dies wird in jedem Fall empfohlen, da somit auch alle Reste bisheriger Lagerlayouts entfernt werden. Die erfolgreiche Erzeugung wird mit einer Meldung bestätigt. Das Skript hierzu befindet sich in Anhang A.10 dieser Arbeit.

Anschließend ist es möglich, über „4. Create Procedures“ die für den Betrieb notwendigen Stored Procedures zu erzeugen. Sie werden u.a. zur Berechnung der Entfernung zweier Lagerplätze für die Doppelspiel-Optimierungsstrategie verwendet (s. Abs. 5.2.2). Da diese Prozeduren in PL/pgSQL, der PostgreSQL-eigenen Programmiersprache, geschrieben wurden, muss vor Ausführung dieses Schrittes sichergestellt sein, dass diese Programmiererweiterung bereits manuell eingerichtet wurde. Dieser Schritt kann betriebssystemabhängig sein. Möglicherweise müssen Bibliotheken (z. B. über die Linux-Paketverwaltung) nachinstalliert werden. Zudem muss die Sprache über einen Ausdruck wie den in Listing 6.3 gezeigten registriert werden. Hierauf weist ein Hinweisfenster den Anwender nach Auswahl des Menüpunktes hin.

Listing 6.3: Einrichtung der Programmiersprache PL/pgSQL.

```
1 CREATE FUNCTION plpgsql_call_handler()  
2   RETURNS OPAQUE AS '/usr/lib/postgresql/8.4/lib/plpgsql.so'  
   LANGUAGE 'C';  
3  
4 CREATE LANGUAGE 'plpgsql' HANDLER plpgsql_call_handler  
   LANCOMPILER 'PL/pgSQL';
```

Bislang wurden fast ausschließlich allgemeine Strukturen in der Datenbank erzeugt. Mit dem Menüpunkt „5. Write Master Data“ hingegen werden die konfigurierten Stammdaten in die Tabellen `cells` und `gates` geschrieben. Da diese Aktion auch ausgeführt werden kann, wenn die vorigen übersprungen wurden – z. B. da sich die Dimension eines Lagerplatzes geändert hat –, wird der Anwender zunächst gefragt, ob alle bereits bestehenden Stammdaten zunächst entfernt werden sollen.

Um die Sicherheit des Systems zu erhöhen, ist es empfehlenswert, nur diese Einrichtungsvorgänge mit dem oben erzeugten Datenbankbenutzer durchzuführen. Alle anderen Operationen sollten unterschieden werden in lesende und schreibende Operationen. So benötigt der Agent zur Aktualisierung des Lagerbestandes in jedem Fall Schreibrechte, während der Datenbankbenutzer zur Visualisierung ausschließlich Lesezugriff besitzen sollte. Über den Menüpunkt „6. Create Regular Users“ werden daher zwei weitere Benutzer erzeugt. Ihre Namen sind mit `warehouseactor` für den Lageragenten und

`warehouseobserver` bereits vordefiniert. Lediglich die entsprechenden Passwörter sind durch den Anwender noch zu vergeben. Da im Anschluss auch die Rechte an entsprechenden Tabellen um die neu erzeugten Benutzer erweitert und somit Abhängigkeiten gebildet werden, wird ein Überschreiben der Benutzer durch erneutes Aufrufen dieses Menüpunkts fehlschlagen.

Abschließend kann die Verbindung zur Datenbank über den Menüpunkt „Disconnect“ wieder geschlossen werden. Ausführlichere Beschreibungen zu den in der Datenbank verwalteten Daten sind in Abschnitt 5.2.2 zu finden.

### 6.1.5 Umsetzung in Java

Der Programmeinstiegspunkt (Methode `main`) befindet sich in Klasse `de.i2ar.warehousedirector.Main`. Wurden Kommandozeilenparameter übergeben, so wird versucht, eine entsprechende Datei bereits beim Start zu öffnen, ansonsten wird das Hauptfenster `de.i2ar.warehousedirector.gui.FormMain` im Ausgangszustand geladen. Dazu bietet der Konstruktor der Klasse `FormMain` einen Parameter, um den Dateinamen der zu öffnenden Datei zu übergeben.

Das Dateiformat einer Lagerkonfiguration wurde bereits in Listing 6.1 gezeigt. Im Hauptspeicher existiert es als ein Objekt der Klasse `HRWConfig` aus dem Package `de.i2ar.warehousedirector.data`, in dem nahezu alle Klassen angesiedelt sind, die Teil der Datenstrukturen einer Lagerkonfiguration sind. In dieser Klasse befinden sich Felder für die Metadaten des abgebildeten Lagers, die im Hauptfenster unter der Registerkarte „General“ definiert werden können. Ebenso werden hier Datenbankeinstellungen und im Feld `zones` alle Lagerzonen – repräsentiert durch je ein Objekt der Klasse `HRWZone` – als Wertepaare Lagerzonen-UID und `HRWZone`-Objekt zum schnellen Zugriff als `HashMap` abgelegt. Für jede in Abb. 6.2 aufgelistete Lagerzone existiert ein solcher Eintrag in der Map. Eine `HRWZone` umfasst neben ihrer eigenen UID Angaben zu allen in ihr befindlichen Lagerzugängen und Lagerplätzen. Diese sind ebenfalls als `HashMap`-Felder, als Wertepaare aus jeweiliger UID und dem zugehörigen Objekt zur näheren Beschreibung, hinterlegt. Diese Objekte sind Instanzen der Klassen `HRWGate` und `HRWCell`. Die Lagerzugänge `HRWGate` besitzen lediglich Felder zur Definition ihrer virtuellen Position (s.o.) und ihre eigene UID. Neben der eigenen UID besitzen Lagerplätze als `HRWCell`-Objekt sämtliche Eigenschaften, die über den Konfigurator definiert werden können (s. Abs. 6.1.1) und zudem ein Feld des Typs `HRWCellVis`, das temporäre, dynamische Informationen für die Visualisierung bereitstellt. Die Beziehungen sind in Abbildung 6.6 als UML-Klassendiagramm dargestellt.

Die Datenstruktur-Klassen besitzen zumeist Hilfsfunktionen, die Suchen nach bestimmten Werten ermöglichen wie z. B. die Suche nach einem Lagerzugang an einer als Argument gegebenen virtuellen Position. Jede Klasse besitzt zudem eine Methode `toXml`, um den durch sie beschriebenen Teil der XML-Konfigurationsdatei selbst zu erzeugen. Diese wird durch die Methode `toXml` der Klasse `HRWConfig` zusammengesetzt.

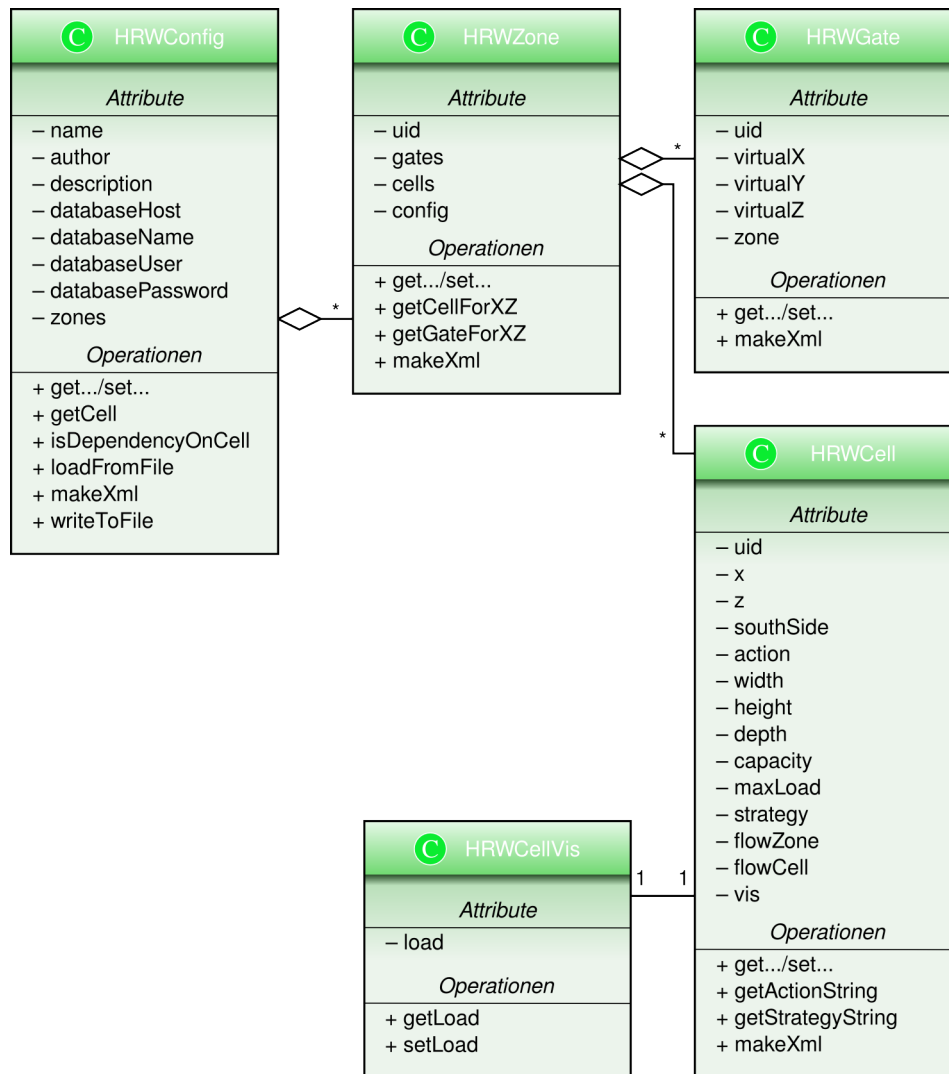


Abbildung 6.6: UML-Klassendiagramm der Lagerlayout-Datenstrukturen des Warehouse Directors.

Bei Instanziierung der Klasse `FormMain` werden alle grafischen Elemente des Fensters erzeugt und ihre Referenzen in Variablen gespeichert. Erst am Ende wird eine leere Konfiguration erzeugt oder eine bestehende anhand ihres Dateinamens über die Methode `doLoadFile` der Klasse `FileActions` geladen. In beiden Fällen wird die in einem Fenster angezeigte Konfiguration im Feld `hrw` der Klasse vorgehalten.

Die Interaktion mit dem Benutzer läuft hauptsächlich über `ActionListener` ab, die für alle GUI-Elemente registriert werden, die z. B. auf Klicks reagieren sollen. In Anlehnung an die `ActionListener` erfolgte die Namensgebung der Klassen, in denen dieses Interface implementiert ist. Die Klassenbezeichnungen enden auf `Action` und befinden sich im Package `de.i2ar.warehousedirector.actions`. Um die Übersichtlichkeit zu erhöhen, wurde dabei für jeden Bereich des Konfigurators ein eigener `ActionListener`



implementiert:

**BenchmarkActions** Diese Implementation verarbeitet alle GUI-Ereignisse, die im Bezug zum Benchmarking stehen. Diese Klasse wird in Abschnitt 6.3 näher erläutert.

**CellMouseActions** Diese Klasse implementiert nicht das Interface `ActionListener`, sondern `MouseListener`. Damit werden alle Ereignisse, die von Bewegungen und Klicks der Maus stammen verarbeitet. Von Interesse ist dabei die Methode `mousePressed`, die dann von der GUI aufgerufen wird, sobald eine Maustaste heruntergedrückt wurde. Sie dient zum Vorbereiten und Anzeigen des Kontextmenüs im Layoutbereich des Warehouse Directors, um Lagerplätze/-zugänge zu erstellen und zu bearbeiten. Wurde nicht die rechte Maustaste gedrückt, wird die Methode sofort wieder verlassen. Dies geschieht ebenso, wenn sich der Mauszeiger zu dem Zeitpunkt nicht im Layoutbereich befand, da ansonsten keine Anwendung für diese genaue Auswertung per `MouseListener` besteht.

Zunächst wird die aktuelle Mauszeigerposition in das Layoutraster übertragen, also unter Berücksichtigung der Scrollleisten und der gewählten Darstellungsform berechnet, welche Positionsindizes sich unter dem Mauszeiger befinden. Dabei werden auch Bereiche berücksichtigt, die außerhalb des bereits benutzen Bereichs liegen, um eine schnelle Konfiguration zu ermöglichen. Anschließend wird die in der gleichen Klasse befindliche Methode `makeAndShowConfigureCellPopup` aufgerufen, die prüft, ob sich am übergebenen Positionsindex ein Lagerplatz oder ein Lagerzugang befindet. Diese Informationen werden daraufhin über die Methode `set` des Feldes `cpal` der Klasse `FormMain` übergeben. `cpal` ist vom Typ `CellPopupActions`, wurde bei der Instanziierung von `FormMain` ebenso instanziiert und dient einerseits zur Vorbereitung des Kontextmenüs und andererseits zur Handhabung der Ereignisse des Kontextmenüs. Nach dieser Vorbereitung wird das Kontextmenü an der aktuellen Mauszeigerposition angezeigt.

**CellPopupActions** Wie bereits erwähnt, dient diese Klasse zur Vorbereitung und Verarbeitung der Ereignisse, die in Verbindung mit dem Kontextmenü des Layoutbereichs auftreten. Die Methode `set` bestimmt anhand ihrer Argumente die anzuzeigenden Menüeinträge und speichert daneben Angaben zu den Positionsindizes und zugehörigem Lagerplatz bzw. -zugang, da alle GUI-Ereignisse ohne manuelle Anpassungen in dieser Hinsicht zustandslos, da nicht parametrierbar, wären. Bei einem eingetretenen Klick-Ereignis eines Menüpunkts wird der Programmablauf zu je einer eigenen Methode verzweigt. Sie verarbeitet die mit dem Menüpunkt verknüpfte Aktion und greift dabei jeweils auf die eben gespeicherten Angaben zurück. So wird in erster Linie eines der Fenster zur Bearbeitung der Eigenschaften von Lagerplatz oder -zugang mit den aktuellen Werten belegt und angezeigt, aber es werden bei Einfügeereignissen auch neue `HRWCell`- oder `HRWGate`-Objekte erzeugt und mit den aktuellen Positionsindizes versehen, bevor ein Eigenschaften-Fenster angezeigt wird. Durch Aufruf des Menüpunkts „Copy“ werden alle Felder des Lagerplatzes an den gewählten Positionsindizes in ein neues `HRWCell`-Objekt kopiert und als globale Variable im Objekt der Klasse `FormMain` gespeichert. Wird an ande-

rer Stelle der Menüpunkt „Paste“ aufgerufen, dann wird auf dieses `HRWCell`-Objekt zurückgegriffen und seine Felder entweder in ein neues Objekt des gleichen Typs kopiert oder die Felder des `HRWCell`-Objekts ersetzt, das zu den Positionsindizes unter dem Mauszeiger gehört. Im ersten Fall wird das neue Objekt schließlich zur aktuell gewählten Lagerzone hinzugefügt, die als globale Variable `currentZone` im `FormMain`-Objekt hinterlegt ist. Lediglich die Positionsindizes werden nicht kopiert, da in diesem Fall mit der Mausposition auch die Positionsindizes des einzufügenden Lagerplatzes bestimmt werden sollen.

**ConfigurationActions** Dieser `ActionListener` reagiert bei einem Klick auf den Menüpunkt „Generate...“ unter „Configuration“ des Hauptmenüs. Befindet sich die Anwendung im Konfigurationsmodus, dann wird nach Bestätigung des Anwenders und nach Eingabe horizontaler und vertikaler Dimensionen ein neues Lagerlayout erstellt. Es enthält einen Lagerzugang an der virtuellen Position  $x = 0$  und  $z = 0$  sowie Lagerplätze auf beiden Regalseiten entsprechend der Eingaben. Alle Felder der Lagerplätze sind mit Null belegt, bzw. es ist LIFO als „Strategy“ und INOUT als „Action“ voreingestellt. Nach Ausführung dieser Aktion befindet sich in der globalen Variablen `hrw` der des `FormMain`-Objekts eine reguläre, jedoch automatisch generierte Lagerkonfiguration.

**DatabaseActions** In dieser Klasse werden alle Ereignisse des Punktes „Database“ im Hauptmenü verarbeitet. Für jeden Menüpunkt wird dabei eine eigene Methode aufgerufen. Punkt 1 besteht dabei lediglich aus einer Abfrage bezüglich Datenbankname und Benutzername sowie der anschließenden Anzeige einer Information mit angepassten SQL-Kommandos. Für den Verbindungsaufbau mit Punkt 2 wird die globale Variable `databaseSetupConnection` des `FormMain`-Objekts herangezogen. Sie ist vom Typ `DatabaseConnection` des Packages `de.i2ar.warehousedirector.connectivity` und besitzt Methoden zum Pollen der Lagerbelegung im Visualisierungs- und Benchmarkingmodus sowie Methoden zur Durchführung aller im „Database“-Menü vorhandenen Funktionen. Viele davon beginnen mit `create` und enden entsprechend ihrer Intention. Es finden allgemein übliche SQL-Ausdrücke geringer Komplexität Anwendung, sodass hierauf nicht näher eingegangen werden soll. Es gilt lediglich zu beachten, dass die Reihenfolge der Löschoperationen an den Abhängigkeiten der Datenbankressourcen untereinander ausgerichtet ist und dass die Menüpunkte unter „Database“ daher auch nicht in beliebiger Reihenfolge oder beliebig oft ausgeführt werden können. Ebenfalls erwähnt werden soll auch, dass sich alle Methoden auf die lokale Variable `conn` der `DatabaseConnectivity`-Instanz beziehen. Diese Variable stellt eine Referenz auf die hergestellte Datenbankverbindung dar. Jede Instanz der Klasse baut somit eine eigene Verbindung auf und kann daher auch auf eine andere Datenbank verweisen oder andere Benutzerkennungen verwenden. Dies wird aus Sicherheitsgründen zur Unterscheidung des Einrichters (Menüpunkt „Database“) und des Beobachters (Visualisierung) genutzt. Die übrigen Punkte 3 bis 6 greifen auf die genannten `create`-Methoden zu, der letzte Punkt auf die Methode `disconnect`.

**FileActions** Über die Klasse `FileActions` werden alle Funktionen realisiert, die über

den Punkt „File“ des Hauptmenüs aufgerufen werden können. Neben der `action`-Methode finden sich hier dedizierte Methoden für alle Menüpunkte sowie Methoden zur Handhabung der geöffneten Lagerkonfiguration bzw. zum Laden einer solchen. Die Methode `loadFile` wird als Ereignisbehandlung bei einem Klick auf den Menüeintrag `Open` ausgeführt. Sie übernimmt die Benutzerinteraktion zum Öffnen einer Datei. Eine Datei kann nur geöffnet werden, wenn sich die Anwendung nicht im Benchmarking-Modus befindet. Wurden Änderungen an der bereits geöffneten Datei gemacht, so wird zunächst hierauf hingewiesen und die direkte Speicherung angeboten sowie der Abbruch der Aktion und das Ignorieren des Umstands angeboten. Soll gespeichert werden, wird die aktuelle Konfiguration einfach überschrieben, falls sie bereits einen Dateinamen besitzt. Ansonsten wird der Anwender aufgefordert, einen Dateinamen zu vergeben. Schließlich wird ein Öffnen-Dialog zur Dateiwahl angezeigt und die Auswahl an die Methode `doLoadFile` weitergereicht. Sie ist für die Verwaltung der Datei, Ändern der Fensterbeschriftung und natürlich dem eigentlichen Laden der Datei über die statische Methode `loadFromFile` der Klasse `HRWConfig` zuständig. `loadFromFile` gibt ein Objekt der Klasse `HRWConfig` zurück, das im Anschluss in der globalen Variable `hrw` des `FormMain`-Objekts hinterlegt wird.

Zum Speichern von Dateien verhält es sich ähnlich. Über die Ereignisbehandlung der GUI wird die Methode `saveFile` aufgerufen. Sie besitzt ein Argument, über welches definiert werden kann, ob eine Rückfrage in Bezug auf den Dateinamen durchgeführt werden soll. Dies wird von „Save as...“ genutzt. Nach Sicherheitsabfragen wird schließlich die Methode `doSaveFile` aufgerufen, die sich wiederum der Methode `writeToFile` des globalen `HRWConfig`-Objekts `hrw` bedient, um die aktuelle Konfiguration in der Datei mit dem gewählten Namen zu speichern.

Auch das Verlassen des Warehouse Directors über „File/Quit“ oder durch Schließen des Fensters wird über `FileActions` behandelt. Hierzu existiert die Methode `quitApp`. Sie prüft zunächst, ob eine geöffnete Datei noch nicht gespeichert wurde und fordert erwartungsgemäß dazu auf.

Wurden Daten der Lagerkonfiguration geändert, so wird die Methode `setChangesMade` dieser Klasse aufgerufen. Sie führt zu einem entsprechenden Vermerk und zur Veränderung der Titelzeile des Hauptfensters. Die Methode `loadFile` reagiert entsprechend auf diesen Vermerk. `doLoadFile` und `doSaveFile` nutzen `setChangesMade` selbst, um den Vermerk bei neu geöffneter Datei oder gerade gespeicherter Datei zurückzusetzen.

**KeyActions** behandelt die Tastendruck-Ereignisse der drei Eingabefelder auf der Registerkarte „General“ des Hauptfensters. Eine Eingabe muss zum Vermerk führen, dass die aktuelle Konfiguration geändert wurde und vor dem Schließen der Datei bzw. des Fensters eine Rückfrage erfolgt, ob die Datei gespeichert werden soll.

**ModeActions** Diese Klasse ist für die Umschaltung zwischen den drei Modi „Configuration“, „Visualisation“ und „Benchmarking“ zuständig. Die Umschaltung wird durch die Auswahl der Einträge im Menü „Mode“ ausgelöst. Dabei wird jeder

Modus durch eine eigene Methode in dieser Klasse eingeleitet. Wird in den Konfigurationsmodus gewechselt, erfolgt zunächst eine Nachfrage, ob eine bestehende Verbindung zunächst abgebaut werden soll. Im Anschluss wird bei den ersten beiden Modi dafür gesorgt, dass ein laufender Benchmark abgebrochen wird. Einzelne Menüpunkte und andere GUI-Elemente werden je nach Modus für die Benutzung aktiviert oder deaktiviert, da nicht in jedem Modus alle Optionen zur Verfügung stehen wie beispielsweise die Wahl der Darstellungsgröße: Nur im Rahmen der Konfiguration kann die detaillierte Ansicht aktiviert werden. Erfordert es ein Modus, dann wird mit Hilfe der Methoden `userReconnectToOperator`, `userReconnectToDatabase` und `userReconnectToAgent` dafür gesorgt, dass entsprechende Verbindungen wiederhergestellt oder erstmalig aufgebaut werden. Für den Visualisierungsmodus wird eine Verbindung zu einem Operator und zu einer Datenbank hergestellt, für den Benchmarkingmodus zudem noch eine Verbindung zu einem Agenten. Die hierfür notwendigen Benutzereingaben werden in den eben genannten Methoden realisiert. Sie führen außerdem den Verbindungsaufbau durch und nutzen dazu die Klassen `DatabaseConnection`, `OperatorConnection` und `AgentConnection` aus dem Package `de.i2ar.warehousedirector.connectivity`. Wird der Benchmarkingmodus aufgerufen, erfolgt die Instanziierung und Anzeige eines weiteren Fensters, in dem der Benutzer das Benchmarking parametrieren und steuern kann (s. Abb. 6.10).

**VisualActions** Ein Großteil des Codes dieser Klasse befasst sich mit dem Zeichnen der aktuellen Lagerkonfiguration im Layoutbereich des Hauptfensters, jedoch befindet sich hier auch die Ereignisbehandlung für die Unterpunkte des Menüs „View“. Findet an einer Stelle im Programm eine Veränderung des Lagerlayouts statt und muss es anschließend auf dem Bildschirm aktualisiert werden, so wird der entsprechende Programmteil die Methode `updateImage` dieser Klasse aufrufen. Sie befasst sich mit der Aktualisierung der GUI-Komponenten und ruft selbst die Methode `buildImage` auf, welche für die Erzeugung des grafischen Abbilds des Lagerlayouts zuständig ist. Dort werden zunächst die Dimensionen des Lagers neu berechnet, da sie sich durch die letzte Änderung verändert haben könnten. Im Anschluss wird die neue Breite und Höhe der Grafik einschließlich aller Abstände berechnet und die Grafik als Instanz der Klasse `BufferedImage` erzeugt. Anhand der als Argument übergebenen Variablen `cfg` und `zone` sind die Lagerkonfiguration auf oberster Ebene sowie die abzubildende Lagerzone bekannt, die neu dargestellt werden muss. Über die in ihr referenzierten Lagerzugänge und -plätze (Objekte der Typen `HRWGate` und `HRWCell`) wird iteriert und zu jedem einzelnen Objekt eine entsprechende Repräsentation mit elementaren Zeichenoperationen in das zuvor erzeugt `BufferedImage` gezeichnet. Dabei finden die Menüoptionen zur den beiden Größenstufen wie auch Spiegelung und Ein-/Ausblendung von Lagerplätzen und -zugängen Berücksichtigung. Die Methode `buildImage` wird schließlich mit dem `BufferedImage` als Rückgabewert beendet.

Die Methode `updateVisualisationImage` entspricht weitestgehend der Methode `updateImage`. Hier wird jedoch speziell auf die Anforderungen der Visualisierung

eingegangen, sodass vor der Aktualisierung der GUI-Elemente die mit `buildImage` regulär gezeichnete Grafik entsprechend einem als Argument übergebenen Objekt der Typs `ReceivedObject` abgeändert wird, um die durch das Argument beschriebene Lageroperation in die grafische Darstellung zu integrieren. Dazu wird die Methode `alterVisualisationImage` – ebenfalls in der Klasse `VisualActions` befindlich – mit dem gleichen `ReceivedObject` als Argument aufgerufen. Sie nimmt schließlich die Anpassungen anhand der Felder im `ReceivedObject` vor.

Überdies existiert eine Methode `updateOutlineTree`, die für eine Aktualisierung der Baumansicht der Lagerzonen auf der „Visualisation“-Registerkarte zuständig ist. Sie wird nach Änderungen über die sich unterhalb befindlichen Buttons aufgerufen und führt zu einem Neuaufbau der Datenstrukturen der Baumansicht.

**WindowActions** definiert lediglich, dass beim Versuch, das Hauptfenster zu schließen, die gleiche Aktion durchgeführt wird, als hätte der Benutzer im Hauptmenü „File/-Quit“ geklickt. Damit wird bei verändertem Layout zunächst eine Rückfrage durchgeführt und die Lagerkonfiguration ggf. gespeichert, bevor das Fenster geschlossen wird.

## 6.2 Visualisierung

Zur Lagerkonfiguration und nach deren Erstellung kann diese visualisiert werden. Während im Entwurfsmodus nur die Zellen einer Lagerzone mit ihren Parametern dargestellt werden, können während des Lagerbetriebs auch aktuelle Vorgänge beobachtet werden. So kann die aktuelle Belegung aller dargestellten Zellen und die aktuelle Position der Aufnehmereinheit bzw. deren aktuelle Route visualisiert werden.

### 6.2.1 Warehouse Operator Integration

#### Funktionsweise

Das Systemmodell bei der Visualisierungsfunktion ist in Abbildung 6.7 dargestellt.

Die Integration der Visualisierung erfolgt mit Hilfe eines Warehouse Experts, der statt einem üblichen Experten zur Hardwaresteuerung im Operator eingebunden wird. Dieser erlaubt zum einen die Weiterleitung der durchgeführten Lageroperationen, zum anderen jedoch auch, Simulationenwerte einzuspielen, um damit die Ergebnisse eines echten, hardwaregebundenen Warehouse Expert vorzutäuschen. Der Simulationsexperte `SimulationExpert` besitzt eine Konfiguration (siehe Beispiel in Listing 6.4), die auf einen zweiten Experten verweist. Dieser *Second-Level-Expert* ist derjenige, der in einer nicht visualisierten Umgebung geladen würde. Er wird durch den Simulationsexperten nachgeladen und alle Anforderungen bezüglich Lageroperationen und Konfiguration werden an ihn

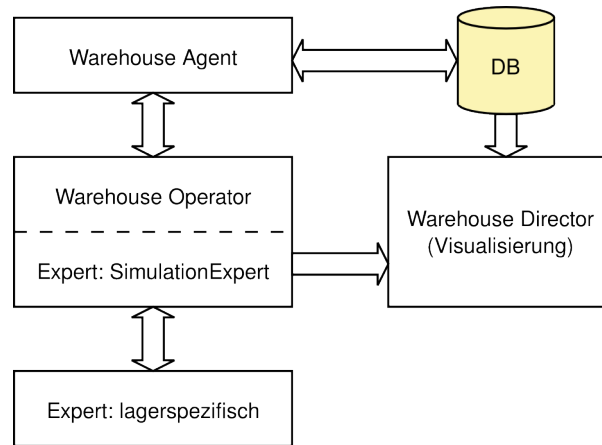


Abbildung 6.7: Systemmodell der Visualisierungsfunktion.

weitergeleitet. Der Ablauf ist als UML-Sequenzdiagramm in Abbildung 6.8 dargestellt und zeigt, wie die Aufrufe vom Agenten über das Agenten-Operator-Protokoll zum Operator und von dort aus zunächst zur Warehouse Extension, dann zum Simulationsexperten und schließlich zum Second-Level-Experten gelangen. Ebenfalls dargestellt ist der Warehouse Director in seiner Aufgabe als Visualisierung, sodass hier auch gezeigt wird, wie dieser an die Informationen zu aktuellen Lageroperationen gelangt.

Listing 6.4: Konfigurationsdatei des Simulation Expert.

```

1 <simulationExpertConfig>
2   <simulation-host>127.0.0.1</simulation-host>
3   <simulation-port>2012</simulation-port>
4   <expert-class>de.i2ar.warehouseop.experts.dummy.
      WarehouseExpert </expert-class>
5   <expert-config>null</expert-config>
6 </simulationExpertConfig>

```

Der Simulationsexperte ist neben seiner Eigenschaft als Warehouse Expert auch ein Serverprogramm. Es erlaubt bis zu 16 gleichzeitige Verbindungen von Visualisierungsclients. Jede der vom Interface `IWarehouseExpert` implementierten Methoden führt dazu die entsprechende Kommunikation mit den Clients durch. Das Kommunikationsprotokoll hierzu ist in Abschnitt 6.2.1 beschrieben. Bei der Anmeldung eines Clients am Simulationsexperten muss dieser übermitteln, ob eine reine Visualisierung oder eine Simulation über diese Verbindung durchgeführt werden soll. Da es sich bei der Visualisierung um eine unidirektionale Kommunikation handelt und bei der Simulation Daten in anderem Umfang und bidirektional ausgetauscht werden, müssen diese Fälle auch bei der Ausführung jeder Lageroperation unterschieden werden. Bei Visualisierung wird zunächst eine Information nach dem Kommunikationsprotokoll an alle Clients versandt und im Anschluss die gleiche, aber dann blockierende Methode des Second-Level-Experten aufgerufen. Im Simulationsbetrieb wird dieser Second-Level-Experte hingegen nicht konsultiert, sondern

nach Senden der Information auf eine Antwort vom Simulationsclient gewartet, die daraufhin nach Kommunikationsprotokoll und Lageroperation ausgewertet wird. Dieser Vorgang ist auch noch einmal als Programmablaufplan in Abbildung 6.9 dargestellt.

**Da die Simulationsfunktion architekturbedingt erst nach Starten des Warehouse Operators aktiviert werden kann, ist bei Simulationen ausdrücklich davon abzuweichen, den Warehouse Operator mit einem hardwaregekoppelten Experten zu starten. Hierfür kann der Dummy Experte `WarehouseExpert` aus dem Package `de.i2ar.warehouseop.experts.dummy` eingesetzt werden, der zwar alle Methoden des Interface `IWarehouseExpert` implementiert, jedoch keine Aktionen durchführt.**

Die Darstellung erfolgt automatisch im Hintergrund. Durch das ständige Pollen der Datenbank liegen immer aktuelle Werte für alle momentan angezeigten Lagerplätze vor. Die Anzeigeroutine, die zyklisch aufgerufen wird, kann zwischen Konfigurationsmodus und den beiden anderen Modi unterscheiden. In letzterem Fall wird die Belegung der Lagerplätze entsprechend farbig dargestellt. Dabei steht rot für eine komplette Belegung, orange eine teilweise Belegung (tritt nur bei mehrfachtiefen Lagerplätzen auf) und grün für gar keine Belegung.

### Umsetzung in Java

Der Simulationsexperte ist eine Implementation des Interfaces `IWarehouseExpert`, wie alle hardwaregebundenen Warehouse Experts auch. Beim Start des Operator Skeletons wird die Warehouse Extension instanziiert und mit ihrer Methode `setConfigurationFileName` initialisiert. Diese wiederum sieht die Instanziierung des konfigurierten Warehouse Experts und den Aufruf dessen Methode `initExpert` vor. Beim Simulationsexperten wird diese Methode zunächst ihre eigene Konfiguration laden, die schon in der Konfigurationsdatei der Warehouse Extension angegeben wurde. Anschließend wird ein Objekt der privaten, sich in `SimulationExpert` befindlichen Klasse `Communicator` erzeugt, unter Angabe des konfigurierten Simulationshosts und dessen Portnummer, und in der Klassenvariablen `com` hinterlegt, um allen Methoden Zugriff auf die Kommunikation mit Clients zu ermöglichen.

Im Konstruktor der Klasse `Communicator` wird ein `ServerSocket` mit obigen Angaben erstellt und ein Thread der Klasse `Acceptor` gestartet, der von nun an auf eingehende Verbindungen lauscht. Die verbundenen Clients werden zunächst mit `HELLO` begrüßt und müssen dann mit `SIMULATION` oder `VISUALISATION` antworten, je nachdem, ob sie selbst als Simulator dienen wollen oder nur die Lageroperationen visualisieren möchten.

Die `Socket`-Objekte aller verbundenen Clients werden mit ihren In- und Output-Streams in einem Array gehalten, sodass in Zukunft direkter Zugriff hierauf besteht. Da es jedoch nur einen einzigen Client geben darf, der als Simulator fungiert, wird der Index dessen `Socket`-Objekts im Array in der Variablen `simulationSocket` festgehalten und später wieder herangezogen. Die Klasse `Communicator` besitzt des Weiteren zwei `send`-Methoden, die für den Versand eines Strings an alle angemeldeten Clients zuständig sind. Eine der beiden Methoden erwartet ein zweites Argument neben dem String, das angibt, ob auf eine Empfangsbestätigung jedes einzelnen Clients gewartet werden soll. Um einer Verklemmung vorzubeugen, falls ein Client bereits nicht mehr existiert (das Senden der Nachricht ist dennoch erfolgreich), wurde ein Timeout eingeführt. Ist diese Zeit verstrichen, wird der jeweilige Client aus dem Array entfernt und in Zukunft nicht weiter berücksichtigt. Die erste `send`-Methode geht stillschweigend davon aus, dass eine Empfangsbestätigung erwünscht ist und ruft das andere `send` entsprechend selbst auf. Die Methode `rcvd` dient dem Empfang einer Nachricht, jedoch nur vom Client, der sich als Simulator angemeldet hatte und falls dieser überhaupt noch mit dem `SimulationExpert` verbunden ist. Alle Methoden zum Senden und Empfangen von Nachrichten lösen im Fehlerfall unterschiedliche Exceptions aus, sodass dem Aufrufer bekanntgegeben wird, welcher Fehler bei der Kommunikation aufgetreten ist.

Die in `IWarehouseExpert` deklarierten und in `SimulationExpert` implementierten Methoden versenden zunächst per Methode `com.send` eine Information an den Simulator bzw. die Visualisierung. Ihr Aufbau entspricht dem in Absatz 6.2.1 erläuterten Kommunikationsprotokoll. Im Fall, dass keine Simulation vorliegt (liegt ab der ersten Anmeldung eines Simulators bis zum Programmende vor), wird die gleichnamige Methode mit exakt den gleichen Argumenten aufgerufen, die von der Methode selbst empfangen wurden. Der Client wird mit einer Nachricht über den erfolgreichen oder fehlgeschlagenen Abschluss der Operation wieder über `com.send` bzw. `throwException` benachrichtigt. Die Ergebnisse des Aufrufs werden als eigene Ergebnisse zurückgegeben und die Methode damit verlassen. Liegt jedoch eine Simulation vor, so wird nach dem Senden der Information auf eine Antwort des Simulators mit `com.rcvd` gewartet. Der Rückgabewert von `rcvd` ist ein String. Je nach implementierter Methode des Interfaces `IWarehouseExpert` werden andere Datentypen als Rückgabewert erwartet. Daher ist eine spezifische Verarbeitung der empfangenen Daten notwendig. Hierzu existieren die Methoden `longLongObjectOrException`, `StringOrException`, `StringListOrException`, `StringStringListOrException` und `voidOrException`. Sie versuchen aus den empfangenen Daten entweder zwei durch ein Dollarzeichen getrennte Long-Werte zu extrahieren und als `Object`-Array zurückzugeben, einen String zu extrahieren und zurückzugeben, eine Liste Strings, durch Dollarzeichen getrennt, zu extrahieren, eine Liste von String-Wertepaaren, ebenfalls durch Dollarzeichen voneinander getrennt, zu extrahieren (dabei muss die Anzahl der String-Elemente gerade sein) oder einfach nur festzustellen, dass kein Wert in den empfangenen Daten enthalten ist. Diese Methoden sind generell anhand der Kommunikationsprotokolls in der Lage, zwischen einer erfolgreichen Ausführung (Daten beginnen mit OK) und einem aufgetretenen Fehler (Daten beginnen mit EXC-) über die Methode `checkException` zu unterscheiden. Bei einem Fehler



wird zudem zwischen drei unterschiedlichen differenziert und dann eine entsprechende Exception ausgelöst, die den weiteren Programmablauf beeinflussen wird. Da diese verarbeitenden Methoden bereits die gleichen Rückgabetypen besitzen wie die im Interface `IWarehouseExpert` deklarierten Methoden, können diese die Rückgabewerte abschließend einfach selbst zurückgeben.

### Kommunikationsprotokoll

Jede Kommunikation besteht aus genau einer Textzeile, die mit dem Zeichen `0x0d` (New-line) abgeschlossen sein muss.

1. Nach Verbindungsaufbau sendet der Simulationsexperte `HELLO`.
2. Der Client sendet seinen Wunsch, ob er Simulator oder nur Visualisierung sein möchte, mit `SIMULATOR` bzw. `VISUALISATION`.
3. Bei eingetretener Lageroperation sendet der Simulationsexperte  
`UNIX-Timestamp$Operation-START`  
Für jedes Argument der Methode wird `$` und das als String kodierte Argument angehängt.
4. Der Client sendet `RCVD` als Empfangsbestätigung.
5. **Simulator** Nach Berechnung der Ergebnisse der simulierten Operation sendet der Client `OK` und wenn dem Interface `IWarehouseExpert` nach für diese Operation Ergebnisse erwartet werden, dann werden diese jeweils mit `$` und einem als String kodierten Wert angehängt. Die für die Methode `reportZones` erwartete String-Liste wird mit  
`$Wert 1$Wert 2$...`  
kodiert. Ebenso die String-Wertepaar-Liste für die Methoden `reportEntrances` und `reportExits`: Hier sind die gesendeten Werte abwechselnd Schlüssel und Wert eines Wertepaars. Trat ein Fehler bei der Berechnung auf, so kann mit `EXC-HF$Fehlerbeschreibung` geantwortet werden. Dies wird im `SimulationExpert` eine `HardwareFailureException` auslösen. Statt `HF` kann auch `PF` zur Auslösung einer `ProcessingFailureException` oder `IA` zur Auslösung einer `InvalidArgumentException` verwendet werden.  
**Visualisierung** Nach Ausführung der entsprechenden Methode des Second-Level-Experts wird im Erfolgsfall  
`UNIX-Timestamp$Operation-RESULTS`  
an den Client gesendet, im Fehlerfall hingegen  
`UNIX-Timestamp$Operation-EXCEPTION$Fehlerbeschreibung`
6. Die Prozedur wird bei 3. fortgesetzt.

## 6.2.2 Warehouse Director Integration

### Funktionsweise

Die Visualisierungsfunktion ist eng in den Warehouse Director und die Konfigurationsoberfläche integriert. Wird im Menü „Mode“ der Punkt „Visualisation“ ausgewählt, dann wird der Benutzer, wie oben bereits beschrieben, zur Eingabe der Datenbankverbindungsdaten und der IP-Adresse des für die Lagersteuerung zuständigen Operators aufgefordert. Beide Verbindungen werden daraufhin hergestellt und im Hintergrund der Anwendung verarbeitet. Bei der Datenbankverbindung findet dabei ein ständiges Polling aller zur Zeit auf dem Bildschirm dargestellten Lagerplätze hinsichtlich ihres Belegungsgrades statt, um ein aktuelles grafisches Abbild der Lagerbelegung erzeugen zu können.

### Umsetzung in Java

Die für die Datenbankverbindung zuständige Klasse `DatabaseConnection` wurde bereits im Rahmen der Menüpunkte zur Datenbankeinrichtung erläutert, jedoch sind noch nähere Ausführungen hierzu und zu den Verbindungen mit Operator und Agent notwendig. Diese sollen ebenso an den entsprechenden Klassen geschehen:

**DatabaseConnection** Sobald eine Datenbankverbindung zum Polling genutzt wird (Instanziierung mit entsprechendem Argument), ruft diese zyklisch in einem Thread alle Belegungsinformationen zu einer als Argument übergebenen Lagerzone ab. Um diese Informationen den übrigen Programmteilen bereitzustellen, damit sie mit der geladenen Lagerkonfiguration verknüpft werden können, bedient sich der zyklisch aufgebaute Thread der Methoden des Java-Interfaces `IDatabaseCallbacks`. Sie müssen durch eine anwendungsspezifische Klasse implementiert werden und dienen der Verarbeitung der eingelesenen Daten. Auch ein manuelles Pollen ist möglich. Hierzu muss lediglich die Methode `poll` der `DatabaseConnection` aufgerufen werden, wobei die eingelesenen Daten abermals über das erwähnte Interface bereitgestellt werden. Die implementierte Klasse des Interfaces muss zuvor mit der Methode `setCallbacks` als Referenz übergeben werden. Die Abbildung der Daten erfolgt anhand der Klasse `HRWCellVis`. Sie enthält zur Zeit ein einziges Feld `load`, das den Füllgrad des Lagerplatzes widerspiegelt.

**DatabaseCallbacks** ist eine anwendungsspezifische Implementation des Interfaces `IDatabaseCallbacks` und ordnet Lagerbelegungsdaten den Lagerplätzen der Konfiguration zu. Hierzu existieren die Methode `updateCells`. Sie erwartet als Argumente die UID des betroffenen Lagerplatzes sowie das über das Polling der Datenbank zusammengestellte `HRWCellVis`-Objekt. Der Lagerplatz mit der gegebenen UID wird gesucht und die Referenz auf die Daten in dessen Feld `vis` gespeichert. Die Auswertung – oder im Fall der Visualisierung die Anzeige – übernimmt die Klasse `VisualActions` mit ihren Methoden zur Generierung des grafischen Lagerlayouts.

**OperatorConnection** Zur Kommunikation mit einem Operator kann nicht die von den Agenten bekannte Bibliothek `libop` verwendet werden. In diesem Fall findet kein Datenaustausch zur Veranlassung von Aktionen durch den Operator statt, sondern es wird eine Verbindung mit dem Warehouse Expert `SimulationExpert` hergestellt (s. Abs. 6.2.1). Genau genommen wird somit keine Verbindung zum Operator Skeleton hergestellt wie es sonst üblich ist. Die Klasse `OperatorConnection` besitzt eine Methode `connect`, die vom Hauptprogramm beim Moduswechsel zur Verbindungsherstellung aufgerufen wird und als Argumente die IP-Adresse sowie Portnummer erwartet und zusätzlich eine Angabe, ob es sich um eine Simulationsverbindung handelt oder um eine Visualisierungsverbindung. Bei ersterer Variante handelt es sich um eine bidirektionale Kommunikation, sodass der Warehouse Director statt dem eigentlichen Warehouse Expert mit der Ausführung von Lageroperationen betraut wird und dieser somit eine Simulation durchführen kann. Die Visualisierungsverbindung ist unidirektional und dient ausschließlich der Information des Warehouse Directors über aktuelle Lageroperationen. Nach Verbindungsaufbau wird `SIMULATION` bzw. `VISUALISATION` an den Operator gesendet, um die Art der Verbindung bekannt zu geben. Im Anschluss wird ein Thread eingerichtet, der die weitere Kommunikation übernimmt. Es wird zyklisch eine Zeile vom Operator eingelesen und der Empfang mit `RCVD` quittiert. Die empfangene Textzeile besitzt je einen definierten Aufbau, bestehend aus mehreren Datenfeldern, die durch ein Dollarsymbol voneinander getrennt sind. Sie werden ausgewertet und in den Feldern einer neuen Instanz der Klasse `ReceivedObject` gespeichert. Je nach Art der Nachricht (Ausführungsbeginn, Ausführungsergebnisse, Fehler) wird eine entsprechende Methode `handleStartObject`, `handleResultsObject` oder `handleExceptionObject` aufgerufen, die die Verarbeitung spezifischer Felder der Textzeile übernimmt. Alle Informationen werden samt Fehlerzuständen anschließend als `ReceivedObject` an eine Callback-Methode übergeben. Dieses Verfahren entspricht dem bei `DatabaseConnection` genannten. Es existiert dazu ein Interface `IOperatorCallbacks` mit den Methoden `inform` und `react`, wobei `inform` zur Benachrichtigung neuer Ereignisse aufgerufen wird, während mit `react` im Benchmarkingmodus aufgefordert wird, einen Simulationswert zurück an den Operator zu senden.

**OperatorCallbacks** Diese spezifische Implementation des erwähnten Interfaces bedient sich in der Methode `inform` der Methoden `updateVisualisationImage` und `doCleanVisualisation` der Klasse `VisualActions`, um das als Argument übergebene `ReceivedObject`-Objekt zur grafischen Anzeige zu bringen, falls es sich dabei um einen Ausführungsbeginn handelt bzw. in den anderen Fällen die visualisierungsspezifischen Grafikausgaben wieder zu entfernen. Die Methode `react` hingegen erzeugt eine eigene `ReceivedObject`-Instanz, um die Simulationsergebnisse der jeweiligen eingeleiteten Lageroperation letztendlich als Ergebnis der Methode zurückzugeben. Hierzu wird zwischen den verschiedenen Lageroperationen unterschieden. Interessant sind die Operationen zur Ein-, Aus- und Umlagerung von Gütern. Hier wird anhand der aktuellen RBG-Position, die noch vom vorher-

rigen Ereignis in einer Klassenvariablen gespeichert ist, die horizontale und vertikale Differenz zur Zielposition der Lageroperation bestimmt. Diese Differenzen werden daraufhin mit den Benutzereingaben (s. Abb. 6.10) zur simulierten Geschwindigkeit und dem simulierten Energieverbrauch des RBG in der Methode `calculateResults` verrechnet, um somit eine Fahrzeit und die dabei aufgewendete Energie zu erhalten und an den Operator zurücksenden zu können. Die Berechnung basiert auf einfachen physikalischen Gesetzen, wobei der einfachste Fall angenommen wird, in dem Geschwindigkeit und Energieverbrauch an jedem Punkt der zurückgelegten Strecke konstant sind. Den Rückversand der Rechenergebnisse übernimmt jedoch die `OperatorConnection` im Anschluss an den Aufruf der `react`-Methode.

**ReceivedObject** Ein `ReceivedObject` repräsentiert einen Satz zwischen Operator und Warehouse Director ausgetauschter Daten. Dabei enthält jede Instanz eine Angabe im Feld `command`, um welche Art von Lageroperation bzw. Antwort vom Warehouse Director es sich handelt. Prinzipiell sind alle Felder vorhanden, die auch als Parameter in die Operatorkommunikation mit der Warehouse Extension eingehen. Felder wie `command` oder `exceptionType` dienen der Abbildung des Agenten-Operator-Kommunikationsprotokolls an sich.

## 6.3 Benchmarking

### 6.3.1 Funktionalität

#### Warehouse Operator

Wenn basierend auf einer Lagerkonfiguration eine Simulation durchgeführt werden soll, so ist dies für den Warehouse Operator ebenso transparent wie die Einbindung der Visualisierungserweiterung für den Operator. Lediglich diese ist über die zu erfolgende Simulation unterrichtet, unterbindet die weiterführenden Aufrufe der Methoden des Second-Level-Experten und kommuniziert stattdessen mit dem Warehouse Director. Das Systemmodell bei Einsatz des `SimulationExpert`, der sich Simulationsmodus befindet, ist in Abbildung 6.11 dargestellt. In rot hervorgehoben sind die Unterschiede im Vergleich zu Abb. 6.7, die beim Simulationsbetrieb entstehen.

#### Warehouse Agent

Der Warehouse Agent erstellt mit seiner `SimulationBehaviour` einen Server-Socket. Sobald sich ein Client hierzu verbindet, schaltet das `StorageFace`, das weiterhin für alle Lageroperationen verantwortlich ist, in den Simulationsmodus. Das heißt, dass aus

Sicherheitsgründen nur noch Anfragen, die von der `SimulationBehaviour` stammen, ausgeführt werden. Damit ist bei einer Simulation im nicht-isolierten Umfeld eine Mischung mit regulären, vom Agentensystem stammenden Anfragen ausgeschlossen. Ein neu verbundener Client wird zunächst mit `HELLO` begrüßt. Anschließend werden zyklisch Anfragen entgegengenommen. Der Client kann eine Einlagerungs- oder eine Auslagerungsanforderung senden. Diese besteht aus dem Schlüsselwort `IMPORT$` bzw. `EXPORT$`, gefolgt von einem als XML-Dokument kodierten `Storage`-Objekt. Dazu wurde auf Seiten des Clients die Methode `toXml` eines `Storage`-Objekts aufgerufen. Hieraus ist dann bekannt, ob eine Ein- oder Auslagerung stattfinden soll.

Da abgesehen von oben genannter Sicherheitsvorkehrung im `StorageFace` keinerlei spezifische Anpassungen für eine Simulation getätigt wurden, erwartet diese für eine Einlagerung, dass der Übernahmehaltepunkt für das Gut gesperrt ist bzw. dass der Übergabehaltepunkt bei einer Auslagerung nicht gesperrt ist. Da bei der Simulation jedoch die ansonsten stattfindenden Transporte ausbleiben, sorgt die `SimulationBehaviour` in beiden Fällen zunächst dafür, dass der Haltepunkt bedingungslos gesperrt bzw. entsperrt wird. Im Anschluss wird die `execute`-Methode des `StorageFace` mit zweitem Argument aufgerufen, das auffordert, den gesetzten Hinweis zum Simulationsmodus zu ignorieren und die Anforderung auszuführen.

Nach erfolgreicher Ausführung der Anforderung wird der Client mit einer `DONE`-Nachricht informiert. Sie enthält folgende Angaben:

- Lagerzellen-UID,
- zugehörige Metrik,
- X-Positionsindex,
- Y-Positionsindex,
- Z-Positionsindex des Lagerplatzes,
- Ausführungszeit der Optimierungsstrategien und
- Ausführungszeit des Operators.

Im Fehlerfall erfolgt eine Benachrichtigung über `FAILED$`, gefolgt von der Nachricht der `Exception`, die den Fehlerfall beschreibt. Unabhängig von Erfolg oder Misserfolg wartet die `SimulationBehaviour` anschließend auf eine neue Anfrage eines Clients.

### **Warehouse Director**

Wechselt ein Anwender im Warehouse Director in den Benchmarkingmodus, dann ist es zunächst erforderlich, dass dieser Angaben zu Operator-, Datenbank- und Agent-Host macht. Sind diese Verbindungen hergestellt (Klasse `ModeActions`), wird ein Fenster mit diversen Einstellungen zum Benchmarking angezeigt (s. Abb. 6.10). Hier muss neben

einer einheitlichen Höhe, Breite und Tiefe der Lagerplätze eine Geschwindigkeit für horizontale, vertikale und quer verlaufende Bewegungen des RBG in  $m/s$  sowie Energieverbräuche dieser Bewegungen definiert werden. Diese werden bei der Simulation des Warehouse Expert zur Bestimmung der Ausführungsdauer und des Energieverbrauchs herangezogen. Um die Komplexität gering zu halten, wird auf eine Berücksichtigung von Beschleunigungsverhalten, Reibung usw. verzichtet. Auch muss ein Übergabepunkt angegeben werden, der der gewählten Lagerzone in der StorageFace-Konfiguration eindeutig zugeordnet ist. Dieses umständliche Verfahren ist nötig, da so der Aufwand bei der Implementation des StorageFace gering bleibt und eine nahezu gleiche Ausführung gegenüber des regulären Betriebs, der auch eine Zuordnung von Übergabepunkt zu Lagerzone vornimmt, garantiert werden kann. Des Weiteren wird die Wahl eines Simulationsmusters erwartet. Dieses bestimmt die ein- und auszulagernden Güter und die Reihenfolge, in der die Lageroperationen durchgeführt werden sollen. Damit lassen sich verschiedene Szenarien (s. Abs. 6.3.3) abbilden.

Durch Klick auf „Run“ werden der Benchmark und die Simulation gestartet. Zur Protokollierung der einzelnen Schritte, die letztendlich alle auf den Warehouse Operator und damit auf das Simulationsmodul des Warehouse Directors abgebildet werden, wird noch um die Angabe eines Dateinamens gebeten. Daten werden im CSV-Format (*Comma Separated Values*) gespeichert, sodass eine Betrachtung z. B. mit OpenOffice.org Spreadsheet problemlos möglich ist. Die enthaltenen Daten geben pro Zeile Auskunft über die Anforderung und das Ergebnis der Lageroperation, wie es der Agent an den Warehouse Director zurückliefert. In der ersten Zeile der CSV-Datei sind die Spalten entsprechend beschriftet, sodass eine Interpretation auch ohne tiefgreifende Systemkenntnisse möglich sein sollte. Zwar generiert das Simulationsmodul des Warehouse Director die Operator-Ausführungszeit selbst und gibt sie zusätzlich als Ergebnis an den SimulationExpert und damit an den Operator und Agenten zurück, jedoch ist es am einfachsten, wenn der Warehouse Director alle Werte, die zu einer Anfrage gehören aus einer einzigen Quelle bezieht, nämlich der Antwort des Agenten auf eine Anfrage. Der Fortschritt und Vorgang des endlichen Benchmarks kann einerseits anhand des Fortschrittbalkens im gleichen Fenster und anhand der Visualisierung im Hauptfenster des Warehouse Directors verfolgt werden, welche genauso abläuft wie im reinen Visualisierungsmodus. Der Vorgang kann im Übrigen jederzeit durch einen Klick auf den Button „Abort“ unterbrochen werden. Unter Umständen ist eine Neuanswahl des Benchmarkingmodus notwendig, um abgebrochene Verbindungen erneut aufzubauen. Nach Abschluss des Benchmarks wird eine zusammenfassende Statistik bezüglich der gemessenen Ausführungszeiten von Optimierungsstrategien und Hardwaresteuerung ausgegeben.

Überdies ist anzumerken, dass die zur Simulation notwendig Systemkonfiguration folgende Punkte umfasst:

- Einrichtung einer Datenbank anhand der Lagerkonfiguration,
- Konfiguration des Warehouse Agent auf die Simulationsdatenbank und
- Konfiguration des Warehouse Operator auf den SimulationExpert sowie des Second-Level-Experts auf den Dummy-Experten.

Das Benchmarking beschränkt sich auf die Messung und Übertragung von Zeitwerten, da sich dieser Arbeit nicht näher mit der energetischen Optimierung befasst. Mit diesem Thema haben sich BUSCH und OSTERMANN [6] näher auseinander gesetzt.

Abbildung 6.12 zeigt noch einmal anhand eines UML-Sequenzdiagramms, wie das komplette Zusammenspiel zwischen Agent, Operator und Warehouse Director als Simulator aussieht.

### 6.3.2 Umsetzung in Java

Da die Umsetzung im Operator bzw. in der `SimulationExpert` bereits im Rahmen der Visualisierung (s. Abs. 6.2) und die Umsetzung der Agenten-Behaviour `SimulationBehaviour` (s. Abs. 5.2.3) ausgiebig dargestellt wurden, soll hier noch auf die Implementation im Warehouse Director eingegangen werden.

Neben der für die GUI notwendige Klasse `FormBenchmark` und der darin realisierten Erzeugung und Initialisierung des Fensterlayouts mit seinen Komponenten, enthält das Java-Package `de.i2ar.warehousedirector.actions` mit der Klasse `BenchmarkActions` sowie `de.i2ar.warehousedirector.connectivity` die für die Simulation bzw. das Benchmarking relevanten Klassen:

**BenchmarkingActions** Diese Klasse enthält die Ereignisbehandlung für die auf `FormBenchmark` befindlichen Buttons zur Steuerung des Benchmarks. Wird ein Benchmark durch Klick auf „Run“ eingeleitet, so ist eine Methode dieser Klasse dafür zuständig, zunächst alle Benutzereingaben syntaktisch zu prüfen und sicherzustellen, dass alle notwendigen Verbindungen zu Operator, Datenbank und Agent hergestellt sind. Danach wird ein `JFileChooser`-Dialog erzeugt, konfiguriert und angezeigt, sodass dieser die Wahl der Protokolldatei ermöglicht. Die Protokollierung wird durch die Klasse `BenchmarkLogger` realisiert, die an dieser Stelle instanziiert und der zusammengesetzte Dateiname durch Aufruf ihrer Methode `begin` bekanntgegeben wird. Ist zu diesem Zeitpunkt noch keine Lagerzone ausgewählt oder besitzt diese keine Lagerzugänge, so wird der Vorgang an dieser Stelle abgebrochen. Sind hingegen alle Voraussetzungen erfüllt, wird durch Aufruf der Methode `makeRequests`, mit dem gewählten Szenario als Argument, eine Ausgangsbelegung des Lagers zusammengestellt und als Liste von `Object`-Array-Elementen zurückgegeben. Diese Elemente enthalten folgende Angaben:

0. Materialtyp-UID,
1. Breite,
2. Farbe und
3. Zeitpunkt der Anfrage.

Bevor diese als bereits abgeschlossene Anfragen in die Datenbank eingetragen werden können (denn nur so werden sie durch die Optimierungsstrategien berücksichtigt), muss die Methode `clearDynamicData` der Klasse `DatabaseConnection`

aufgerufen werden, um alle bisherigen Anfragen und vorhandenen Güter samt ihren Materialien zu löschen. Erst dann werden diese durch Aufruf der Methode `initRequests` der Klasse `DatabaseConnection` in die Datenbank eingefügt und es wird eine Methode `benchmark...` in Abhängigkeit des gewählten Szenarios ausgeführt, welche für die Erzeugung der Anfragen für den Agenten zuständig ist. Nach Rückkehr aus dieser Methode ist der Benchmark abgeschlossen und es wird noch eine letzte Aktualisierung der Visualisierung durch neue Datenbankinformationen erzwungen. Der Rückgabewert der spezifischen Benchmarking-Methode ist ein `Object`-Array, das Angaben zu Anzahl und Gesamtdauer aller Lageroperationen enthält und aus denen Durchschnittswerte gebildet und in einem `JOptionPane`-Dialog angezeigt werden. Im Anschluss wird der Benchmark durch Aufruf der Methode `abort` – sie wird im Übrigen auch bei einem Klick auf `abort` aufgerufen – beendet, der alle Eingabefelder wieder aktiviert und die Protokolldatei schließt sowie die Klassenvariable `run` zurücksetzt, um einem laufenden Benchmarkingprozess zu signalisieren, dass die Vorgänge abgebrochen werden sollen.

Die Methode `makeRequests` sorgt je nach gewähltem Szenario dafür, dass eine Liste mit `Object`-Arrays (s.o.) erzeugt wird, unter Berücksichtigung der in Abschnitt 6.3.3 aufgeführten Materialhäufigkeiten. Die Liste kann bei einer Gleichverteilung also so viele Elemente enthalten, wie unterschiedliche Materialien existieren, aber bei einer ABC-Verteilung z. B. wesentlich mehr, da jedes Material entsprechend seiner Häufigkeit unter Berücksichtigung des größten gemeinsamen Teilers auftreten muss. Dieses Verfahren findet Anwendung, in Verbindung mit dem Verfahren zur Durchführung der Lageroperationen: Es wird je nach Szenario z. B. zufällig ein Element aus der Materialliste entnommen und eine Lageroperation damit durchgeführt. Ist die Materialliste schließlich leer, wird eine neue Kopie von ihr erzeugt und mit den hierin enthaltenen Elementen fortgefahren usw.

`benchmark...` sind Methoden, die die Lageroperationen anhand einer zuvor zusammengestellten Materialliste bestimmen und ausführen. Dabei ist die Reihenfolge entscheidend. Es wird stets von einem zu Beginn gänzlich leeren Lager ausgegangen, so dass in jedem Fall die Einlagerung eines Guts durchgeführt werden muss, bevor es wieder ausgelagert werden kann. Ziel des Benchmarks ist es, unter definierten Bedingungen reproduzierbar Lagervorgänge durchzuführen und deren Ergebnisse festzuhalten. Hierzu wird u.a. auch eine Liste mit bereits durch den Benchmark eingelagerten Gütern erzeugt, sodass hieraus Güter zur Durchführung von Auslagerungen gewählt werden können, z. B. im Wechsel. Um Materialabfolgen nicht fest einprogrammieren zu müssen, wird ein Zufallsgenerator verwendet. Jedoch ist hierbei zu bemerken, dass dieser mit der numerischen, für ein Szenario repräsentativen Konstante initialisiert wird. Dadurch erzeugen mehrere Durchläufe immer dasselbe Ergebnis. Dies ist jedoch von Vorteil, wenn unterschiedliche Optimierungsparameter oder Lagerlayouts gewählt werden. Da die `benchmark...`-Methoden in erster Linie zyklisch aufgebaut sind, wird hier als Schleifenbedingung auch die Klassenvariable `run` ausgewertet, um bei einem Abbruch den Benchmark unterbrechen zu können. Auf die Vorgänge der einzelnen `benchmark...`



Methoden soll nicht näher eingegangen werden, da die grundlegenden Konzepte dargelegt wurden und die Implementation einfach gehalten ist. Es sei jedoch auf Abschnitt 6.3.3 verwiesen, in dem die einzelnen Benchmarking-Szenarien erläutert werden.

**AgentConnection** implementiert das Agent – Warehouse Director Kommunikationsprotokoll (s. Abs. 6.3.4) für den Warehouse Director. Dazu besitzt sie die Methode `connect` zum Verbindungsaufbau zum Agenten. Hier wird zudem eine `LinkedBlockingQueue` für Objekte der privaten Klasse `Request` erzeugt, die zur Pufferung von Anforderungen vom Benchmarkingmodul vorgesehen ist. Dieses ruft zum Einfügen einer Anforderung die Methode `execute` dieser Klasse auf und übergibt ein `Storage`-Objekt als Argument, das bereits aus vergangenen Erläuterungen bekannt ist. `execute` fügt das Objekt in eine neue `Request`-Instanz ein, zusammen mit einem `Semaphore`-Objekt. `execute` fordert die `Semaphore` zunächst zweimal an und gibt sie danach einmal wieder frei. Dies führt jedoch vor der zweiten Anforderung zu einem Blockieren, was beabsichtigt ist. Mit Aufruf der `connect`-Methode wurde auch ein Thread der Klasse `AgentConnection` zur Ausführung gebracht, der zyklisch auf neue Elemente in der `LinkedBlockingQueue` wartet, diese entnimmt, die Anfrage an den Agenten gemäß dem Kommunikationsprotokoll zusammenstellt, ausführt, die Ergebnisse vom Agenten empfängt und zuordnet, sie bzw. eine Fehlermeldung in Felder des zuvor abgerufenen `Request`-Objekts aus der Queue einfügt und die dann ebenfalls darin enthaltene `Semaphore` freigibt. Sofort im Anschluss wird sie durch die oben genannte zweite Anforderung beansprucht, jedoch gleich wieder freigegeben, sodass hiermit das blockierende Verhalten der `execute`-Methode erreicht wird. Da `execute` das `Request`-Objekt kennt, kann sie hieraus die Ergebnisse der Ausführung entnehmen und sie selbst als Rückgabewert bzw. `null` im Fehlerfall verwenden.

**BenchmarkLogger** Der `BenchmarkLogger` ermöglicht den `benchmark...`-Methoden eine einfache Protokollierung der Ausführungsergebnisse für die einzelnen Benchmarkschritte. Zu Beginn muss eine Instanz des `BenchmarkLogger` erzeugt werden. Daraufhin muss die Methode `begin` unter Angabe eines Dateinamens als Argument aufgerufen werden. Sie erzeugt und öffnet die entsprechende Datei, legt den Stream zum Schreiben in die Datei an und speichert diesen als Klassenvariable `out`. Zur Speicherung eines einzelnen Benchmarkergebnisses wird die Methode `append` aufgerufen. Sie erwartet ein `Object`-Array als Argument. Alle Elemente des Arrays werden jeweils durch ein Semikolon getrennt in die Datei geschrieben, gefolgt von einem Zeilenumbruchzeichen `0x0d`. Dabei wird die Methode `toString` zur Transformation jedes einzelnen `Object` in eine menschenlesbare Darstellung genutzt. Durch Aufruf der Methode `end` wird der Datei-Ausgabestrom wieder geschlossen.

### 6.3.3 Benchmarking-Szenarien

Die im Folgenden erläuterten Erwartungen bezüglich der dargestellten Szenarien setzen eine einheitliche Gewichtung aller Optimierungsstrategien voraus. Zudem werden alle Szenarien nur unter Berücksichtigung einfachtiefer Lagerung und mit einem einzigen belegten Regal betrachtet. Zudem ist für jedes Szenario sicherzustellen, dass keine oder minimale Abhängigkeiten zwischen dem allgemeinen Fall des Szenarios und einem konkreten Lagerlayout bestehen. Das heißt insbesondere, dass gerade die Lagerplatzanzahl nicht in die Vorbereitung des Benchmarks einfließen darf, da die Benchmarks unterschiedlicher Lager ansonsten nicht länger vergleichbar wären.

**ABC** Das ABC-Szenario ist der ABC-Methode aus dem betriebswirtschaftlichem Bereich entlehnt. Es sollen drei verschiedene Güter mit einer unterschiedlichen Auftrittshäufigkeit bei

**Gut A** mit 15%,

**Gut B** mit 35% und

**Gut C** mit 50%

unter allen Lageroperationen berücksichtigt werden. Dies impliziert, dass gleich viele Ein- wie auch Auslagerungen je Gut stattfinden müssen. Um kontrollierte Bedingungen zu schaffen darf zu Anfang nur von einem leeren Lager ausgegangen werden. Da zu diesem Zeitpunkt jedoch auch keine Abrufstatistik mehr zur Verfügung steht, die der ABC-Methode entsprechen würde, kann an dieser Stelle zwischen zwei Varianten unterschieden werden:

1. Solang keine Auslagerungen stattgefunden haben, erfolgen alle Einlagerungsvorgänge mit gleicher Bewertung. Es herrschen wahrlich chaotische Lagerverhältnisse. Nach vollständiger Füllung des Lagers kann das weitere Verhalten beobachtet werden, bei zufälliger Reihenfolge der Auslagerungsvorgänge. Die Statistik wird dabei erzeugt und mit jeder weiteren Auslagerung ergänzt. Sie ist dabei jedoch erst zum Abschluss des Benchmarks vollständig, sodass sich zwischenzeitlich zwar eine Tendenz zu oben genannten Werten abzeichnet, diese jedoch vorher nicht erreicht werden. Da die Statistik jedoch nur Relevanz bei Einlagerungen besitzt, ist dieser Umstand unerheblich. Bei einer abschließenden, erneuten Einlagerung aller Güter nach dem ABC-Schema kommt dann diese Statistik zum Tragen und die Güter werden entsprechend ihrer Auftrittshäufigkeit im Lager positioniert.
2. Es ist auch möglich, Ein- und Auslagerungsvorgänge von Beginn an zu mischen. Dabei wird für dieses Szenario ein festes Verhältnis von 1:1 zusammenhängender Operationen angenommen. Das heißt, es werden zunächst  $n$  Güter eingelagert und im Anschluss wieder  $n$ -Güter ausgelagert. Die Wahl eines Guts bei der Einlagerung erfolgt pseudozufällig anhand der ABC-Methode.  $n$  bestimmt dabei, wie schnell

sich die Einlagerungsvorgänge der Statistik anpassen, denn je größer  $n$  ist, desto später werden die Auslagerungsvorgänge der soeben eingelagerten Güter eingeleitet und damit auch die Statistik erweitert, die erst bei zukünftigen Einlagerungen berücksichtigt wird. Für die Auslagerung kommen beim 1:1-Verhältnis nur die gerade eingelagerten  $n$  Güter in Frage. Die Veränderungen bei der Lagerplatzzuordnung der Einlagerungsvorgänge können hierbei jedoch verfolgt werden. So werden die Positionen der Lagerplätze innerhalb eines Einlagerungsblocks gutweise ähnlich verhalten, nach dem nächsten Auslagerungsblock jedoch etwas springen, vor allem bei noch gleichen Verteilungen unter den ersten wenigen Einlagerungen, und dann zunehmend gutweise einem Bereich annähern.

Für  $n$  muss ein fester, unabhängiger Wert gewählt werden. Dieser sei an dieser Stelle auf 5 festgesetzt. Daraus resultiert, dass dieses Verfahren nur mit Lagern durchgeführt werden kann, die mindestens fünf Lagerplätze besitzen. Die Auslagerungsstatistik wird dabei etappenweise aufgebaut und nähert sich auch den oben genannten Werten der ABC-Methode an. Da die Operationen wiederholt werden müssen, werden  $r = 20$  Wiederholungen angesetzt.  $5 \cdot 20 = 100$  und zur Erzeugung der ABC-Verteilung wird ein Pool mit insgesamt 100 Gütern angelegt, sodass bei  $n = 5$  und  $r = 20$  die vorgegebene Verteilung vollständig berücksichtigt wird.

**Doppelspiele** Mit einem einfachen Experiment kann der Effekt der Doppelspielstrategie experimentell nachgewiesen werden. Dazu sollen nur zwei unterschiedliche Güter betrachtet und vor dem Start der Simulation als Statistik initialisiert werden. Damit liegt das Verhältnis der Abrufhäufigkeiten zu diesem Zeitpunkt bei 1:1. Die Güter sollten dann im gleichen Verhältnis abwechselnd eingelagert werden. Damit sollte eine gute Vermengung der Güter im Lager erreicht werden. Im Anschluss werden diese wechselweise wieder ausgelagert. Zu Beginn wird das Verhältnis der Auslagerungen dabei stark schwanken, jedoch wieder gegen 1:1 gehen.

Der Bezug zum Doppelspiel wird sich zeigen, da bei den Auslagerungsvorgängen zu beobachten sein wird, wie sich die Positionen der zur Auslagerung bestimmten Lagerzellen symmetrisch um den Lagerzugang entwickeln. Dieses Phänomen kann jedoch nur unbeeinflusst auftreten, wenn alle übrigen Optimierungsstrategien eine sehr geringe oder eine Null-Gewichtung besitzen, da ansonsten in erster Linie die Entfernung zum Lagerzugang und der Einlagerungszeitpunkt bei der FIFO-Strategie berücksichtigt mit einfließen. Die Auslagerungsanforderungen werden auf Grund der Auftragssteuerung nicht in einer Warteschlange vom Agenten vorgehalten und passend als Doppelspiel gewählt, sondern immer sofort ausgeführt. Dennoch sollte die implementierte Doppelspielstrategie den zur letzten Einlagerung naheliegendsten Lagerplatz für die Auslagerung im Doppelspiel wählen.

**Zufall** Neben allen Szenarien, die bereits eine bestimmte Absicht besitzen, eine Vermutung bezüglich der Effektivität oder anderen Eigenschaften zu belegen, ist es auch interessant, zu beobachten, was bei zufälligen Ein- und Auslagerungsvorgängen geschieht. Um

den Vorgang immer gleich wiederholen zu können, wird der Zufallsgenerator dazu mit der internen, numerischen Repräsentation dieses Szenarios initialisiert, sodass vielmehr von einem Pseudo-Zufallsgenerator zu sprechen ist. Dazu wird zunächst die Statistik des Lagers beeinflusst, indem so viele bereits bearbeitete Anfragen nach zufälligen Gütern aus einem Pool eingefügt werden, wie Lagerplätze vorhanden sind. Um unabhängig von der Lagergröße zu sein, muss der Zufallsgenerator erneut initialisiert werden. Hierzu wird der oben genutzte und um 1 inkrementierte Wert verwendet, um bei den folgenden Einfügeoperationen nicht die gleichen Güter (in der gleichen Reihenfolge) einzulagern wie schon zur Initialisierung der Statistik. Während des Einfügens wird mitgeschrieben, um welche Güter es sich handelte. Diese Liste wird anschließend in eine zufällige Reihenfolge gebracht. Dazu wird sich eines Zufallsgenerators bedient, der mit dem oben genannten und um 2 inkrementierten Wert initialisiert wurde. Diese Liste wird von oben nach unten durchlaufen und das entsprechende Gut aus dem Lager angefordert. Damit ist sichergestellt, dass nur Güter angefordert werden, die sich auch tatsächlich noch virtuell im Lager befinden. Die Anzahl der Güterklassen kann nicht als prozentualer Anteil der Lagerplatzanzahl bestimmt werden, da sich somit eine Abhängigkeit von jedem speziellen Lager ergäbe und die Benchmarks nicht mehr vergleichbar wäre. Aus diesem Grund wird definiert, dass zehn Güterklassen verwendet werden sollen.

### 6.3.4 Kommunikationsprotokoll Agent – Warehouse Director

Das Protokoll ist sehr minimalistisch und umfasst nur zwei unterschiedliche Befehle. Der Ablauf ist wie folgt:

1. Der Agent sendet nach dem Verbindungsaufbau: `HELLO`
2. Der Warehouse Director sendet für eine Ein- oder Auslagerungsanforderung `EXECUTE$`, gefolgt von einem als XML-Dokument kodierten `Storage`-Objekt. Dabei ist zu beachten, dass das Dokument keine Zeilenumbrüche enthalten darf.
3. Der Agent antwortet nach erfolgreicher Ausführung mit `DONE$`, gefolgt von sieben als String kodierten Werten in dieser Reihenfolge und jeweils durch ein Dollarzeichen getrennt:
  - a) Lagerplatz-UID
  - b) Metrik des Lagerplatzes
  - c) X-Positionsindex,
  - d) Y-Positionsindex und
  - e) Z-Positionsindex des Lagerplatzes
  - f) Ausführungszeit der Optimierungsstrategien
  - g) Ausführungszeit des Operators

Im Fehlerfall hingegen antwortet der Agent mit `FAILED$` und einer angeschlossenen Fehlerbeschreibung.

4. Die Prozedur wird bei 2. fortgesetzt.

Alle Befehle und Nachrichten sind einzeilige Textnachrichten und enden mit dem Zeichen 0x0d (Newline).

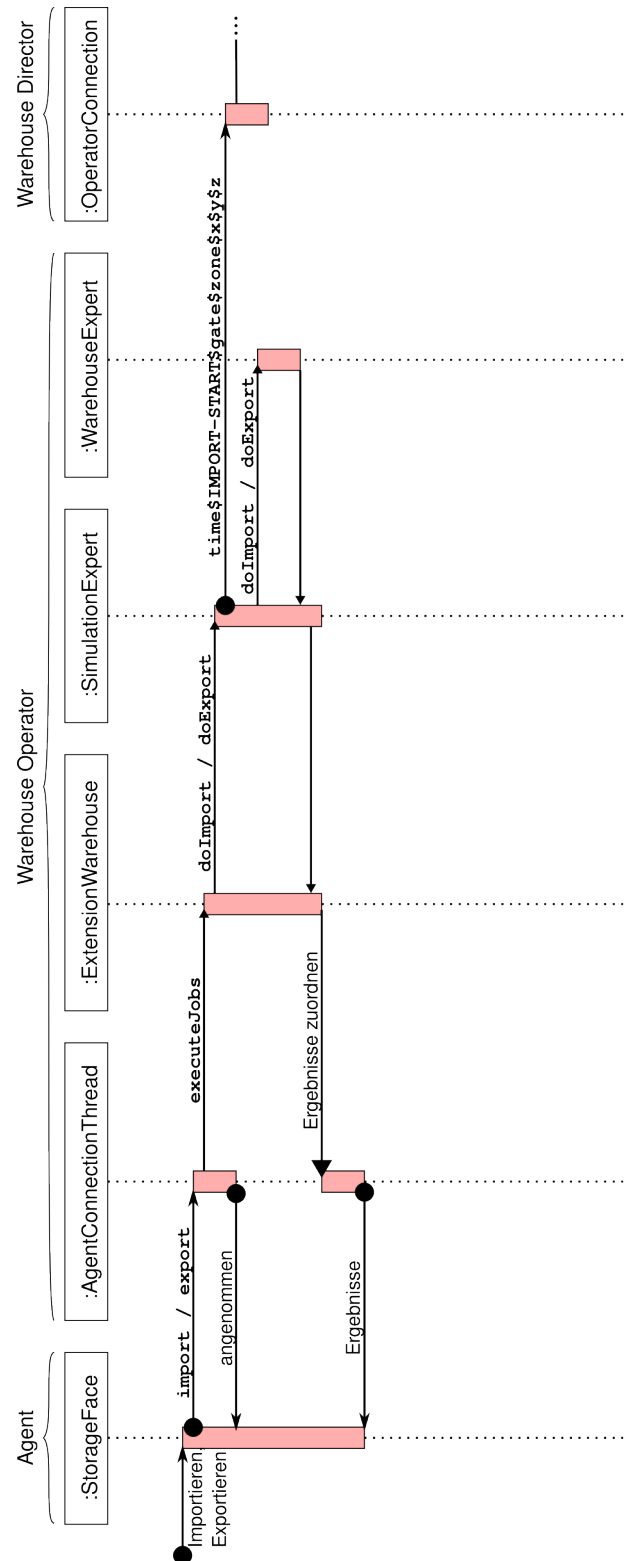
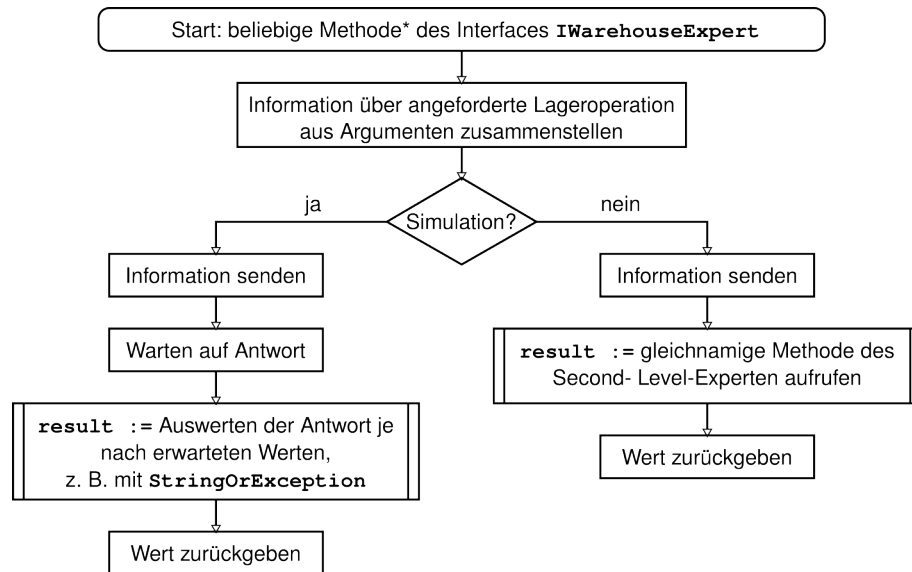


Abbildung 6.8: UML-Sequenzdiagramm zur Darstellung der Verarbeitung und Visualisierung eines Einlagerungsauftrags.



\*ausgenommen `initExpert`, da hier auch der `SimulationExpert` selbst initialisiert werden muss.

Abbildung 6.9: Programmablaufplan aller Methoden des Interface `IWarehouseExpert` für die spezielle Implementation im `SimulationExpert`.

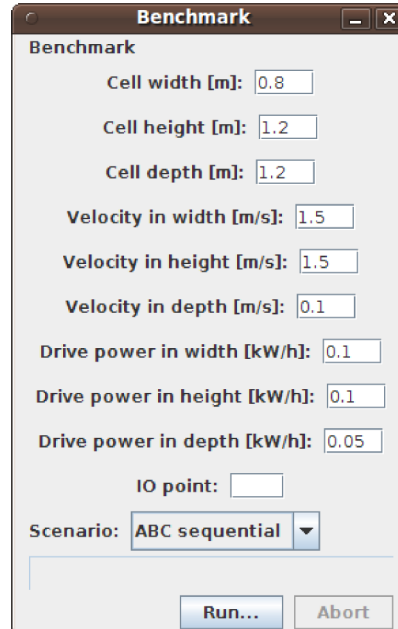


Abbildung 6.10: Fenster zur Parametrierung eines Benchmarks.

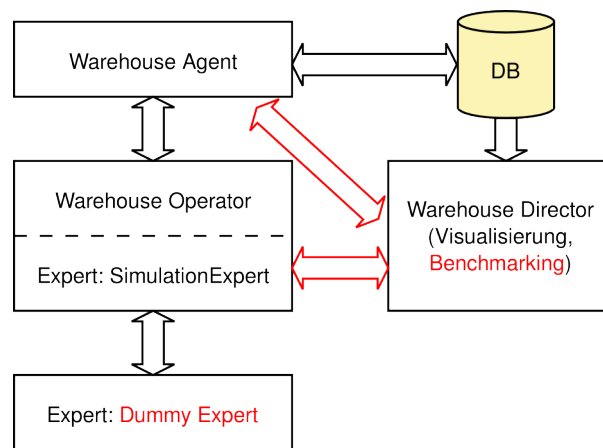


Abbildung 6.11: Systemmodell bei Simulationsfunktion.



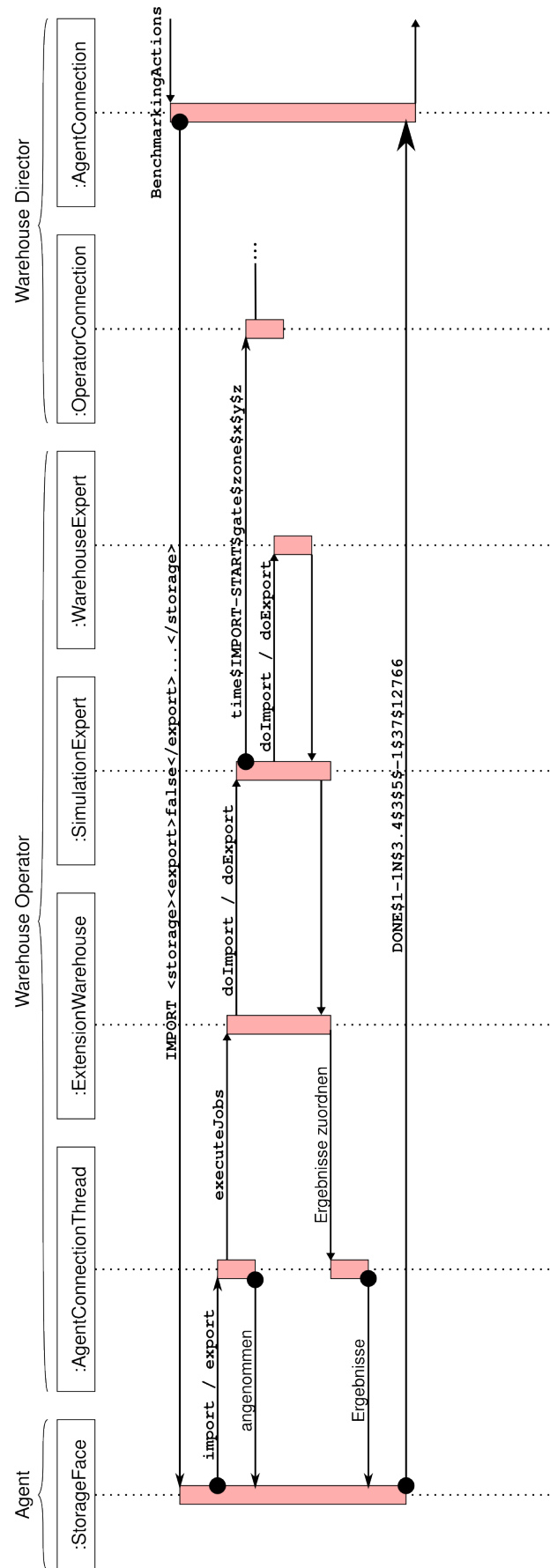


Abbildung 6.12: UML-Sequenzdiagramm zum Komponenten-Zusammenspiel. 145

# 7 Benchmarking

Wie bereits in der Dokumentation des Master-Vertiefungsprojekts von VIRKUS [28] angekündigt, wurde im Rahmen dieser Arbeit ein Simulations- bzw. Benchmarking-Tool umgesetzt, das bei der Ermittlung der Parameter für Optimierungsstrategien behilflich sein soll. Wie zuvor beschrieben, führt das Tool keine selbstständige Analyse durch. Es speichert jedoch relevante, greifbare Daten, die während der Durchführung der Simulation entstehen. In diesem Kapitel soll erörtert werden, was die für einen Benchmark entscheidenden Kriterien sind und es soll anhand von Beispielen die Interpretation der Datenaufzeichnung und damit auch die Analyse einiger Optimierungsverfahren gezeigt werden.

## 7.1 Vorbereitungen

Zu den Arbeiten, die im Vorfeld eines Benchmarks stattfinden müssen, zählen:

- Auswahl des Benchmarks
- Auswahl geeigneter Lagerstrukturen
- Auswahl der zu verwendenden Güter und ihre Verteilung
- Auswahl der Reihenfolge von Ein- und Auslagerungen

**Auswahl des Benchmarks** Auf den ersten Blick ist möglicherweise nicht ersichtlich, dass unterschiedliche Benchmarks durchgeführt werden können. Zum einen kann die Effektivität der Optimierungsverfahren mit ihren Parametern anhand des physikalischen Ergebnisses bestimmt werden. Hierzu ist eine automatisierte oder manuelle Analyse der durch die Tools zur Verfügung gestellten, aufgezeichneten Werte notwendig. Ein anderer Benchmark kann über der Ausführungsgeschwindigkeit der Optimierungsalgorithmen durchgeführt werden bzw. es kann der Einfluss der einzelnen Optimierungsparameter auf die Rechenzeiten bestimmt werden.

**Auswahl geeigneter Lagerstrukturen** Es muss zunächst klargestellt werden, welcher Zweck mit der Durchführung eines Benchmarks in einem konkreten Fall verfolgt werden soll. Ist es die generelle Analyse der Einflüsse von Optimierungsparametern, so bietet sich eine sehr einfache Lagerstruktur mit einer stark begrenzten Menge Lagerplätzen an. Die Arbeit des Analysten wird hierdurch übersichtlicher und damit vereinfacht. In einem anderen Fall kann ein Benchmark auch mit Abbildungen real vorhandener Lagerstrukturen durchgeführt werden, um die spezifischen Auswirkungen unterschiedlicher Optimierungsparameter zu untersuchen. Schließlich kann für jede einzelne Ein-/Auslagerung geprüft werden, ob die durch die Optimierungsverfahren berechneten Werte sinnvoll sind. Hieraus ergibt sich letztendlich auch die endgültige Umschlagleistung, die es in der Regel zu maximieren gilt.

**Auswahl der zu verwendenden Güter und ihre Verteilung** Einfluss auf das Ergebnis eines Benchmarks haben auch die Güter, die testweise ein- und ausgelagert werden. Bei der Einlagerung wird ein statistischer Wert für die Abfragehäufigkeit des einzulagernden Guts gebildet. Dieser entscheidet maßgeblich über die Positionierung bei der Einlagerung. Er verändert sich für jedes Gut mit jeder Auslagerung stetig, daher ist auch die Auswahl der Reihenfolge bei Ein- und Auslagerungen zu beachten.

**Auswahl der Reihenfolge von Ein- und Auslagerungen** Die Reihenfolge, in der Güter ein- und ausgelagert werden, entscheidet auch über die Positionierung von Gütern im Lager. Da die Abfragestatistik mit jeder Auslagerung um eine Erhebung bereichert wird, ist die Reihenfolge relevant. Dies macht sich insbesondere dann bemerkbar, wenn sich die Statistik erst weniger Erhebungen bedienen kann. Es entsteht ein Phänomen großer Sprünge in Bezug auf die Wahl des optimalen Lagerplatzes für eine Einlagerung. Dieses ist begründet in der Spreizung der relativen Anfragehäufigkeit jedes einzelnen Guts auf ein Intervall  $[0, 1]$ , mit 0 für beste Lagerposition auf Grund der häufigsten Anfragen für ein Gut und 1 für schlechteste Lagerposition auf Grund der seltensten Anfragen für ein Gut. Gerade solange noch wenig Anfragen getätigt wurden, differieren die relativen Anfragehäufigkeiten der einzelnen Güter stark. Die in der Abbildung 7.1 gezeigte relative Anfragehäufigkeit sei basierend auf einer durchschnittlichen Abfrageverteilung von 5:2 auf Gut 1 : Gut 2. Die Verteilung ergibt sich statistisch erst nach einigen Vorgängen, falls im Ausgangszustand noch keinerlei Anfragen bekannt sind. Zum Phänomen der Sprünge sollte auch noch einmal darauf hingewiesen werden, dass die Abfragehäufigkeit eines Guts linear mit der Entfernung der Lagerzellen zum Lagerein-/ausgang korreliert wird.

In Abschnitt 6.3.3 wurden bereits Benchmarking-Szenarien dargestellt und erläutert, bei denen die oben genannten Punkte berücksichtigt wurden. Mit diesen Szenarien wurden Benchmarks auf einwändigen Lagern mit  $5 \times 5$  und  $10 \times 10$  Lagerplätzen (s. Abb. 7.2) durchgeführt. Die Ergebnisse sollen hier diskutiert werden. Dazu werden zwei unterschiedliche Diagrammtypen zur Illustration herangezogen. Diese sind u.a. in Abb. 7.6 dargestellt. Links ist ein zweidimensionales kartesisches Koordinatensystem abgebildet. Alle durch entsprechende Lageroperationen berührte Lagerplätze sind hier markiert und der

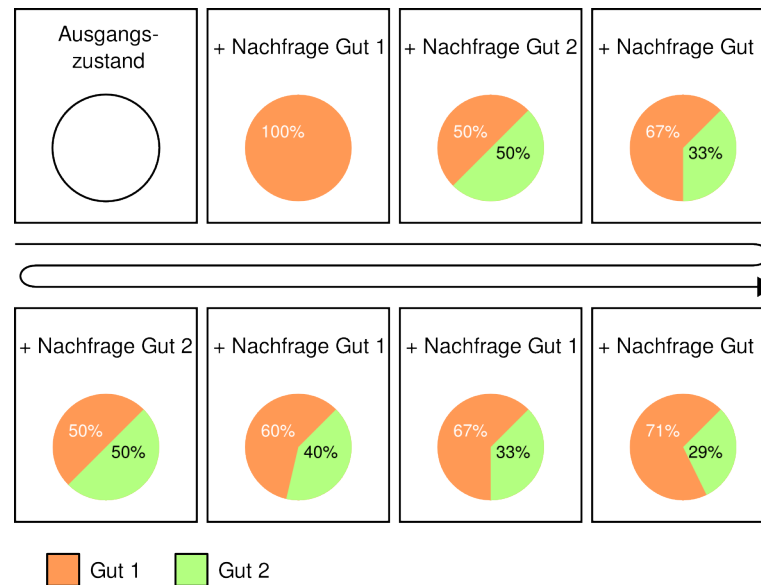


Abbildung 7.1: Entwicklung der relativen Anfragehäufigkeit bei zwei unterschiedlichen Gütern.

Reihenfolge nach miteinander verbunden. Damit kann ein Überblick über die Orte der Lageroperationen gegeben werden. Im rechten Teil der Grafik ist ein Netzdiagramm dargestellt. Im Außenbereich sind die untersuchten Lageroperationen, repräsentiert durch einen Index, ringförmig aufgetragen, beginnend oben und gegen den Uhrzeigersinn fortführend. Radial sind die Lagerplatzindizes für X- (rot) und Z-Index (blau) aufgetragen. Ihre Werte sind vereinzelt entlang des senkrechten Radius zu entnehmen. Dabei entspricht die Kombination des X- mit dem Z-Index auf einem Strahl den Lagerplatzindizes der entsprechenden Lageroperation. Mit Hilfe dieses Diagramms lassen sich räumlich zusammenhängende Operationen einfach identifizieren, um die erste Darstellung zu unterstützen. Alle Diagramme sind mitsamt den zugrunde liegenden Daten auf dem beiliegenden Datenträger als PDF-, OpenOffice- und CSV-Format enthalten.

## 7.2 ABC – Sequentiell

Ziel ist es, für das sequentielle ABC-Szenario (Lager befüllen, leeren, erneut befüllen) zu zeigen, dass anhand der Auslagerungsstatistik die Lagerplätze bei erneuter Einlagerung entsprechend der Relevanz eines Gutes in der Statistik vergeben werden. Zunächst ist in Abb. 7.3 der erstmalige Einlagerungsvorgang im  $5 \times 5$ -Lager gezeigt. Es ist deutlich anhand des linken Diagramms eine schrittweise Belegung des Lagers vom entferntesten bis zum dichtesten am Lagerzugang gelegenen Lagerplatz zu erkennen, denn zu Beginn existieren noch keine Einträge in der Statistik, sodass vom entferntesten Punkt für unbekannte (alle) Güter begonnen wird. Am rechten Diagramm ist dies nur schwer zu erkennen, mit Kenntnis des Ausgangspunktes ist jedoch eine Verfolgung der Reihenfolge möglich.

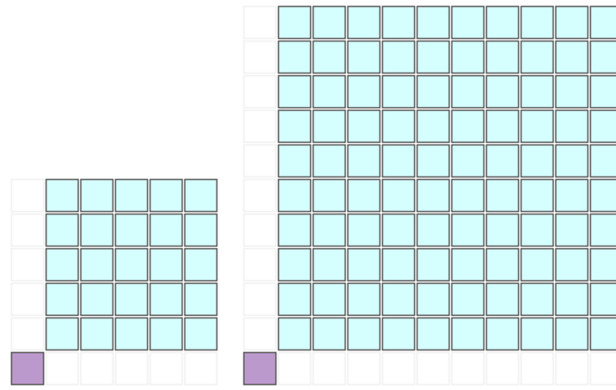
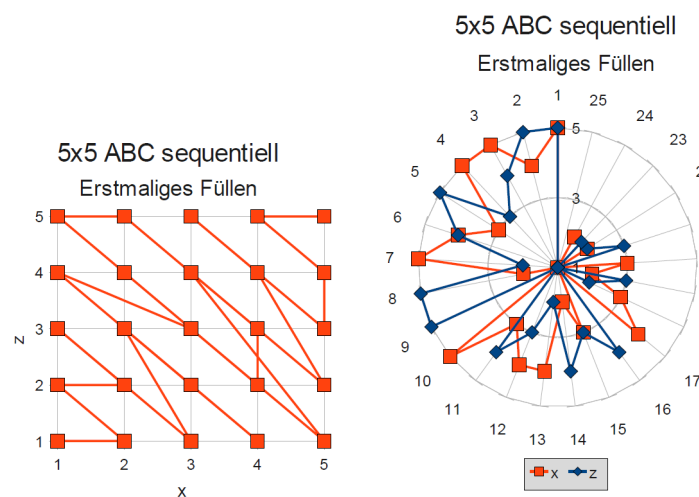
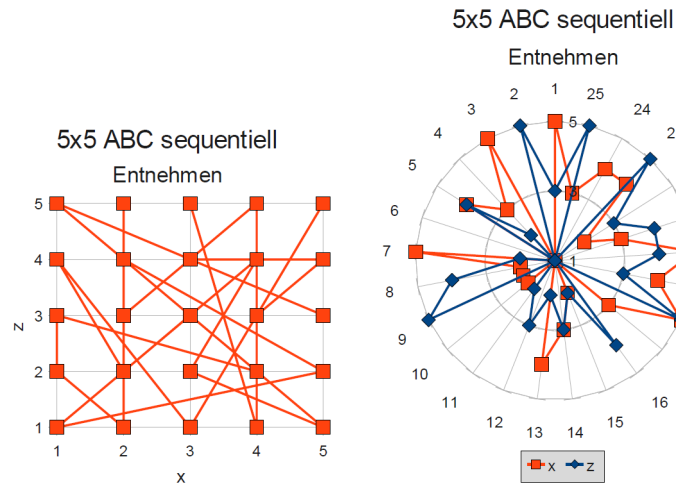


Abbildung 7.2: Zum Benchmarking verwendete Lagerlayouts.

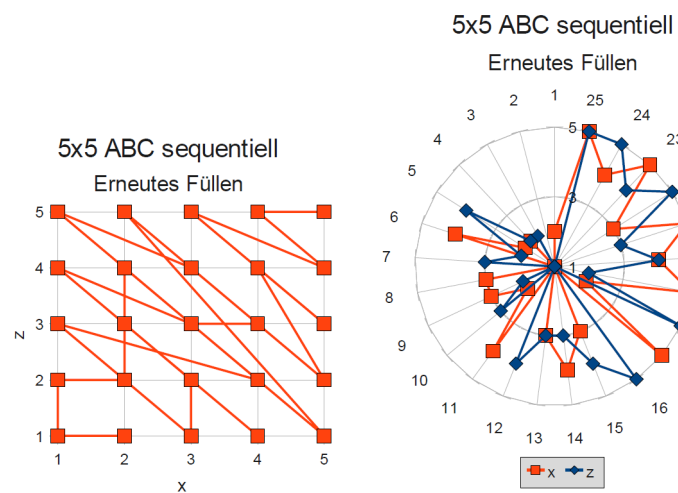
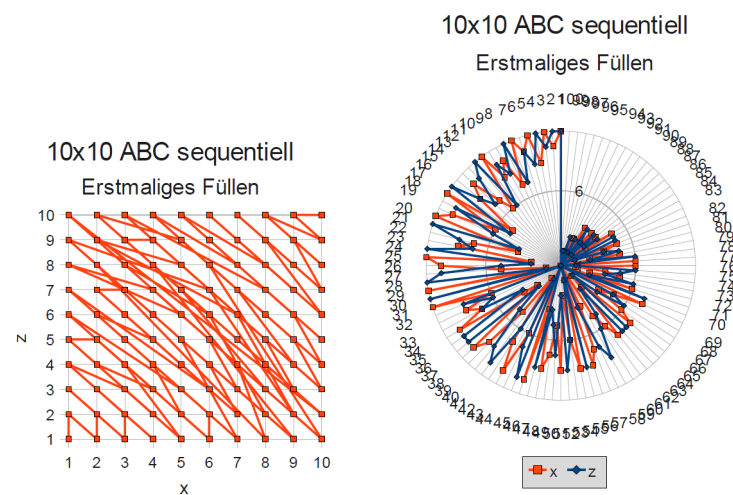
Da alle eingelagerten Güter zwischengespeichert und zur Auslagerung in eine zufällige Reihenfolge gebracht werden, ist in Abb. 7.4 hier kein Schema zu erkennen. Es existieren jedoch bereits Hinweise auf die Doppelspielstrategie, die später noch ausgeführt werden (s. Abs. 7.4).

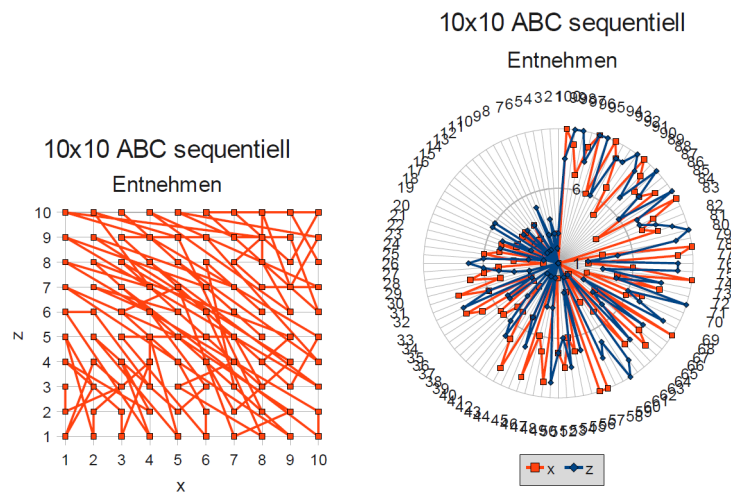
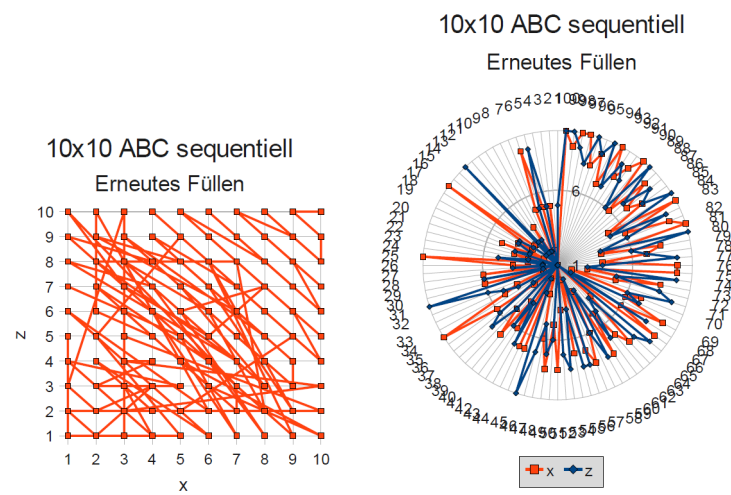
Bei der abschließenden, erneuten Einlagerung der Güter ist anhand der Abbildung 7.5 zu sehen, dass zunächst sehr viele Einlagerungen nahe des Lagerzugangs stattfinden. Dies wird durch den Typ C (50% Anteil) ausgelöst. Mit der Zeit sind jedoch keine Lagerplätze mehr in der nahen Lagerzugangsregion verfügbar, sodass der Einlagerungsbereich ausgeweitet wird. Bereits früh ist bei den Indizes 5 und 6 ein *Ausbrechen* zu erkennen. An dieser Stelle werden zwei Güter des Typs A (15% Anteil), also der seltensten Kategorie, eingelagert und sind damit für die hinteren Lagerplätze prädestiniert. Diese Aufteilung ist in Abschnitt 7.3 noch einmal genauer dargestellt.

Abbildung 7.3:  $5 \times 5$  ABC sequentiell, erstmaliges Füllen

Abbildung 7.4:  $5 \times 5$  ABC sequentiell, Entnehmen

Im Diagramm zum sequentiellen ABC-Szenario mit einem  $10 \times 10$ -Lagerlayout ist bei der erstmaligen Befüllung wesentlich deutlicher eine Schneckenform im Netzdiagramm zu erkennen (s. Abb. 7.6), die sich wieder ergibt, da noch keine Statistik vorliegt. Im Gegenteil zur  $5 \times 5$ -Variante ist bei der anschließenden Auslagerung (s. Abb. 7.7) wieder deutlich eine Schneckenform zu erkennen, die jedoch *unruhiger* wirkt, als bei der erstmaligen Einlagerung. Dies ist mit einer zufällig gewählten Reihenfolge der auszulagernden Güter zu erklären. Bei der erneuten Einlagerung findet die Statistik nun Berücksichtigung, wie u.a. an den *Ausreißern* 5, 6, 13, 17 usw. zu erkennen ist, denn die dort eingelagerten Güter sind vom Typ A und sollten bei der Einlagerung einen vom Lagerzugang entfernten Lagerplatz zugewiesen bekommen. Dass ca. ab Vorgang 70 die Entfernung aller gewählten Lagerplätze zum Lagerzugang auch für die am häufigsten vorkommenden Güter zunimmt, liegt an der inzwischen vollständigen Belegung aller näheren Lagerplätze.

Abbildung 7.5:  $5 \times 5$  ABC sequentiell, erneutes FüllenAbbildung 7.6:  $10 \times 10$  ABC sequentiell, erstmaliges Füllen

Abbildung 7.7:  $10 \times 10$  ABC sequentiell, EntnehmenAbbildung 7.8:  $10 \times 10$  ABC sequentiell, erneutes Füllen



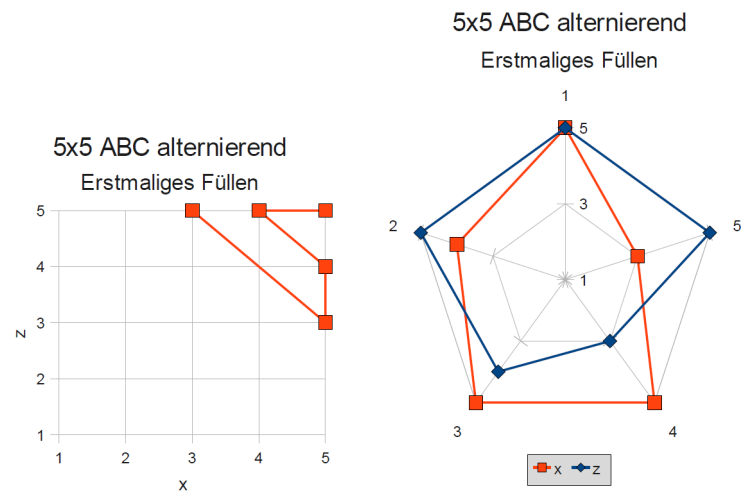
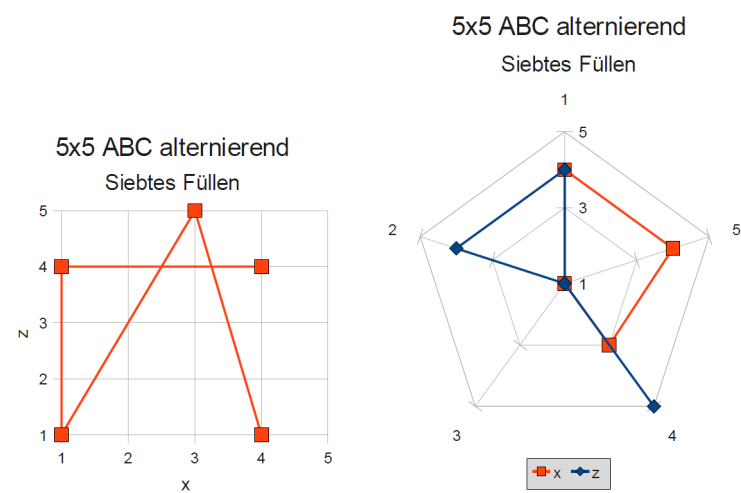
### 7.3 ABC – Alternierend

Eine Variante des eben gezeigten Szenarios sieht die alternierende Ein- und Auslagerung von Fünfergruppen der Gütern aus den drei Klassen A, B und C vor. Dabei wird die Auslagerungsstatistik nach und nach aufgebaut, sodass sich diese auch erst im Laufe der Zeit auswirkt, wie anhand der Diagramme in den Abbildungen 7.9, 7.10, 7.11 und 7.12 zu sehen ist, die für die Auslagerungen genutzt werden. Da im Wechsel eben so viele Güter aus- wie eingelagert werden und keine weiteren im Lager existieren sind die Lagerplätze identisch, lediglich die Reihenfolge kann unterschiedlich sein.

Beim erstmaligen Füllen werden alle Güter noch schlechtestmöglich (vgl. oben) eingestuft. Da die Hälfte der eingelagerten Güter nach dem ABC-Schema bestmöglich zu platzieren sind, wird sich die Statistik entsprechend entwickeln, was an den Diagrammen verfolgt werden kann. An den Netzdiagrammen ist deutlich eine Entwicklung der gewählten Lagerplätze in Richtung der Netzmittelpunkte, also einer sehr geringen Entfernung zum Lagerzugang, zu erkennen. In den zweidimensionalen Koordinatensystemen ist zudem die Bildung von symmetrischen Lagerbelegungen zu erkennen bzw. bei Fortsetzung einer höheren Gruppengröße zu erahnen.

Bei den Diagrammen des  $10 \times 10$ -Layouts (s. Abb. 7.13, 7.14, 7.15 und 7.16) bestätigt sich diese Annahme ebenfalls. Zudem ist hier stärker zu erkennen, dass sich Gürtel bilden, um die sich Güter einer Klasse bevorzugt ansammeln. So ist in Abb. 7.15 eine Ansammlung belegter Lagerplätzen um die (gedachte) Gerade durch die Lagerplätze  $((7, 1), (1, 7))$  sowie durch die dazu parallele Gerade durch den Lagerplatz  $(1, 1)$  zu erkennen. Ähnlich verhält es sich auch in Abb. 7.14, wo sich Gürtel um die Geraden durch die Lagerplätze  $((14, 1), (1, 14))$ ,  $((8, 1), (1, 8))$  und  $(1, 1)$  gebildet haben. Diese Annahmen bestätigen sich, wenn das zugrunde liegende Benchmarkprotokoll betrachtet wird, denn hier ist z. B. für Abb. 7.14 festgehalten, dass zwei der Lagerplätze mit Gütern des Typs A, zwei mit Gütern des Typs B und einer mit einem Typ-C-Gut belegt wurden.

Es kann einerseits argumentiert werden, dass diese Verteilung der Güter im Lager keine 50% des Lagervolumens für Güter des Typs C vorsieht, jedoch gilt es zu beachten, dass Typ B mit 35% nur unwesentlich schwächer vertreten ist (Faktor 1.4), im Vergleich mit 50%:15% (Faktor 3.3) oder 35%:15% (Faktor 2.3).

Abbildung 7.9:  $5 \times 5$  ABC alternierend, erstmaliges FüllenAbbildung 7.10:  $5 \times 5$  ABC alternierend, siebtes Füllen

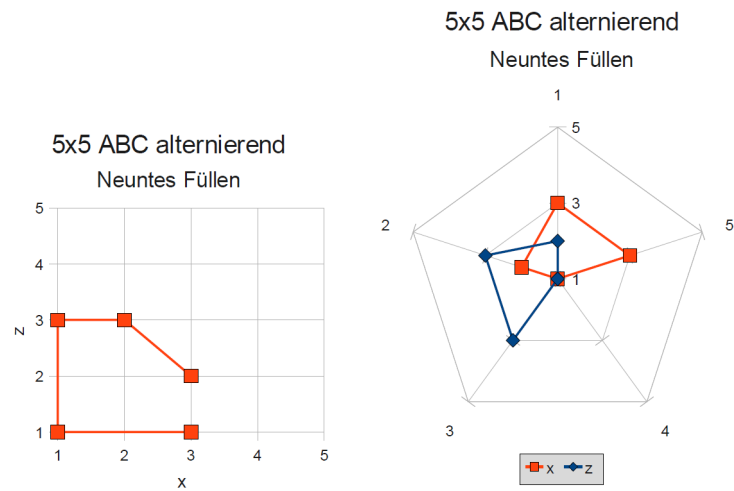


Abbildung 7.11:  $5 \times 5$  ABC alternierend, neuntes Füllen

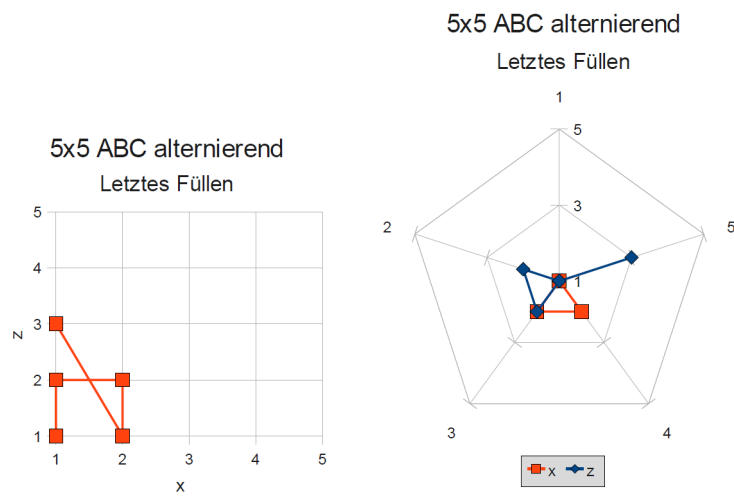


Abbildung 7.12:  $5 \times 5$  ABC alternierend, letztes Füllen

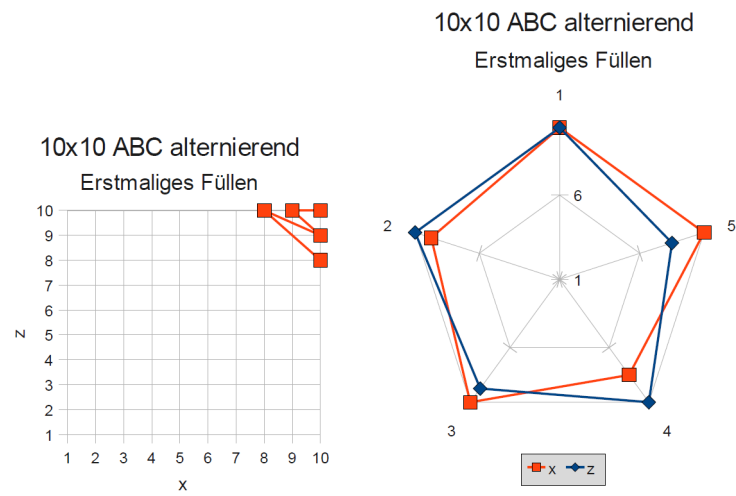


Abbildung 7.13:  $10 \times 10$  ABC alternierend, erstmaliges Füllen

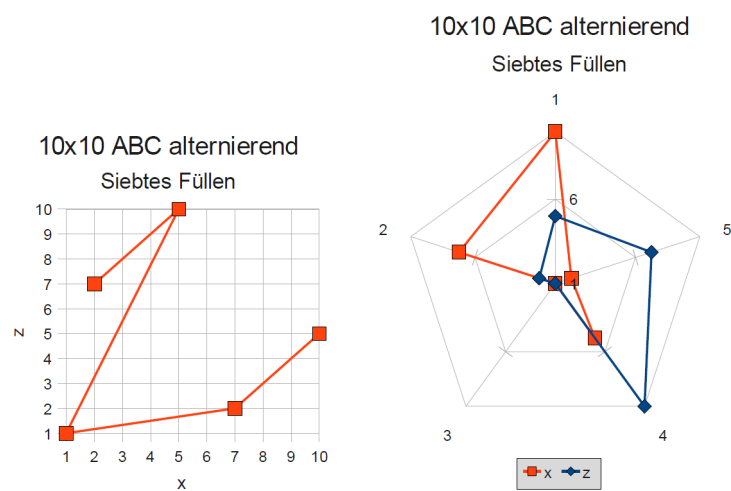


Abbildung 7.14:  $10 \times 10$  ABC alternierend, siebtes Füllen

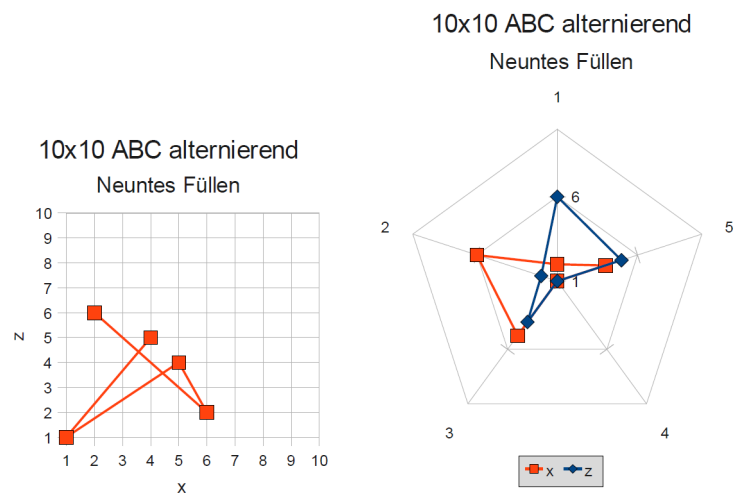


Abbildung 7.15:  $10 \times 10$  ABC alternierend, neuntes Füllen

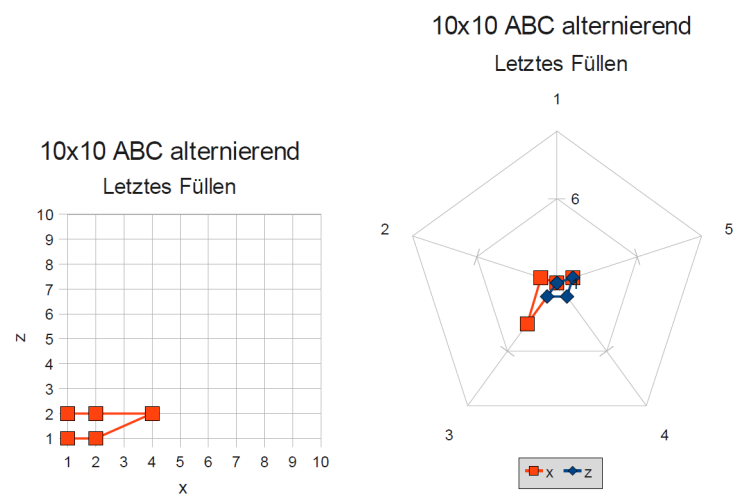


Abbildung 7.16:  $10 \times 10$  ABC alternierend, letztes Füllen

## 7.4 Doppelspiel

Ein Szenario ist speziell konzipiert worden, um Doppelspiele explizit auszuführen. Dass heißt, nach Vordefinition der Statistik für nur zwei unterschiedlichen Gütern wird das Lager komplett abwechselnd mit beiden Gütern gefüllt und dann die Hälfte hiervon wieder ausgelagert. Damit wird eine gute Vermengung der Güter sichergestellt (s. 7.17 für  $5 \times 5$ - und 7.19 für  $10 \times 10$ -Lager).

Bei der folgenden abwechselnden Ein- und Auslagerung (s. Abb. 7.18 und 7.20) der beiden Güter zeigt sich der Effekt der Doppelspielstrategie. Zwei aufeinander angefahrene Lagerpunkt liegen dicht beieinander, wenn sich sowohl X- wie auch Z-Index nur minimal auf dem Radiant bewegen. Es findet u.a. eine *Kronenbildung* im Indexbereich 44 bis 46 sowie 54 bis 59 statt. Hier finden sich aber auch Lagerplatzverläufe, bei denen eine der Positionsgrößen nahezu konstant ist, während sich die andere stetig verringert oder erhöht 57 bis 60.

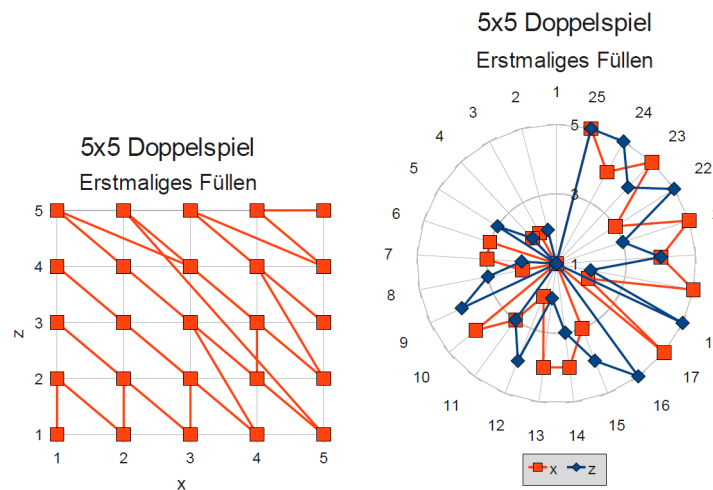


Abbildung 7.17:  $5 \times 5$  Doppelspiel, Füllen

Bei der oben beschriebenen Durchführung wurde eine gleiche Gewichtung der Auslagerungsstrategien gewählt. Wird diese für alle auf Null gesetzt und nur das Doppelspiel voll gewichtet, ergeben sich die in den Abbildungen 7.21 und 7.22 für  $5 \times 5$ - sowie 7.23 und 7.24 für  $10 \times 10$ -Lager dargestellten Reihenfolgen. Dabei schlägt sich die Fixierung auf die Doppelspielstrategie sichtbar nieder. Gerade im Vorgangsbereich 43 bis 50 und 66 bis 72 liegen die Lagerplätze dichter beieinander als noch bei dem Benchmark mit gleich gewichteten Optimierungsstrategien.

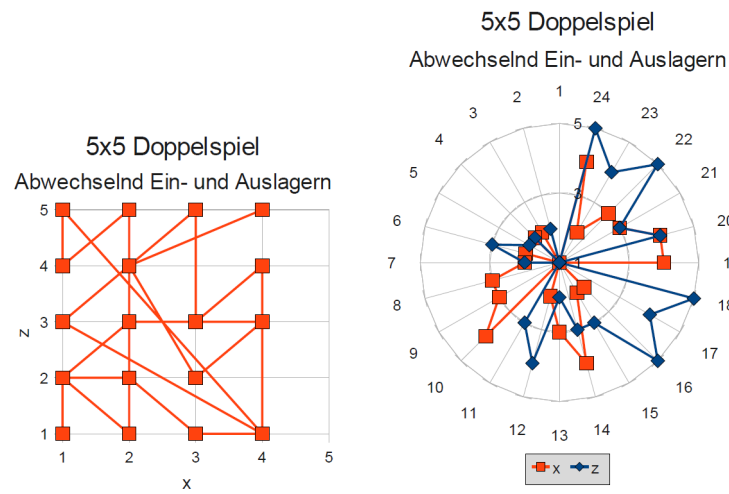


Abbildung 7.18:  $5 \times 5$  Doppelspiel, abwechselnd Ein- und Auslagern

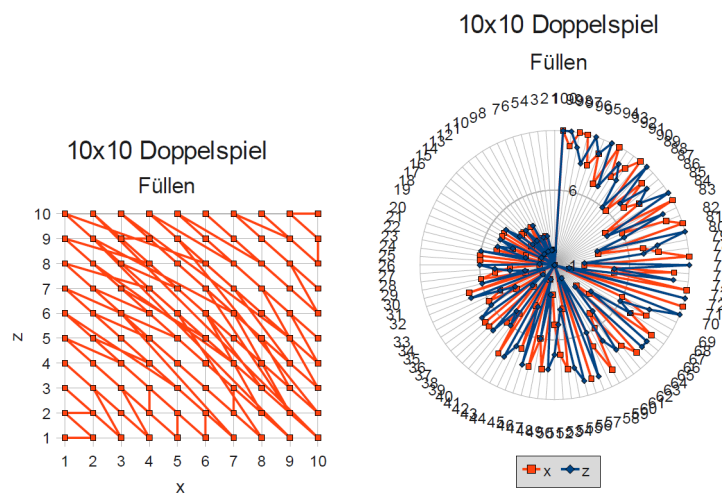


Abbildung 7.19:  $10 \times 10$  Doppelspiel, Füllen

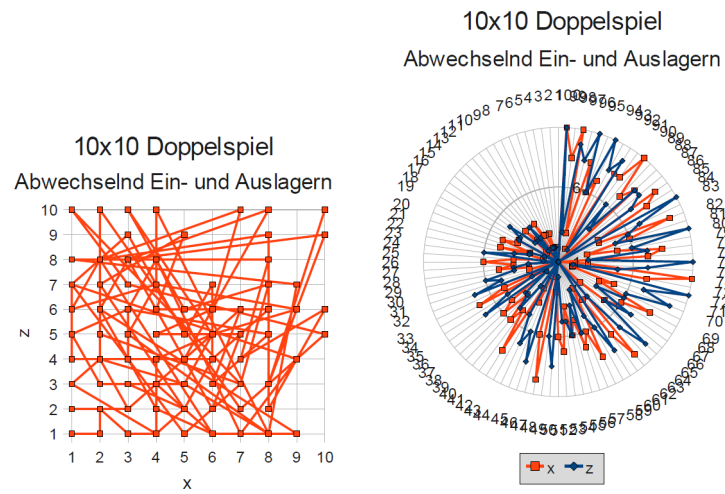


Abbildung 7.20:  $10 \times 10$  Doppelspiel, abwechselnd Ein- und Auslagern

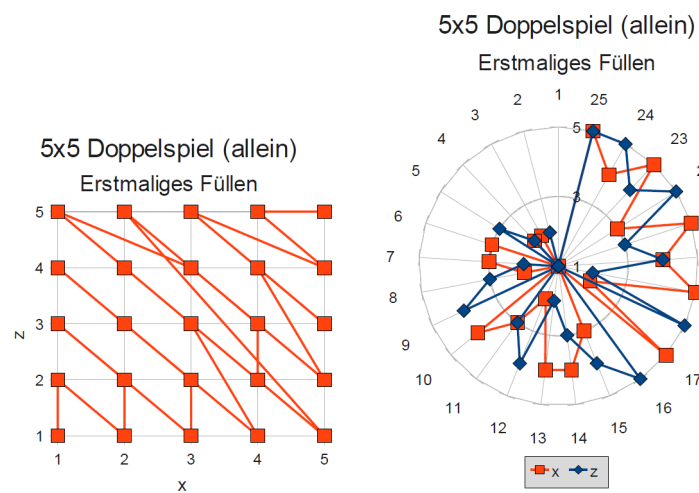


Abbildung 7.21:  $5 \times 5$  Doppelspiel mit ausschließlicher Gewichtung, Füllen



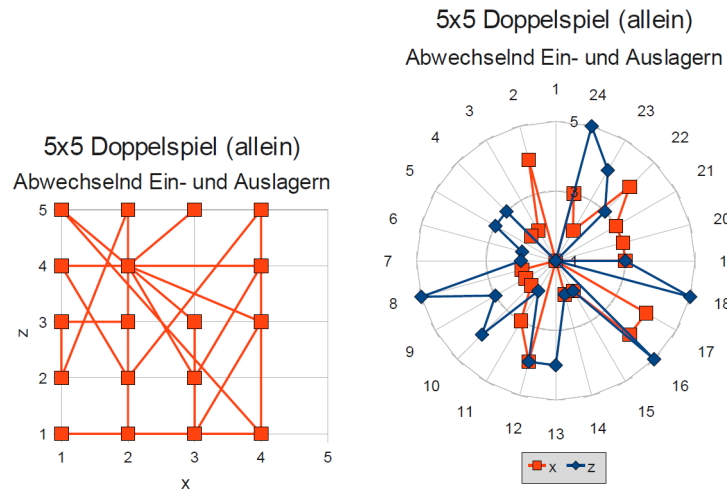


Abbildung 7.22:  $5 \times 5$  Doppelspiel mit ausschließlicher Gewichtung, abwechselnd Ein- und Auslagern

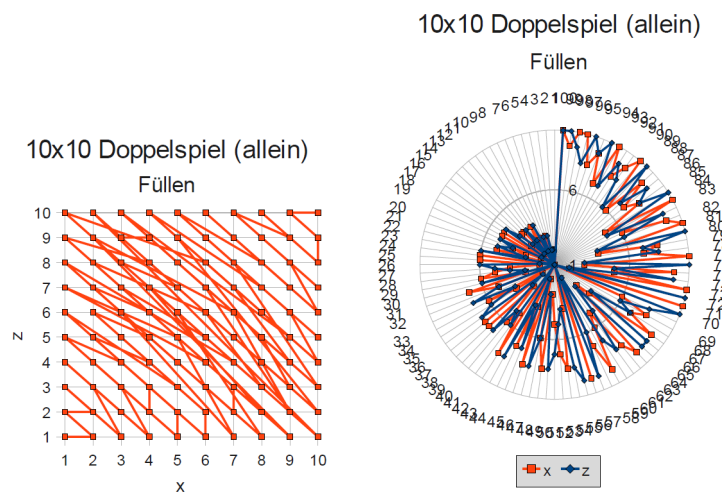


Abbildung 7.23:  $10 \times 10$  Doppelspiel mit ausschließlicher Gewichtung, Füllen

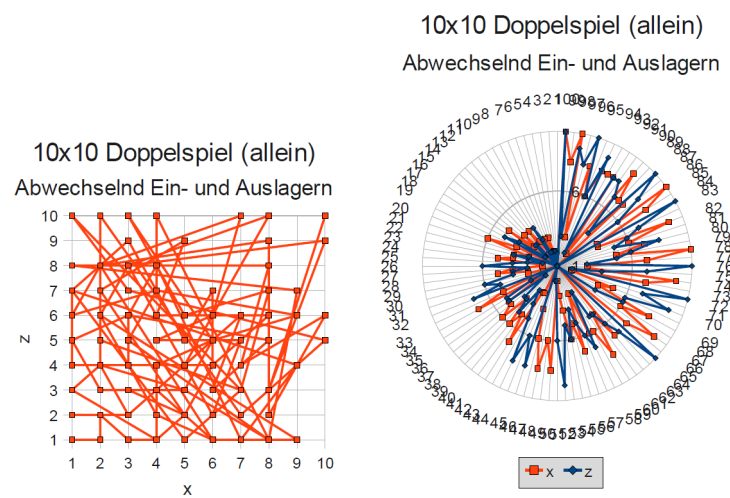


Abbildung 7.24:  $10 \times 10$  Doppelspiel mit ausschließlicher Gewichtung, abwechselnd Ein- und Auslagern

## 7.5 Zufall

In einem letzten Szenario werden zunächst alle Lagerplätze mit einem zufällig gewählten Gut aus einem Schatz zehn unterschiedlicher Güter belegt. Im Anschluss erfolgt die zufällige Auslagerung aller Güter. Zufall heißt in diesem Zusammenhang eben nicht, eine chaotische Lagerplatzzuweisung, als eine zufällige Lagerplatzwahl, durchzuführen, sondern ein zufälliges Gut zur Einlagerung, und im begrenzten Maß, zur Auslagerung zu bestimmen. Daher finden die üblichen Optimierungsalgorithmen hier auch durchaus Anwendung. Die Doppelspielstrategie ist beispielsweise in Abb. 7.26 bei den Operationsindizes 20 bis 22 und auch in Abb. 7.28 bei den Indizes 63 bis 69 wieder zu erkennen. Die anfängliche Einlagerung zeigt keine Auffälligkeiten (s. Abb. 7.25, 7.27).

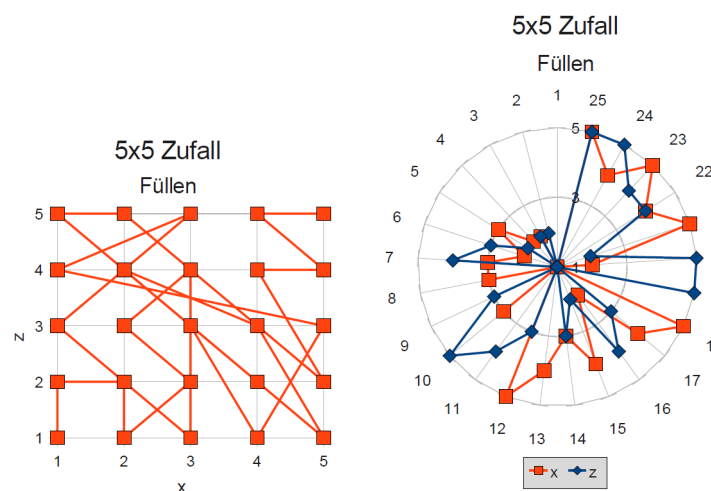


Abbildung 7.25:  $5 \times 5$  Zufällig

## 7.6 Berechnungszeiten

Für die obigen Szenarien wurden auch die Zeiten der Ausführung und der Berechnungen für die Optimierung gemessen. So erhöht sich beim Doppelspiel als alleiniges Optimierungskriterium die durchschnittliche Ausführungsdauer je Fahrt des RBG erwartungsgemäß bei einem Auslagerungsvorgang um ca. 0,5 Sekunden bei einem  $5 \times 5$ -Lager und um ca. 0,2 Sekunden bei einem  $10 \times 10$ -Lager. Dafür sinkt die Rechenzeit um ca.  $\frac{3}{4}$  der Rechenzeit auch mit allen Optimierungsstrategien bei einem  $5 \times 5$ -Lager und um  $\frac{2}{3}$  bei einem  $10 \times 10$ -Lager. Bei einem  $20 \times 20$ -Lager hingegen sinkt die benötigte Rechenzeit mit allen Optimierungsstrategien wieder, sodass hier keine fundierte Interpretation der Werte abgegeben werden kann.

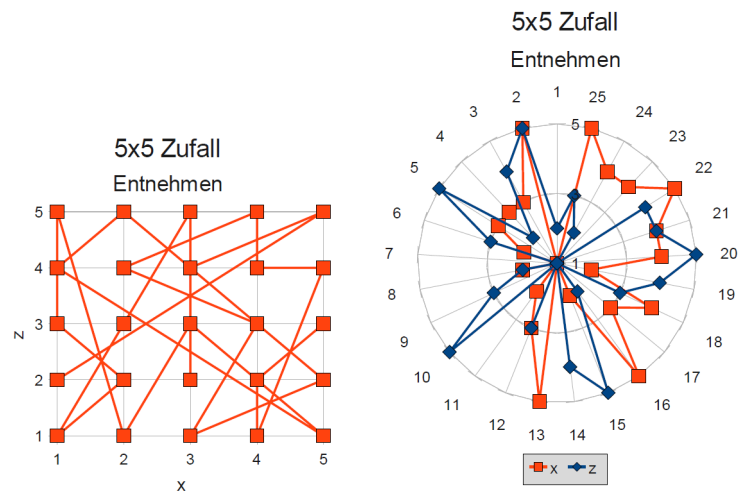


Abbildung 7.26:  $5 \times 5$  Zufällig

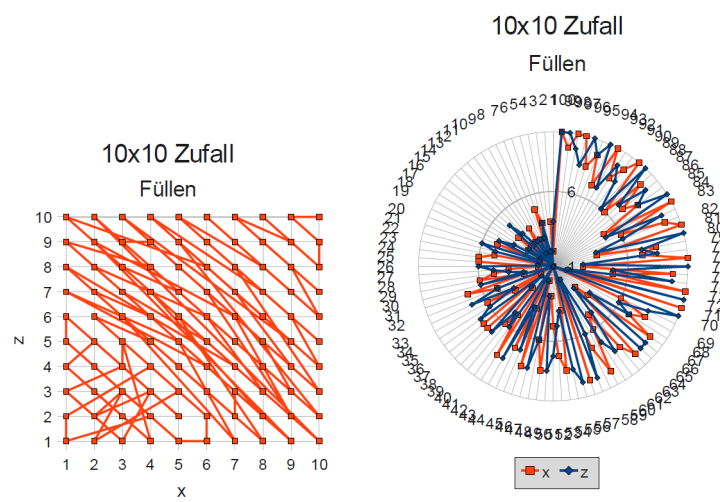


Abbildung 7.27:  $10 \times 10$  Zufällig

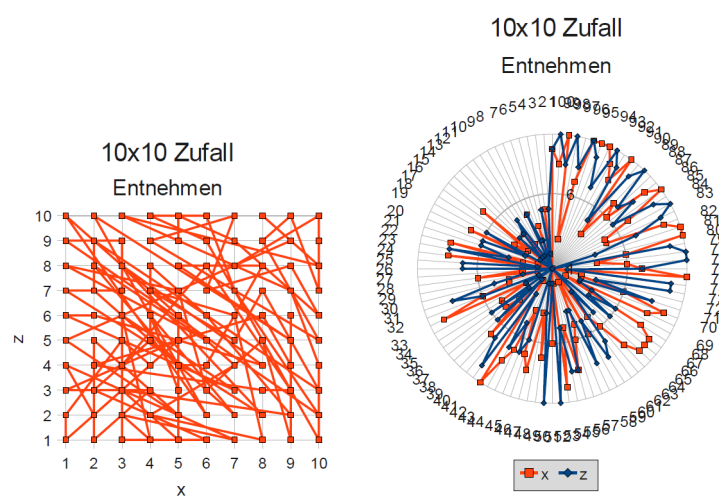


Abbildung 7.28:  $10 \times 10$  Zufällig

## 8 Abschließende Betrachtungen

### Rückblick

Die hardwareseitige Steuerung des Hochregallagers in der Digitalen Fabrik der Hochschule Emden/Leer ist bereits seit geraumer Zeit verfügbar und wurde seitdem stetig verbessert. Es existieren auch einige überliegende Ansätze im Zusammenhang mit der kompletten Automatisierung der Fabrik, die das Lager mit in den Betrieb einbinden. Dennoch wurde im Rahmen dieser Arbeit erstmalig eine Lagerverwaltung geschaffen, die Optimierungsalgorithmen nutzt, um einen noch schnelleren Zugriff auf die Güter im Hochregallager zu ermöglichen. Durch die lose gekoppelten Teilkomponenten, die sich nach den Schichten im LK<sup>3</sup>S-Modell ergeben, ist es auch möglich, dass fremde Projekte Teile der Arbeit nutzen können. So kann beispielsweise die Verwaltungsschicht (Warehouse Agent) ausgetauscht werden. Dann stehen jedoch sämtliche Optimierungsprozesse nicht länger zur Verfügung. Die abstrakte Hardwaresteuerung über den Warehouse Operator bliebe jedoch erhalten. Andererseits können auch die unteren Schichten getauscht werden, um neuartige Lager oder Lager mit anderer logischer und physikalischer Ansteuerung zu integrieren. Aus diesem Grund wurde das Konzept des Stubs und der Warehouse Experts eingeführt.

Wird das gesamte LK<sup>3</sup>S-Agentensystem mit allen seinen hardwarefernen und -nahen Komponenten betrachtet, das im Rahmen dieser Arbeit und auch in deren Vorfeld mit allen LK<sup>3</sup>S-Mitarbeitern entwickelt wurde, so ist es ein komplexer Verbund lose gekoppelter Einzelsysteme. Die aufwendige Entwicklung dieses Systems rentiert sich jedoch durch seine Flexibilität und Erweiterbarkeit. So lässt es sich mit den Warehouse Experts, den Agentengesichtern, Operatoren, Stubs und weiteren generellen Schnittstellen unproblematisch für spezifische Anwendungen erweitern und schnell über Konfigurationsanpassungen in Betrieb nehmen.

In die entwickelten und implementierten Optimierungsalgorithmen, fließen die äußeren Randbedingungen wie die geometrischen Gegebenheiten, Zugriffshäufigkeiten und die Kombinationsmöglichkeiten mit anderen Lageroperationen (Doppelspiele) ein. Diese Algorithmen wurden datenbankseitig realisiert und sind ebenso erweiterbar wie auch andere Komponenten. Möglich macht dies eine sog. Metrik, die von jedem Optimierungsalgorithmus einzeln für alle vorhandenen Lagerplätze gebildet wird. Schließlich reicht ihre einfache Summierung, um eine Gesamtmetrik zu erhalten, die entscheidend für die Bestimmung des einen Lagerplatzes ist, der für eine Ein- oder Auslagerung herangezogen wird.

Ein anderer Teil dieser Arbeit hat sich softwareseitig mit der Konfiguration und der Einrichtung von Hochregallagern befasst. Hierzu wurde das Werkzeug Warehouse Director geschaffen. Es ermöglicht eine komfortable visuelle Modellierung von Lagerstrukturen mit mehreren Lagerzonen, jeweils zwei Regalwänden, beliebig positionierbaren Lagerzugängen und die Parametrierung der Lagerplätze in Dimension und Belastbarkeit. Zudem wurden bereits Vorkehrungen getroffen, um Durchlaufregale in Betrieb zu nehmen. Diese können bereits rudimentär konfiguriert werden, eine entsprechende Lagerverwaltung auf logischer Ebene mit dem Warehouse Agent ist jedoch noch nicht umgesetzt.

Neben der Modellierung unterstützt der Warehouse Director auch maßgeblich bei der Einrichtung der vom Warehouse Agent genutzten Datenbank, die sämtliche Stammdaten, aber auch Bewegungs- und Bestandsdaten umfasst. So werden Relationen automatisch erzeugt und Benutzer für die Bedienung über andere Softwarewerkzeuge eingerichtet, sodass hier auch ein Grundmaß an Sicherheit gegenüber Eindringlingen geschaffen wurde.

Da der Warehouse Director bereits eine visuelle Darstellung des Lagerlayouts besitzt, lag es nah, eine Visualisierungsfunktion zu integrieren. So kann sich der Warehouse Director einerseits zur selbst erzeugten Datenbank und andererseits zum Warehouse Operator verbinden, um aktuelle Belegungsdaten abzurufen und um über neu eingetretene Ereignisse auf RBG-Ebene informiert zu werden. Dazu wurde eine für den Warehouse Operator transparente Warehouse Extension geschaffen, die Ereignisse verzweigt und entsprechend zum Warehouse Director und zum eigentlichen Warehouse Expert weiterleitet, der die reale Hardwaresteuerung übernimmt. Die Darstellung erfolgt im Warehouse Director über die Einzeichnung der aktuellen Fahrwege des RBG als Überlagerung des bereits sichtbaren Lagerlayouts. Überdies erfolgt eine Belegungsanzeige der einzelnen Lagerplätze.

Um den Effekt der Optimierungsalgorithmen prüfen zu können, wurde eine weitere Funktion in den Warehouse Director integriert. Dieser kann sich mit Datenbank, Warehouse Operator und Warehouse Agent verbinden und selbstständig Operationen im Lager anhand vordefinierter Szenarios auslösen, indem Befehle an den Agenten gesendet werden. Da insbesondere bei einer Leistungsmessung der Algorithmen jedoch nicht die reale Lagerhardware bedient werden soll – auch aus Sicherheitsgründen –, unterstützt der bereits von der Visualisierung bekannte, transparent arbeitende Warehouse Expert auch die Simulation. Wird dieser vom Agenten mit einem Fahrauftrag versehen, leitet er die entsprechenden Informationen an den Warehouse Director weiter, sodass dieser über eine Formel anhand voriger Benutzereingaben die virtuellen Fahrzeiten berechnen und zurück an den Operator geben kann, der diese wiederum an den Agenten zurück sendet. Zeitgleich erfolgt die hiervon getrennt arbeitende Visualisierung und alle Ereignisse werden in einer Logdatei zur späteren Auswertung festgehalten.

### Ausblick

Es existieren Punkte, in denen das System noch erweitert und verbessert werden kann. Hierzu zählt z. B. die Integration des Handlings für Durchlaufregale im Warehouse Agent. So könnte die FESTO-Fabrik im Echtzeitlabor der Hochschule Emden/Leer als reale Testumgebung dienen, wenn die vorhandene Lagermatrix virtuell in Spalten geteilt und diese entsprechend wie Durchläufe bedient werden.

Die implementierten Optimierungsstrategien bieten grundlegende Optimierungsfunktionen, jedoch sind diese gänzlich unabhängig voneinander (Einlagerung) bzw. sie können maximal die Menge potentieller Lagerplätze weiter verringern (Auslagerung). Sollen jedoch komplexere Optimierungsstrategien implementiert werden, die Abhängigkeiten untereinander besitzen, so müssen hier Veränderungen vorgenommen werden.

Der Warehouse Director ist im Stande, Benchmarks durchzuführen. Neben der Ermittlung der durchschnittlichen Ausführungs- und Berechnungsdauer für alle durchgeführten Lageroperationen besitzt er allerdings keine weiteren Funktionen zur Auswertung eines Benchmarks. Es wäre somit denkbar, dass hier eine Erweiterung stattfindet, die die Ergebnisse automatisch verarbeitet und eventuell sogar in die Optimierungsparameter des Warehouse Agent eingreift, um so ein Optimum für ein oder mehrere Szenarien zu finden. Somit kann diese Arbeit als Grundlage für weitere Forschungen in diesem Bereich wie z. B. zur Erweiterung und fortführenden Auswertung der Benchmarks oder dem Entwurf von Werkzeugen für die automatische Parameteroptimierung der Optimierungsstrategien dienen.

Sobald das Agentensystem über einen Agenten verfügt, der in der Lage ist, eine weitläufige Produktionsplanung durchzuführen und durchzusetzen, kann die Lagerverwaltung ihr volles Potential z. B. bei der Optimierung nach Doppelspielen unter Verwendung von Deadlines für Auslagerungsanforderung ausschöpfen. Die Lagerverwaltung ist unter diesem Ziel entwickelt worden und es bedarf nur wenigen Änderungen, um diese Funktionalität in die Lagerkomponenten zu integrieren. Eine entsprechende Optimierungsstrategie ist bereits teilweise implementiert und in Kapitel 4 beschrieben worden.



# Literaturverzeichnis

- [1] AMATO, F., AND BASILE, F. Optimal control of warehousing systems with simultaneous crane and shuttle optimization. vol. 2, pp. 95 –104 vol.2.
- [2] BENEFIT OF HINDSIGHT. high bay, Jun. 2006. Creative Commons-Lizenz Attribution-NonCommercial-NoDerivs 2.0 Generic, <http://www.flickr.com/photos/cdell/180841962/>, besucht am 20.08.2010.
- [3] BOLLINGER, J., AND ET AL. *VISIONARY MANUFACTURING CHALLENGES FOR 2020*. National Academy Press, Washington DC, 1998.
- [4] BUNDESMINISTERIUM DER JUSTIZ. Abgabenordnung AO, July 2009. [http://bundesrecht.juris.de/ao\\_1977/index.html](http://bundesrecht.juris.de/ao_1977/index.html) besucht am 29.04.2010.
- [5] BUNDESMINISTERIUM DER JUSTIZ. Handelsgesetzbuch HGB, July 2009. <http://bundesrecht.juris.de/hgb/index.html> besucht am 29.04.2010.
- [6] BUSCH, J. H., AND OSTERMANN, T. Energieeffizientes Hochregallager. Vertiefungsprojekt im Studiengang M.Eng. Industrial Informatics, June 2010.
- [7] CACHAPA, D., COLOMBO, A., FEIKE, M., AND BEPPERLING, A. An approach for integrating real and virtual production automation devices applying the service-oriented architecture paradigm. *Emerging Technologies and Factory Automation, 2007. ETFA. IEEE Conference on* (Sept. 2007), 309–314.
- [8] DÍAZ, B. D. VRP Web, March 2007. <http://neo.lcc.uma.es/radi-aeb/WebVRP/> besucht am 26.04.2010.
- [9] EU-COMMISSION, Ed. *MANUFACTURE — a vision for 2020 Assuring the future of manufacturing in Europe* (Luxembourg, 2004), Office for Official Publications of the European Communities, ISBN 92-894-8322-9.
- [10] FA. LTW LAGERTECHNIK GMBH, WOLFURT, ÖSTERREICH. *Automatisches Regalbediengerät, kurvengängig mit Teleskopgabel für Europaletten*, Feb. 2006. <http://de.wikipedia.org/wiki/Datei:Regalbedienger%C3%A4t.automatisch.kurveng%C3%A4ngig.zweimastig.Teleskopgabel.Euro.jpg>.
- [11] FOUNDATION FOR INTELLIGENT PHYSICAL AGENTS (FIPA). FIPA Communicative Act Library Specification. <http://www.fipa.org/specs/fipa00037/SC00037J.html> besucht am 07.07.2010.
- [12] GLASS, M. *Schnellläuferstrategien in Lagern und dynamische Zonierung*, Sept. 2005. TU Dresden.

- [13] INTEL CORPORATION, AND SEAGATE TECHNOLOGY. Serial ATA – Native Command Queueing, July 2003. [http://www.seagate.com/content/pdf/whitepaper/D2c\\_tech\\_paper\\_intc-stx\\_sata\\_ncq.pdf](http://www.seagate.com/content/pdf/whitepaper/D2c_tech_paper_intc-stx_sata_ncq.pdf).
- [14] INTERNATIONAL ELECTROTECHNICAL COMMISSION (IEC). IEC 62264-1: Enterprise-control system integration – Part 1: Models and terminology, 2003.
- [15] KOEHLER, J., AND SCHUSTER, K. Elevator control as a planning problem. AAAI Press, Menlo Park, pp. 331–338.
- [16] LENZE DRIVE SYSTEMS GMBH. *Communication Manual CAN*. 2003.
- [17] LEO GMBH. Suche nach „performative“. <http://dict.leo.org/forum/viewUnsolvedquery.php?idThread=693224&idForum=1&lp=ende&lang=de#followup5> besucht am 06.07.2010.
- [18] LEPUSCHITZ, W., VALLEE, M., MERDAN, M., VRBA, P., AND RESCH, J. Integration of a heterogeneous low level control in a multi-agent system for the manufacturing domain. In *IEEE Conference on Emerging Technologies & Factory Automation, 2009. ETFA 2009* (Sept. 2009).
- [19] MALMBORG, C., AND KRISHNAKUMAR, B. Optimal storage assignment policies for multiaddress warehousing systems. *Systems, Man and Cybernetics, IEEE Transactions on* 19, 2 (Mar/Apr 1989), 197 –204.
- [20] MARTIN, H. *Transport und Lagerlogistik*. Vieweg+Teubner, 2009.
- [21] POSTGRESQL. PostgreSQL: Documentation: Manuals: PostgreSQL 8.4: Procedural Languages. <http://www.postgresql.org/docs/8.4/interactive/xplang.html> besucht am 26.06.2010.
- [22] RHODEN, F. Programmierung und Visualisierung eines Hochregallagers, Aug. 2008.
- [23] SCHUMANN, M. *Zur Bestimmung der Umschlagleistung von Hochregallagern unter besonderer Berücksichtigung der Lagerorganisation*, Sept. 2007. Technische Universität Chemnitz.
- [24] STILL GMBH. Pressedownloads. <http://www.still.de/599.0.43.html> besucht am 10.05.2010, Genehmigung siehe Anhang B.
- [25] TEN HOMPEL, M., AND HEIDENBLUT, V. *Taschenlexikon Logistik*. Springer-Verlag Berlin Heidelberg, 2008.
- [26] VDMA. FEM 9.851, 2003.
- [27] VIRKUS, K. Agent-Operator-Protokoll, Aug. 2010.
- [28] VIRKUS, K. Strategien zur dynamischen Lageroptimierung, June 2010. Vertiefungsprojekt im Studiengang M.Eng. Industrial Informatics.
- [29] VIRKUS, K., BARKHOFF, M., SCHMIDTMANN, U., KREUTZ, G., STÖCKMANN, T., AND JOVIC, M. Leicht konfigurierbare Hardware-Abstraktionsschicht für flexible Automatisierung. In *Software-intensive verteilte Echtzeitsysteme* (Jan 2010), Springer Berlin Heidelberg.

- [30] VIRKUS, K., BARKHOFF, M., STÖCKMANN, T., SCHMIDTMANN, U., AND KREUTZ, G. Applied shop floor scheduling and sequencing in an agent-controlled production, July 2010. to be published.
- [31] WERMANN, J., LÜCKE, P., AND KYEK, J. Visualisierung des Hochregallagers (Arbeitstitel). 2010.
- [32] WIKIPEDIA DEUTSCHLAND. Regalbediengerät, Feb. 2010. <http://de.wikipedia.org/wiki/Regalbedienger%C3%A4t> besucht am 27.04.2010.
- [33] WOLF, K.-H. *Hochregallager im Bau*, July 2004. <http://de.wikipedia.org/wiki/Datei:Hochregallager.im.Bau.jpg>.

# A Konfigurationen für die Digitale Fabrik

Alle gezeigten Konfigurationen beziehen sich ausschließlich auf die Lagerfunktionalitäten. Es sind weitere Konfigurationen notwendig, um einen Arbeitsplatz in Betrieb zu nehmen.

## A.1 LK<sup>3</sup>S-Agent

Listing A.1: Konfiguration des Agenten.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <agent-config>
3     <name>WarehouseAgent</name>
4     <description>Warehouse Agent</description>
5     <xsd-files>/usr/local/etc/lk3s/xsd</xsd-files>
6     <logging>
7         <host>192.168.157.220</host>
8         <name>logging</name>
9         <table>logbook</table>
10        <user>logger</user>
11        <password>logger</password>
12    </logging>
13    <faces>
14        <face class="DFFederation" config="/usr/local/etc/lk3s/
15            agents/ap5/face-dffederation.xml" />
16        <face class="operators" config="/usr/local/etc/lk3s/
17            agents/ap5/face-operators.xml" />
18        <face class="links" config="/usr/local/etc/lk3s/agents/
19            ap5/face-links.xml" />
20        <face class="transport" config="/usr/local/etc/lk3s/
21            agents/ap5/face-transport.xml" />
22        <face class="storage" config="/usr/local/etc/lk3s/agents
23            /ap5/face-storage.xml" />
24        <face class="orders" config="/usr/local/etc/lk3s/agents/
25            ap5/face-orders.xml" />
26    </faces>
27 </agent-config>
```

## A.2 OperatorsFace

Listing A.2: Konfiguration des OperatorsFace.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <operators-config>
3   <port>11005</port>
4   <backlog>1</backlog>
5   <max-operators>0</max-operators>
6 </operators-config>
```

## A.3 LinksFace

Listing A.3: Konfiguration des LinksFace.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <links-config>
3   <links>
4     <link link-uid="I02" stop-uid="I02" incoming="incoming"
        outgoing="outgoing" />
5     <link link-uid="I03" stop-uid="I03" incoming="incoming"
        outgoing="outgoing" />
6   </links>
7 </links-config>
```

## A.4 TransportFace

Listing A.4: Konfiguration des TransportFace.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <transport-config>
3   <accept-all-items>true</accept-all-items>
4   <pathop-name>pathop</pathop-name>
5   <stops>
6     <stop>I02</stop>
7     <stop>I03</stop>
8   </stops>
9 </transport-config>
```

## A.5 StorageFace

Zunächst bedarf es einer Einbindung des StorageFace in den LK<sup>3</sup>S-Agenten. Dazu wird dessen Konfiguration folgende Zeile im Abschnitt `faces` hinzugefügt:

```
<face class="storage"
      config="/etc/lk3s/agents/warehouse/face-storage.xml" />
```

Sie zeigt zudem auf die Konfigurationsdatei, die im Anschluss durch das StorageFace geladen wird:

Listing A.5: Konfiguration des StorageFace.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <storage-config>
3   <simulation-port>2013</simulation-port>
4   <database>
5     <host>127.0.0.1</host>
6     <name>warehouse</name>
7     <user>warehouseactor</user>
8     <password>warehouseactor</password>
9   </database>
10  <weights>
11    <import-geometry>1</import-geometry>
12    <import-filldegree>1</import-filldegree>
13    <import-samematerial>1</import-samematerial>
14    <import-statisticalcellposition>1</import-
15      statisticalcellposition>
16    <export-fifo>1</export-fifo>
17    <export-deadlines>1</export-deadlines>
18    <export-dualcommand>1</export-dualcommand>
19    <export-material>1</export-material>
20  </weights>
21  <assignments>
22    <assign link-uid="I02" zone-uid="main" gate-uid="I02" />
23    <assign link-uid="I03" zone-uid="main" gate-uid="I03" />
24  </assignments>
25</storage-config>
```

## A.6 Operator Skeleton – skeleton-warehouse.xml

Die Konfiguration des Operator Skeleton zu einem Warehouse Operator geschieht über folgende Konfiguration:

Listing A.6: Konfiguration des Operator Skeleton.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <skeleton-config>
3   <xsd-files>/usr/local/etc/lk3s/xsd</xsd-files>
4   <extension-class>
5     de.i2ar.warehouseop.ExtensionWarehouse
```

```
6 </extension-class>
7 <extension-config>
8   /usr/local/etc/lk3s/operators/ap5/warehouse/extension.xml
9 </extension-config>
10 <dummy-execution>false</dummy-execution>
11 <maintenance port="10005" backlog="1" />
12 <agents>
13   <agent-registration-timeout>
14     1000
15   </agent-registration-timeout>
16   <agent-job-timeout>0</agent-job-timeout>
17   <agent ip="192.168.157.220" port="11005" />
18 </agents>
19 </skeleton-config>
```

## A.7 Warehouse Extension – extension-warehouse.xml

Über diese Konfigurationsdatei wird die Warehouse Extension des Operators konfiguriert:

Listing A.7: Konfiguration der Warehouse Extension.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <warehouseconfig>
3   <name>WarehouseOperator</name>
4   <description>Warehouse Operator</description>
5   <expert-class>de.i2ar.warehouseop.experts.simulation.
     SimulationExpert</expert-class>
6   <expert-config>/usr/local/etc/lk3s/operators/ap5/warehouse/
     expert-simulation-dummy.xml</expert-config>
7 </warehouseconfig>
```

## A.8 Warehouse Expert – expert-hrw.xml

Die Konfigurationsdatei für den HRW-Expert (Digitale Fabrik) besitzt einen knappen Aufbau, der nur den Kontakt zum zuständigen Stub definiert:

Listing A.8: Konfiguration des HRW-Expert.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <expert-hrw-config>
3   <stub>
```

```
4         <ip>192.168.157.226</ip>
5         <port>2008</port>
6     </stub>
7 </expert-hrw-config>
```

## A.9 Simulation Expert – expert-simulation.xml

Listing A.9: Konfiguration des Simulation Expert.

```
1 <simulationExpertConfig>
2     <simulation-host>127.0.0.1</simulation-host>
3     <simulation-port>2012</simulation-port>
4     <expert-class>de.i2ar.warehouseop.experts.dummy.
        WarehouseExpert </expert-class>
5     <expert-config>null</expert-config>
6 </simulationExpertConfig>
```

## A.10 Datenbank – cells

Listing A.10: Skript zur Erzeugung der Datenbanktabellen.

```
1 -- Table: cells
2
3 -- DROP TABLE cells;
4
5 CREATE TABLE cells
6 (
7     zoneuid character varying(32) NOT NULL,
8     celluid character varying(32) NOT NULL,
9     reserved character varying(45) DEFAULT NULL::character varying
10    ,
11     "action" character varying(5) NOT NULL DEFAULT 'INOUT'::
        character varying,
12     x integer NOT NULL,
13     y integer NOT NULL,
14     z integer NOT NULL,
15     width integer NOT NULL,
16     height integer NOT NULL,
17     depth integer NOT NULL,
18     maxload integer NOT NULL,
19     strategy character varying(4) NOT NULL DEFAULT 'LIFO'::
        character varying,
20     capacity integer NOT NULL,
```



```

20     flowzoneuid character varying(32) DEFAULT NULL::character
        varying,
21     flowcelluid character varying(32) DEFAULT NULL::character
        varying,
22     CONSTRAINT cells_pkey PRIMARY KEY (zoneuid, celluid)
23 )
24 WITH (
25     OIDS=FALSE
26 );
27 ALTER TABLE cells OWNER TO director;
28 GRANT ALL ON TABLE cells TO director;
29 GRANT SELECT ON TABLE cells TO warehouseactor;
30 GRANT SELECT ON TABLE cells TO warehouseobserver;
31
32 -- Table: gates
33
34 -- DROP TABLE gates;
35
36 CREATE TABLE gates
37 (
38     zoneuid character varying(32) NOT NULL,
39     gateuid character varying(32) NOT NULL,
40     virtualx integer NOT NULL,
41     virtualy integer NOT NULL,
42     virtualz integer NOT NULL,
43     CONSTRAINT gates_pkey PRIMARY KEY (zoneuid, gateuid)
44 )
45 WITH (
46     OIDS=FALSE
47 );
48 ALTER TABLE gates OWNER TO director;
49 GRANT ALL ON TABLE gates TO director;
50 GRANT SELECT ON TABLE gates TO warehouseactor;
51 GRANT SELECT ON TABLE gates TO warehouseobserver;
52
53 -- Table: items
54
55 -- DROP TABLE items;
56
57 CREATE TABLE items
58 (
59     itemuid character varying(45) NOT NULL,
60     chipuid character varying(45) NOT NULL,
61     authority character varying(45) NOT NULL,
62     orderuid character varying(45) DEFAULT NULL::character varying
        ,

```

```

63  entered bigint NOT NULL,
64  "left" bigint,
65  expires bigint,
66  import bigint NOT NULL,
67  export bigint,
68  priority integer DEFAULT 0,
69  zoneuid character varying(32) NOT NULL,
70  celluid character varying(32) NOT NULL,
71  material boolean NOT NULL DEFAULT false
72 )
73 WITH (
74     OIDS=FALSE
75 );
76 ALTER TABLE items OWNER TO director;
77 GRANT ALL ON TABLE items TO director;
78 GRANT SELECT, UPDATE, INSERT, DELETE ON TABLE items TO
    warehouseactor;
79 GRANT SELECT ON TABLE items TO warehouseobserver;
80
81 -- Table: material
82
83 -- DROP TABLE material;
84
85 CREATE TABLE material
86 (
87     itemuid character varying(45) NOT NULL,
88     typeuid character varying(45) NOT NULL,
89     x integer NOT NULL,
90     y integer NOT NULL,
91     z integer NOT NULL,
92     angle integer NOT NULL,
93     width integer NOT NULL,
94     colour character varying(45) NOT NULL
95 )
96 WITH (
97     OIDS=FALSE
98 );
99 ALTER TABLE material OWNER TO director;
100 GRANT ALL ON TABLE material TO director;
101 GRANT SELECT, UPDATE, INSERT, DELETE ON TABLE material TO
    warehouseactor;
102 GRANT SELECT ON TABLE material TO warehouseobserver;
103
104 -- Table: requests
105
106 -- DROP TABLE requests;

```

```

107
108 CREATE TABLE requests
109 (
110     id bigint NOT NULL DEFAULT nextval('requestid'::regclass),
111     itemuid character varying(45),
112     typeuid character varying(45),
113     width integer,
114     colour character varying(45),
115     amount integer NOT NULL,
116     date bigint NOT NULL,
117     deadline bigint,
118     authority character varying(45) NOT NULL,
119     gateuid character varying(32) NOT NULL,
120     done boolean NOT NULL DEFAULT false,
121     CONSTRAINT requests_pkey PRIMARY KEY (id)
122 )
123 WITH (
124     OIDS=FALSE
125 );
126 ALTER TABLE requests OWNER TO director;
127 GRANT ALL ON TABLE requests TO director;
128 GRANT SELECT, UPDATE, INSERT, DELETE ON TABLE requests TO
    warehouseactor;
129 GRANT SELECT ON TABLE requests TO warehouseobserver;

```

## **B Genehmigung STILL**

### **Anfrage**

Sehr geehrter Herr Wrusch,

ich möchte Sie höflichst um Erlaubnis bitten, einige der unter Pressedownloads zur Verfügung gestellten Bilder in meiner Master Thesis zum Thema Hochregallageroptimierung abzu drucken.

Mit freundlichen Grüßen Kristian Virkus

### **Antwort**

Hallo Herr Virkus,

Uns freut, dass Sie STILL bei Ihrer Masterarbeit einbeziehen moechten. Gerne duerfen Sie die Bilder nutzen. Viel Erfolg wuenscht Ihnen Mit freundlichen Grüßen

Jürgen Wrusch International Corporate Communications \_\_\_\_\_  
\_\_\_\_\_ STILL GmbH Berzeliusstraße 10 D-22113 Hamburg [www.still.de](http://www.still.de) Tel.: +49  
(0)40/73 39 - 1508 Fax: +49 (0)40/73 39 - 16 18 E-Mail: [juergen.wrusch@still.de](mailto:juergen.wrusch@still.de)